

3X-UI 面板使用者手冊

 العربية ·  English ·  Español ·  فارسی ·  Bahasa Indonesia ·  日本語 ·  Português ·  Русский ·  Türkçe ·  Українська ·  Tiếng Việt ·  简体中文 ·  繁體中文

3X-UI 版本：3.5.0。本手冊依據此版本撰寫，內容適用於該版本。3.5.0 相對於 3.4.2 的變更摘要，請見「[3.5.0 新功能](#)」一節。

本手冊是 **3X-UI** 網頁面板（管理 Xray-core）的詳細使用說明，涵蓋 功能介紹、設定方法與日常操作，並逐一說明介面中的每個欄位與 開關。

各名稱與標籤均與面板介面保持一致。inbound / outbound 二詞 不作翻譯。

目錄

- 3.5.0 新功能
- 1. 簡介、需求與安裝
 - 1.1. 什麼是 3X-UI
 - 1.2. 支援的作業系統與架構
 - 1.3. 安裝方式
 - 1.4. 首次啟動與預設憑證
 - 1.5. 檔案位置
 - 1.6. 管理指令 `x-ui` (腳本選單)
 - 1.7. `x-ui` 子命令 (無互動選單)
 - 1.8. 從 SQLite 遷移至 PostgreSQL
- 2. 登入面板與存取安全
 - 2.1. 登入表單
 - 2.2. 雙重要素驗證 (2FA / TOTP)
 - 2.3. 登入嘗試限制 (login limiter / 暴力破解防護)
 - 2.4. 變更管理員登入名稱與密碼
 - 2.5. 秘密路徑 (URI 路徑 / webBasePath) 與面板連接埠
 - 2.6. 工作階段存活時間 (逾時)
 - 2.7. LDAP (同步與身分驗證)
- 3. 概覽 / 儀表板
 - 3.1. 資料收集的一般原則
 - 3.2. CPU
 - 3.3. 記憶體 (RAM)
 - 3.4. 交換空間 (Swap)
 - 3.5. 磁碟 (Storage)
 - 3.6. 系統運行時間 (Uptime)
 - 3.7. 系統負載 (Load average)
 - 3.8. 網路：速率與總流量
 - 3.9. 伺服器 IP 位址
 - 3.10. TCP/UDP 連線
 - 3.11. Xray 狀態與程序管理
 - 3.12. 面板更新 (3X-UI)
 - 3.13. 地理位置檔案更新 (GeoIP / GeoSite)
 - 3.14. 資料庫備份與還原
 - 3.15. 其他介面元素
- 4. Inbounds：建立與通用參數
 - 4.1. 通用表單欄位
 - 4.2. Sniffing (嗅探)
 - 4.3. Allocate (連接埠分配策略)
 - 4.4. External Proxy (外部代理)
 - 4.5. Fallbacks (Fallback)
 - 4.6. 定期重設流量
 - 4.7. Inbound 的 JSON (advanced)
 - 4.8. inbound 的操作：QR / Edit / Reset / Delete 與統計

- 5. 協議
 - 5.1. 支援協議列表
 - 5.2. 各協議對 TLS / REALITY / 傳輸的支援
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (透明轉發器)
 - 5.8. SOCKS / HTTP (mixed 協議)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (預設 v2)
 - 5.11. MTPROTO (Telegram 代理)
 - 5.12. 協議選擇速查表
- 6. 傳輸 (Stream Settings)
 - 6.1. 選擇傳輸網路
 - 6.2. RAW / TCP (tcpSettings)
 - 6.3. mKCP (kcpSettings)
 - 6.4. WebSocket (wsSettings)
 - 6.5. gRPC (grpcSettings)
 - 6.6. HTTPUpgrade (httpupgradeSettings)
 - 6.7. XHTTP / SplitHTTP (xhttpSettings)
 - 6.8. Hysteria 傳輸 (hysteriaSettings)
 - 6.9. 相關參數
- 7. 連線安全性：TLS、XTLS 與 REALITY
 - 7.1. 差異說明：TLS vs XTLS vs REALITY
 - 7.2. 「無」模式 (none)
 - 7.3. TLS 模式
 - 7.4. REALITY 模式
 - 7.5. 設定實務建議
- 8. 客戶端
 - 8.1. 客戶端欄位
 - 8.2. 綁定至 inbound
 - 8.3. 客戶端操作
 - 8.4. 批量操作
 - 8.5. 搜尋、篩選與排序
 - 8.6. 狀態標誌與狀態
- 9. 客戶端群組
 - 9.1. 什麼是客戶端群組，以及為何需要它
 - 9.2. 群組與客戶端、inbound、節點及協定的關聯
 - 9.3. 群組目錄與「空」群組
 - 9.4. 群組的欄位與資料表欄位
 - 9.5. 建立群組
 - 9.6. 重新命名群組
 - 9.7. 將客戶端新增到群組
 - 9.8. 將客戶端從群組移除 (不刪除客戶端本身)

- 9.9. 重設群組流量
- 9.10. 刪除群組與刪除群組客戶端
- 9.11. 與「客戶端」頁面的關聯
- 9.12. API 端點彙總
- 9.13. 依群組的流量
- 10. 訂閱 (Subscription)
 - 10.1. 什麼是 subId 以及連結的組成方式
 - 10.2. 訂閱伺服器設定
 - 10.3. 輸出格式
 - 10.4. 訂閱資訊頁面與 QR 碼
 - 10.5. 自訂訂閱頁面範本
- 11. Xray: 路由、outbounds、DNS 與擴充功能
 - 11.1. 編輯器結構：分頁/模式
 - 11.2. 主要設定 (General)
 - 11.3. 路由規則 (routing)
 - 11.4. Outbounds (外出連線)
 - 11.5. 負載均衡器 (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. 反向代理與 TUN
 - 11.10. 日誌與統計 (Stats, metrics)
 - 11.11. 儲存、重啟與自動轉換
 - 11.12. 訂閱的 Outbound (含自動更新)
 - 11.13. WARP 的 IP 輪換
- 12. 節點 (多面板, master/slave)
 - 12.1. 清單頂部摘要
 - 12.2. 新增和編輯節點
 - 12.3. TLS 驗證 (適用於 https 節點)
 - 12.4. 每個節點顯示的內容
 - 12.5. 節點操作
 - 12.6. 指標歷史
 - 12.7. inbound 和用戶端如何同步
 - 12.8. 節點鏈 (子節點/傳遞節點)
 - 12.9. 3.5.0 修正 (節點穩定性)
 - 12.10. 節點：3.3.0 版本的新功能
- 13. 面板設定
 - 13.1. 儲存與重新啟動面板
 - 13.2. 一般設定 (「面板」分頁 / General)
 - 13.3. 面板存取：IP、連接埠、路徑、網域、憑證
 - 13.4. 工作階段、面板代理與受信任代理 (「代理與伺服器」分頁 / Proxy and Server)
 - 13.5. Telegram 機器人 (「Telegram 機器人」分頁 / Telegram Bot)
 - 13.6. 日期與時間 (「日期與時間」分頁 / Date and Time)
 - 13.7. 外部流量與 Xray 行為 (「外部流量」分頁 / External Traffic)
 - 13.8. 其他：Xray 組態範本與檢查 URL
 - 13.9. 管理員帳號與 API 權杖

- 13.10.3.3.0 中的 API 變更（對整合很重要）
- 14. Telegram 機器人
 - 14.1. 啟用與設定機器人
 - 14.2. 主選單與按鈕
 - 14.3. 機器人指令
 - 14.4. 用戶端管理（僅限管理員）
 - 14.5. 通知與報告
 - 14.6. 備份與記錄
 - 14.7. 運作特性
- 15. 地理資料庫（geoip / geosite 與自訂）
 - 15.1. geoip.dat 與 geosite.dat 是什麼
 - 15.2. 標準地理檔案及其更新
 - 15.3. 透過 Xray 自動更新地理資料（Geodata Auto-Update）
 - 15.4. 驗證與限制
 - 15.5. 面板啟動時的自動檢查
 - 15.6. 在路由規則中使用地理資料庫
- 16. 日常運維：備份、日誌、更新、CLI
 - 16.1. 資料庫備份與還原
 - 16.2. 日誌檢視
 - 16.3. Xray 日誌等級與設定
 - 16.4. 管理 Xray：停止與重啟
 - 16.5. 重啟與更新面板
 - 16.6. 週期性任務（cron）
 - 16.7. 主控台選單與 CLI（`x-ui`）
 - 16.8. 移除面板
 - 16.9. `x-ui migrateDB` 命令

3.5.0 新功能

3.5.0 是一次重大更新：MTProto 改採多客戶端模式（mtg-multi 引擎、個人 secret、配額與 ad-tag），受管主機改為群組式（一筆記錄可含多個 inbound 與多個地址），PostgreSQL 面板的還原功能可接受 SQLite 備份，outbound 新增「Target Strategy」、「Real delay」測試與 Egress/Country 欄位，負載均衡器可將另一個負載均衡器用作 fallback。隨附 Xray 核心 26.7.11。以下列出相對於 3.4.2 的變更，依手冊章節分組整理。

第 1 節的變更 – 簡介、需求與安裝

- Xray 核心更新至 **26.7.11**。隨附的自動遷移：Shadowsocks 的 `none / plain` 與 VMess 的 `none / zero` 加密方式已從核心移除（已儲存的設定會自動改寫），指向公網地址的未加密 VLESS/Trojan outbound 在儲存時會被拒絕。
- 新增 `x-ui pgclient [版本]` 指令與 PostgreSQL 選單中的 **10. Install/Upgrade client tools (pg_dump/pg_restore)** 項目——安裝 / 更新 PostgreSQL 客戶端工具。
- 腳本修正：RHEL 系上的 PostgreSQL 與 fail2ban 安裝（EPEL）、Arch 不再執行完整的 `pacman -Syu`、32 位元 ARM 上 Xray 二進位檔名稱正確（`xray-linux-arm32`）、簽發 IP 憑證前確認自動偵測的 IPv4、修正選擇預設 ACME 連接埠時誤報「Your input is invalid」的問題。

第 2 節的變更 – 登入面板與存取安全

- IP 限制：「死」連線現在只會被封鎖一次，而非每次 10 秒掃描都觸發——fail2ban 計數器不再虛增，也無需調高 `maxretry`。

第 4 節的變更 – Inbounds：建立與通用參數

- Inbound 清單新增**搜尋**功能（依備註、連接埠與協定），節點下拉選單（「Deploy to」、「節點」篩選器）也支援輸入搜尋。

第 5 節的變更 – 協議

- **MTProto 改採多客戶端模式**（mtg-multi 引擎）：MTProto 使用者現在是一般客戶端，擁有自己的 secret、配額、有效期、ad-tag 和個人 `tg://proxy` 連結。inbound 層級的「Secret」欄位已移除（既有 inbound 會自動轉換），「FakeTLS domain」成為新 secret 的預設域名。新的 inbound 欄位：**Max connections**（連線數限制）、**Public IPv4/IPv6**（用於 ad-tag middle proxy）。客戶端變更「即時」套用，不會中斷其他人的 Telegram 工作階段。
- WireGuard：inbound 選單獲得完整的客戶端操作集（Export All URLs、繫結/解除繫結、群組），匯出視窗拆分為 **Config** 與 **Links** 兩個分頁，「WireGuard 允許的 IP」欄位改為可編輯（多筆記錄以逗號分隔），節點 inbound 的客戶端設定中 `Endpoint` 現在指向節點地址。

第 7 節的變更 – 連線安全性：TLS、XTLS 與 REALITY

- **Finalmask + REALITY** 組合在儲存時會被拒絕（它會導致 Xray-core 在首次連線時崩潰）；minClientVer 的輸入提示更新為 26.3.27。
- 新的 Finalmask TCP 遮罩類型——**XMC (Minecraft)**：將流量偽裝成 Minecraft 流量（Hostname、Usernames、必填的 Password 且支援自動生成）。

第 8 節的變更 – 客戶端

- 新增**「速度」**欄位——每個客戶端的即時速度（↑/↓，約 5 秒滑動平均）。
- 客戶端搜尋再次支援 **Telegram ID**；客戶端表單中已停用的 inbound 從綁定清單中隱藏；修正「解綁」視窗中重複項目累積的問題。
- MTPROTO 客戶端擁有專屬欄位：「**MTPROTO secret**」（可重新生成）與「**Ad-tag（贊助頻道）**」（32 個 hex 字元）；配額與有效期現在對 MTPROTO 實際生效。

第 9 節的變更 – 客戶端群組

- 客戶端資訊視窗現在會顯示其群組。

第 10 節的變更 – 訂閱 (Subscription)

- 受管主機改為群組式：一筆記錄可涵蓋多個 inbound（多重選取）與多個地址（標籤式輸入，每筆記錄可帶自己的 `:?/?/?`；地址自動完成；空白——繼承 inbound 地址）。清單欄位以標籤顯示地址與 inbound（帶「+N」），操作與 API 以群組（`groupId`）為單位運作，並新增批量端點 `POST /panel/api/hosts/bulk/add`。主機排序現在是全域的（先依順序，再依備註）。
- 公告文字（`subAnnounce`）現在會以橫幅顯示在訂閱資訊頁面上；自訂範本可使用 `announce` 變數。
- 資訊頁面現在也可透過 **JSON/Clash** 連結在瀏覽器中開啟（不再僅限主連結）。
- 主機設定 **Final Mask** 與 **Allow insecure** 現在分別也對 raw 連結（`fm=`）和 **Hysteria2**（`insecure=1 / skip-cert-verify: true`）生效。
- 「訂閱更新間隔」（`subUpdates`）範圍修正為 **0-525600**（先前的介面上限 168 會在從 2.x 升級後阻擋設定儲存）。
- 原生 **WireGuard** 客戶端現在會納入 **Clash** 與 **JSON** 訂閱（先前僅 raw）。

第 11 節的變更 – Xray：路由、outbounds、DNS 與擴充功能

- Outbound 編輯器：新增「**Target Strategy**」欄位（從 `AsIs` 到 `ForceIPv4` 共 11 個值）、「**Real delay**」測試模式（含隧道建立的完整時間；HTTP 模式現在以「暖」連線測量），以及 HTTP/Real 測試後的 **Egress**（出口 IP，隱藏於「眼睛」圖示後）與 **Country**（旗幟 + 國家，WARP 標記）欄位。
- 負載均衡器的 **Fallback** 可指向另一個負載均衡器：面板會自行建立隱藏的 loopback 物件（`_bl_...`），並防範循環依賴與刪除使用中的負載均衡器；`_bl_` 前綴為保留字。
- 「基本路由」分頁新增「**Default Outbound**」選擇器——決定未匹配任何規則的流量由哪個 outbound 處理（所選項目會移至首位）。
- 私有 IP 上的 DNS 伺服器不再被 `geoip:private` 規則封鎖——面板會自行維護受管的放行規則。
- Dial 設定（sockopt）中的 Happy Eyeballs 現在會確實啟用；「Try delay」預設為 250 毫秒，明確設定的 0 會被保留。
- Outbound 訂閱匯入：帶 `?plugin=` / 結尾 / 的 `ss://` 連結現在能正確解析連接埠。

第 12 節的變更 – 節點（多面板，master/slave）

- 修正合集：未變更的客戶端儲存不再中斷節點 inbound 的即時流量；節點的 Host 覆寫在首次接受時會匯入主面板；自動續期會開啟新的配額週期；在主面板刪除客戶端會在節點上完整刪除；節點的

inbound 在首次接受前不會被清除；單一異常 inbound 不會中止節點流量同步；連接埠衝突檢查僅限於自身節點。

第 14 節的變更 – Telegram 機器人

- 機器人指令選單新增 `/usage`、`/inbound`、`/restart`，以及新的管理員指令 `/clearall`（重設所有客戶端的流量，需確認）。
- 線上客戶端清單以 `email - inbound` [?] [?] 標示；備份與封鎖日誌訊息包含主機名稱；依 Telegram ID 搜尋不受設定格式影響。

第 16 節的變更 – 日常運維：備份、日誌、更新、CLI

- PostgreSQL 面板的還原功能可接受 SQLite 檔案：一般 `.db` 備份或遷移用 `.dump` 可直接匯入 PostgreSQL（單一交易，並在停止 Xray 前先行檢查）。檔案選擇對話框在兩種資料庫上均接受 `.dump, .db`；「下載遷移檔案」僅保留於 PostgreSQL 面板。
- 還原 `pg_dump` 封存檔前，面板會檢查轉儲的可讀性，版本不符時會提示確切的 `x-ui pgclient <[?] [?] >` 指令。
- 啟動時自動修復：溢位的流量計數器會被箝制並復原；移除 inbound 連接埠上過時的 UNIQUE 限制（它妨礙 multi-node）。
- Xray 日誌：新的任務每 10 分鐘檢查一次，當 `access` 或 `error` 日誌超過 **64 MiB** 時將其截斷；每日清理現在會同時清理兩者。
- Docker：憑證自動續期已修復（`crond` 會啟動，`acme.sh` 狀態保存在磁碟區中）。

1. 簡介、需求與安裝

1.1. 什麼是 3X-UI

3X-UI 是一個用於 [Xray-core](#) 伺服器的開源網頁管理面板。面板提供統一的多語言網頁介面，用於部署、設定和監控多種代理及 VPN 協議，支援從單台 VPS 到多節點分散式配置的使用場景。

3X-UI 是原始 X-UI 專案的進階分支版本。相較於原版，新增更多協議支援、提升了穩定性、實現了按客戶端計量的流量統計，以及眾多實用功能。

主要功能：

- **多協議 inbound** – VLESS、VMess、Trojan、Shadowsocks、WireGuard、Hysteria2、HTTP、SOCKS (Mixed)、Dokodemo-door / Tunnel、TUN 以及 **MTPROTO** (Telegram 代理，於 3.3.0 版本新增)。
- **現代化傳輸與加密** – TCP (Raw)、mKCP、WebSocket、gRPC、HTTPUpgrade 與 XHTTP，並可搭配 TLS、XTLS 和 REALITY 進行保護。
- **Fallback** – 透過 Xray 的 fallback 機制，在同一埠上同時服務多個協議（例如在 443 埠同時支援 VLESS 和 Trojan）。
- **按客戶端管理** – 流量配額、到期日期、IP 限制、線上狀態顯示、一鍵邀請連結、QR 碼與訂閱功能。
- **流量統計** – 按每個 inbound、客戶端和 outbound 統計，並支援重置。
- **多節點支援** – 從單一面板管理和擴展至多台伺服器。
- **Outbound 與路由** – WARP、NordVPN、自訂路由規則、負載均衡器、代理鏈。
- **內建訂閱伺服器**，支援多種輸出格式。
- **Telegram 機器人**，用於遠端監控與管理。
- **REST API**，並內建 Swagger 文件。
- **靈活的儲存方案** – SQLite (預設) 或 PostgreSQL。
- **13 種介面語言**，支援深色和淺色主題。
- **整合 Fail2ban**，按客戶端執行 IP 限制。

重要：本專案僅供個人使用。不建議用於非法目的或生產環境。

1.2. 支援的作業系統與架構

作業系統

安裝腳本會從 `/etc/os-release` (或 `/usr/lib/os-release`) 的 `ID` 欄位識別發行版本。官方支援的系統包括：

Ubuntu、Debian、Armbian、Fedora、CentOS、RHEL、AlmaLinux、Rocky Linux、Oracle Linux、Amazon Linux、Virtuozzo、Arch、Manjaro、Parch、openSUSE (Tumbleweed / Leap)、Alpine，以及 Windows。

Alpine 系列系統使用 OpenRC 服務 (`rc-service` / `rc-update`)，其他系統則使用 systemd。CentOS 7 透過 `yum` 安裝套件，較新版本則使用 `dnf`。若未能識別發行版本，腳本預設嘗試使用 `apt-get` 套件管理器。

處理器架構

架構由 `uname -m` 的輸出決定，並對應至支援的值之一：

uname -m 的值	3X-UI 架構
x86_64, x64, amd64	amd64
i*86, x86	386
armv8*, arm64, aarch64	arm64
armv7*, arm	armv7
armv6*	armv6
armv5*	armv5
s390x	s390x

若架構不在此列表中，腳本將輸出「Unsupported CPU architecture!」並終止安裝。

基本依賴

在安裝面板之前，腳本會自動安裝一組基本套件（各發行版的名稱可能不同）：`cron` / `cronie` / `dcron`、`curl`、`tar`、`tzdata/timezone`、`socat`、`ca-certificates`、`openssl`。

1.3. 安裝方式

方式一：安裝腳本（推薦）

以 root 身份執行以下一行指令即可完成安裝：

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

腳本必須以 root 權限執行：若非以 root 執行，將顯示「Please run this script with root privilege」並以錯誤結束。

安裝程式的逐步流程：

1. 識別作業系統和架構。
2. 安裝基本依賴。
3. 下載發行版壓縮包 `x-ui-linux-<arch>.tar.gz` 並解壓至 `/usr/local/x-ui` 目錄。
4. 下載管理腳本 `x-ui.sh` 並以 `/usr/bin/x-ui` 指令的形式安裝。
5. 建立日誌目錄 `/var/log/x-ui`。
6. 執行初始設定：選擇資料庫、生成憑證、選擇埠號、可選的 SSL 設定。
7. 安裝並啟動自啟動服務（systemd 服務單元 `x-ui.service` 或 Alpine 的 OpenRC 初始化腳本）。

安裝時選擇資料庫。 安裝程式提供以下選項：

- 1) SQLite (預設，建議客戶端數量 < 500 時使用) – 單一檔案 `/etc/x-ui/x-ui.db`，無需額外設定。
- 2) PostgreSQL (建議客戶端數量龐大或使用多節點時採用)。PostgreSQL 可本地安裝 (建立專用使用者和名為 `xui` 的資料庫)，也可指定 DSN 連線至現有伺服器。連線參數以 `XUI_DB_TYPE` 和 `XUI_DB_DSN` 變數的形式寫入服務環境檔案 (依發行版不同，分別為 `/etc/default/x-ui`、`/etc/conf.d/x-ui` 或 `/etc/sysconfig/x-ui`)。

範例：將 PostgreSQL 參數寫入服務環境檔案。 選擇 PostgreSQL 並指定 DSN 後，安裝程式將在環境檔案中加入類似以下的內容：

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

此處 `xui` 為使用者名稱和資料庫名稱，`127.0.0.1:5432` 為伺服器地址和埠號，`sslmode=disable` 適用於本地連線 (遠端伺服器通常使用 `require`)。

安裝特定 (舊) 版本。 可明確指定版本標籤 – 安裝程式將下載對應的發行版本：

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

此方式安裝的最低版本為 `v2.3.5`；若指定更舊的版本，將顯示「Please use a newer version (at least v2.3.5)」。

安裝 dev 版本。 除版本標籤外，安裝程式還接受 `dev-latest` 參數 (別名 `dev`) – 此選項將安裝基於 `main` 分支最新提交的持續滾動 dev 版本：

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) dev-
latest
```

dev 版本是每次提交的預發佈版本 (標籤 `dev-latest`)，而非穩定版本，因此不執行最低版本檢查。安裝時將顯示警告：「Installing the rolling dev build (tag: dev-latest). This is a per-commit pre-release, not a stable version.」。若不帶任何參數，安裝程式將安裝最新穩定版本。dev 版本僅建議用於測試尚未正式發佈的修復；日常使用請安裝穩定版本。

方式二：Docker

使用預設 SQLite 資料庫啟動：

```
docker compose up -d
```

若要使用內建的 PostgreSQL 服務，需在 `docker-compose.yml` 中取消 `XUI_DB_*` 相關行的注釋，並以指定配置檔啟動：

```
docker compose --profile postgres up -d
```

映像包含 Fail2ban（預設啟用），用於按客戶端執行 IP 限制。Fail2ban 透過 iptables 封鎖違規者，需要 NET_ADMIN 能力。docker-compose.yml 中已透過 cap_add 授予該能力。自 3.5.0 起，容器中透過 SSL 選單簽發之憑證的自動續期已修復：entrypoint 會啟動 crond 並重新登錄 acme.sh 的 cron 任務，且 docker-compose.yml 新增了存放 acme.sh 狀態的獨立磁碟區——續期可在容器重建後存續（先前憑證會在約 90 天後靜默過期）。若手動使用 docker run 啟動，需自行添加此能力，否則封鎖操作僅會寫入日誌而不會實際執行：

範例：完整的 docker run 指令。 包含面板埠映射、網路能力和持久化資料庫磁碟區的最小配置：

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

磁碟區 /etc/x-ui 用於在容器重啟之間保存 x-ui.db 檔案，否則設定和帳號將會遺失。

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

在 Docker 中，面板是容器的主要進程：自啟動由容器的重啟策略控制（例如 restart: unless-stopped），而非容器內的服務。

1.4. 首次啟動與預設憑證

首次安裝時（尚未修改過預設憑證），安裝程式會隨機生成使用者名稱、密碼、網頁路徑及埠號：

參數	安裝時的生成方式	備註
使用者名稱 (Username)	隨機 10 字元字串	自動生成
密碼 (Password)	隨機 10 字元字串	自動生成
面板網頁路徑 (WebBasePath)	隨機 18 字元字串	防止面板透過根 URL 被探測發現
面板埠號 (Port)	預設為 1024–62000 範圍內的隨機埠；也可手動指定	webPort 的「出廠」預設值為 2053，但安裝程式會覆蓋此值

安裝結束時，腳本會輸出摘要資訊：使用者名稱、密碼、埠號、網頁路徑、API 令牌以及格式如下的登入連結 (Access URL)：

```
https://<[?][?][?]IP>:<[?][?]>/<[?][?][?]>
```

若未設定 SSL 憑證，連結將使用 http://，腳本將顯示需要設定 SSL 的警告（選單第 19 項）。

務必更換憑證。由於登入名稱和密碼為隨機生成，請在安裝完成後立即儲存。可隨時透過選單中的「Reset Username & Password」選項（見下文）或從網頁介面的面板設定中更改。重置後，腳本將提示：「Please use the new login username and password to access the X-UI panel. Also remember them!」。

安裝完成後，使用 `x-ui` 指令開啟管理選單（詳見第 1.6 節）。

1.5. 檔案位置

路徑	用途
<code>/usr/local/x-ui/</code>	面板安裝目錄（二進位檔 <code>x-ui</code> 、腳本 <code>x-ui.sh</code> ）
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	Xray-core 二進位檔（在 <code>armv5/armv6/armv7</code> 上重新命名為 <code>xray-linux-arm</code> ）
<code>/usr/bin/x-ui</code>	管理腳本（ <code>x-ui</code> 指令）
<code>/etc/x-ui/x-ui.db</code>	SQLite 資料庫檔案（預設）
<code>/var/log/x-ui/</code>	面板日誌目錄
<code>/etc/systemd/system/x-ui.service</code>	systemd 服務單元（Alpine 除外）
<code>/etc/init.d/x-ui</code>	OpenRC 初始化腳本（僅限 Alpine）
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	服務環境變數檔案（路徑依發行版而定）； <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code> 寫入此處

資料庫目錄可透過環境變數 `XUI_DB_FOLDER` 覆蓋（預設為 `/etc/x-ui`），Xray 二進位檔目錄可透過 `XUI_BIN_FOLDER` 覆蓋（預設為面板目錄下的 `bin` 子目錄）。資料庫檔案名為 `x-ui.db`。

範例：將資料庫移至獨立磁碟。 若不想將 `x-ui.db` 存放在 `/etc/x-ui`，而是存放在已掛載的磁碟（例如 `/data`），可在服務環境檔案中設定變數並重啟面板：

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

資料庫的完整路徑將變為 `/data/x-ui/x-ui.db`。

主要環境變數

變數	用途	預設值
<code>XUI_DB_TYPE</code>	資料庫後端： <code>sqlite</code> 或 <code>postgres</code>	<code>sqlite</code>
<code>XUI_DB_DSN</code>	PostgreSQL 連線字串（當 <code>XUI_DB_TYPE=postgres</code> 時）	—
<code>XUI_DB_FOLDER</code>	SQLite 資料庫檔案目錄	<code>/etc/x-ui</code>

變數	用途	預設值
XUI_INIT_WEB_BASE_PATH	網頁面板的初始 URI 路徑（僅在首次初始化時生效）	/
XUI_DB_MAX_OPEN_CONNS	最大開啟連線數（PostgreSQL 連線池）	—
XUI_DB_MAX_IDLE_CONNS	最大閒置連線數（PostgreSQL 連線池）	—
XUI_ENABLE_FAIL2BAN	透過 Fail2ban 啟用 IP 限制	true
XUI_LOG_LEVEL	日誌級別（debug、info、warning、error）	info
XUI_DEBUG	除錯模式	false

範例：暫時啟用詳細日誌記錄。 若要診斷問題，可將日誌級別提升至 `debug` 並重啟服務：

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log      # [?] [?] [?] [?] [?]
```

診斷完成後，請將值恢復為 `info`，以防日誌檔案過度增長。

透過環境設定初始網頁面板路徑。 變數 `XUI_INIT_WEB_BASE_PATH` 用於在首次初始化設定時指定網頁面板的 URI 路徑（`webBasePath`）。此功能在 Docker 或透過 `systemd` 部署時非常便利，可從一開始就固定面板的登入路徑。該值會自動正規化 – 必要時自動添加前導和尾隨斜線；若值為空或僅包含空格，則予以忽略（此時套用預設路徑 `/`）。此變數僅在首次初始化時生效：若設定已存在，需透過網頁介面或選單中的「Reset Web Base Path」選項來更改路徑。

1.6. 管理指令 `x-ui`（腳本選單）

安裝完成後，`x-ui` 指令（以 `root` 執行）會開啟互動式選單「3X-UI Panel Management Script」。透過輸入對應編號（範圍 0–27）來選擇選項。許多選項也可作為腳本使用的子命令（詳見第 1.7 節）。

選單分為以下功能區塊。

安裝與更新

- **1. Install** – 安裝面板（執行 `install.sh`）。安裝前會先檢查面板是否尚未安裝。
- **2. Update** – 將所有 `x-ui` 元件更新至最新版本。更新過程中資料不會遺失；更新完成後面板將自動重啟。需要確認。
- **3. Update Menu** – 僅更新管理腳本（`x-ui.sh` / `x-ui` 指令）至最新版本，不重新安裝面板。
- **4. Legacy Version** – 安裝指定的（舊）版本面板。腳本會要求輸入版本號（例如 `2.4.0`）並下載對應的發行版本。
- **5. Uninstall** – 完全移除面板及 `Xray`。停止並停用服務、刪除 `/etc/x-ui/` 和 `/usr/local/x-ui/` 目錄、服務環境檔案以及管理腳本本身。需要確認（預設為「否」）。

憑證與設定

- **6. Reset Username & Password** – 重置面板的使用者名稱和密碼。可輸入自訂值，或留空以隨機生成（隨機使用者名稱 10 字元，隨機密碼 18 字元）。若已設定雙因素驗證（2FA），還會提示是否停用。重置後面板將重啟。
- **7. Reset Web Base Path** – 重置面板網頁路徑：生成新的隨機路徑（18 字元），面板重啟。適用於原路徑已洩露或遺忘的情況。
- **8. Reset Settings** – 將所有面板設定重置為預設值。憑證（使用者名稱和密碼）及帳號資料不會遺失。需要確認；重置後面板將重啟。
- **9. Change Port** – 更改網頁面板的埠號。輸入埠號（1-65535）；設定後需重啟才能生效。
- **10. View Current Settings** – 查看目前設定（`x-ui setting -show`）。顯示內容包括使用的資料庫後端（SQLite 或 DSN 中密碼已遮罩的 PostgreSQL）以及登入連結（Access URL）。若未設定 SSL，系統會提示為 IP 地址申請 Let's Encrypt 憑證。

服務管理

- **11. Start** – 啟動面板服務。若面板已在運行，顯示訊息說明無需重複啟動。
- **12. Stop** – 停止面板服務。
- **13. Restart** – 重啟面板服務。
- **14. Restart Xray** – 僅重啟 Xray-core 核心而不重啟面板本身（透過 `systemctl reload x-ui`；在 Docker 中則向面板進程發送 `USR1` 信號）。
- **15. Check Status** – 查看服務狀態（`systemctl status x-ui` 或 `rc-service x-ui status`）。
- **16. Logs Management** – 日誌管理：查看除錯日誌（Debug Log，透過 `journalctl`），以及（Alpine 除外）清除所有日誌（Clear All logs）。

自動啟動

- **17. Enable Autostart** – 啟用作業系統啟動時自動啟動面板（`systemctl enable x-ui` 或 `rc-update add`）。
- **18. Disable Autostart** – 停用作業系統啟動時的自動啟動。

在 Docker 中，自動啟動由容器的重啟策略控制，因此這些選項僅顯示相應的提示說明。

安全性與網路

- **19. SSL Certificate Management** – 透過 `acme.sh` 管理 SSL 憑證：為網域申請憑證、撤銷、強制更新、查看現有網域、為面板指定憑證路徑，以及為 IP 地址申請短期憑證（約 6 天，自動更新）。
- **20. Cloudflare SSL Certificate** – 透過 Cloudflare DNS 驗證申請 SSL 憑證。
- **21. IP Limit Management** – 管理按客戶端的 IP 數量限制（基於 Fail2ban）：查看和解除封鎖等操作。
- **22. Firewall Management** – 管理防火牆（開放/關閉埠號及查看規則）。
- **23. SSH Port Forwarding Management** – 設定 SSH 埠轉發，以便透過 SSH 隧道從本地電腦訪問面板。

效能與維護

- **24. Enable BBR** – 啟用/停用 TCP BBR 壅塞控制演算法（子選單包含 Enable BBR / Disable BBR）。

- **25. Update Geo Files** – 更新地理資料庫（.dat 檔案），可選擇來源：Loyalsoldier（geoip.dat、geosite.dat）、chocolate4u（geoip_IR.dat、geosite_IR.dat）、runetfreedom（geoip_RU.dat、geosite_RU.dat）或 All（全部）。更新後面板重啟。
- **26. Speedtest by Ookla** – 透過 Speedtest by Ookla 執行網路速度測試。
- **27. PostgreSQL Management** – 管理內建/關聯的 PostgreSQL 實例（啟用及相關操作）。
- **0. Exit Script** – 退出選單。

1.7. x-ui 子命令（無互動選單）

為方便在腳本中使用，x-ui 指令支援直接輸入子命令（不帶參數執行 x-ui 則開啟選單）：

命令	動作
x-ui	開啟管理選單
x-ui start	啟動面板
x-ui stop	停止面板
x-ui restart	重啟面板
x-ui restart-xray	重啟 Xray
x-ui status	查看目前服務狀態
x-ui settings	查看目前設定
x-ui enable	啟用作業系統啟動時自動啟動
x-ui disable	停用自動啟動
x-ui log	查看日誌
x-ui banlog	查看 Fail2ban 封鎖日誌
x-ui update	更新面板
x-ui update-all-geofiles	更新所有地理資料檔案
x-ui migrateDB [file]	轉換 .db 為 .dump (SQLite)
x-ui legacy	安裝舊版本
x-ui install	安裝面板
x-ui uninstall	移除面板

1.8. 從 SQLite 遷移至 PostgreSQL

可將現有的 SQLite 安裝遷移至 PostgreSQL：

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# /etc/default/x-ui XUI_DB_TYPE XUI_DB_DSN
systemctl restart x-ui
```

原始 SQLite 檔案保持不變 – 請在確認新後端正常運作後再手動刪除。

範例：確認已切換至 PostgreSQL。 遷移後，透過查看設定指令確認面板確實在使用新後端 – 輸出中應顯示 PostgreSQL（DSN 中的密碼會被遮罩）：

```
x-ui settings | grep -i -E 'db|dsn'
```

若面板可正常打開且帳號資料完整，則可刪除原始的 `x-ui.db`。

2. 登入面板與存取安全

本節說明與 3X-UI 面板管理員身分驗證相關的一切：登入表單、雙重要素驗證（TOTP）、密碼暴力破解防護、變更帳號憑證、修改面板的秘密路徑與連接埠、工作階段存活時間，以及透過 LDAP 進行同步 / 身分驗證。

2.1. 登入表單

登入頁面會在面板秘密路徑（`webBasePath`）的根路徑提供。若使用者已通過授權，會自動轉向 `.../panel/`。頁面上有主題切換、介面語言選擇，以及表單本身。

表單欄位：

欄位	提示 / 標題 (RU)	必填	說明
使用者名稱	「Имя пользователя」	是	管理員登入名稱。空值在用戶端即被拒絕，在伺服器端則以「Введите имя пользователя」訊息拒絕。
密碼	「Пароль」	是	管理員密碼。空值會以「Введите пароль」訊息拒絕。
Код 2FA	「Код 2FA」	僅在啟用 2FA 時	此欄位僅在面板啟用雙重要素驗證時出現。為驗證器應用程式產生的 6 位數代碼。

「Войти」按鈕會將表單送往 `POST /login`。

行為與訊息：

- 登入成功時顯示「Вход выполнен успешно」並轉向 `.../panel/`。
- 對於任何帳號憑證錯誤或 2FA 代碼錯誤，伺服器都會回傳統一訊息：「Неверные данные учетной записи。」（英文：*Invalid username or password or two-factor code.*）。這是刻意設計的——面板不會提示究竟是哪一項錯誤（登入名稱、密碼或代碼），以免讓暴力破解更容易。
- 面板會根據 `POST /getTwoFactorEnable` 請求來顯示或隱藏「Код 2FA」欄位，該請求會在授權之前回傳目前的 2FA 狀態。
- 若伺服器端工作階段已過期，下次請求時會顯示「Сессия истекла. Войдите в систему снова」，並將使用者轉向登入頁面。

關於 CSRF 的說明：在送出表單之前，用戶端會取得 CSRF 權杖（`GET /csrf-token`）；`/login` 與 `/logout` 請求受 CSRF 檢查保護。

範例：透過 API 登入。 當 2FA 關閉時，只需登入名稱與密碼即可；啟用 2FA 時則加上 `twoFactorCode` 欄位：

```
# Без 2FA
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль'

# С включённой 2FA — добавляется 6-значный код
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

成功時伺服器會回傳帶有工作階段 cookie 的 Set-Cookie ——後續對 /panel/api/... 的請求都需要帶上它。

2.2. 雙重要素驗證 (2FA / TOTP)

3X-UI 中的 2FA 依照 **TOTP** 標準實作，並相容於任何驗證器應用程式（Google Authenticator、Aegis、FreeOTP 等）。參數為硬編碼固定值：演算法 **SHA1**、6 位數、週期 **30** 秒、簽發者 (issuer) **3x-ui**、標籤 **Administrator**。

範例：用於編碼 QR 碼的 **otpauth-URI**。若驗證器應用程式無法掃描相機，可透過下列連結手動新增權杖（請將你自己的 Base32 秘密代入，取代 **JBSWY3DPEHPK3PXP**）：

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

參數 **algorithm=SHA1**、**digits=6**、**period=30** 對應面板的硬編碼固定值——無需修改。

設定位於 **Настройки** → **Учетная запись** 區段的**「Двухфакторная аутентификация」**分頁。

元素	文字 (RU)	說明
切換開關	「Включить 2FA」	啟用 / 停用雙重要素驗證。
說明	「Добавляет дополнительный уровень аутентификации для повышения безопасности。」	切換開關下方的提示。

如何啟用 2FA

啟用切換開關時，面板會在本機產生新的秘密——一段 Base32 編碼的隨機字串（字母表為 **A-Z** 與 **2-7**）。隨即開啟「Включить двухфакторную аутентификацию」視窗，內含逐步說明：

1. **「Отсканируйте этот QR-код в приложении для аутентификации или скопируйте токен рядом с QR-кодом и вставьте его в приложение」**。QR 碼下方會以文字形式顯示秘密本身——點選 QR 碼即可將秘密複製到剪貼簿（會彈出「Скопировано」）。
2. **「Введите код из приложения」**——需輸入應用程式產生的 6 位數代碼。代碼會在瀏覽器端驗證：面板自行依剛產生的秘密計算當前 TOTP，並與輸入值比對。若代碼錯誤——「Неверный код」；欄位只接受剛好 6 位數字。

只有在成功確認之後，秘密與啟用旗標才會被儲存。儲存時會顯示「Двухфакторная аутентификация была успешно установлена」。

重要：設定區段中的變更須透過共用的**「Сохранить」按鈕套用，之後通常需要重新啟動面板（「Сохраните изменения и перезапустите панель для их применения」）。首次啟用 2FA 時，伺服器還會讓所有作用中的工作階段失效**（遞增「login epoch」），因此套用設定後需要重新登入——此時就要帶上 2FA 代碼。

如何停用 2FA

再次按下切換開關會開啟「Отключить двухфакторную аутентификацию」視窗，並提示「Введите код из приложения, чтобы отключить двухфакторную аутентификацию.」。輸入正確代碼後（自 3.4.2 起此代碼會在伺服器端驗證），旗標與秘密會被清除，並顯示「Двухфакторная аутентификация была успешно удалена」。

登入時的代碼驗證

登入時，伺服器會取出已儲存的秘密，並將當前 TOTP 與送來的 2FA 代碼比對。不相符視為登入失敗，但向使用者顯示的是同一則合併訊息「Неверные данные учетной записи.」。

復原存取 (recovery)

3X-UI 沒有獨立的「復原代碼」機制。若失去對驗證器應用程式的存取，就無法透過面板介面復原登入。唯一的途徑是直接在此伺服器的資料庫中停用 2FA：在設定資料表中將 `twoFactorEnable` 鍵重設為 `false`（必要時清空 `twoFactorToken`），之後重新啟動面板。因此建議在啟用 2FA 時，將秘密（Base32 權杖）保存在安全的地方。

範例：在伺服器上緊急停用 2FA。 透過 SSH 取得伺服器存取後，停止面板、重設設定資料表中的鍵，然後再次啟動面板：

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

完成後，登入只需登入名稱與密碼即可，2FA 可視需要重新設定。

與變更帳號憑證的關聯：變更登入名稱 / 密碼時（見 2.4），2FA 會在伺服器上自動停用，以免舊秘密阻擋以新帳號憑證登入。

2.3. 登入嘗試限制 (login limiter / 暴力破解防護)

面板內建失敗登入限制器（相當於應用層級的 fail2ban）。參數在程式碼中固定，無法透過介面設定：

參數	值	用途
最大失敗次數	5	在時間視窗內允許的失敗嘗試次數。
計數視窗	5 分鐘	累計失敗的滑動視窗（較舊的會被捨棄）。
封鎖（cooldown）	15 分鐘	超過門檻後該鍵被封鎖的時間長度。

運作方式：

- 封鎖鍵由**「IP + 登入名稱」組合**構成（登入名稱轉為小寫，並去除空白）。也就是說，封鎖只套用於特定的「位址 + 使用者名稱」組合，而非整個面板。
- 每次失敗嘗試（登入名稱 / 密碼錯誤或 2FA 代碼錯誤）都會使計數器增加。在 **5 分鐘** 內累積達 **5 次** 失敗後，該鍵會被封鎖 **15 分鐘**。封鎖期間，該組合的任何嘗試都會立即以同一則「Неверные данные учетной записи。」訊息被拒絕，即使資料正確也一樣。
- 成功登入會立即重設計數器並解除該組合的封鎖。
- 用戶端 IP 位址的判定會考量受信任的代理（見 `trustedProxyCIDRs`）：只有當請求來自受信任位址時，才接受 `X-Real-IP` 與 `X-Forwarded-For` 標頭。否則使用實際連線位址，而當無法取得時則為字串 `unknown`。

所有嘗試都會被記錄。對於失敗的嘗試，會在伺服器日誌中寫入一則警告，包含使用者名稱、IP、原因，以及（若被封鎖）`blocked_until` 時間。若啟用了透過 Telegram 機器人的登入通知（`tgNotifyLogin` —— 「Уведомление о входе」），管理員還會額外收到成功、失敗與被封鎖嘗試的使用者名稱、IP 與時間。

範例：Telegram 中的登入通知。 啟用 `tgNotifyLogin` 後，每次嘗試後管理員都會收到大致如下的訊息：

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

對於被封鎖的「IP + 登入名稱」組合，狀態中會註明該嘗試被限制器拒絕。

2.4. 變更管理員登入名稱與密碼

Настройки → **Учетная запись** 區段的**「Учетные данные администратора」**分頁。欄位：

欄位	文字 (RU)	說明
目前登入名稱	「Текущий логин」	現行使用者名稱。必須與目前登入名稱相符，否則變更會被拒絕。
目前密碼	「Текущий пароль」	用於確認身分的現行密碼。
新登入名稱	「Новый логин」	新的使用者名稱。不能為空。
新密碼	「Новый пароль」	新密碼。不能為空。

變更透過**「Подтвердить」**按鈕套用，並送往 `POST /panel/setting/updateUser`。

伺服器邏輯與訊息：

- 若「Текущий логин」與實際不符，或「Текущий пароль」錯誤——「Произошла ошибка при изменении учетных данных администратора。」並附說明「Неверное имя пользователя или пароль」。
- 若新登入名稱或新密碼為空——說明「Новое имя пользователя и новый пароль должны быть заполнены」。
- 成功時——「Вы успешно изменили учетные данные администратора。」密碼以 bcrypt 雜湊形式儲存。

2FA 代碼確認（自 3.4.2 起）。若面板已啟用雙重驗證，變更登入名稱 / 密碼還需要額外輸入驗證器應用程式中的當前代碼。會開啟「變更帳號憑證」視窗，提示「Введите код из приложения, чтобы изменить учётные данные администратора。」；代碼（twoFactorCode）會在伺服器端驗證，代碼錯誤或為空時不會套用變更。停用 2FA 時現在也需要相同的確認（詳見 2.2）。

範例：透過 API 變更帳號憑證。此請求需要有效的工作階段 cookie（登入時取得），並確認目前的登入名稱 / 密碼：

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=ВАША_СЕССИОННАЯ_КОOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

成功後，目前的工作階段會被作廢——需要以新資料重新登入。

變更帳號憑證的重要影響：

- **所有既有工作階段都會被作廢**（使用者的 login_epoch 計數器遞增），因此變更後面板會自動登出並轉向登入頁面——需要重新登入。
- 若變更當下已啟用 **2FA**，則會自動停用（旗標與秘密被重設）。變更登入名稱 / 密碼後，雙重要素驗證須重新設定。

若已啟用 2FA，在送出表單前會開啟「Изменить учетные данные」視窗，並提示「Введите код из приложения, чтобы изменить учетные данные администратора。」——只有確認目前的 2FA 代碼後才能變更帳號憑證。

2.5. 秘密路徑（URI 路徑 / webBasePath）與面板連接埠

這些參數位於 **Настройки** → **Панель** 區段，直接影響面板的「隱蔽性」與可存取性。儲存並重新啟動面板後生效。

欄位	文字 (RU)	預設值	說明
面板連接埠	「Порт панели」 (panelPort)，提示「Порт, на котором работает панель」	2053	網頁介面的 TCP 連接埠。
URI 路徑	「URI-путь」 (panelUrlPath)，提示「Должен начинаться с '/' и заканчиваться '/'」	/	秘密基底路徑 (webBasePath)。面板僅能透過它存取（例如 /мой-секрет/）。

欄位	文字 (RU)	預設值	說明
面板管理用 IP 位址	「IP-адрес для управления панелью» (panelListeningIP) , 提示 «Оставьте пустым для подключения с любого IP»	空	面板監聽的位址。空 = 所有介面。
面板網域	「Домен панели» (panelListeningDomain) , 提示 «Оставьте пустым для подключения с любых доменов и IP.»	空	依網域 (Host) 限制存取。
面板憑證公鑰路徑	publicKeyPath , 提示 «Введите полный путь, начинающийся с '/'»	空	用於面板 HTTPS 存取的 TLS 憑證。
面板憑證私鑰路徑	privateKeyPath , 相同提示	空	TLS 私鑰。

基底路徑 (webBasePath) 的行為：

- 值會自動正規化：若未以 / 開頭，會在開頭加上該字元；若未以 / 結尾，會在結尾加上。也就是說，路徑實際上一律為 /.../ 形式。
- 基底路徑套用於面板本身、資源 (assets) 以及工作階段 cookie (cookie 僅針對此路徑發放)。

安全建議 (「Предупреждения безопасности」區段)：若設定「過於公開」，面板會自行顯示警告：

- «Панель работает по обычному HTTP — настройте TLS для продакшна.»
- «Стандартный порт 2053 широко известен — измените его на случайный.»
- «Базовый путь по умолчанию "/" широко известен — измените его на случайный.»

換言之，對於正式環境的伺服器，應設定非標準連接埠、非顯而易見的 URI 路徑以及 TLS 憑證。

範例：正式環境的「隱蔽」面板設定。在 **Настройки** → **Панель** 區段中設定大致如下的值：

欄位	值
面板連接埠	34571 (隨機，取代 2053)
URI 路徑	/aXf9Qm2/ (非顯而易見，以 / 開頭與結尾)
面板憑證公鑰路徑	/etc/letsencrypt/live/panel.example.com/fullchain.pem
面板憑證私鑰路徑	/etc/letsencrypt/live/panel.example.com/privkey.pem

儲存並重新啟動後，面板將僅能透過 `https://panel.example.com:34571/aXf9Qm2/` 存取，且安全警告會消失。

2.6. 工作階段存活時間 (逾時)

** «Продолжительность сессии» **欄位 (sessionMaxAge) 位於面板 / 間隔設定之中。

欄位	文字 (RU)	預設值	單位	說明
工作階段時長	「Продолжительность сессии」，提示「Продолжительность сессии в системе (значение: минута)」	360	分鐘	管理員工作階段 cookie 的存活期限。

行為：

- 值以分鐘指定（預設 360 分鐘 = 6 小時），在設定 cookie 時換算為秒。
- 若值大於 0，工作階段 cookie 會設定相應的 MaxAge。逾期後 cookie 失效，下次請求時使用者會收到「Сессия истекла. Войдите в систему снова」。
- 工作階段也會在變更帳號憑證或首次啟用 2FA 時提前失效（透過 login_epoch 機制，見 2.4 與 2.2），以及在明確登出（POST /logout）時失效。
- 工作階段 cookie 標記為 HttpOnly，採用 SameSite=Lax 政策；在直接以 HTTPS 存取面板時會設定 Secure 旗標。

除了逾時本身，還有一項相關通知：「Задержка уведомления об истечении сессии」（expireTimeDiff，提示「Получение уведомления об истечении срока действия сессии до достижения порогового значения (значение: день)」，預設 0）——可讓你提前收到警告。

2.7.LDAP（同步與身分驗證）

LDAP 區段提供兩項功能：(1) 當本機密碼不符時，透過 LDAP 驗證管理員登入；(2) 定期從目錄同步用戶端的狀態（VLESS 旗標的啟用 / 停用）。

登入時的使用方式：伺服器會先檢查本機 bcrypt 密碼雜湊。若不符且已啟用 LDAP，面板會嘗試在目錄中驗證使用者：在指定 Bind DN 時執行服務性 bind，接著依篩選器與屬性搜尋使用者記錄，並嘗試以找到的 DN 與輸入的密碼進行 bind。成功即表示登入。（LDAP 驗證成功後，若已啟用 2FA，仍會檢查 TOTP 代碼。）

區段欄位：

欄位	文字 (RU)	預設值	說明
啟用 LDAP 同步	「Включить LDAP-синхронизацию」 (enable)	false	LDAP 整合的主開關。
LDAP 主機	「LDAP-хост」 (host)	空	LDAP 伺服器位址。
LDAP 連接埠	「Порт LDAP」 (port)	389	連接埠。LDAPS 通常為 636。
使用 TLS (LDAPS)	「Использовать TLS (LDAPS)」 (useTls)	false	啟用時使用 ldaps:// 配置（預設會驗證伺服器憑證）。

欄位	文字 (RU)	預設值	說明
略過 TLS 憑證驗證	「Пропускать проверку TLS-сертификата」 (ldapInsecureSkipVerify)	false	於 3.4.2 新增。在 LDAPS 時停用伺服器憑證驗證——適用於內部 / 自簽 CA。提示： 「Небезопасно — отключает проверку сертификата сервера. Используйте только с внутренними/недоверенными CA」。此切換開關僅在已啟用「使用 TLS (LDAPS)」時可用。
Bind DN	「Bind DN」 (bindDn)	空	用於初次 bind / 搜尋的服務帳號 DN。若為空——不執行 bind (匿名搜尋)。
Bind 密碼	提示：「Настроено; оставьте пустым, чтобы сохранить текущий пароль。」 / 「Не настроено。」 / 「Настроено — введите новое значение для замены」	空	Bind DN 的密碼。獨立儲存；若要保留原值，將欄位留空。
Base DN	「Base DN」 (baseDn)	空	進行搜尋的子樹根 (搜尋為遞迴，涵蓋整個子樹)。
使用者篩選器	「Фильтр пользователя」 (userFilter)	(objectClass=person)	用於選取帳號的 LDAP 篩選器。驗證時，登入名稱會經跳脫後代入篩選器。
使用者屬性 (username/email)	「Атрибут пользователя (username/email)」 (userAttr)	mail	與登入名稱 / 用戶端識別碼對應的屬性 (例如 mail 或 uid)。
VLESS 旗標屬性	「Атрибут VLESS-флага」 (vlessField)	vless_enabled	用於判定是否應啟用用戶端 VLESS 存取的屬性。
通用旗標屬性 (選用)	「Общий атрибут флага (опц.)」 (flagField)，提示「Если задано, переопределяет флаг VLESS — напр. shadowInactive.」	空	若指定，會用來取代 vless_enabled。
Truthy 值	「Truthy-значения」 (truthyValues)，提示「Через запятую; по умолчанию: true,1,yes,on」	true,1,yes,on	視為「啟用」的旗標屬性值清單。
反轉旗標	「Инвертировать флаг」 (invertFlag)，提示「Включите, когда атрибут означает «отключено» (напр. shadowInactive).」	false	反轉旗標的意義。
同步排程	「Расписание синхронизации」 (syncSchedule)，提示「Строка типа cron, напр. @every 1m」	@every 1m	以 cron 類格式表示的同步週期。

欄位	文字 (RU)	預設值	說明
inbound 標籤	「Теги входящих」 (inboundTags)，提示 「Входящие, на которых LDAP-синхронизация может авто-создавать или авто-удалять клиентов.»	空	限制可在哪些 inbound 上執行自動操作。若沒有 inbound: 「Входящие не найдены. Сначала создайте входящий.»
自動建立用戶端	「Авто-создание клиентов」 (autoCreate)	false	當用戶端出現在目錄中時，在指定的 inbound 上建立用戶端。
自動刪除用戶端	「Авто-удаление клиентов」 (autoDelete)	false	當用戶端從目錄中消失時刪除它。
預設流量 (GB)	「Объём по умолчанию (ГБ)」 (defaultTotalGb)	0	自動建立用戶端的流量上限 (0 = 無上限)。
預設期限 (天)	「Срок по умолчанию (дни)」 (defaultExpiryDays)	0	自動建立用戶端的有效期限 (0 = 永久)。
預設 IP 上限	「Лимит IP по умолчанию」 (defaultIpLimit)	0	同時連線 IP 數量的限制 (0 = 無限制)。

旗標同步邏輯的特點：讀取旗標屬性 (flagField ，預設 vless_enabled) 時，若其值落在 truthy 值清單中，則視為「啟用」；若啟用反轉，結果會變為相反。使用者屬性 (userAttr) 用作對應鍵 (email / 名稱) ——沒有此屬性值的記錄會被略過。

安全：建議啟用 **TLS (LDAPS)**，以免 bind 密碼與待驗證密碼以明文傳輸；並為 Bind DN 使用僅具最低必要讀取權限的帳號。

範例：典型的 LDAP 同步設定 (Active Directory)。 為下列目錄填寫區段欄位：存取狀態儲存在類似 userAccountControl 的旗標屬性中，且依電子郵件對應：

欄位	值
LDAP 主機	ldap.example.com
LDAP 連接埠	636
使用 TLS (LDAPS)	啟用
Bind DN	CN=svc-3xui,OU=Service,DC=example,DC=com
Base DN	OU=Users,DC=example,DC=com
使用者篩選器	(objectClass=person)
使用者屬性 (username/email)	mail
VLESS 旗標屬性	vless_enabled
Truthy 值	true,1,yes,on
同步排程	@every 5m

採用此設定後，面板每 5 分鐘會遍歷 `OU=Users` 子樹，依 `mail` 對應用戶端，並依 `vless_enabled` 的值啟用 / 停用 VLESS 存取。

3. 概覽 / 儀表板

儀表板（「儀表板」，英文介面為 *Overview*）是面板的起始頁面，以即時方式顯示伺服器與 Xray 程序的狀態。所有指標均來自伺服器端。背景排程器每隔 **2 秒** 重新收集一次快照，並透過 WebSocket 廣播至所有已開啟的分頁；每分鐘將累積的指標資料列寫入磁碟。HTTP 端點 `GET /status` 回傳最後快取的快照。

以下逐一說明頁面上的每個指標與控制項目。

自 3.4.2 起，面板側邊選單（以及行動版抽屜）有一個「文件」按鈕（書本圖示）——開啟官方文件 <https://docs.sanaei.dev/>。此外，3.4.2 全面提升了**無障礙性**：圖示按鈕與可點擊元素均加上了 aria 標籤，並可透過 Enter/Space 啟用（供螢幕報讀器等鍵盤導覽使用）。

3.1. 資料收集的一般原則

- 快照由 `gopsutil` 函式庫收集。若某項量測失敗，該欄位保持為零，並在日誌中寫入警告（`get cpu percent failed`、`get uptime failed` 等）——這不會導致整個儀表板失效，僅對應的磁磚顯示 0/N/A。
- 「瞬時」速率（CPU %、網路、磁碟 I/O）的計算方式為：當前快照與上一次快照的差值除以時間間隔（秒）。因此，頁面首次載入時，速率值可能為零，直到收集到第二次量測為止。
- 歷史資料可在「系統歷史」（*System History*）區段查看——圖表依據下文描述의相同資料列繪製（見第 3.12 節）。

3.2. CPU

「CPU」磁磚顯示當前的處理器使用率百分比，以及處理器本身的參數。

指標	說明
CPU 使用率，%	在最後一個時間間隔內，已使用的處理器時間佔比。以指數移動平均（EMA，係數 <code>alpha = 0.3</code> ）進行平滑，以避免指標因劇烈波動而跳動。數值始終鎖定在 0-100% 的範圍內。首次量測時回傳 0（初始化基準點）。
邏輯處理器	邏輯核心數，即包含 Hyper-Threading 在內。
實體核心	實體核心數。
頻率	處理器的基礎頻率（MHz）。採惰性查詢並進行快取：首次成功量測後儲存結果，重試間隔不短於 5 分鐘，且每次查詢本身有 1.5 秒的逾時限制（在某些系統上，頻率查詢回應較慢）。

CPU 使用率的計算邏輯如下：若平台提供原生實作，則優先使用；否則依據處理器時間計數器的差值（`busy / total`）計算。Guest 與 GuestNice 時間被排除，以避免重複計算。

3.3. 記憶體（RAM）

「記憶體」（RAM）磁磚顯示已使用量與總量，以「已使用 / 總量」及/或使用率百分比的形式呈現。歷史記錄中儲存的是百分比。

3.4. 交換空間 (Swap)

「交換空間」 (Swap) 磁磚顯示已使用量與總量。若未設定交換檔案/分割區 (總量 = 0)，則指標為零；歷史資料列中，無交換空間時寫入 0。

3.5. 磁碟 (Storage)

「磁碟」 (Storage) 磁磚顯示已使用量與總量，其中僅統計根分割區 `/`。「磁碟使用量」 (Disk Usage) 歷史記錄中寫入使用率百分比。另外獨立收集磁碟 I/O (讀取 / 寫入，位元組/秒)，作為時間間隔內計數器的差值——顯示在歷史記錄的「磁碟 I/O」分頁中。

3.6. 系統運行時間 (Uptime)

「系統運行時間」 (Uptime) 指標為整台伺服器自開機以來的時間 (秒)，而非面板或 Xray 的運行時間。Xray 程序的運行時間另行儲存 (見第 3.9 節)，面板執行緒數也單獨記錄 (「執行緒」 / Threads)。

面板佔用的記憶體

在面板程序指標旁邊，顯示 3X-UI 程序本身所佔用的記憶體量。此值取自程序的實際 RSS (如作業系統所見)，與系統工具所顯示的數值一致。記憶體釋放後，數值會隨之下降。過去面板顯示的是 Go 內部計數器，該計數器高估了記憶體使用量 (例如，在只有一個客戶端的閒置伺服器上顯示約 300 MB)，且從不減少——現在這個問題已不存在。此外，定期執行的後台程序會將未使用的記憶體歸還給作業系統，使指標反映實際消耗。

3.7. 系統負載 (Load average)

「系統負載」 (System Load) 區塊為包含三個數值的陣列 [Load1, Load5, Load15]。提示說明：「過去 1、5、15 分鐘的系統平均負載」 (System load average for the past 1, 5, and 15 minutes)。歷史圖表名稱為「系統平均負載 (1 / 5 / 15 分鐘)」。歷史資料列中，各值分開記錄：load1、load5、load15。

這是標準的 Unix 指標：處於執行佇列中的平均程序數。參考標準是與核心數相比：若負載持續超過實體核心數，代表系統過載。

3.8. 網路：速率與總流量

僅統計實體網路介面。虛擬與通道介面被排除：包括 lo / lo0，以及所有以 loopback、docker、br-、veth、virbr、tun、tap、wg、tailscale、zt 開頭的介面。所有剩餘介面的數值加總統計。

整體速率 (Overall Speed) —— 瞬時速率，為時間間隔內計數器的差值：

指標	說明
上傳 (標籤「上傳」 / Upload)	出站速率，位元組/秒。
下載 (標籤「下載」 / Download)	入站速率，位元組/秒。

總流量 (Total Data) —— 自系統啟動以來的累積計數器：

指標	說明
已發送（標籤「已發送」/ <i>Sent</i> ）	總發送位元組數。
已接收（標籤「已接收」/ <i>Received</i> ）	總接收位元組數。

另外收集封包速率（封包/秒）與封包總計數器——顯示在歷史記錄的「網路封包」（*Network Packets*）分頁中。網路歷史資料列：`netUp`、`netDown`、`pktUp`、`pktDown`。

3.9. 伺服器 IP 位址

「伺服器 IP 位址」（*IP Addresses*）區塊顯示 IPv4 與 IPv6。外部位址透過第三方服務查詢（`api4.ipify.org`、`ipv4.icanhazip.com`、`v4.api.ipinfo.io/ip`、`ipv4.myexternalip.com/raw`、`4.ident.me`、`check-host.net/ip` 用於 IPv4，IPv6 類似）。清單依序嘗試，直到第一個成功回應；每個請求的逾時為 3 秒。

注意事項：

- 結果在程序生命週期內**快取**：成功查詢到的位址不會重複請求。
- 若所有服務均未回應，欄位保持 N/A。對於 IPv6，首次出現 N/A 後，IPv6 請求將完全停用，以避免在無 IPv6 的網路上浪費時間。
- 旁邊有一個「眼睛」按鈕用於隱藏/顯示位址——提示說明「隱藏或顯示伺服器 IP 位址」（*Toggle visibility of the IP*）。這僅是介面上的視覺隱藏（例如用於截圖），不影響位址本身。

3.10. TCP/UDP 連線

「連線數」（*Connection Stats*）區塊顯示伺服器上活躍的 TCP 和 UDP 連線總數（涵蓋整個系統，而非僅 Xray）。歷史圖表為「活躍連線（TCP / UDP）」（*Active Connections*），資料列 `tcpCount`、`udpCount`。

3.11. Xray 狀態與程序管理

「Xray」卡片顯示 Xray-core 程序的狀態，並提供管理功能。

狀態

值	標籤	對應說明	設定時機
running	「執行中」	<i>Running</i>	Xray 程序已啟動。
stop	「已停止」	<i>Stopped</i>	程序未執行，且無已記錄的啟動錯誤。
error	「錯誤」	<i>Error</i>	程序未執行，但已記錄啟動錯誤。錯誤訊息顯示在標題為「Xray 執行時發生錯誤」（ <i>An error occurred while running Xray</i> ）的彈出視窗中。
—	「未知」	<i>Unknown</i>	在尚未取得狀態時顯示。

狀態旁邊顯示 **Xray 版本**。

控制按鈕

- **停止 (Stop)**。呼叫 `POST /stopXrayService`。成功時，面板透過 WebSocket 廣播新狀態 `stop` 及通知「Xray 已成功停止」 (*Xray service has been stopped*)；失敗時廣播狀態 `error` 及錯誤訊息。注意：若面板透過 Xray 本身存取，停止 Xray 可能中斷與面板的連線——直接連接面板則不受影響。
- **重新啟動 (Restart)**。呼叫 `POST /restartXrayService`。執行前顯示確認提示「重新啟動 xray?」，說明「以已儲存的設定重新載入 xray 服務」。成功時顯示狀態 `running` 及通知「Xray 已成功重新啟動」 (*Xray service has been restarted successfully*)。重新啟動會套用當前已儲存的設定——請在修改設定後使用此功能。

注意。在此分支中，儀表板新增了針對所有授權類型的完整 Start / Stop / Restart 控制；在原版 3x-ui 介面中沒有獨立的「啟動」按鈕——啟動透過重新啟動來執行。

Xray 日誌查看按鈕

Xray 卡片上有一個查看 Xray 日誌的按鈕 (*Logs*)。只有在 Xray 設定中啟用了 `access` 日誌時，此按鈕才會出現：內建的查看器讀取的正是該日誌檔，因此無 `access` 日誌時按鈕隱藏。按鈕的可見性與獨立的 `accessLogEnable` 屬性綁定，不再依賴 IP 限制——線上清單與 IP 位址限制功能即使在無 `access` 日誌的情況下仍可正常運作（見第 8 節）。

選擇 Xray 版本

「選擇版本」 (*Version*) 區段允許將 Xray-core 切換至其他版本。版本清單透過 `GET /getXrayVersion` 載入：

- 資料來源為 GitHub API 中 `XTLS/Xray-core` 儲存庫的 `/releases`。請求快取 **15 分鐘**；若 GitHub 發生故障，則回傳最後成功獲取的清單，以免選擇器出現空白。
- 清單中僅包含格式為 `X.Y.Z` 且不早於 **26.4.25** 的版本。

提示說明：「選擇您想切換的版本」 (*Choose the version you want to switch to.*) 以及警告「重要：舊版本可能不支援當前設定」 (*Choose carefully, as older versions may not be compatible with current configurations.*)。

切換：`POST /installXray/:version`。流程：

範例。切換至特定版本的 Xray-core (Cookie 工作階段需已透過授權取得)：

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

其中 `v25.6.8` 為 `GET /getXrayVersion` 回傳清單中的標籤。版本必須存在於該清單中，否則面板將拒絕請求。自 3.4.2 起，可安裝的最低 Xray 版本提升至 **26.6.27**，而 3.5.0 隨附 **Xray 核心 26.7.11**。核心的相關變更：Shadowsocks 的 `none / plain` 與 VMess 的 `none / zero` 加密方式已被移除（已儲存的設定會自動改

寫：SS 改為受支援的加密方式，VMess 改為 `auto`），且指向公網地址的未加密 VLESS/Trojan outbound 在儲存時會被拒絕——帶有此類設定的核心根本無法啟動。

1. 所選版本在當前版本清單中進行驗證（否則拒絕）。
2. Xray 停止。
3. 根據當前作業系統和架構從 GitHub 下載 `Xray-<os>-<arch>.zip` 壓縮檔（支援 amd64/64、arm64-v8a、arm32-v7a/v6/v5、386/32、s390x；Windows 使用 `xray.exe`）。壓縮檔與二進位檔的大小限制為 200 MB。
4. 二進位檔以原子方式替換（透過臨時檔案 + 重新命名），並標記為可執行。
5. Xray 重新啟動。

切換前顯示對話框「切換 Xray 版本」（*Do you really want to change the Xray version?*），說明「這將把 Xray 版本更改為 #version#」。成功時顯示通知「Xray 已成功更新」（*Xray updated successfully*）。

3.12. 面板更新（3X-UI）

面板更新檢查區塊。資料透過 `GET /getPanelUpdateInfo` 取得：

欄位	說明
目前面板版本	已安裝的面板版本。
最新面板版本	從 GitHub 取得的最新 3x-ui 版本。
有可用更新	表示最新版本比目前版本新。若無需更新，則顯示「面板已是最新版本」/「已更新」。

「更新面板」（*Update Panel*）按鈕觸發 `POST /updatePanel`。提示說明：「這將把 3X-UI 更新至最新版本並重新啟動面板服務」。執行前顯示確認「您確定要更新面板嗎？」，說明文字「這將把 3X-UI 更新至 #version# 版本並重新啟動面板服務」。

注意事項與限制：

- 自動更新僅在 **Linux** 上受支援（其他作業系統將回傳錯誤）。
- 更新腳本從官方儲存庫下載（`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`，限制 2 MB）並透過 `bash` 執行，如可能則透過 `systemd-run` 進行隔離。
- 成功啟動時顯示「面板更新已開始」（*Panel update started*）；若更新檢查失敗則顯示「面板更新檢查失敗」。安裝期間顯示警告「安裝進行中，請勿重新整理頁面」。

3.13. 地理位置檔案更新（GeoIP / GeoSite）

地理資料庫更新按鈕/對話框呼叫 `POST /updateGeofile`（所有檔案）或 `POST /updateGeofile/:fileName`（單一檔案）。更新依據嚴格的名稱與來源白名單進行：

檔案	來源
<code>geoip.dat</code> 、 <code>geosite.dat</code>	<code>Loyalsoldier/v2ray-rules-dat</code> （最新版）
<code>geoip_IR.dat</code> 、 <code>geosite_IR.dat</code>	<code>chocolate4u/iran-v2ray-rules</code> （最新版）
<code>geoip_RU.dat</code> 、 <code>geosite_RU.dat</code>	<code>runetfreedom/russia-v2ray-rules-dat</code> （最新版）

行為：

- 檔案名稱會被驗證：禁止 `..`、斜線、絕對路徑；僅允許 `[a-zA-Z0-9._-]+.dat`。不在白名單中的檔案不會被下載。
- 使用條件式請求 `If-Modified-Since`：若來源伺服器上的檔案未更動（HTTP 304），則不重新下載，僅更新時間戳記。
- 下載完成後，Xray 會重新啟動（以載入新的資料庫）。

範例。僅更新俄羅斯地理資料庫，不影響其他檔案：

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoip_RU.dat' -b cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b cookie.txt
```

若要一次更新白名單中的所有檔案，請呼叫不帶檔案名稱的 `POST /updateGeofile`。

- 對話框：單一檔案顯示「您確定要更新地理位置檔案嗎？」及「這將更新 #filename# 檔案」；「全部更新」按鈕顯示「這將更新所有地理位置檔案」。成功時顯示「地理位置檔案已成功更新」。

3.14. 資料庫備份與還原

「備份與還原」（*Backup & Restore*）區塊。行為取決於所使用的資料庫系統（預設為 SQLite，或 PostgreSQL）。

匯出資料庫（備份）

「匯出資料庫」/「備份」（*Back Up*）按鈕呼叫 `GET /getDb`。檔案以附件形式回傳：

- SQLite**：先執行 checkpoint（清除 WAL），然後下載 `x-ui.db` 檔案。提示說明：「點擊下載包含當前資料庫備份的 .db 檔案.....」。
- PostgreSQL**：下載自訂格式的轉儲檔 `x-ui.dump`（`pg_dump --format=custom --no-owner --no-privileges`）。伺服器需安裝 PostgreSQL 客戶端工具；否則顯示缺少 `pg_dump` 的錯誤訊息。

匯入資料庫（還原）

「匯入資料庫」/「還原」（*Restore*）按鈕透過 `POST /importDB`（表單欄位 `db`）上傳檔案。提示說明：「點擊選擇並上傳 .db 檔案.....以從備份還原資料庫」。

SQLite 的還原流程安全且具備回滾機制：

- 驗證檔案格式是否為 SQLite 並儲存至臨時檔案，然後進行完整性檢查。
- 停止 Xray，關閉並將當前資料庫重新命名為 `*.backup`（備用方案）。
- 新檔案取代工作資料庫，執行初始化與遷移。若發生任何錯誤，則還原備用資料庫。
- 重新啟動 Xray。

自 3.5.0 起，**PostgreSQL** 的「還原」接受三種檔案（類型依內容判斷，而非副檔名）：`pg_dump` 封存檔（PGDMP）——透過 `pg_restore --clean --if-exists --single-transaction ...` 套用；**SQLite** 資料庫

.db（一般備份）與 **SQLite 遷移 .dump** ——它們會經過驗證、必要時重組，並以與 `x-ui migrate-db --dsn` 相同的引擎、單一交易匯入 PostgreSQL（出錯時 PostgreSQL 保持不變）。檔案選擇對話框在兩種資料庫上均接受 `.dump, .db`；將 `pg_dump` 封存檔上傳至 SQLite 面板會得到明確的錯誤訊息。提示說明明確警告：「這將替換所有當前資料」。

訊息：「資料庫已成功匯入」、「匯入資料庫時發生錯誤」、「.....讀取資料庫時」、「.....取得資料庫時」。

遷移檔案（SQLite 與 PostgreSQL 之間）

「下載遷移檔案」（*Download Migration*）按鈕呼叫 `GET /getMigration`。自 3.5.0 起，它僅顯示於 **PostgreSQL 面板**，並下載 `x-ui.db` ——從 PostgreSQL 資料構建的現成 SQLite 資料庫（「回到 SQLite」的路徑）。SQLite 面板上的此按鈕已作為多餘功能移除：一般 `.db` 備份現在本來就能直接還原至 PostgreSQL 面板。

3.15. 其他介面元素

- **線上客戶端指示器**。儀表板維護 `online` 資料列（*Online Clients* / 「線上客戶端」）——具有活躍連線的客戶端數量。在 Xray 執行時計算（否則為 0），以相同的 2 秒週期寫入歷史記錄。圖表在「線上」分頁顯示。
- **系統歷史（圖表）**。「圖表」→「系統歷史」按鈕/區段，包含以下分頁：「頻寬」、「封包」、「磁碟 I/O」、「線上」、「負載」、「連線」、「磁碟使用量」。資料透過 `GET /history/:metric/:bucket` 取得；允許的聚合間隔（bucket，秒）：**2、30、60、180、360、720、1440、2880、10080**，每個分頁最多回傳 60 個資料點。頁面上的時間範圍選擇器提供以下按鈕：**2m、1h、3h、6h、12h、24h、2d、7d**（對應 bucket 值 `2[?]60[?]180[?]360[?]720[?]1440[?]2880[?]10080`）。在較長的時間範圍 **2d** 和 **7d** 中，X 軸的時間標籤會附加 `MM-DD HH:MM` 格式的日期。儲存採三層抽稀（rollup）機制：最新資料以 2 秒步長保留最後一小時，之後平均至 1 分鐘步長保留 **48 小時**，再平均至 10 分鐘步長保留 **7 天**。因此，圖表（CPU、RAM、流量、封包、連線、磁碟、線上、負載）可查看最長 **7 天** 的資料（以前為 48 小時），且越往過去，資料粒度越粗。允許的指標：`cpu`、`mem`、`swap`、`netUp`、`netDown`、`pktUp`、`pktDown`、`diskRead`、`diskWrite`、`diskUsage`、`tcpCount`、`udpCount`、`online`、`load1`、`load5`、`load15`。「最近 2 分鐘」對應 bucket = 2（即時模式）。

範例。取得最近約 2 分鐘的 CPU 使用率資料列（bucket = 2 秒，最多 60 個資料點），以及同一資料列以 5 分鐘聚合（bucket = 300 秒）：

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

指標可替換為任何允許的值（`mem`、`netUp`、`tcpCount`、`load1` 等）。不在白名單 `2[?]30[?]60[?]180[?]360[?]720[?]1440[?]2880[?]10080` 中的 bucket 值將被拒絕。

- **Xray 指標**——獨立區塊，顯示 Xray 的記憶體消耗與垃圾回收資料（資料列 `xrAlloc`、`xrSys`、`xrHeapObjects`、`xrNumGC`、`xrPauseNs`）以及「Observatory」（出站連線狀態）。僅在 Xray 設定中設定了 `metrics` 區塊（`listen 127.0.0.1:11111`，標籤 `metrics_out`）時有效；否則顯示「Xray 指標端點未設定」。Xray 指標視窗有自己的時間範圍選擇器，包含按鈕 **2m、1h、3h、6h、12h**（對應 bucket 值 `2[?]60[?]180[?]360[?]720`）。

範例——啟用 Xray 指標磁磚的區塊設定。Xray 設定中需同時包含帶標籤的 `metrics` 以及監聽該標籤的 inbound：

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

位址 `127.0.0.1:11111` 刻意不對外公開——面板在本機查詢。

- **深色主題切換**。位於通用選單/頁首，而非儀表板本身。選項：「主題」（*Theme*），包含「深色」與「超深色」（*Ultra Dark*）兩種選項。這是純視覺的介面外觀設定，不影響面板運作。
- **儀表板周邊的其他連結**（來自選單/底部欄）：「日誌」、「設定」——查看最終 Xray JSON（`GET /getConfigJson`）、「說明文件」。

4. Inbounds：建立與通用參數

「Inbounds」（英文 *Inbounds*）區段是所有 Xray 入口點的清單，用戶端透過這些入口點進行連線。每個 inbound 同時保存「面板」欄位（備註、流量上限、重設排程）以及 Xray 組態的原始 JSON 區塊（`settings`、`streamSettings`、`sniffing`）。

建立透過「Add Inbound」（*Add Inbound*）按鈕完成，編輯則透過「Modify Inbound」（*Modify Inbound*）按鈕。兩項操作分別送往 API 端點 `POST /add` 與 `POST /update/:id`。

以下說明表單中所有不屬於特定協定設定（用戶端、加密、REALITY/TLS）、且不屬於傳輸/串流（「Stream」、**「Security」**分頁）的欄位－後者是其他章節的主題。

4.1. 通用表單欄位

Remark（備註）

參數	值
欄位	<code>remark</code>
類型	字串
預設值	空

inbound 的人類可讀名稱，會顯示在清單與對話框標題中（例如「刪除連線 "{remark}"？」等）。欄位標籤為「Remark」。它不影響 Xray 的運作，僅用於管理上的便利；建議設定唯一且有意義的名稱，因為它們會被代入匯出檔案的名稱與大量操作的確認訊息中。

Protocol（協定）

參數	值
欄位	<code>protocol</code>
標籤	「Protocol」
驗證	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel</code>

inbound 協定的下拉選單。允許的值：

值	備註
<code>vmess</code>	
<code>vless</code>	
<code>trojan</code>	

值	備註
shadowsocks	
wireguard	
hysteria	Hysteria v2 即 hysteria 搭配 streamSettings.version = 2，並沒有獨立的協定
http	
mixed	同一連接埠上的 socks/http
tunnel	
tun	驗證器接受此值，但沒有對應的獨立協定常數

此欄位為必填（required）。所選協定決定哪些用戶端設定欄位以及哪種傳輸方式可用（見各協定專屬章節）。

重要：儲存時服務會將 streamSettings 正規化。傳輸設定只會保留給 vmess、vless、trojan、shadowsocks、hysteria 這幾種協定；其餘協定（http、mixed、tunnel、wireguard、tun）的 streamSettings 欄位會被強制清空。

對於 tunnel/TProxy 類型的 inbound，其 streamSettings 區塊不含 security 鍵（無傳輸變體），表單可正常開啟並儲存，不會出現 streamSettings.security Invalid input 驗證錯誤。

Listen IP（監聽 IP）

參數	值
欄位	listen
類型	字串
預設值	空 → Xray 監聽於 0.0.0.0（所有 IP）

inbound 接受連線所使用的 IP 位址。欄位提示：

「留空以監聽所有 IP 位址」。

在產生 Xray 組態時，空值會被替換為 0.0.0.0。除了 IP 之外，此欄位也接受 Unix 通訊端路徑 – 提示如下：

「也可以指定 Unix 通訊端路徑（例如 /run/xray/in.sock）或以 @ 為前綴的抽象通訊端名稱（例如 @xray/in.sock），以監聽通訊端而非 TCP 連接埠 – 此情況下請將連接埠設為 0」。

因此，此欄位接受兩種 Unix 通訊端形式：檔案系統中的路徑（/run/xray/in.sock）以及以 @ 為前綴的抽象通訊端名稱（@xray/in.sock）。兩種情況下都請將 Port 設為 0。

當需要將 inbound 限制在單一介面上時（例如將只作為 Nginx 後方 fallback 目標運作的 inbound 設為 127.0.0.1），或當 inbound 監聽 Unix 通訊端時，才需要變更此欄位。

範例。 只監聽本機介面（典型的 Nginx 後方 fallback 目標）以及 Unix 通訊端的 inbound：

```
listen = 127.0.0.1    port = 8443
listen = /run/xray/in.sock    port = 0
```

Port（連接埠）

參數	值
欄位	port
標籤	「Port」
驗證	gte=0,lte=65535
預設值	–（由使用者設定）

監聽用的 TCP/UDP 連接埠。允許 0 到 65535 之間的值。值 0 僅在搭配監聽 Unix 通訊端時使用（見上文）。

儲存時服務會檢查連接埠衝突：兩個 inbound 不能同時佔用同一傳輸方式（TCP/UDP）下相互重疊的 listen:port。傳輸方式由協定與 streamSettings/settings 推算得出：例如 hysteria 與 wireguard 一律佔用 UDP，kcp/quic 佔用 UDP，而其餘大多數佔用 TCP。發生衝突時，儲存會被拒絕並回報錯誤。

此外，面板不允許佔用內部 Xray API 的保留連接埠（標籤 api，預設為 127.0.0.1 上的 62789）：若本機 TCP inbound 的監聽位址在 loopback 上與此連接埠重疊，會以相同的連接埠衝突錯誤被拒絕。實際的 API 連接埠是從 Xray 組態範本讀取（備援值為 62789）。在節點（nodes）上此限制不適用 – 它們有各自的 Xray。

Xray 標籤（Tag，唯一）會自動依連接埠與傳輸方式以 in-<[?][?][?]>-<tcp|udp|tcpudp|any> 格式產生；對於部署在節點上的 inbound，會加上 n<nodeId>- 前綴。若發生衝突，會在標籤後附加 -2、-3 等。使用者通常不需編輯標籤。

Total traffic（總流量，GB）

參數	值
欄位	total（單位為位元組）
標籤	「Total traffic」
預設值	0

inbound 的總流量上限。表單中以 GB 為單位輸入，資料庫中以位元組儲存。欄位提示：

「= 無限制。（單位：GB）」。

也就是說 **0** 代表無限制。這是整個 inbound 層級的上限（而非各別用戶端）；實際已用流量保存在 **up**（已傳送）與 **down**（已接收）欄位中，並與 **total** 比較。

Expiry date / Duration（到期日期 / 期限）

參數	值
欄位	<code>expiryTime</code> （Unix 時間戳記）
標籤	「 Expiry date 」（英文 <i>Duration</i> ）
預設值	空 / 0

inbound 的有效期限。提示：

「留空以設為永久」。

空值（0）表示 inbound 永不過期。值以 Unix 時間戳記保存；表單既可設定明確的日期，也可設定以天數計的期限（從目前時刻起相對計算 – 英文欄位標籤為 *Duration*）。

Enabled（啟用）

參數	值
欄位	<code>enable</code>
標籤	「 Enabled 」（英文 <i>Enabled</i> ）
預設值	於建立時設定

inbound 的啟用狀態標記。在清單中切換此旗標是由一個獨立的「輕量」端點 `POST /setEnabled/:id` 處理，而非完整更新 – 這是為了避免在每次切換含有數千個用戶端的 inbound 的開關時，重新序列化整個 `settings` 區塊（所有用戶端）。停用 inbound 時，它會從運作中的 Xray 移除；啟用時則重新加回。

Node / Deploy to（節點 / 部署至）

參數	值
欄位	<code>nodeId</code>
標籤	「 Deploy to 」、「 Local panel 」
預設值	空（本機面板）

選擇 inbound 實際運作的位置：在本機面板上，或在其中一個已註冊的節點上。自 3.5.0 起，「Deploy to」清單（以及 inbound 清單上方的「節點」篩選器）支援輸入搜尋，且 inbound 清單上方新增了搜尋欄位（依

備註、連接埠與協定搜尋；可與節點篩選器搭配使用）。實作細節：nodeId = 0 會被正規化為 nil，因為 0 並非有效的節點 id，而是表單繫結的產物；nil/0 代表本機面板。將 inbound 儲存到離線節點時，可能會出現「變更將在節點重新連線時同步」的提示訊息。自 3.4.2 起，此提示僅在節點確實離線或關閉時才顯示（先前在線上節點儲存時也可能出現）。

連結用位址策略 (Share address strategy)

參數	值
欄位	策略 + (選用) 自訂位址
標籤	「Share address strategy」 (英文 Share address strategy)
預設值	「Inbound listen」 (Inbound listen)

此下拉選單決定該 inbound 的匯出分享連結與 QR 碼中要代入哪個位址。值：

值	標籤	代入的內容
node	「Node address」 (Node address)	inbound 所運作節點的位址
listen	「Inbound listen」 (Inbound listen)	inbound 本身的監聽位址
custom	「Custom」 (Custom)	來自「Custom share address」 (Custom share address) 欄位的自訂位址

選擇「Custom」時，會出現「Custom share address」欄位；在其中輸入不含協定與連接埠的主機名稱或 IP（值會被驗證）。「Node address」選項只有在存在已啟用、且此 inbound 可在其上運作的節點時才會出現在清單中；否則會被隱藏，且值會回退為「Inbound listen」。

此策略只影響直接分享連結與 QR 碼。它不影響訂閱輸出 – 訂閱中的位址仍由面板的一般邏輯決定。

4.2. Sniffing (嗅探)

「Sniffing」分頁編輯 Xray 組態的 sniffing 區塊，該區塊以原始 JSON 保存。Sniffing 讓 Xray 能「窺探」連線內部真正的網域名稱/協定，以供路由判斷之用。

子欄位	標籤	用途
enabled	(分頁開關)	啟用/停用 inbound 的嗅探
destOverride	—	要攔截目的位址的協定清單：http、tls、quic、fakedns
metadataOnly	「Metadata only」	僅使用連線中繼資料，不讀取有效負載
routeOnly	「Route only」	嗅探結果僅用於路由，不改寫目的位址
domainsExcluded	「Excluded domains」	排除於嗅探之外的網域

子欄位	標籤	用途
(排除的 IP)	「Excluded IPs」	排除於嗅探之外的 IP 位址

- **destOverride** — 一組嗅探器：`http`（從 HTTP 標頭 Host 判斷網域）、`tls`（從 SNI 判斷）、`quic`（從 QUIC ClientHello 判斷）、`fakedns`（與 FakeDNS 池比對）。判斷網域時通常啟用 `http` 與 `tls`。

sniffing 區塊範例（依 HTTP 與 TLS 判斷網域，結果僅用於路由，不影響本地網路）：

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **metadataOnly** — 啟用時，Xray 不讀取第一個封包的內容，僅依靠中繼資料；對於不可「窺探」資料的協定，這有助於避免破壞協定。
- **routeOnly** — 嗅探結果僅供路由規則使用；outbound 中的連線位址不會被改寫為辨識出的網域。

備註：面板將 `sniffing` 當作不透明的 JSON 區塊保存，儲存時不會對其新增任何內容 — 這些核取方塊的所有預設值皆由用戶端應用程式那一側產生。原始形式的區塊可透過「Inbound 的 JSON」區段編輯（見下文）。

4.3. Allocate（連接埠分配策略）

`streamSettings` 中的 `allocate` 區塊控制 Xray 如何分配監聽連接埠。這是 Xray 組態的一部分；面板會將其當作 `streamSettings / inbound` JSON 的一部分保存並傳遞。參數（依 Xray-core 的術語）：

子欄位	用途	值 / 預設值
<code>strategy</code>	連接埠配置策略	<code>always</code> — 一律監聽指定的連接埠（預設）； <code>random</code> — 在範圍內定期更換所監聽的連接埠
<code>refresh</code>	<code>random</code> 時更換連接埠的間隔（分鐘）	整數分鐘（建議 5；最小為 2）
<code>concurrency</code>	<code>random</code> 時同時保持開啟的連接埠數量	整數（預設 3；不超過連接埠範圍寬度的三分之一）

`strategy: always` 讓 inbound 維持在單一連接埠上（標準模式）。`strategy: random` 用於反封鎖情境，此時 inbound 會定期在連接埠範圍內「跳動」；這種情況下 `refresh` 與 `concurrency` 才有意義。只有在刻意使用隨機連接埠模式時，才需變更這些值。

`streamSettings` 中的 **allocate 區塊範例**（隨機連接埠模式：保持 3 個連接埠開啟，每 5 分鐘更換一次）：

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

為使其運作，inbound 的 `port` 需設為範圍（例如 `20000-20100`）。

4.4. External Proxy（外部代理）

「External Proxy」欄位屬於邀請連結產生的設定，保存在 inbound 的 `streamSettings` 中。它設定一份替代外部位址清單（主機/連接埠，必要時可強制 TLS – 「**Force TLS**」），這些位址會被代入用戶端連結，取代 inbound 真實的 `listen:port`。

當用戶端不應直接連到伺服器、而需透過外部代理/反向代理/CDN 連線時使用：此時分享連結中會填入該前端的公開位址。這不影響 Xray 接受連線的程序本身 – 它只是所產生連結的「外觀美化」。相關表單欄位：「**Force TLS**」、「**Fingerprint**」，以及各筆記錄的標籤。

4.5. Fallbacks（Fallback）

「Fallbacks」區段設定那些未匹配 inbound 任何用戶端的連線之轉送規則。此功能適用於 TLS 傳輸（VLESS/Trojan TCP-TLS）上的主 inbound。透過 `GET /:id/fallbacks` / `POST /:id/fallbacks` 端點管理。

區段提示：

「當此 inbound 上的連線未匹配任何用戶端時，它會被轉送到別處。在下方選擇一個子 inbound，路由欄位（SNI / ALPN / Path / xver）便會自動從其傳輸設定填入；或者保持空白並直接設定 `Dest`（例如 `8080` 或 `127.0.0.1:8080`），以轉送到外部伺服器（如 Nginx）。每個子 inbound 都必須監聽於 `127.0.0.1` 且 `security=none`」。

fallback 區段只會在 RAW (TCP) 上、且安全性為 TLS 或 REALITY 的 VLESS/Trojan inbound 上顯示。新的 inbound 以 `security=none` 啟動，因此該區段一開始可能看起來不存在。在此狀態下（VLESS/Trojan、RAW/TCP、尚未設定安全性），會以內嵌提示取代區段：在 **「Security」**分頁選擇 TLS 或 Reality 之後，fallback 才會可用。

fallback 列的欄位

欄位	預設值	說明
（子 inbound）	—	選擇子 inbound（標籤「 Select inbound 」）。選定後，Name/Alpn/Path/Dest 欄位可從其傳輸設定自動填入
Name	空（= 任意）	依名稱（SNI/名稱）匹配的條件。「任意」標籤為「 any 」

欄位	預設值	說明
Alpn	空	依 ALPN 匹配的條件
Path	空	依路徑匹配的條件（適用於子 inbound 的 WS/HTTP 傳輸）
Dest	自動	轉送目的地。佔位文字「 auto (子的 listen:連接埠)」。可指定連接埠（ 8080 ）或 host:port （ 127.0.0.1:8080 ）
Xver	0	PROXY 協定版本（「 Xver 」）： 0 – 關閉， 1 或 2 – 對應的 PROXY protocol 版本
（順序）	依位置	規則套用順序；以「 上移 」/「 下移 」按鈕設定

儲存邏輯：主 inbound 的整份 fallback 清單會原子性地替換。若某一行既沒有選定子 inbound（`childId <= 0`），也沒有設定 **Dest**，則會被略過。若選定的子 inbound 等於主 inbound 自身的 id，則會被歸零。產生最終 JSON 時：若 **Dest** 為空，會從子 inbound 算出 **listen:port**，其中 `0.0.0.0 / :: / ::0` 會被替換為 `127.0.0.1`；空的 **name / alpn / path** 欄位不會出現在輸出 JSON 中；**xver** 只有在大於 0 時才會加入。

最終 settings.fallbacks 範例（**alpn=h2** 的流量經由路徑 **/ws** 送往 WS 目標，其餘全部送往連接埠 **8080** 上的本機 Nginx）：

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

最後一行沒有 **name / alpn / path** – 這是「預設」規則，捕捉其餘所有流量。

fallbacks 區段的按鈕與提示

- 「**Add fallback**」 – 新增一行；「**尚無 fallback**」 – 空狀態。
- 「**快速加入所有符合的**」 / 「**Add all**」 – 為每個符合且尚未連接的 inbound 新增一行 fallback。結果：「已新增 {n} 個 fallback」或「沒有新的符合 inbound」。
- 「**從子填入**」 – 從選定子 inbound 的傳輸設定重新拉取路由欄位（SNI/ALPN/Path/xver）；執行後出現「已從子填入」。
- 「**編輯路由欄位**」 / 「**隱藏進階**」 – 顯示/隱藏列的細部欄位。
- 「**Routes when**」與「**預設 – 捕捉其餘所有**」標籤說明每一行的觸發條件。

儲存 fallback 之後，伺服器會觸發 Xray 重新啟動，使新的 **settings.fallbacks** 生效。

4.6. 定期重設流量

「重設流量」區塊設定依排程自動重設 inbound 流量計數器。說明：

「依指定的間隔自動重設流量計數器」。

參數	值
欄位	trafficReset
驗證	omitempty,oneof=never hourly daily weekly monthly
預設值	never
相關欄位	lastTrafficResetTime – 最後一次重設的時間戳記（標籤「Last reset」）

下拉選單：

值	標籤
never	「Never」
hourly	「Hourly」
daily	「Daily」
weekly	「Weekly」
monthly	「Monthly」

每個週期都註冊了一個依對應排程執行的 cron 工作（@hourly、@daily、@weekly、@monthly）。該工作會選出所有設定了相應 trafficReset 的 inbound，並對每一個重設 inbound 自身的計數器（up=0、down=0）以及其所有用戶端的流量。也就是說，定期重設會同時影響 inbound 與其用戶端。

欄位值範例。 若要讓計數器在每月一日歸零，在表單中選擇「Monthly」，儲存為：

```
{ "trafficReset": "monthly" }
```

值 never（預設）會完全關閉自動重設。

4.7. Inbound 的 JSON (advanced)

****「Inbound 的 JSON 區段」** 區段提供對 inbound 原始 JSON 區塊的直接存取。說明：

「inbound 的完整 JSON，以及 settings、sniffing 與 streamSettings 的個別編輯器」。

可用的編輯器：

分頁	標籤	編輯內容
All	「在單一編輯器中呈現含所有欄位的完整 inbound 物件」	整個 Inbound 物件
Settings	「Xray settings 區塊的包裝」	settings 欄位
Sniffing	「Xray sniffing 區塊的包裝」	sniffing 欄位
Stream	「Xray stream 區塊的包裝」	streamSettings 欄位

這些欄位會序列化為巢狀 JSON 物件：空區塊會以 `null` 輸出，而非有效 JSON 的文字會被包裝成字串，以免資料遺失。儲存時的解析錯誤會以「進階 JSON」前綴輸出。

「Inbound 的 JSON」檢視視窗與 inbound 匯入視窗一樣，使用具備 JSON 語法高亮的完整程式碼編輯器（而非一般文字欄位）：檢視組態時為唯讀的高亮模式，匯入時為可編輯模式，這讓閱讀與修改更為輕鬆。

4.8. inbound 的操作：QR / Edit / Reset / Delete 與統計

在清單與 inbound 卡片中可使用下列操作（**「選單」**選單）：

流量統計

顯示 inbound 的彙總流量：「已傳送/已接收」（up / down 欄位）、「總流量」、「總連線數」。卡片中還有「建立時間」、「更新時間」、「到期日期」。

inbounds 清單中有一個 **Speed** 欄位，顯示每個 inbound 目前的流量速率（上傳/下載），由兩次輪詢之間的計數器增量計算而得；同樣的即時速率也會顯示在 inbound 統計視窗中。當某次輪詢沒有增量時，速率值會重設。

inbounds 頁面的用戶端摘要中，狀態依「已耗盡/已結束」優先順序判定：到期或流量耗盡（且自動工作已取消其 `enable`）的用戶端歸類為「已耗盡/已結束」（*Depleted/Ended*）狀態，而非灰色的「已停用」（*Disabled*），且不會被重複計算。此分類與用戶端本身卡片中所顯示的一致，並正確處理繫結到多個 inbound 的用戶端。

QR 碼與複製連結

- 「詳情」－展開連線與訂閱連結。
- 用戶端 QR 碼：提示「點擊 QR 碼以複製」。
- 「複製連結」（英文 *Copy URL*）、「匯出連結」。自 3.4.2 起，頁面動作「匯出所有連結」（GET /panel/api/inbounds/allLinks；視窗「匯出所有 inbound 連結」，檔案「All-Inbounds」）會透過訂閱引擎生成連結——對每個用戶端套用備註範本，並優先使用受管的 Host 端點（先前使用固定備註 inbound-email）。

Edit（編輯）

「Modify Inbound」－開啟編輯表單（POST /update/:id）。更新時服務會重新讀取既有的資料列、套用變更後的欄位、必要時重新產生標籤（若舊標籤是自動產生的），並同步 Xray 執行階段。成功時出現提示「連線已成功更新」。

Reset Traffic（重設流量）

「重設流量」－將此 inbound 的 up / down 計數器歸零（POST /:id/resetTraffic，設定 up=0, down=0）。確認訊息：

「重設 "{remark}" 的流量？」 / 「將此連線的傳送/接收計數器重設為 0」。

重設 inbound 流量不會動到其用戶端的計數器（用戶端有獨立的「重設用戶端流量」操作）。重設後會觸發 Xray 重新啟動。成功時出現提示「**inbound 流量已重設**」。也有大量操作的版本 – 「**重設所有連線的流量**」（`POST /resetAllTraffics`）。

Delete（刪除）

「刪除連線」（`POST /del/:id`）。確認訊息：

「刪除連線 "{remark}"？」 / 「此連線及其所有用戶端都將被刪除。此操作無法復原」。

刪除會將 inbound 從運作中的 Xray 移除（必要時伴隨重新啟動）。成功時出現提示「**連線已成功刪除**」。大量刪除為 `POST /bulkDel`，具備逐項回報，且最多只觸發一次 Xray 重新啟動。

inbound 用戶端的其他操作

選單中也可使用：「複製」（建立 inbound 副本，使用新連接埠且用戶端清單為空）、「刪除所有用戶端」（`POST /:id/delAllClients` – 刪除所有用戶端，inbound 本身保留）、「刪除已停用的用戶端」、「繫結/解除繫結用戶端」、「匯入」/「匯出連線」（`POST /import`）。用戶端操作的細節屬於用戶端章節。

5. 協議

建立 inbound 時，第一步選擇的是**協議**（「Protocol」）。協議決定了 Xray-core 對此 inbound 採用何種身份驗證與流量加密方式、**settings** 中需要填寫哪些欄位，以及可用的傳輸方式（**network**）與安全層（TLS / REALITY）。

協議欄位在建立 inbound 時設定一次，**編輯時無法更改**（編輯表單中下拉選單為鎖定狀態）。若要更換協議，需建立新的 inbound。

5.1. 支援協議列表

伺服器接受 **Protocol** 欄位的以下值：

```
oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

自版本 **3.3.0** 起，列表中新增了 **mtproto**（Telegram 代理）。

設定值	用途	客戶端模型
vless	主要代理協議（建立 inbound 時的預設值）	以 UUID 為憑證的客戶端，支援 flow 與後量子加密
vmess	Xray 的經典代理協議	以 UUID 與 security 參數為憑證的客戶端
trojan	偽裝成普通 HTTPS 的代理	以密碼為憑證的客戶端
shadowsocks	Shadowsocks 代理（含 SIP022 / 2022-blake3）	單用戶或多用戶（2022）
wireguard	WireGuard inbound	Peer（非客戶端）
hysteria	Hysteria inbound（預設版本 2）	以 auth token 為憑證的客戶端
http	傳統 HTTP 正向代理（forward proxy）	用戶名/密碼帳號，不計流量
mixed	合併 SOCKS + HTTP 代理	用戶名/密碼帳號
tunnel	透明轉發器（xray dokodemo-door）	無客戶端
tun	TUN 介面（僅供現有介面渲染）	無客戶端
mtproto	Telegram 代理（MTPROTO），3.3.0 新增；由獨立的 mtg 行程管理，非 Xray	無客戶端（透過 secret 存取）

關於 **tun**：此值保留在列表中是為了相容性及顯示先前儲存的 inbound，但在當前版本中後端不建議建立此類型——支援已被標記為過時。建立此類型的新 inbound 沒有實際意義。

關於 Hysteria 2：不存在獨立的「hysteria2」協議。這是 hysteria 協議，其 `streamSettings.version = 2`。當串流版本等於 2 時，生成分享連結時會自動選擇 `hysteria2://` 格式。

並非所有協議都支援節點（nodes）分發。可部署至節點的協議僅限：`vless`、`vmess`、`trojan`、`shadowsocks`、`hysteria`、`wireguard`。協議 `http`、`mixed`、`tunnel`、`tun`、`mtproto` 僅在本地面板上運作。

5.2. 各協議對 TLS / REALITY / 傳輸的支援

能否啟用特定安全層與傳輸方式，取決於協議和所選網路（`streamSettings.network`）：

功能	可用協議	允許的網路（network）
TLS	<code>vmess</code> 、 <code>vless</code> 、 <code>trojan</code> 、 <code>shadowsocks</code> （以及 <code>hysteria</code> 始終可用）	<code>tcp</code> 、 <code>ws</code> 、 <code>http</code> 、 <code>grpc</code> 、 <code>httpupgrade</code> 、 <code>xhttp</code>
REALITY	<code>vless</code> 、 <code>trojan</code>	<code>tcp</code> 、 <code>http</code> 、 <code>grpc</code> 、 <code>xhttp</code>
flow（ <code>xtls-rprx-vision</code> ）	僅 <code>vless</code>	僅 <code>tcp</code> ，且 <code>security = tls</code> 或 <code>reality</code>
Stream / 傳輸（「串流」分頁）	<code>vmess</code> 、 <code>vless</code> 、 <code>trojan</code> 、 <code>shadowsocks</code> 、 <code>hysteria</code>	—

對於 `http`、`mixed`、`tunnel`、`tun`、`wireguard` 協議，傳輸分頁不可用——它們沒有 Xray 的 stream 設定。

5.3. VLESS

用途：現代主要代理協議。支援 XTLS-Vision（`flow`）、REALITY，以及 VLESS 自身層級的後量子加密（`decryption` / `encryption` 欄位）。為新建 inbound 的預設選擇。

`settings` 區塊的欄位：

欄位	預設值	說明
<code>clients</code>	<code>[]</code>	客戶端列表。每個客戶端包含： <code>id</code> （UUID）、 <code>email</code> （必填）、 <code>flow</code> 、限制（ <code>limitIp</code> 、 <code>totalGB</code> 、 <code>expiryTime</code> ）、 <code>enable</code> 、 <code>tgId</code> 、 <code>subId</code> 、 <code>comment</code> 、 <code>reset</code>
<code>decryption</code>	<code>none</code>	伺服器端解密參數。介面標籤：「解密」（英文「Decryption」）
<code>encryption</code>	<code>none</code>	對應的加密參數（含入客戶端連結）。標籤：「加密」（英文「Encryption」）
<code>fallbacks</code>	<code>[]</code>	fallback 列表（參見 fallback 章節）；在 <code>network = tcp</code> 且 <code>security</code> 為 TLS 或 REALITY 時可用

欄位	預設值	說明
testseed	(4 個數字： 900、500、900、 256)	「Vision testseed」——4 個正整數，用於 XTLS-Vision padding。僅對 flow 為 xtls-rprx-vision 的客戶端生效，否則忽略

flow (xtls-rprx-vision)

flow 在客戶端上設定，而非 inbound，可取以下三個值之一：

值	含義
'' (空)	無 XTLS-flow (預設)
xtls-rprx-vision	XTLS-Vision——VLESS over TCP+TLS/REALITY 的推薦模式
xtls-rprx-vision-udp443	與 Vision 相同，但額外處理 UDP/443 (QUIC)

只有同時滿足以下條件時，flow 欄位才可選：協議為 vless、network = tcp 且 security 為 tls 或 reality。**Vision testseed** 欄位也僅在相同條件下顯示。

XHTTP 的例外情況：當 VLESS over network = xhttp 並啟用了 VLESS 後量子身份驗證 (encryption/decryption，即 vlessenc) 時，xtls-rprx-vision 也是允許的——與安全層無關，包括使用 REALITY 的情況。此時面板會正確地將 xtls-rprx-vision 傳遞至分享連結及訂閱（包括 Clash/Mihomo 格式），使客戶端獲得確切帶 Vision 的設定。

解密 / 加密 (VLESS 後量子身份驗證)

decryption 和 encryption 欄位是 VLESS 協議自身層級的身份驗證（與傳輸層 TLS/REALITY 分開）。預設兩者均為 none。表單中這兩個欄位下方有**「金鑰生成」區塊——一個模式下拉選單和「生成」按鈕（旁邊還有「清除」**按鈕）。下拉選單包含六個選項：**X25519 (native)**、**X25519 (xorpub)**、**X25519 (random)**、**ML-KEM-768 (native)**、**ML-KEM-768 (xorpub)**、**ML-KEM-768 (random)**——即兩種金鑰類型（傳統 X25519 和後量子 ML-KEM-768），每種各有三種模式：

- **native**——所選類型的基本金鑰對；
- **xorpub**——對公鑰部分進行額外處理的衍生模式；
- **random**——帶有隨機成分的衍生模式。

在列表中選擇所需模式，點擊**「生成」：面板會自動填入兩個欄位（decryption 和 encryption），生成對應模式的金鑰對。「清除」**按鈕可將兩個欄位重設回 none。

欄位下方會顯示**「已選：...」**狀態欄，它能根據生成的字串識別金鑰類型（X25519 或 ML-KEM-768）和模式（native / xorpub / random）並顯示。空欄位或 none 顯示為「None」。

技術上，這些按鈕呼叫 GET /panel/api/server/getNewVlessEnc（透過 xray vlessenc 生成金鑰），並以 mlkem768x25519plus.native.<rtt>.<role> 格式（例如 decryption = mlkem768x25519plus.native.600s.server-x25519、encryption =

mlkem768x25519plus.native.0rtt.client-x25519) 的配對值填入**兩個**欄位。decryption 參數留在伺服器端，encryption 進入客戶端連結。

重要：在為 Xray 生成 inbound 設定時，面板會清理多餘項目：若 settings 中存在 encryption (屬客戶端側)，它會從伺服器設定中被**移除**。伺服器上僅保留 decryption。

何時選擇 VLESS：這是新建 inbound 的推薦預設選項，特別適合與 REALITY (無需自有憑證) 或 TLS + XTLS-Vision 組合使用。

範例：含單一客戶端及 XTLS-Vision 的 VLESS inbound settings 區塊。flow 欄位在客戶端上，decryption 留在伺服器端：

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

搭配 REALITY 時，對應的 streamSettings 區塊 (「Transport」分頁 → Security: REALITY) 如下：

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<приватный ключ X25519>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

用途：Xray 的經典代理協議。以 UUID 進行身份驗證，客戶端可額外設定有效載荷加密方式 (security)。

settings 區塊的欄位：

欄位	預設值	說明
clients	[]	客戶端列表

每個 VMess 客戶端除通用欄位（email、限制、enable、tgId、subId、comment、reset）外，還有：

客戶端欄位	預設值	說明
id	—	客戶端 UUID
security	auto	VMess 有效載荷加密方式。允許值：aes-128-gcm、chacha20-poly1305、auto、none、zero

security 的值說明：

- auto ——Xray 根據平台自動選擇加密演算法（推薦）；
- aes-128-gcm、chacha20-poly1305 ——固定的 AEAD 加密演算法；
- none ——無有效載荷加密（僅在 TLS 上有意義）；
- zero ——無有效載荷加密且無身份驗證。

歷史相容性：舊記錄中可能存有 security: "" ——讀取時空字串會被轉換為 auto。在生成伺服器設定時，VMess 客戶端的 security 欄位會從 settings 中移除，因為 inbound 側不需要它。

何時選擇 VMess：用於與舊客戶端或現有設定相容。新部署通常優先選擇 VLESS。

5.5. Trojan

用途：模仿普通 HTTPS 流量的代理。以密碼進行身份驗證。與 VLESS 一樣，支援 fallback 及（在 network = tcp 時）REALITY/TLS。

settings 區塊的欄位：

欄位	預設值	說明
clients	[]	客戶端列表
fallbacks	[]	fallback 列表（在 network = tcp 且 TLS/REALITY 時可用）

每個 Trojan 客戶端的關鍵欄位：

客戶端欄位	預設值	說明
password	—	客戶端密碼（必填，至少 1 個字元）
email	—	客戶端唯一識別符

其餘客戶端欄位為通用欄位（limitIp、totalGB、expiryTime、enable、tgId、subId、comment、reset）。

何時選擇 Trojan：需要在 443 埠偽裝成 HTTPS，並為未授權連線設置 fallback 至網頁伺服器（Nginx）時。

範例：含 fallback 至本地網頁伺服器的 Trojan settings 區塊。未授權連線（無有效密碼）將被導向監聽 127.0.0.1:8080 的 Nginx：

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

使用 fallback 需要 network = tcp 且 Security 為 TLS 或 REALITY；否則 fallbacks 欄位不可用。

5.6. Shadowsocks

用途：輕量級 Shadowsocks 代理。同時支援舊版 AEAD 加密算法和新版 SIP022 方法（2022-blake3-*）。可在單用戶或多用戶模式下運作。

settings 區塊的欄位：

欄位	預設值	說明
method	2022-blake3-aes-256-gcm	inbound 加密方式。介面標籤：「加密方式」（英文「Encryption method」）
password	''	inbound 密碼（2022 方式下會根據所選方式自動生成）
network	tcp,udp	傳輸方式。標籤：「網路」（英文「Network」）。選項：tcp,udp（TCP、UDP）、tcp、udp
clients	[]	客戶端列表
ivCheck	false（關閉）	「ivCheck」開關——防止 IV 重複使用

加密方式（method）

允許的值集合：

方式	類別
aes-256-gcm	舊版 AEAD
chacha20-poly1305	舊版 AEAD
chacha20-ietf-poly1305	舊版 AEAD
xchacha20-ietf-poly1305	舊版 AEAD
2022-blake3-aes-128-gcm	SS 2022（推薦）

方式	類別
2022-blake3-aes-256-gcm	SS 2022 (預設)
2022-blake3-chacha20-poly1305	SS 2022, 僅單用戶

面板對各方式的處理邏輯：

- **2022 方式** (2022-blake3-*) 被視為「SS 2022」。2022-blake3-chacha20-poly1305 方式為**單用戶** (不支援多用戶)；其他 2022 方式允許多個客戶端。密碼欄位 (帶生成按鈕, 會根據方式調整金鑰長度) 僅在 2022 方式下顯示於表單中。
- **舊版加密算法** (aes-*, chacha20-*) 採用經典的「單一方式 + 單一密碼」模式。

啟動 Xray 前的規範化處理：對於舊版加密算法, 每個客戶端的 method 必須與 inbound 的方式一致 (否則 Xray 會出現「unsupported cipher method:」錯誤而崩潰)。對於 2022 方式, 情況相反——客戶端的 method 欄位必須為空 (否則 Xray 會以「users must have empty method」拒絕 inbound)。切換方式時, 面板會自動整理資料。

切換金鑰長度時重新生成客戶端金鑰：對於 Shadowsocks-2022, 當加密方式切換至金鑰長度不同的方式時 (例如在 2022-blake3-aes-256-gcm 和 2022-blake3-aes-128-gcm 之間切換), 面板在儲存 inbound 時會自動為客戶端 PSK 重新生成新長度的金鑰。否則舊金鑰長度不變, Xray 將拒絕接受。後果：受影響的客戶端需要重新獲取訂閱——舊連結將無法連接。

何時選擇 Shadowsocks：適用於無需 TLS 偽裝的簡單部署；現代首選方式為 2022-blake3 系列。

範例：2022-blake3 方式的 Shadowsocks settings 區塊 (多用戶模式)。inbound 有自己的密碼 (所需長度的 base64 金鑰), 每個客戶端有自己的密碼, 客戶端的 method 欄位為空：

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWlcmU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMl1ieXRlcyl1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

對於舊版加密算法 (aes-256-gcm 等) ——情況相反：inbound 使用單一密碼, 且客戶端的 method 必須與 inbound 的方式一致。

5.7. Dokodemo-door / Tunnel（透明轉發器）

用途：透明轉發器（在面板中為 `tunnel` 協議，實現 `dokodemo-door` 行為）。接收流量並將其轉發至指定的地址/埠，無需身份驗證，無客戶端概念。

`settings` 區塊的欄位：

欄位	預設值	說明
<code>rewriteAddress</code>	(無)	「改寫地址」（英文「Rewrite address」）——流量被重定向到的目標地址
<code>rewritePort</code>	(無)	「改寫埠」（英文「Rewrite port」）——目標埠（0-65535）
<code>allowedNetwork</code>	<code>tcp,udp</code>	「允許的網路」（英文「Allowed network」）。選項： <code>tcp,udp</code> 、 <code>tcp</code> 、 <code>udp</code>
<code>portMap</code>	<code>{}</code>	「埠映射」——埠到埠的映射（Record<string,string>）
<code>followRedirect</code>	<code>false</code> （關閉）	「跟隨 redirect」（英文「Follow redirect」）——使用攔截連線中的原始目標地址

Tunnel 的「Transport」分頁：此類型 inbound 可使用**「Transport」**分頁，但僅限 `sockopt` 設定——這已足夠支援 **TProxy** 模式（透過 `sockopt.tproxy` 進行透明代理/redirect）。傳輸方式選擇下拉選單（`network`）和 Tunnel 的「Security」分頁被隱藏，因為此類型不支援 TLS/REALITY。

何時選擇：用於透明代理/將埠轉發至內部服務。

「改寫埠」欄位（`rewritePort`）可以留空：清除後該值只是從 inbound 設定中排除，而不會導致儲存錯誤。（以前清除此欄位會導致 `settings.rewritePort` 驗證錯誤並阻止儲存，包括透過 JSON 分頁操作時。）

5.8. SOCKS / HTTP（`mixed` 協議）

此版本中沒有獨立的 `socks` 協議——SOCKS 和 HTTP 代理合併為 `mixed` 協議（合併 SOCKS + HTTP）。此外，還有獨立的純 `http` 代理。

5.8.1. Mixed（SOCKS + HTTP）

`settings` 區塊的欄位：

欄位	預設值	說明
<code>auth</code>	<code>password</code>	「Auth」——身份驗證模式。選項： <code>password</code> （用戶名/密碼）或 <code>noauth</code> （無需授權）
<code>accounts</code>	(選填)	「帳號」——用戶名/密碼對的列表。 <code>auth = noauth</code> 時不寫入設定
<code>udp</code>	<code>false</code> （關閉）	「UDP」開關——透過 SOCKS 支援 UDP

欄位	預設值	說明
ip	127.0.0.1	「UDP IP」——UDP 關聯的本地地址。僅在啟用 udp 時顯示

帳號透過「新增」按鈕添加；新增時會自動生成隨機用戶名（8 個字元）和密碼（12 個字元），可手動編輯。

5.8.2. HTTP（純代理）

用途：傳統 HTTP 正向代理。在 Xray 層面不將客戶端視為「計費」用戶（無 email/限制）——只有帳號列表。

settings 區塊的欄位：

欄位	預設值	說明
accounts	[]	「帳號」——用戶名/密碼對的列表（兩個欄位均必填）
allowTransparent	false（關閉）	「允許透明」（英文「Allow transparent」）——以原始 Host 標頭轉發請求

何時選擇 SOCKS/HTTP：用於本地或服務性代理存取，無需複雜偽裝。mixed 的便利之處在於單一埠同時服務 SOCKS 和 HTTP 客戶端。

5.9. WireGuard（inbound）

用途：WireGuard inbound——WireGuard VPN 隧道，而非偽裝代理。傳輸方式和 TLS/REALITY 不適用。

自 3.4.2 起，WireGuard 改採多用戶端模式（如同 VLESS/VMess/Trojan/Shadowsocks/Hysteria）。inbound 本身僅保存伺服器部分；peer 設備現在以一般用戶端形式新增（詳見第 8 節）——在隧道中自動配發地址，支援訂閱、流量 / 時限限制與群組。inbound 表單中先前內嵌的「Peer / 新增 peer」清單已移除：新 inbound 建立時不含任何 peer，每個用戶端的地址會自動配發。

inbound 欄位（settings 區塊）：

欄位	預設值	說明
secretKey	—	伺服器私鑰（必填）。旁邊有生成按鈕；公鑰（Public Key）會由它自動推導（唯讀；單獨儲存的 pubKey 已移除）
mtu	1420	介面 MTU
dns	1.1.1.1, 1.0.0.1	3.4.2 新增。寫入用戶端 .conf 中 DNS = 行的 DNS
noKernelTun	false（關閉）	「TUN 無 kernel」——使用 userspace TUN 而非核心 TUN
domainStrategy	（選填）	「Domain Strategy」：ForceIP、ForceIPv4、ForceIPv4v6、ForceIPv6、ForceIPv6v4

用戶端的 WireGuard 參數——位於新增/編輯用戶端視窗的「憑證」分頁（當綁定的 inbound 為 WireGuard 時顯示）：

欄位	說明
「WireGuard 私鑰」	用戶端私鑰（可編輯，有「重新生成」按鈕）；輸入後會由它自動推導公鑰
「WireGuard 公鑰」	唯讀；由私鑰推算
「WireGuard 共享金鑰」（PSK）	選填的額外預共享金鑰
「WireGuard 允許的 IP」	自 3.5.0 起——可編輯（輸入提示 <code>10.0.0.2/32</code> ，提示「留空以自動配發；多筆記錄以逗號分隔」）。留空——地址自動配發；伺服器會驗證每筆記錄（IP 或 CIDR），單一地址會正規化為 <code>/32</code> ，並拒絕已被此 inbound 其他用戶端佔用的地址

隧道內地址由伺服器從 inbound 的子網路中自動配發（預設 `10.0.0.0/24`：伺服器佔用 `.1`，用戶端從 `.2` 起算）；若既有用戶端使用其他 `/24`，則新地址也在同一子網路中配發。

Domain Strategy 與 Transport 分頁

除上述欄位外，WireGuard inbound 還有 **Domain Strategy** 欄位（域名解析策略：`ForceIP`、`ForceIPv4`、`ForceIPv4v6`、`ForceIPv6`、`ForceIPv6v4`）。此欄位為選填，僅在設定後才寫入設定。

Workers 欄位（workers，工作執行緒數）已從 WireGuard 表單（包括 inbound 和 outbound）中移除：自 xray-core v26.6.22 起，引擎不再使用此欄位，而是依賴 wireguard-go 的內部機制。先前儲存的設定無需任何修改即可正常運作——解析時此欄位會被直接忽略，無需遷移。

WireGuard 也有**「Transport」**分頁——但以精簡形式呈現：僅可設定 `sockopt` 和 **Finalmask** 混淆。傳輸方式選擇下拉選單（`network`）被隱藏，因為 WireGuard 始終在 UDP 上監聽。在 Finalmask 的雜訊記錄（noise）中，**Rand Range** 作為獨立欄位設定（範圍 0–255 位元組，帶驗證），WireGuard 和 Hysteria 可用的混淆方式還包括 **Salamander**。

Inbound 操作與匯出（自 3.5.0 起）

WireGuard inbound 的列選單現在包含完整的「多用戶端」操作集——「匯出所有連結」、依訂閱匯出、**繫結/解除繫結用戶端**、加入群組與刪除所有用戶端（先前僅有匯出 / 重設 / 複製 / 刪除），且清單中會顯示用戶端計數。對 WireGuard，「匯出所有連結」視窗分為兩個分頁：**Config**（串接的 `.conf` 區塊，每個用戶端一個）與 **Links**（個人 `wireguard://` 連結）；「複製」與「下載」作用於目前開啟的分頁。對部署在節點上的 inbound，用戶端 `.conf` / QR 中的 `Endpoint` 現在指向**節點地址**，而非主面板。

WireGuard 用戶端設定與分享

WireGuard 用戶端在「資訊」/QR 視窗中有一張可收合的設定卡片：完整的 `.conf`（[Interface] 含 `PrivateKey / Address / DNS / MTU`，[Peer] 含 `PublicKey / PresharedKey / AllowedIPs = 0.0.0.0/0, ::/0 / Endpoint / PersistentKeepalive`），並附 **複製**、**下載** 和 **QR** 按鈕。亦支援 `wireguard://` / `wg://` 形式的連結，用戶端應用程式可將其解析為完整的 `.conf`。

何時選擇 WireGuard：需要真正的 WireGuard VPN 隧道而非偽裝代理時。

5.10. Hysteria（預設 v2）

用途：基於 QUIC 的 Hysteria inbound。面板預設使用版本 2。每個客戶端以 auth token 進行身份驗證，而非 UUID/密碼。TLS 對 Hysteria 始終可用（參見 5.2 中的功能對照表）。

settings 區塊的欄位：

欄位	預設值	說明
version	2	協議版本（最低 1；面板預設 2）
clients	[]	客戶端列表

每個客戶端的關鍵欄位為 auth（token，必填），以及通用欄位（email、限制、enable、tgId、subId、comment、reset）。

額外參數在 streamSettings.hysteriaSettings 中設定：

欄位	值 / 選項	說明
version	固定為 2（欄位鎖定）	「版本」（英文「Version」）
udpIdleTimeout	（整數 ≥ 1，秒）	「UDP idle timeout（秒）」——UDP 閒置逾時時間
masquerade	預設關閉	「Masquerade」——對「未授權」請求偽裝成普通網頁伺服器

啟用 masquerade 時，可選擇類型（type）：

- ``——預設（404 頁面）；
- proxy ——反向代理（欄位「Upstream URL」、「改寫 Host」、「跳過 TLS verify」）；
- file ——提供目錄（欄位「目錄」，例如 /var/www/html）；
- string ——固定回應（欄位「狀態碼」、「Body」、「標頭」）。

何時選擇 Hysteria：需要 QUIC 傳輸及在不穩定/行動網路上保持穩定性時；偽裝功能可提升入口隱蔽性。

5.11. MTProto（Telegram 代理）

在版本 3.3.0 中新增。協議值為 mtproto。

MTProto 是 Telegram 自有代理協議。在 3X-UI 中，此類 inbound 不由 Xray 管理，而由獨立的 mtg 行程管理，面板本身控制該行程。面板會定期將已啟用的 MTProto inbound 與正在運行的 mtg 行程進行核對：啟動缺失的、停止多餘的，並從 mtg 的指標中讀取流量計數。因此，此類 inbound 的流量統計與普通協議一樣正常運作。

自 3.5.0 版起，MTPROTO 是多客戶端協議（引擎改為 `mtg-multi` 分支）：一個 inbound 可服務多位使用者，每位使用者都是一般的客戶端（見第 8 節），擁有自己的 FakeTLS secret、流量配額、有效期、啟用開關、選填的 ad-tag 和個人 `tg://proxy` 連結。先前「一個 inbound = 一個 secret」的模式已廢除：表單中 inbound 層級的「Secret」欄位已移除，面板更新時，既有的單一 secret 會自動轉換為該 inbound 的第一個客戶端（無需手動操作）。

相應說明：

- 「Transport」（Stream Settings）分頁不適用於此 inbound；客戶端則與其他協議一樣管理——繫結/解除繫結、限制、群組、訂閱。
- 客戶端變更（新增、刪除、更換 secret、啟用/停用、ad-tag）會透過 `mtg` 的 management API **「即時」** 套用，不會中斷其他使用者的 Telegram 工作階段；只有 inbound 的結構性變更（地址、domain fronting、連線數限制、公網 IP）會重啟行程。
- MTPROTO inbound 僅在主面板上運行；不會部署至子節點（nodes）（設有 `NodeID` 的 inbound 會被跳過）。
- **「Sniffing」** 分頁對 MTPROTO 隱藏——此協議由 `mtg` 行程管理，而非 Xray，因此 sniffing 不適用。

欄位。儲存於 inbound 的 `settings` 中：

介面欄位	鍵	說明
Remark	<code>remark</code>	Inbound 標籤。
Listen IP	<code>listen</code>	監聽 IP（空 = 所有介面）。
Port	<code>port</code>	代理埠。
偽裝域名 (FakeTLS)	<code>settings.fakeTlsDomain</code>	預設 <code>www.cloudflare.com</code> 。自 3.5.0 起——生成新客戶端 secret 時使用的預設域名（提示：「Default FakeTLS domain used to generate a new client's secret. Each client can front its own domain」）；每個客戶端都可透過自己的 secret 使用自己的偽裝域名。
Max connections	<code>settings.throttleMaxConnections</code>	3.5.0 新增。所有使用者的同時連線數上限（公平分配）；0 —— 不限制。
Public IPv4 / Public IPv6	<code>settings.publicIpv4</code> / <code>settings.publicIpv6</code>	3.5.0 新增。伺服器的公網地址，用於 ad-tag middle proxy（輸入提示 <code>1.2.3.4</code> / <code>2001:db8::1</code> ）；空白 —— 由 <code>mtg</code> 自行偵測。

Secret 格式 (FakeTLS)。secret 現在屬於每個客戶端（客戶端表單中的「MTPROTO secret」欄位，帶重新生成按鈕；旁邊是選填的「Ad-tag（贊助頻道）」，恰為 32 個 hex 字元）。secret 的格式不變：`ee` + 32 個 hex 字元 + 偽裝域名的 hex 編碼（`ee<hex32><hex(🔍)>`）；將新客戶端繫結至 MTPROTO inbound 時，secret 會依 inbound 的預設域名自動生成。自 3.5.0 起，客戶端的流量配額與有效期實際生效（透過 `mtg-multi` 的限制）：已耗盡或已過期的客戶端會被停用，重置流量後立即恢復存取。

Domain-fronting 與 mtg 進階選項

MTPROTO inbound 有 `mtg` 行程的額外參數。**Domain fronting IP**、**Domain fronting port** 和 **Domain fronting PROXY protocol** 欄位指定 `mtg` 將非 Telegram 流量（例如發送至偽造的 NGINX 網站）傳送到何處：若 IP 留空，則透過 DNS 使用 FakeTLS 域名，埠預設為 443。另外還有 **Accept PROXY protocol**（用於監聽器）、**IP preference**（`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`）和 **Debug logging**。每個值只有在設定後才會寫入 `mtg-<id>.toml` 檔案。

透過 Xray 路由 Telegram 流量

「Route through Xray」開關（預設關閉）和選填的 **Outbound** 欄位允許將 Telegram 的出口流量交由 Xray 路由器管理。啟用後，面板會在 Xray 設定中注入一個帶有該 inbound 標籤的本地 SOCKS 橋接，`mtg` 透過它發送 Telegram 流量。之後可在「Routing」分頁使用規則匹配流量，或透過 **Outbound** 欄位強制導向指定的 outbound 或負載均衡器（若欄位為空，則由路由規則決定）。

如何分享給用戶。對於 MTPROTO inbound，面板會生成邀請連結：

範例：FakeTLS secret 及完整連結。 若偽裝域名為 `www.cloudflare.com`，secret 的組成為 `ee` + 32 個 hex 字元 + 域名的 hex 編碼，例如：

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

完整的邀請連結（此連結和 QR 碼透過 Telegram 發送給用戶）：

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f756466c6172652e636f6d
```

```
tg://proxy?server=<[?]>&port=<[?]>&secret=<secret>
```

（等效形式為 `https://t.me/proxy?server=...&port=...&secret=...`）。自 3.5.0 起，此連結為個人專屬——由具體客戶端的 secret 生成，且其中的 `#remark` 片段已移除（Telegram 部分不嚴謹的解析器會將其與 secret 黏在一起而破壞匯入）；備註改以獨立標籤顯示在資訊視窗中。將此連結和 QR 碼發送給 Telegram 用戶——打開後代理會立即添加至應用程式。此連結也透過訂閱伺服器提供。

何時使用。 這是繞過 Telegram 封鎖的標準方式；FakeTLS 偽裝（偽裝域名）使流量看起來像對指定網站的普通訪問。

5.12. 協議選擇速查表

- **VLESS**——預設首選；與 REALITY 或 TLS + XTLS-Vision 組合時最佳，支援後量子身份驗證。
- **Trojan**——偽裝成 HTTPS，可設置 fallback 至網頁伺服器。
- **VMess**——與舊客戶端相容。
- **Shadowsocks**——無 TLS 偽裝的簡單代理；現代首選方式為 `2022-blake3-*`。
- **Hysteria**——QUIC，在不穩定網路上保持穩定。
- **mixed / http**——服務性 SOCKS/HTTP 代理。

- **WireGuard**——完整的 VPN 隧道。
 - **tunnel**——透明埠轉發。
 - **MTProto**——繞過 Telegram 封鎖的代理 (FakeTLS) ；由獨立的 `mtg` 行程管理。
-

6. 傳輸 (Stream Settings)

傳輸（在面板介面中為「**Транспорт**」欄位，英文 *Transmission*）決定 Xray-core 在 inbound 內傳遞資料的方式：在 TLS/Reality 之上使用哪種網路協定，以及流量如何分幀。這些參數會儲存到 Xray 設定的 `streamSettings` 物件中，並在 inbound 編輯器的傳輸分頁中設定。加密（TLS / Reality）會在另一節中討論——這裡只描述網路的選擇及其參數。

6.1. 選擇傳輸網路

網路在「**Транспорт**」下拉選單（`streamSettings.network`）中選擇。預設值為 `tcp`（在列表中顯示為 **RAW**）。可用的選項如下：

列表中的值	network 欄位	傳輸
RAW	<code>tcp</code>	普通 TCP（在新版 Xray 中更名為 RAW），可選擇搭配 HTTP 混淆
mKCP	<code>kcp</code>	可靠的 UDP 傳輸 mKCP
WebSocket	<code>ws</code>	基於 HTTP(S) 的 WebSocket
gRPC	<code>grpc</code>	gRPC 隧道 (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP——現代的可多工傳輸

切換值時，面板會清除前一個網路的設定區塊，並用該網路結構的預設值填充新網路的區塊，因此子表單的每個欄位始終會有合理的初始值。

重要。 在此版本的面板中，列表中沒有 **HTTP/2 (h2) 傳輸**——它已從網路集合中移除；對於雙向類 HTTP/2 隧道請使用 gRPC，對於現代的 HTTP 偽裝傳輸請使用 XHTTP。**Hysteria 傳輸 (hysteria)** 不透過此列表選擇：它與 Hysteria 協定緊密綁定，並在 inbound 本身以 Hysteria 協定建立時自動出現（見第 6.8 節）。

下面分別解析每個網路及其每個欄位。

6.2. RAW / TCP (`tcpSettings`)

基礎 TCP 傳輸。預設情況下流量「按原樣」傳遞；可選擇將其偽裝成普通的 HTTP/1.1 交換。

欄位	預設值	描述
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (關閉)	接受來自上游負載均衡器/代理的 PROXY protocol 標頭
HTTP 混淆 (<code>header.type</code>)	<code>none</code> (關閉)	開啟將流量偽裝成 HTTP/1.1

Proxy Protocol

「Proxy Protocol」 切換開關（`acceptProxyProtocol`）。開啟時，Xray 會在傳入連線上等待 PROXY protocol 標頭，並從中提取客戶端的真實 IP。僅當面板前面有反向代理/負載均衡器（例如帶 `send-proxy` 的 HAProxy 或 nginx）會添加此標頭時才開啟。預設為關閉。

HTTP 混淆（camouflage）

「HTTP Обфускация」 切換開關。控制 `header` 欄位：

- 關閉 → `header.type = "none"`（在傳輸線上 `header` 欄位直接不存在）。純 TCP。
- 開啟 → `header.type = "http"`。流量會以 HTTP/1.1 請求與回應的形式分幀。開啟時面板會立即用預設值填充 `request` 與 `response` 子物件。

開啟 HTTP 混淆時會出現用於設定模擬請求與回應的欄位。

請求標頭（`header.request`）：

欄位	鍵	預設值	描述
請求版本	<code>request.version</code>	1.1	請求起始行中的 HTTP 版本
請求方法	<code>request.method</code>	GET	模擬請求的 HTTP 方法
請求路徑	<code>request.path</code>	/	路徑。以逗號分隔的值列表輸入；在傳輸線上是字串陣列。若留空則填入 /
請求標頭	<code>request.headers</code>	{ } (空)	HTTP 標頭的「名稱/值」表格。儲存為 [?] → [?] 映射（一個名稱可對應多個值）

回應標頭（`header.response`）：

欄位	鍵	預設值	描述
回應版本	<code>response.version</code>	1.1	回應起始行中的 HTTP 版本
回應狀態	<code>response.status</code>	200	模擬回應的 HTTP 狀態碼
回應原因	<code>response.reason</code>	OK	狀態的文字描述（reason-phrase）
回應標頭	<code>response.headers</code>	{ } (空)	回應標頭的「名稱/值」表格（ [?] → [?] 映射）

標頭欄位逐行編輯——每行設定標頭名稱（Имя）及其值（Значение）。這些參數僅用於偽裝流量的外觀；對加密沒有影響。預設值（GET / HTTP/1.1，回應 200 OK）適用於大多數情境——只有在需要模擬特定網站/服務時才值得修改。

帶 HTTP 混淆的 RAW 之 `streamSettings` 範例：

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

請注意：在傳輸線上 `path` 是字串陣列，而每個標頭是值的陣列（`Host` → `["www.example.com"]`）。

6.3. mKCP（`kcpSettings`）

mKCP 是基於 UDP 的可靠傳輸。在丟包率高、延遲大的通道上很有用，但會產生較高的額外流量。所有預設值都與 xray-core 中推薦的值一致。

欄位	鍵	預設	允許範圍	描述
MTU	<code>mtu</code>	1350	576–1460	最大封包大小（位元組）。出現分片問題時調小
TTI（毫秒）	<code>tti</code>	20	10–100	傳輸間隔（ms）。越小延遲越低，但額外開銷越高
Uplink（MB/c）	<code>uplinkCapacity</code>	5	≥ 0	上傳的估算吞吐量（MB/c）
Downlink（MB/c）	<code>downlinkCapacity</code>	20	≥ 0	下載的估算吞吐量（MB/c）
CWND 乘數	<code>cwndMultiplier</code>	1	≥ 1	壅塞視窗（congestion window）乘數
最大發送視窗	<code>maxSendingWindow</code>	2097152	≥ 0	最大發送視窗大小

欄位說明：

- **Uplink / Downlink capacity** 設定 mKCP 佔用通道的積極程度。應依照實際的通道頻寬設定：值過高會導致多餘流量，值過低則導致通道未被充分利用。
- **TTI** 直接影響「延遲 額外開銷」的權衡：較小的值降低延遲，但增加服務性封包的數量。
- **MTU** 限制單個 mKCP 封包的大小；調低有助於在大型 UDP 封包被切割或丟失的通道上使用。

在此版本中，mKCP 子表單裡的「seed」欄位（mKCP 混淆密碼）以及標頭/混淆類型下拉選單（none、srtp、utp、wechat-video、dtls、wireguard）不作為獨立欄位存在——傳輸層混淆已移至通用的「FinalMask」機制（包括 mkcp-legacy 模式），在相應的章節中描述。「congestion」參數也未作為獨立勾選框列出；壅塞控制透過 cwndMultiplier 與 maxSendingWindow 設定。

6.4. WebSocket (wsSettings)

基於 HTTP(S) 的 WebSocket 傳輸。能很好地穿過 CDN 與反向代理，偽裝成普通的網頁流量。

欄位	鍵	預設	描述
Proxy Protocol	acceptProxyProtocol	false	接受來自上游代理的 PROXY protocol 標頭（見第 6.2 節）
Хост	host	"" (空)	HTTP 標頭 Host 的值。在透過 CDN/域前置工作時指定
Путь	path	/	WebSocket 握手請求行中的路徑
heartbeat 週期	heartbeatPeriod	0	發送 heartbeat 幀的間隔（秒）。0 會停用 heartbeat
標頭	headers	{ } (空)	握手的額外 HTTP 標頭。「名稱 → 值」映射（僅字串值，無陣列）

說明：

- **Путь** 必須在伺服器（inbound）與客戶端上一致。常常會在 web 伺服器側用此路徑來偽裝入口點。
- **Хост** 在 inbound 位於 CDN 之後或使用域前置時才有意義；否則可以留空。
- **heartbeat 週期** 在穿過會積極斷開不活躍連線的代理/CDN 時保持連線「活著」。預設（0）heartbeat 為關閉。
- 與 RAW 不同，WebSocket 的標頭表格使用「扁平」格式 `key → value`（每個標頭一行值）。

位於 CDN 之後的 WebSocket 之 streamSettings 範例：

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

host 與 path 的值必須與客戶端一致；與 RAW 不同，這裡的標頭值是普通字串，而非陣列。

6.5. gRPC (`grpcSettings`)

按參數數量計算最「輕量」的傳輸。在 gRPC 呼叫內部（基於 HTTP/2）對流量進行隧道封裝；與支援 gRPC 的 CDN 良好相容。沒有標頭混淆。

欄位	鍵	預設	描述
服務名稱 (Service Name)	<code>serviceName</code>	<code>""</code> (空)	gRPC 服務的名稱（實際上是隧道的「路徑」）。必須在伺服器與客戶端上一致
Authority	<code>authority</code>	<code>""</code> (空)	偽標頭 <code>:authority</code> 的值 (HTTP/2 中相當於 <code>Host</code>)。在透過 CDN/域工作時指定
Multi Mode	<code>multiMode</code>	<code>false</code> (關閉)	開啟在單一連線內多工多個並行 gRPC 流

說明：

- **Service Name** 是 gRPC 通道的主要識別碼；它在兩端必須相同。空值是允許的，但通常會設定一個不明顯的字串來偽裝。
- **Authority** 影響在 HTTP/2 幀中發送哪個 `:authority`；主要在透過 CDN 代理時需要。
- **Multi Mode** 允許多個邏輯流透過單一物理連線；在伺服器與客戶端都支援時開啟以提升效能。

gRPC 的 `streamSettings` 範例：

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName`（此處為 `GunService`）扮演隧道「路徑」的角色，必須在伺服器與客戶端上一致。

6.6. HTTPUpgrade (`httpupgradeSettings`)

基於 HTTP Upgrade 機制的傳輸（與 WebSocket 類似，但沒有 WebSocket 協定本身）。同樣能很好地穿過代理與 CDN。欄位集合與 WebSocket 相同，但沒有 heartbeat 週期。

欄位	鍵	預設	描述
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	接受來自上游代理的 PROXY protocol 標頭
Хост	<code>host</code>	<code>""</code> (空)	HTTP 標頭 <code>Host</code> 的值
Путь	<code>path</code>	<code>/</code>	帶 Upgrade 標頭的 HTTP 請求路徑

欄位	鍵	預設	描述
標頭	headers	{ } (空)	額外的 HTTP 標頭。「扁平」映射 [?][?] → [?] (與 WebSocket 相同)

Хост、Путь 與 **標頭** 欄位的用途與 WebSocket 相同 (第 6.4 節)。HTTPUpgrade 未提供 heartbeat——這是 WebSocket 的特性。

6.7. XHTTP / SplitHTTP (xhttpSettings)

XHTTP (又稱 SplitHTTP) 是 xray-core 現代的可多工 HTTP 傳輸。它將上行與下行流分成各自的 HTTP 請求，這很適合 CDN 及對連線持續時間有限制的環境。並非所有欄位都會同時顯示在編輯器中：其中一部分會依所選模式 (mode) 與已開啟的切換開關而出現。

基礎欄位 (始終顯示)

欄位	鍵	預設	描述
Хост	host	"" (空)	HTTP 標頭 Host 的值
Путь	path	/	HTTP 請求的基礎路徑
模式 (Mode)	mode	auto	傳輸模式 (見下文)
Server Max Header Bytes	serverMaxHeaderBytes	0	伺服器上請求標頭大小的限制 (位元組)。0 為 xray-core 的預設值
Padding Bytes	xPaddingBytes	100-1000	隨機「填充」padding 的範圍 (位元組，格式為 мин-макс)，用於阻礙大小分析
標頭	headers	{ } (空)	額外的 HTTP 標頭。「扁平」映射 [?][?] → [?]
HTTP 方法 Uplink	uplinkHTTPMethod	"" (Default = POST)	上行請求的 HTTP 方法。選項：空 (預設 POST)、POST、PUT、GET (後者僅在 packet-up 模式可用)
Padding Obfs Mode	xPaddingObfsMode	false (關閉)	開啟進階的 padding 混淆並展開額外欄位 (見下文)
No SSE Header	noSSEHeader	false (關閉)	不發送 Content-Type: text/event-stream (SSE) 標頭。若它妨礙穿過中間節點則開啟

「模式」欄位 (mode)

下拉選單，值如下：

值	描述
auto	自動選擇模式 (預設)

值	描述
packet-up	上行流被拆分成各自的 HTTP 請求（每個請求一個封包）
stream-up	上行流以單一長時間串流請求傳輸
stream-one	單一共用的雙向串流請求

模式的選擇決定哪些額外欄位會變為顯示。

僅在 `mode = packet-up` 時顯示的欄位：

欄位	鍵	預設	描述
最大緩衝上傳	scMaxBufferedPosts	30	上行流同時緩衝的 POST 請求上限
最大上傳大小 (位元組)	scMaxEachPostBytes	1000000	單個上行 POST 請求的最大大小 (位元組)
Uplink Data Placement	uplinkDataPlacement	"" (Default = body)	上行流資料的放置位置：body、header、cookie、query
Uplink Data Key	uplinkDataKey	""	uplink 資料的鍵/標頭名稱。僅在 uplinkDataPlacement 已設定且不等於 body 時出現

僅在 `mode = stream-up` 時顯示的欄位：

欄位	鍵	預設	描述
Stream-Up Server	scStreamUpServerSecs	20-80	伺服器串流連線的保持時間範圍 (秒，格式為 мин-макс)

padding 混淆欄位 (在 `xPaddingObfsMode = 開啟` 時顯示)

欄位	鍵	預設	描述
Padding Key	xPaddingKey	"" (placeholder x_padding)	padding 的鍵名稱
Padding Header	xPaddingHeader	"" (placeholder X-Padding)	傳遞 padding 的 HTTP 標頭名稱
Padding Placement	xPaddingPlacement	"" (Default = queryInHeader)	padding 的放置位置：queryInHeader、header、cookie、query
Padding Method	xPaddingMethod	"" (Default = repeat-x)	padding 的產生方法：repeat-x 或 tokenish

工作階段與序列的放置（始終顯示）

欄位	鍵	預設	描述
Session ID Placement	<code>sessionIDPlacement</code>	"" (Default = path)	傳遞工作階段識別碼的位置：path、header、cookie、query
Session ID Key	<code>sessionIDKey</code>	"" (placeholder x_session)	工作階段鍵名稱。僅在 sessionIDPlacement 已設定且不等於 path 時出現
Session ID Table	<code>sessionIDTable</code>	"" (placeholder Base62)	用於產生工作階段識別碼的字元集。可從自動補全下拉選單中選擇預定義值（ALPHABET、Alphabet、BASE36、Base62、HEX、alphabet、base36、hex、number），或輸入任意 ASCII 字串。空為 xray-core 的預設值
Session ID Length	<code>sessionIDLength</code>	"" (空)	產生的識別碼的長度或範圍（例如 8-16）。僅在已設定 Session ID Table 時顯示；最小值必須大於 0
Sequence Placement	<code>seqPlacement</code>	"" (Default = path)	傳遞封包序號的位置：path、header、cookie、query
Sequence Key	<code>seqKey</code>	"" (placeholder x_seq)	序列鍵名稱。僅在 seqPlacement 已設定且不等於 path 時出現

工作階段欄位已依 xray-core v26.6.22 更名：以前稱為 **Session Placement / Session Key**（`sessionPlacement / sessionKey`）——現在為 **Session ID Placement / Session ID Key**（`sessionIDPlacement / sessionIDKey`）；核心已不再識別舊名稱。更新前儲存的 inbound 會自動遷移到新鍵——無需重新儲存。

建議：

- 對於大多數安裝，將模式 = **auto**、設定 **Путь/Хост**，並（在透過 CDN 工作時）與客戶端協調這些值即可。
- 放置欄位（*Placement / *Key）與 padding 混淆僅在針對特定反 DPI/CDN 情境進行精細調整時才需要；當值為空時會使用括號中標示的 xray-core 預設值。
- 與客戶端/outbound 端相關的參數（例如重複 POST 的間隔、區塊大小）不會顯示在 inbound 表單中——監聽伺服器會忽略它們。相反地，XMUX 多工器在 inbound 表單中可用（見下文）。
- 不再填入服務性預設值**。面板不再在 XHTTP 設定中寫入服務性預設值 `scMaxEachPostBytes` 與 `scMinPostsIntervalMs` ——會套用 xray-core 的內部值。這消除了一個固定的 DPI 簽名（字面量 `scMinPostsIntervalMs=30`），先前流量可能因此被封鎖。對於已儲存的 inbound，與 xray-core 預設值相符的值不會輸出到連結與訂閱中（無需重新儲存 inbound）；手動設定的值會被保留。

XHTTP（auto 模式）的 streamSettings 範例：

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

對於大多數安裝，這四個欄位就足夠了；工作階段/序列放置與 padding 混淆欄位留空——此時會使用 xray-core 的預設值。

XMUX（連線多工）

XMUX 切換開關（`enableXmux`）會開啟多工層，將並行請求分配到一個小型物理連線池中。開啟時會展開六個可設定的欄位（儲存於 `xhttpSettings.xmux`）：

欄位	鍵	預設	描述
Max Concurrency	<code>maxConcurrency</code>	16-32	單個連線的並行請求上限（範圍 мин-макс）
Max Connections	<code>maxConnections</code>	6	物理連線的上限（0 為無限制）。自 3.4.2 起，新 inbound 以 6 建立；既有 inbound 保留其原值。
Max Reuse Times	<code>cMaxReuseTimes</code>	""（空）	連線重複使用的次數
Max Request Times	<code>hMaxRequestTimes</code>	600-900	單個連線的請求上限（範圍）
Max Reusable Secs	<code>hMaxReusableSecs</code>	1800-3000	連線可供重複使用的時間（秒，範圍）
Keep Alive Period	<code>hKeepAlivePeriod</code>	""（空）	保持連線的 keep-alive 週期

重要。 不能同時設定 **Max Connections** 與 **Max Concurrency**——xray-core 會拒絕這樣的設定。預設情況下，開啟 XMUX 時面板會填入 Max Concurrency = 16-32；如果您設定了 **Max Connections**（值大於 0），面板會移除預設的 Max Concurrency 值以避免衝突。

6.8. Hysteria 傳輸（`hysteriaSettings`）

Hysteria 傳輸不在「Транспорт」列表中選擇：它在 inbound 以 Hysteria 協定建立時自動啟用，對其他協定則隱藏（離開 Hysteria 協定時網路會強制回到 tcp）。參數：

欄位	鍵	預設	描述
版本	<code>version</code>	2（已固定，欄位鎖定）	Hysteria 版本。僅支援 Hysteria 2
UDP idle timeout（秒）	<code>udpIdleTimeout</code>	60	UDP 工作階段的閒置逾時（秒）。允許範圍為 2-600；xray-core 在啟動時會拒絕此區間外的值
Masquerade	<code>masquerade</code>	關閉（不存在）	開啟在探測時將監聽器偽裝成 HTTP/3 伺服器

開啟 **Masquerade** 時會出現類型選擇（`type`）以及依其而定的欄位：

- `""` — **default (404 page)**：傳回標準的 404 頁面（無需額外欄位）。
- **proxy (reverse proxy)**：反向代理到外部網站。
 - `url`（**Upstream URL**, placeholder `https://www.example.com`）——目標位址；
 - `rewriteHost`（**Переписать Host**，預設 `false`）——替換 `Host` 標頭；
 - `insecure`（**Пропустить TLS verify**，預設 `false`）——不驗證上游的 TLS 憑證。
- **file (serve directory)**：從目錄提供檔案。
 - `dir`（**Директория**, placeholder `/var/www/html`）。
- **string (fixed body)**：固定的 HTTP 回應。
 - `statusCode`（**Код статуса**，預設 `0`，範圍 0-599）；
 - `content`（**Body**）——回應主體；
 - `headers`（**Заголовки**）—— [?] [?] → [?] 映射。

Masquerade 讓基於 Hysteria 的 inbound 對主動探測看起來像普通的 HTTP/3 伺服器，從而提高隱蔽性。預設情況下偽裝為關閉。

帶反向代理（`masquerade` → `proxy`）的 `hysteriaSettings` 範例：

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

此處在探測時監聽器會傳回來自 `https://www.example.com` 的回應，偽裝成普通的 HTTP/3 網站。

6.9. 相關參數

除了選擇網路之外，同一個分頁中還有兩個不依賴於特定傳輸的通用區塊（詳見相應章節）：

- **External Proxy**（`externalProxy`）——一組外部位址/連接埠，會在訂閱連結中替換面板本身的位址。
- **Sockopt**（`sockopt`）——低階的 socket 選項（TCP Fast Open、mark、域策略、透明代理等）。

Real client IP（在 CDN/中繼之後判定真實 IP）

當 inbound 位於中介之後（如 Cloudflare 這類 CDN、L4 隧道/中繼或另一個面板）時，Xray 看到的是中介的位址，而非真實訪客。此位址會進入線上客戶端列表，並依它計算每個客戶端的 IP 限制，因此兩者在代理之後都變得無用。為了還原真實 IP，inbound 表單的 **Sockopt** 區段中有 **Real client IP** 預設集的選擇，它整合了 `acceptProxyProtocol` 與 `trustedXForwardedFor` 的設定：

預設集	作用	何時套用
Off / direct	清除兩個欄位。	inbound 直接對客戶端開放
Cloudflare CDN	設定 <code>sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> 。	位於 Cloudflare CDN（橙色雲）之後的 WebSocket / HTTPUpgrade / XHTTP / gRPC
L4 relay / Spectrum (PROXY)	開啟 <code>acceptProxyProtocol = true</code> 。	inbound 前的 L4 隧道/中繼，或 Cloudflare Spectrum

各預設集互斥：選擇一個會清除另一個的欄位，因此過時的 `trustedXForwardedFor` 不會蓋過依 PROXY 協定還原的 IP。預設集下方仍會顯示「原始」的 **Proxy Protocol** 切換開關與 **Trusted X-Forwarded-For** 列表——預設集只是替您填寫它們，必要時可手動修改。如果所選預設集不被目前的傳輸支援（例如在 mKCP 上使用 PROXY 協定），表單會顯示警告。這些欄位僅與伺服器端相關，絕不會在訂閱中發送給客戶端。

只用其中一種。`acceptProxyProtocol` 從 L4 的 PROXY 協定標頭讀取真實 IP，而 `trustedXForwardedFor` 從 HTTP 請求標頭讀取；只有在您的中介鏈確實需要時才值得手動混用它們。

- **FinalMask**（`finalmask`）——通用的傳輸層混淆機制（包括 mKCP 的 legacy 混淆），它取代了網路子表單內部的獨立「seed」/「header type」欄位。

7. 連線安全性：TLS、XTLS 與 REALITY

每個支援透過 transport 串流傳輸的 inbound (VMess、VLESS、Trojan、Shadowsocks、Hysteria) 在編輯器中都有「安全性」標籤頁，用於設定傳輸通道的加密與偽裝方式。共有三種模式，透過單選按鈕切換：

模式	UI 顯示名稱	可用時機
none	無	始終可用 (Hysteria 除外，TLS 為必填)
tls	TLS	適用於 VMess/VLESS/Trojan/Shadowsocks 在 tcp、ws、http、grpc、httpupgrade、xhttp 網路上；Hysteria 始終使用
reality	Reality	僅適用於 VLESS/Trojan 在 tcp、http、grpc、xhttp 網路上

若協議為 Hysteria，無按鈕不會顯示 (TLS 為必填)。Reality 按鈕僅在協議與網路的組合符合允許條件時才會出現 (見上表)。

切換模式時，面板會完整重建 streamSettings 區塊：移除前一模式的 tlsSettings 與 realitySettings，並填入所選模式的預設值。特別是選擇 Reality 時，面板會立即自動：從內建熱門網域清單中隨機填入一組 target + serverNames (SNI)、生成隨機 shortIds，並向伺服器請求最新的 X25519 金鑰對 (privateKey/publicKey)。

7.1. 差異說明：TLS vs XTLS vs REALITY

- **TLS** – 使用 TLS 1.2/1.3 協議的傳統傳輸加密。伺服器上須有有效憑證 (自有域名 + 憑證鏈)。流量看起來與一般 HTTPS 相同，但對主動審查者而言，TLS 握手指向您的域名具有可識別性；若依 SNI 封鎖或憑證不受信任，連線將被封鎖或出現錯誤。
- **XTLS (Vision)** – 這不是「安全性」清單中的獨立模式，而是在 TLS 或 Reality 之上的 flow 機制。透過 inbound 客戶端側的 Flow 欄位設為 xtls-rprx-vision (或 xtls-rprx-vision-udp443) 來啟用。Vision 適用於 VLESS 在 tcp 網路上且 security = tls 或 security = reality 的情況，以及 VLESS 透過 xhttp 傳輸並啟用 VLESS 加密 (vlessenc / ML-KEM) 的情況——此時 Flow 欄位同樣可設為 xtls-rprx-vision，且該值會正確帶入 vless:// 連結 (flow=xtls-rprx-vision)。Vision 透過握手後直接傳遞有效載荷來降低「雙重加密」(TLS-in-TLS)，從而提升傳輸速度並改善流量偽裝。因此，VLESS + Reality + Flow xtls-rprx-vision 的組合被視為現代推薦配置。

Flow Vision 的自動還原。 若 VLESS/XHTTP-inbound 的加密功能 (ML-KEM、decryption/encryption 欄位) 在客戶端加入後才啟用，該 inbound 便具備了使用 flow 的條件。在這種情況下，面板會自動為那些應當擁有 flow = xtls-rprx-vision 但 Flow 欄位卻為空的客戶端還原該設定。過去在這種場景下，Vision 會靜默地從設定、分享連結與訂閱中消失 (在節點型 inbound 上尤為明顯)。無需任何手動操作：修正會在保存 inbound 時自動套用，並在面板更新時執行一次。此行為保守——面板不會憑空新增 flow，也不會覆蓋客戶端明確設定的值。

- **REALITY** – 一種無需自有憑證的偽裝機制。伺服器「借用」真實第三方網站 (target / serverNames) 的 TLS 握手，因此對觀察者而言連線與存取該網站無異，且完全不需要憑證。驗證依賴 X25519 金鑰對

與 `shortIds` 集合。REALITY 對主動探測（active probing）和 SNI 封鎖具有抵抗力，因為 SNI 指向的是真實的外部域名。代價是設定要求更嚴格（需正確填寫含連接埠的 `target`，並與客戶端同步金鑰）。

簡短選擇原則：

- 有自有域名與有效憑證，且只需呈現 HTTPS 外觀 → **TLS**（盡可能搭配 Vision）；
- 無域名/憑證，或需要對 DPI 達到最高隱蔽性 → **REALITY**（對 VLESS/TCP 搭配 Vision）。

7.2. 「無」模式（`none`）

傳輸不包含 TLS 封裝：`streamSettings` 中的 `tlsSettings` 與 `realitySettings` 區塊將被移除。此模式無額外欄位。適用場景：

- inbound 僅監聽 `127.0.0.1` 並作為 fallback 目標（依面板規則，用於 fallback 的子 inbound 須監聽 `127.0.0.1` 且 `security=none`）；
- 加密/偽裝由外部層負責（例如面板前置的 Nginx 反向代理）；
- 在內部網路中使用自帶加密的協議（Shadowsocks）。

對外可存取的 inbound 不建議使用「無」模式。

範例：`tcp` 網路上 TLS 的 `streamSettings` 區塊（VLESS/Trojan/VMess）。以下是選擇 **TLS** 模式並填寫 SNI 和憑證路徑後的結果：

```
"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}
```

7.3. TLS 模式

`tlsSettings` 區塊的欄位。預設值取自面板的設定結構描述。

基本參數

欄位 (標籤)	預設值	說明
SNI (serverName)	"" (空白)	Server Name Indication – TLS 握手中呈現的域名。須與憑證域名一致。英文輸入提示：「Server Name Indication」。
Cipher Suites (cipherSuites)	"" → 自動	允許的加密套件清單。預設為空，由 Xray/Go 自行選擇（自動選項）。僅在需要明確限制加密套件時才修改。
最低/最高版本 (minMaxVersion)	min = 1.2 , max = 1.3	TLS 的最低與最高版本。可用值：1.0、1.1、1.2、1.3。建議保持 1.2 – 1.3；不建議將最低版本降至 1.0/1.1（已過時且不安全）。
uTLS (settings.fingerprint)	chrome (表單中可選 None = "")	模擬的 TLS 用戶端 hello 指紋 (uTLS fingerprint)，使握手看起來像來自主流瀏覽器。詳見下方清單。在 TLS 中，清單第一項為 None ("")，用於停用指紋模擬。
ALPN (alpn)	["h2", "http/1.1"]	在 TLS 中協商的應用層協議清單（多選）。可用值：h3、h2、http/1.1。預設提供 h2 與 http/1.1。

uTLS fingerprint 的可用值 (TLS 與 REALITY 相同)：chrome、firefox、safari、ios、android、edge、360、qq、random、randomized、randomizednoalpn、unsafe。在 TLS 表單中另有空白選項 **None** (不套用指紋模擬)。

Cipher Suites 的可用值 (TLS 1.3 及 ECDHE 套件)：TLS_AES_128_GCM_SHA256、TLS_AES_256_GCM_SHA384、TLS_CHACHA20_POLY1305_SHA256、TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA、TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA、TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA、TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA、TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256、TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384、TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256、TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384、TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256、TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256。

TLS 開關

開關	預設值	說明
拒絕未知 SNI (rejectUnknownSni)	關閉 (false)	啟用後，當客戶端提供的 SNI 與憑證中的域名不符時，伺服器將中斷連線。可提升隱蔽性 (伺服器不回應「陌生」請求)，但要求客戶端的 SNI 必須精確匹配。
停用 System Root (disableSystemRoot)	關閉 (false)	停止使用系統受信任根憑證儲存庫。
工作階段恢復 (enableSessionResumption)	關閉 (false)	啟用 TLS 工作階段恢復 (session resumption / session tickets)。

TLS 進階參數 (vsn、曲線、金鑰日誌、ECH Sockopt)

基本 TLS 設定下方提供了額外欄位。

欄位 (標籤)	預設值	說明
Verify Peer Cert By Name (<code>settings.verifyPeerCertByName</code>)	""	客戶端用來驗證伺服器憑證的名稱 (逗號分隔)，取代 SNI 進行驗證。這是 Xray 於 2026-06-01 後移除的 <code>allowInsecure</code> 欄位的現代替代方案。此值僅供面板使用：不會寫入伺服器的 xray 設定檔，但會帶入邀請連結和訂閱 (<code>vcn=...</code>)，供客戶端自行套用。輸入提示： <code>example.com</code> 。
Curve Preferences (<code>curvePreferences</code>)	""	TLS 金鑰交換曲線的限制與優先順序 (如 <code>X25519MLKEM768</code> 、 <code>X25519</code>)。空白時使用 Xray-core 的預設值。
Master Key Log (<code>masterKeyLog</code>)	""	以 <code>SSLKEYLOGFILE</code> 格式記錄 TLS master key 的路徑 (用於在 Wireshark 中解密流量進行偵錯)。輸入提示： <code>/path/to/sslkeylog.txt</code> 。生產環境請留空——此檔案可解密所有流量。
ECH Sockopt (<code>echSockopt</code>)	關閉	Xray 請求 ECH config list 時所用連線的 socket 選項開關。啟用後可設定： Dialer Proxy (<code>dialerProxy</code> – 透過指定標籤的 outbound 代理該請求)、 Domain Strategy (<code>domainStrategy</code>)、 TCP Fast Open (<code>tcpFastOpen</code>)、 Multipath TCP (<code>tcpMptcp</code>)。非必要請保持關閉。

`verifyPeerCertByName`、`curvePreferences`、`masterKeyLog` 與 `echSockopt` 欄位位於 `tlsSettings` 的頂層，且在保存設定時不會被面板欄位裁剪所移除。

憑證

SSL 憑證區塊 (UI 標題為「SSL 憑證」) 以清單形式呈現：點擊 **+** 按鈕新增憑證記錄，點擊 **- 刪除** 按鈕移除 (僅當記錄超過一筆時刪除按鈕才可用)。啟用 TLS 時，預設會建立一筆空白記錄。

每筆記錄可切換輸入模式 (`useFile`)：

- **憑證路徑** (`useFile = true`，預設) – 指定伺服器上的檔案路徑：
 - **公開金鑰** (`certificateFile`) – 憑證檔案路徑 (`.crt` / `.pem`)；
 - **私密金鑰** (`keyFile`) – 私密金鑰檔案路徑 (`.key`)。
- **憑證內容** (`useFile = false`) – 直接將內容貼入欄位 (多行文字區域)：
 - **公開金鑰** (`certificate`) – 憑證的 PEM 內容；
 - **私密金鑰** (`key`) – 金鑰的 PEM 內容。

「憑證路徑」模式下的欄位底部提供兩個按鈕：

- **設定面板憑證** – 自動填入面板本身的 SSL 憑證路徑。對於中央面板上的 inbound，使用其憑證 (`POST /panel/setting/all` → `webCertFile` / `webKeyFile`)；對於指派到節點的 inbound，則使用該節點本身的憑證 (`GET /panel/api/nodes/webCert/{nodeId}`)，因為中央面板的路徑在節點上並不存在。若未設定憑證，將顯示警告：「面板未設定憑證。請先在設定中安裝。」 (面板本身的憑證在「設定 → 安全性」章節中設定)。
- **清除** – 清空兩個路徑欄位。

每筆憑證記錄的額外欄位：

欄位	預設值	說明
OCSP Stapling (ocspStapling)	0 (關閉)	OCSP stapling 更新間隔 (秒)，最小值為 0。新 inbound 預設關閉 (0)：這可避免不支援 OCSP responder 的憑證 (例如已放棄 OCSP 的 Let's Encrypt) 在 xray 日誌中產生錯誤。僅對支援 stapling 的憑證啟用。
單次載入 (oneTimeLoading)	關閉 (false)	啟用後，憑證僅在啟動時從磁碟讀取一次，檔案變更後不會重新讀取。
使用選項 (usage)	encipherment	憑證用途。可用值：encipherment (加密——一般伺服器憑證)、verify (驗證)、issue (簽發——伺服器自行簽發/發行憑證)。
Build Chain (buildChain)	關閉 (false)	僅在 usage = issue 時顯示。是否建立憑證鏈。

inbound 編輯器中沒有獨立的自簽憑證按鈕：面板不會為 inbound 即時生成自簽憑證。憑證須透過路徑/內容方式指定，或透過「設定面板憑證」按鈕從面板設定中帶入。面板本身的 SSL 憑證申請/取得 (包括檔案上傳與域名綁定) 在 設定 → 安全性 章節中操作；此處不提供針對 inbound 的 ACME/Let's Encrypt 端點。

ECH 與憑證綁定 (TLS 進階欄位)

欄位	預設值	說明
ECH key (echServerKeys)	""	伺服器端的 Encrypted Client Hello 金鑰。
ECH config (settings.echConfigList)	""	ECH config list (客戶端部分，帶入連結)。
對等憑證 SHA-256 (settings.pinnedPeerCertSha256)	[]	對等憑證的 SHA-256 雜湊值 (十六進位字串，逗號分隔)。原文提示：「對等憑證的 SHA-256 雜湊值 (十六進位字串，如 e8e2d3...)，逗號分隔。僅供面板使用——不會寫入 xray 伺服器設定，但會帶入邀請連結，供客戶端綁定憑證。」

按鈕：對等憑證 SHA-256 欄位旁有兩個自動填寫按鈕：

- **Fill from this inbound's certificate** (盾牌圖示) — 填入此 inbound 本身憑證的 SHA-256 雜湊值 (透過 getCertHash 端點在本地取得)。
- **Fetch the hash by pinging the SNI (xray tls ping)** (下載圖示) — 透過執行 TLS 連線 (伺服器端呼叫 getRemoteCertHash) 獲取指定 SNI 的伺服器即時憑證雜湊值。須先填寫 **SNI** (serverName) 欄位，否則會顯示提示：「Set the SNI (serverName) first to ping the remote certificate.」

取得的雜湊值將追加至欄位中 (逗號分隔)，並帶入邀請連結供客戶端綁定憑證。

- **取得新 ECH 憑證** — 向伺服器請求當前 SNI 的新 ECH 金鑰對 (POST /panel/api/server/getNewEchCert，伺服器端執行 xray tls ech --serverName <SNI>)；填入 **ECH key** 與 **ECH config** 欄位。
- **清除** — 清空兩個 ECH 欄位。

7.4. REALITY 模式

`realitySettings` 區塊的欄位。REALITY 不使用 SSL 憑證：取而代之的是借用的外部域名 TLS 握手與 X25519 金鑰對。

偽裝參數

欄位 (標籤)	預設值	說明
顯示 (show)	關閉 (false)	將 REALITY 偵錯資訊輸出至 Xray 日誌。通常保持關閉。
Xver (xver)	0	傳送至後端的 PROXY 協議版本 (0 為關閉)。最小值 0。
uTLS (settings.fingerprint)	chrome	模擬的 TLS 指紋 (清單與 TLS 相同，但無空白的 None 選項)。
目標 (target)	"" (自 3.4.2 起，啟用 REALITY 時保持為空)	必填欄位 。REALITY 借用其 TLS 握手的真實域名。原文提示：「必填。須包含連接埠 (如 <i>example.com:443</i>)。未填連接埠時 <i>Xray-core</i> 無法啟動。」面板驗證會檢查連接埠是否存在且合法；否則顯示錯誤「REALITY 目標為必填」/「REALITY 目標須包含連接埠...」/「REALITY 目標的連接埠無效」。欄位旁有「掃描」按鈕 (「即時」驗證當前目標) 與「尋找目標」按鈕 (開啟 REALITY 目標掃描器)；詳見下文。
SNI (serverNames)	[] (與目標一同填入)	允許的 SNI 清單 (標籤式多重輸入)。須與 目標 中的域名相符。成功掃描目標時，SNI 會依其憑證填入。
最大時間差 (毫秒) (maxTimediff)	0	客戶端與伺服器時鐘允許的最大偏差 (毫秒)，0 為不限制。最小值 0。
客戶端最低版本 (minClientVer)	""	Xray 客戶端的最低版本 (自 3.5.0 起輸入提示為 26.3.27)。空白為不限制。
客戶端最高版本 (maxClientVer)	""	Xray 客戶端的最高版本。空白為不限制。
Short IDs (shortIds)	[] (啟用時生成)	用於區分客戶端的短識別碼清單 (十六進位)。標籤式多重輸入；重新整理按鈕生成隨機組合。
SpiderX (settings.spiderX)	/	模擬存取外部網站時使用的「爬蟲」路徑 (REALITY 客戶端部分)，帶入邀請連結。

自 3.4.2 起，啟用 REALITY 時 目標 (target) 與 SNI (serverNames) 不再自動填入——兩個欄位均保持為空，目標改由即時掃描器挑選 (詳見下文)。請選擇流量大、穩定且支援 TLS 1.3 與 HTTP/2 的第三方 HTTPS 網站，且不可位於您自己的伺服器後端。

尋找 REALITY 目標（即時掃描器）

自 3.4.2 起，原本的「隨機目標」按鈕（搭配靜態內建清單）已被 **目標** 欄位旁的兩個動作取代：

- **「掃描」** — 「即時」驗證當前目標：建立 TLS 連線，若目標合適，則依其憑證填入 SNI。提示訊息：
「Цель пригодна — заполнены цель и SNI.» / 「Цель доступна, но не пригодна для REALITY.» / 「Не удалось просканировать цель REALITY.»。
- **「尋找目標」** — 開啟 **「REALITY 目標掃描器」** 視窗：可指定網域、**IP 或 CIDR 範圍**（此時面板會依存活主機的憑證尋找目標），或將欄位留空（檢查內建候選）。結果會排名；欄位有「狀態」/「合適」、「金鑰交換」、「憑證」、「TLS」、「ALPN」、「延遲」。透過 **「使用」** 按鈕，可將選定列填入 inbound 欄位。

目標若伺服器協商出 **TLS 1.3**、**HTTP/2 (ALPN h2)**、**X25519/X25519MLKEM768** 金鑰交換，並提供**受信任**（非萬用字元）憑證，則視為**合適**。掃描在面板伺服器端執行（POST `/panel/api/server/scanRealityTarget` 及 `.../scanRealityTargets`）。

範例：tcp 網路上 REALITY 的 streamSettings 區塊（VLESS）。無需憑證——以借用域名和 X25519 金鑰對取代：

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": ["", "6ba85179e30d4fc2"],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

此處面板的 **目標**（`target`）欄位對應 Xray 最終設定中的 `dest`。若 REALITY inbound 是以舊版面板、透過 API 或外部工具建立，且 `destination` 存放於 `dest` 鍵中，面板在解析時若 `target` 為空，會將 `dest` 正規化為 `target`——因此此類 inbound 可正確載入，**目標** 欄位不會為空，且重新保存不會破壞正常運作的 REALITY。

REALITY 金鑰（X25519）

欄位	預設值	說明
公開金鑰（ <code>settings.publicKey</code> ）	""	X25519 公開金鑰（客戶端放入自身設定/連結中）。
私密金鑰（ <code>privateKey</code> ）	""	X25519 私密金鑰（僅存放於伺服器端）。

金鑰下方的按鈕：

- **取得新憑證** – 向伺服器請求新的金鑰對（GET /panel/api/server/getNewX25519Cert；伺服器端執行 xray x25519），填入 **私密金鑰** 與 **公開金鑰**。首次啟用 REALITY 模式時也會自動生成此金鑰對。

範例：透過 API 取得 X25519 金鑰對（在表單外，例如用於腳本）。請求會返回私密金鑰和公開金鑰：

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# [?][?][?]
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}
```

cookie.txt – 透過 POST /login 登入後取得的工作階段 cookie 檔案。

- **清除** – 清空兩個金鑰欄位。

後量子簽名 ML-DSA-65 (mldsa65)

REALITY 的額外（選用）後量子驗證層：

欄位	預設值	說明
mldsa65 Seed (mldsa65Seed)	""	伺服器端的 ML-DSA-65 金鑰 seed。
mldsa65 Verify (settings.mldsa65Verify)	""	驗證值（客戶端部分，帶入連結）。

按鈕：

- **取得新 Seed** – 請求新的金鑰對（GET /panel/api/server/getNewmldsa65；伺服器端執行 xray mldsa65），填入 **mldsa65 Seed** 與 **mldsa65 Verify**。
- **清除** – 清空兩個欄位。

REALITY fallback 限速與金鑰日誌

REALITY 設定中提供 fallback 流量限速功能——防止主動探測者將伺服器作為連往借用域名的免費通道。限速分別針對兩個方向設定——**Limit Fallback Upload** 與 **Limit Fallback Download**（limitFallbackUpload / limitFallbackDownload），每個方向的欄位相同：

欄位（標籤）	預設值	說明
After Bytes (afterBytes)	0	開始限速前以全速通過的位元組數。0 表示從第一個位元組即開始限速。
Bytes Per Sec (bytesPerSec)	0	超過閾值後 fallback 流量的速度上限（位元組/秒）。0 表示不限速（停用此方向）。
Burst Bytes Per Sec (burstBytesPerSec)	0	超過穩定速度上限時允許的短暫突發流量（token-bucket 大小）。若小於 Bytes Per Sec，將自動提升至與其相同的值。

同一位置還新增了 **Master Key Log**（masterKeyLog）欄位——以 SSLKEYLOGFILE 格式記錄 TLS master key 路徑，供 Wireshark 偵錯使用；生產環境請留空。

7.5. 設定實務建議

1. **VLESS + Reality (建議)**：建立 `tcp` 網路上的 VLESS inbound，在「安全性」標籤頁選擇 **Reality**——面板會自動填入隨機 `target/SNI`、`shortIds` 並生成 X25519 金鑰。如需使用自己的金鑰對，請點擊「取得新憑證」。為 VLESS 客戶端啟用 **Flow** = `xtls-rprx-vision` (XTLS Vision) ——這將帶來最佳效能與隱蔽性。

範例：VLESS + Reality + Vision 的最終客戶端連結。 以下是面板為此類 inbound 生成的邀請連結外觀（金鑰/ID 值僅供示意）：

```
vless://uuid-клиента@1.2.3.4:443?
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

其中 `pbk` 為 X25519 公開金鑰，`sni` 為 **目標** 中借用的域名，`sid` 為其中一個 **Short IDs**，`flow=xtls-rprx-vision` 表示已啟用 XTLS Vision。

2. **使用自有域名的 TLS**：選擇 **TLS**，在 **SNI** 中填入域名，新增憑證（檔案路徑或內容），或若域名與憑證已在「設定 → 安全性」中設定，則點擊「設定面板憑證」。保持 **最低/最高版本** = `1.2 - 1.3` 及 **uTLS** = `chrome` 以偽裝成一般瀏覽器。
3. 不要對外開放的 inbound 使用 **無** 模式——僅將其用於本地 fallback 目標（`127.0.0.1`），或由外部代理提供 TLS 時使用。
4. 介面中的提示：大多數進階欄位都有「建議保持預設設定」的提示——在充分了解後果前請勿修改。

8. 客戶端

客戶端是 VPN 用戶的帳號：一組認證憑證（UUID 或密碼），綁定至一個或多個 inbound，並擁有各自的流量配額、有效期限及同時連線數限制。在本分支中，客戶端是獨立的實體（`clients` 資料表）：同一個客戶端可同時綁定多個 inbound，共用相同的 UUID / 密碼及流量計數器。**客戶端**區段顯示面板中所有帳號，不區分 inbound，並提供搜尋、篩選、排序及批量操作功能。

8.1. 客戶端欄位

以下逐一說明**新增客戶端** / **編輯客戶端**編輯器中的每個欄位。

客戶端表單分為兩個分頁：**基本**（email、inbound 綁定、限制、期限、群組、備註、反向標籤）和**認證憑證**（UUID / 密碼 / auth、Flow、VMess Security）。欄位標籤中，配額顯示為**流量限制 (GB)**，期限顯示為**有效天數和自動續期 (天)**；**流量限制 (GB)** 和 **IP 限制**欄位均有提示說明 `0` 代表「不限制」。編輯現有客戶端時，隨機 email 產生按鈕會隱藏，而 IP 日誌按鈕則直接顯示於 **IP 限制**欄位旁，並顯示已記錄的位址數量。

欄位	JSON 鍵	預設值	說明
Email	<code>email</code>	—（必填）	客戶端的唯一識別碼
UUID	<code>id</code>	自動產生	VMess/VLESS 的識別碼
密碼	<code>password</code>	自動產生	Trojan/Shadowsocks 的密碼
授權	<code>auth</code>	自動產生	Hysteria 的密碼
Flow	<code>flow</code>	空	Flow control (XTLS)，僅限 VLESS
VMess Security	<code>security</code>	<code>auto</code>	VMess 加密方式
IP 限制	<code>limitIp</code>	<code>0</code> （不限制）	同時在線的 IP 上限
總上傳 / 下載 (GB)	<code>totalGB</code>	<code>0</code> （不限制）	流量配額
到期時間	<code>expiryTime</code>	<code>0</code> （永不過期）	帳號到期日
自動續期	<code>reset</code>	<code>0</code> （停用）	流量重置週期，單位天
Telegram 使用者 ID	<code>tgId</code>	<code>0</code> （無）	Telegram 數字 ID
訂閱 ID	<code>subId</code>	自動產生	訂閱識別碼
群組	<code>group</code>	空	邏輯分組標籤
備註	<code>comment</code>	空	管理員任意備註
已啟用	<code>enable</code>	<code>true</code>	帳號是否啟用

Email（識別碼）

Email 欄位是客戶端的主要必填識別碼。儘管名稱如此，它不一定是電子郵件地址：任何文字標籤（使用者名稱、編號）皆可。該值在整個面板中必須**唯一**；嘗試建立 email 已被使用的第二個客戶端會被拒絕（`email already in use`），除非 `subId` 也相同（此情況視為綁定同一客戶端）。

Email 不能為空（client email is required），且不能包含空格、/、\ 及控制字元（「Email 不能包含空格、/、\ 或控制字元」）。Email 用於流量統計、IP 日誌、在線清單及操作名稱——不建議事後更改。

UUID / 密碼 / 授權（認證憑證）

具體的認證憑證欄位取決於客戶端所綁定的 inbound 協議。若欄位留空，則自動填入預設值：

- **UUID**（id 欄位）——用於 **VMess** 和 **VLESS** 協議。若未指定，則自動產生隨機 UUID v4。
- **密碼**（password 欄位）——用於 **Trojan** 和 **Shadowsocks**。Trojan 預設產生不含連字符的 UUID。Shadowsocks 會依據 inbound 的加密方式產生 Base64 格式的對應長度金鑰：2022-blake3-aes-128-gcm 為 16 位元組，2022-blake3-aes-256-gcm 和 2022-blake3-chacha20-poly1305 為 32 位元組；其餘加密方式則使用不含連字符的 UUID。若手動輸入的金鑰不符合 2022-blake3 的要求，將被替換為自動產生的金鑰。
- **授權**（auth 欄位）——**Hysteria** 的密碼。預設為不含連字符的 UUID。

由於一個客戶端可以綁定至不同協議的 inbound，客戶端記錄中可能同時存在 UUID、密碼和 auth——每種協議各使用其對應的欄位。

範例：客戶端認證憑證在不同 inbound 的 settings 中的呈現方式。 同一客戶端在 VLESS-inbound 中以 id 識別，在 Trojan 中以 password 識別，在 Shadowsocks 中以 password（Base64 金鑰）識別：

```
// [?][?][?]VLESS-inbound [?] settings.clients
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// [?][?][?][?][?] Trojan-inbound
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// [?][?][?][?][?] Shadowsocks-inbound ([?][?][?][?] 2022-blake3-aes-256-gcm)
{ "password": "GPy0aA3f7C00az53eaQ8eqMfRDjmBl0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow（flow 欄位）——XTLS 流量控制。僅適用於 **VLESS**，且只有當 inbound 設定為 XTLS Vision 時才有效：即 VLESS 使用 **TCP** 傳輸且 security 為 **tls** 或 **reality**。有效值為 xtls-rprx-vision（以及歷史值 xtls-rprx-vision-udp443）；空值表示不使用 flow。

若 inbound 不支援 XTLS-flow，設定的 flow 在儲存客戶端時會自動清空：對於綁定多個 inbound 的同一客戶端，flow 僅在支援的 inbound 上生效。只有在明確使用 VLESS-Vision 時才需要修改此欄位。

VMess Security

VMess Security（security 欄位）——VMess 負載的加密方式。預設值為 auto（由 Xray 自動選擇加密算法）。有效值為 VMess 標準值：auto、aes-128-gcm、chacha20-poly1305、none、zero。其他協議不使用此欄位。

IP 限制（同時連線數）

IP 限制（`limitIp` 欄位）——客戶端可同時連線的不同 **IP 位址** 最大數量。預設值為 `0`，表示**不限制**。設定正數值後，面板會追蹤客戶端的活躍 IP，超出限制時，背景任務會停用該帳號。（從 **3.3.1** 版起，IP 計數通過 Xray 核心的 `online-stats` API 進行，**無需**存取日誌；舊版核心時，面板會回退至讀取存取日誌，此時必須啟用存取日誌。）此欄位可用於防止同一訂閱在多裝置上共用：例如 `2` 代表允許兩台裝置。

IP 限制通過 **Fail2ban** 機制實現，因此 **IP 限制** 欄位僅在 Fail2ban 已安裝且正常運作時有效（面板通過 `GET /panel/api/server/fail2banStatus` 檢查其狀態）。若未安裝 Fail2ban，客戶端編輯器（及批量新增表單）中的欄位將被禁用，游標懸停時顯示提示，建議從 `x-ui` `bash` 選單安裝 Fail2ban（「Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option.」）；在 Windows 上提示顯示 Fail2ban 不可用（「Fail2ban is not available on Windows, so the IP limit cannot be enforced.」），若伺服器上此功能已停用則顯示「The IP limit feature is disabled on this server.」。面板更新時，無 Fail2ban 伺服器上客戶端已儲存的 IP 限制值會通過一次性遷移歸零，因為該值本來就未生效。

範例值。 `limitIp: 0` ——不限制；`limitIp: 1` ——同時只允許一台裝置；`limitIp: 3` ——最多三個不同 IP。當第四個 IP 活躍時，背景任務會停用客戶端（`enable = false`），直到您執行**重置 IP 限制**。

相關操作：**IP 日誌**顯示客戶端已記錄的 IP 清單；每筆記錄除 IP 本身外，還包含最後存取時間及節點標籤（@ [?] [?] [?] [?]），顯示連線所通過的節點——在多面板設定中可看出客戶端通過哪個節點連線。**重置 IP 限制**清除已累積的 IP 日誌，使客戶端無需等待記錄自然過期即可重新連線。

總上傳 / 下載 (GB)——流量配額

總上傳 / 下載 (GB)（`totalGB` 欄位）——總流量配額（上傳 + 下載）。預設值 `0` 表示**不限制**。達到配額（`up + down >= total`）後，客戶端被視為**已耗盡**（`depleted`）並停用。UI 中通常以 GB 輸入；資料庫中以位元組儲存。

客戶端清單中的**流量**欄位顯示彩色使用條：已使用流量、限制標籤（或無限制時的 ∞ 符號），以及游標懸停時的詳細提示，顯示上傳 / 下載分別的用量及剩餘量。手機端的客戶端卡片也顯示相同的緊湊指示器。

到期時間（Expiry）

到期時間（`expiryTime` 欄位）設定帳號的到期時間點。此欄位有三種模式：

- **永不過期**——`0`。客戶端永遠不會因時間而過期。
- **具體日期**——正的 Unix 時間戳（毫秒）。到期（`expiryTime <= [?] [?]`）後，客戶端被視為已過期（`expired`）並停用。UI 中通常通過選擇日期或以天數指定**有效天數**來設定。
- **首次使用後開始計時**——**負值**，用於編碼有效期。在客戶端未傳輸任何位元組之前，到期時間保持為負值（「延遲啟動」）。在第一次流量統計時，面板將其轉換為絕對日期：`[?] [?] + |[?] [?] [?] [?]|`。這允許銷售「從首次連線起 30 天」之類的訂閱，無需事先知道客戶端何時激活。轉換按 email 只執行一次，使所有綁定的 inbound 獲得相同的到期時間。

到期時間編碼範例。 固定日期 2026 年 3 月 1 日 00:00 UTC → `expiryTime: 1772323200000`（毫秒正時間戳）。「首次連線後 30 天」→ `expiryTime: -2592000000`（負值， $30 \times 24 \times 60 \times 60 \times 1000$ ）；在首次流量傳輸時面板將其替換為 `[?] [?] + 2592000000`。永不過期 → `expiryTime: 0`。

自動續期（客戶端流量重置週期）

自動續期欄位（`reset` 欄位）——自動續期 / 重置的週期（天）。提示：「到期後自動續期。（0 = 停用）（單位：天）」。

- 0 ——自動續期**停用**（預設值）。到期後客戶端直接變為已耗盡狀態。
- > 0 ——背景任務在到期時將**流量計數器歸零**（`up = down = 0`），將到期時間向後推移 `reset` 天（如有需要，可推移多個週期，直到新到期時間在未來），並在必要時重新啟用客戶端。這實現了定期訂閱（例如按月計費）。自動續期**不適用於節點上的 inbound**（`node_id IS NOT NULL`）。

重要結論：`reset > 0` 的客戶端在批量刪除操作中被排除在「已耗盡」的概念之外——其流量 / 到期時間預期由自動續期重置，而非將帳號標記為刪除候選。

Telegram 使用者 ID

Telegram 使用者 ID（`tgId` 欄位）——用於綁定面板內建 Telegram 機器人（通知、自助查看統計）的 Telegram 數字識別碼。提示：「Telegram 使用者的數字 ID（0 = 無）」。預設值 0 表示無綁定。此欄位支援篩選（有 / 無）。

訂閱 ID（subId）

訂閱 ID（`subId` 欄位）——客戶端包含在訂閱（subscription）中的識別碼。所有具有相同 `subId` 的客戶端通過同一訂閱連結提供。若建立時欄位留空，面板會自動產生隨機 `subId`（UUID）。該值在其他 email 的客戶端中必須**唯一**（`subId already in use`），並遵循與 email 相同的字元限制（「訂閱 ID 不能包含空格、'/'、'\" 或控制字元」）。

若無 `subId`，客戶端的訂閱連結不可用（「此客戶端沒有 `subId`，共享連結不可用。」）。

Links 分頁（外部連結與訂閱）

除**基本**和**認證憑證**分頁外，客戶端編輯器中還有第三個 **Links** 分頁（提示：「Add third-party share links and remote subscription URLs to include in this client's subscription.」）。在此分頁中，可通過 **Add External Link** 按鈕新增第三方分享連結（`vless://`、`vmess://`、`trojan://`、`ss://`、`hysteria2://`、`wireguard://`），並通過 **Add External Subscription** 按鈕新增遠端訂閱的 URL（例如 `https://provider.example/sub/...`）。

所有這些內容都會合併到該客戶端的訂閱輸出中（raw、JSON 和 Clash 格式）：連結原樣新增，遠端訂閱則由面板定期下載（帶快取和短逾時）並將其設定與本地設定合併。這樣在一個客戶端訂閱連結中，可以同時提供自有伺服器 and 外部設定。

群組

群組（`group` 欄位）——用於關聯相關客戶端的邏輯標籤。提示：「用於分組相關客戶端的邏輯標籤（例如團隊、客戶、地區）。可從工具列篩選。」，佔位符為「例如 customer-a」。此欄位為選填（預設為空）。可按群組篩選清單並執行批量操作；使用**取消分組**操作可移除客戶端的群組標籤。

也可直接在單一客戶端編輯器中移除群組：清空**群組**欄位並儲存後，標籤會正確移除，客戶端不再顯示於原群組下。

備註

備註（`comment` 欄位）——供管理員使用的任意文字備注（預設為空）。內容包含在搜尋範圍內，並可篩選（有 / 無備註）。

已啟用

已啟用（`enable` 欄位）——帳號的啟用標誌。預設為**已啟用**（`true`）；建立時即使未傳入此標誌，面板也會強制設為 `true`。已停用的客戶端（`enable = false`）無法連線，在統計中歸為**未啟用**（`deactive`）類別。面板會自動停用流量配額耗盡、已過期或超出 IP 限制的客戶端。

唯讀欄位

客戶端卡片中還顯示服務欄位：**建立時間**（`created_at`）和**更新時間**（`updated_at`）——建立和最後修改的時間戳，自動填入且不可編輯。**反向標籤**（`reverse`）欄位——VLESS 簡單反向代理的選填 Reverse tag（「選填 Reverse tag」）。

8.2. 綁定至 inbound

每個客戶端必須至少綁定一個 inbound——建立時至少需要一個（`at least one inbound is required`）。在編輯器中，此欄位為**已綁定的 inbound**，提示為**選擇一個或多個 inbound**。

- **綁定**——將客戶端新增至選定的 inbound（共用相同的 UUID / 密碼和流量統計）。現有綁定保留不變。
- **解綁**——從選定的 inbound 中移除客戶端。客戶端記錄本身保留（如需完全刪除，請使用**刪除**）。未曾綁定的配對會靜默跳過。

儲存綁定多個 inbound 的客戶端時，與特定協議 / 傳輸不相容的欄位（例如非 VLESS-Vision 的 Flow）會自動調整為每個 inbound 允許的值。

在 inbound 選擇清單上方（客戶端表單、批量新增客戶端視窗，以及批量綁定 / 解綁視窗中）有**全選**和**清除**按鈕。這些清單中，每個 inbound 以其備註（`remark`）為標籤（若已設定），否則使用 inbound 的 tag。

8.3. 客戶端操作

針對單一客戶端（通過**客戶端資訊**卡片或**操作**右鍵選單）可執行：

查看資訊、QR 碼和連結

- **客戶端資訊**——顯示所有欄位、已使用 / 剩餘流量（**剩餘**）、到期時間及綁定的 inbound 的卡片。自 3.5.0 起，若客戶端已設定群組，卡片還會顯示其**群組**（帶標籤的「群組」列，位於「備註」上方）。

通過 API (GET /panel/api/clients/get/:email) 查詢客戶端時，除 `client` 和 `inboundIds` 欄位外，還會返回 `usedTraffic` —— 實際已使用的流量（上傳 + 下載，包含節點數據），方便將實際用量與 `totalGB` 配額進行對比。

- **QR 碼和連結** —— 供客戶端應用程式匯入的設定連結。根據所有綁定的受支援協議 inbound 產生 (GET /links/:email)。若無合適的連結：「沒有可分享的連結——請先將客戶端綁定到支援的協議 inbound。」
- **訂閱連結** —— 通過 `subId` 的訂閱 URL (GET /subLinks/:subId)。僅在客戶端有 `subId` 且面板設定 → 訂閱中已啟用訂閱服務時可用（否則顯示「訂閱服務已停用。」）。另外還提供 **JSON 訂閱 URL**。

重置流量

重置流量 (POST /resetTraffic/:email) 將特定客戶端的上傳 / 下載計數器 (`up`、`down`) 歸零。配額 (`totalGB`) 和到期時間 **不受影響** —— 僅重置已使用的量。提示訊息：「流量已重置」。若客戶端未綁定任何 inbound：「請先將此客戶端綁定到 inbound。」

重置流量 按鈕也可在客戶端編輯表單中使用 —— 位於底部面板的 **取消 / 儲存** 旁（執行前會請求確認）。若客戶端因流量耗盡而被停用，重置（單個或批量）會自動重新 **啟用** 該客戶端 (`enable = true`)，並立即將此變更同步至節點 —— 無需在主面板和各節點上手動重新啟用客戶端。

重置 IP 限制

清除客戶端已累積的 IP 日誌 (POST /clearIps/:email)，以解除因超出同時連線數限制而導致的臨時封鎖。提示訊息：「日誌已清除」。

刪除

刪除 (POST /del/:email) —— 永久刪除客戶端。確認對話框：標題為「刪除客戶端 {email}？」，文字為「此客戶端將從所有已綁定的 inbound 中移除，其流量記錄將被銷毀。此操作無法撤銷。」。刪除會將客戶端從 **所有** inbound 解綁並銷毀其流量記錄。提示訊息：「客戶端已刪除」。

8.4. 批量操作

在客戶端清單中可勾選多筆記錄 (**全選**、**全不選**)；計數器顯示「已選擇 {count} 個」。對選定記錄可執行：

- **刪除 ({count})** (POST /bulkDel) —— 批量刪除。確認：「刪除 {count} 個客戶端？」，「每個選定的客戶端將從所有已綁定的 inbound 中移除，其流量記錄將被銷毀。此操作無法撤銷。」。提示訊息：「已刪除客戶端：{count}」，部分失敗時：「已刪除：{ok}，失敗：{failed}」。
- **修改 ({count}) / 調整** (POST /bulkAdjust) —— 批量修改到期時間和 / 或配額。對話框「修改 {count} 個客戶端」，提示：「正值為增加，負值為減少。無限到期時間或無限流量的客戶端在對應欄位將被跳過。」。欄位：**新增天數**、**新增流量 (GB)** 和 **Set flow**。邏輯：
 - **到期時間**：永不過期 (`expiryTime == 0`) 的客戶端會被跳過 (「unlimited expiry」)；有到期日的客戶端到期時間按指定天數推移；「首次使用後」模式 (負值到期時間) 的客戶端，等待有效期會被調整。超出剩餘時間的減少會被跳過 (「reduction exceeds remaining time/delay window」)。

- **流量**：無限制（totalGB == 0）的客戶端會被跳過（「unlimited traffic」）；否則配額按指定量調整，不低於零。
- **Flow**：**Set flow** 下拉選單允許一次為所有選定客戶端設定或清除 XTLS flow。預設選擇 **No change**（不變更）。**Disable (clear flow)** 選項可清除 flow，而 xtls-rprx-vision 和 xtls-rprx-vision-udp443 值可設定對應的 vision-flow。Vision-flow 的設定僅對支援 flow 的 inbound 生效；不支援的 inbound 保持不變並標記為已跳過，而清除 flow 則始終允許。
- **自動啟用（自 3.4.2 起）**：僅因耗盡（已過期或超出配額）而被停用的客戶端，在延長時會自動重新啟用——本機與其節點上皆然——前提是調整後使其回到限制範圍內。手動停用或仍處於耗盡狀態的客戶端維持停用（enable 欄位現在會明確寫入，因此即使沒有 inbound 的客戶端也能保留狀態）。
- 若未指定天數、流量或 flow：「請在套用前指定天數、流量或 flow。」。提示訊息：「已修改：{count}」 / 「已修改：{ok}，已跳過：{skipped}」。

範例：將選定客戶端延長 30 天並新增 50 GB。在修改對話框中設定新增天數 = 30，新增流量 (GB) = 50。若要反向扣除一週並削減 10 GB 配額，請輸入負值：新增天數 = -7，新增流量 (GB) = -10（無限到期時間或無限流量的客戶端在對應欄位將被跳過）。

- **綁定 ({count}) / 解綁 ({count})** (POST /bulkAttach / bulkDetach) —— 批量將選定客戶端綁定至或從選定 inbound 解綁。目標僅限多使用者 inbound。解綁結果：「已解綁 {detached}，已跳過 {skipped}。」自 3.5.0 起，客戶端表單中的 inbound 選擇器會隱藏已停用的 inbound（客戶端已綁定者除外），且「解綁」視窗中同一客戶端記錄重複累積的問題已修正（資料會在面板啟動時自動修復）。
- **訂閱連結 ({count})** —— 顯示選定客戶端的訂閱 URL 和 JSON 訂閱 URL 彙總表，並提供複製全部按鈕。若無任何客戶端有 subId：「所選客戶端均無訂閱 ID。」
- **加入群組和取消分組** —— 指定和移除群組標籤。
- **啟用 ({count}) / 停用 ({count})** (POST /bulkEnable / bulkDisable) —— 批量啟用和停用選定客戶端。**啟用**會在所有已綁定的 inbound 上激活每個選定的客戶端；流量配額已耗盡或已過期的客戶端會被自動重新停用。**停用**會立即撤銷客戶端的存取權，但其記錄和已累積的流量保留不變。執行前面板會請求確認，操作完成後顯示已處理客戶端數量的通知，以及操作失敗的數量（如有）。

按狀態重置流量和刪除

- **重置所有客戶端流量** (POST /resetAllTraffics) —— 將面板所有客戶端的 up / down 計數器歸零。確認：「重置所有客戶端的流量？」及「所有客戶端的上傳 / 下載計數器將歸零。配額和到期時間不受影響。此操作無法撤銷。」。提示訊息：「所有客戶端的流量已重置」。
- **刪除已耗盡** (POST /delDepleted) —— 刪除所有配額已耗盡（total > 0 and up + down >= total）或已過期（expiry_time > 0 and expiry_time <= 30）且 reset = 0 的客戶端（有自動續期的客戶端不受影響）。確認：「刪除已耗盡的客戶端？」，「將刪除所有流量配額已耗盡或已過期的客戶端。此操作無法撤銷。」。提示訊息：「已刪除已耗盡的客戶端：{count}」。

匯出、匯入和刪除未綁定客戶端

當沒有選取任何記錄時，客戶端頁面上的更多選單提供三個操作。

匯出客戶端（GET /clients/export）開啟檢視器，顯示 {client, inboundIds} 格式的所有客戶端 JSON 清單，並提供複製和下載按鈕（檔案名 clients-export.json）。**匯入客戶端**（POST /clients/import）開啟編輯器，貼入相同格式的 JSON 後點擊 **Import**：帶 inboundIds 的客戶端會被建立並綁定至對應 inbound，無綁定的客戶端會作為獨立的「裸」記錄恢復，而已存在的 email **永遠不會被覆蓋**——它們會出現在跳過清單中。提示訊息：「{count} clients imported」，「{ok} imported, {failed} skipped」。

刪除未綁定客戶端（POST /clients/delOrphans）——危險操作：刪除所有未綁定任何 inbound 的客戶端，連同其流量記錄、IP 日誌和外部連結。確認：「Delete clients without an inbound?」，「Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.」。提示訊息：「{count} unattached clients deleted」。此操作不可撤銷。

8.5. 搜尋、篩選與排序

清單上方有搜尋框（「搜尋 email、備註、sub ID、UUID、密碼、auth、Telegram ID...」）——搜尋範圍涵蓋 email、備註、subId、UUID、密碼、auth，以及（自 3.5.0 起再次支援）**Telegram ID**。結果計數：「顯示 {shown} / {total}」。

客戶端清單會自動更新：面板每隔幾秒拉取當前頁面的最新數據，因此新連線的客戶端和排序順序的變化無需手動重新整理即可呈現（背景輪詢時不顯示閃爍的載入指示器）。

自 3.5.0 起，表格新增**「速度」**欄位（位於「流量」與「剩餘」之間）：顯示客戶端的即時速度——藍色標記 ↑ [?] / ↓ [?]（約 5 秒滑動平均，每 5 秒更新一次）；無流量時顯示灰色破折號。在行動裝置上，速度以單獨一行顯示在客戶端卡片中。

客戶端篩選面板可按狀態（類別）、協議、已綁定的 inbound、到期時間範圍、已使用流量範圍、自動續期有無（有/無）、Telegram ID 有無、備註有無，以及群組進行篩選。在帶有節點的面板中，會出現節點多選框：可將清單限制為選定節點的客戶端；**本地面板**選項可篩選未綁定節點的 inbound 客戶端（僅在有節點時顯示此篩選器）。排序方式：**從舊到新 / 從新到舊、最近更新、最近在線、Email A→Z / Z→A、流量較多、剩餘較多、即將到期**。

8.6. 狀態標誌與狀態

狀態優先順序：已耗盡 / 已過期 → 未啟用 → 即將到期 → 活躍。

- **在線 / 離線**——客戶端有活躍連線（在當前在線清單中）且**已啟用**。在線清單通過單獨的請求更新（/onlines、/onlinesByGuid）。
- **已耗盡**（depleted）——配額已用盡（up + down >= totalGB）或已過期（expiryTime <= [?]）。此類客戶端會自動停用，並納入**刪除已耗盡**操作的範圍。
- **即將到期 / 耗盡**（expiring）——已啟用的客戶端，距到期時間不足閾值時間間隔，或距配額耗盡不足閾值用量（閾值在面板設定中設定）。若客戶端已耗盡或已停用，則不計為此狀態。
- **未啟用**（deactive）——enable = false 的客戶端（手動停用或由背景任務停用）。
- **活躍**（active）——已啟用、未耗盡、未過期，且距各閾值仍有餘裕。

9. 客戶端群組

這是本 3X-UI 分支的功能。在原始的 3x-ui 專案 (MHSanaei) 中並沒有「客戶端群組」的概念——這裡新增了獨立的群組資料表、面板選單中的 **Группы** (群組) 頁面以及對應的 API 方法。如果您將設定遷移到原始的 3x-ui，群組標籤就不會在任何地方被納入考量。

9.1. 什麼是客戶端群組，以及為何需要它

群組是一個具名的邏輯標籤 (label)，可以掛到一個或多個客戶端上。它不會建立新的連線方式，也既不是 inbound、也不是節點——它純粹是一個組織用的標記，藉此可以方便地對客戶端進行篩選與批次處理。

本分支客戶端模型核心理念：**客戶端是頂層實體，以 email 來識別** (clients 資料表中的 email 欄位帶有唯一索引)。同一個客戶端 (同一個 email、同一組憑證) 可以同時隸屬於多個 inbound，甚至分布在多個節點上，並且可以使用不同的協定。群組標籤**每個客戶端只儲存一次**，因此它會自動套用到該客戶端的所有 inbound 綁定上。

群組標籤是用於分組的邏輯標籤：

層級	儲存位置	欄位
客戶端記錄 (資料庫)	clients 資料表	group_name (預設為空字串 '')
群組目錄 (資料庫)	client_groups 資料表	name (唯一索引，不可為空)
inbound 設定 (Xray)	JSON settings.clients[].group	在該客戶端所屬的每個 inbound 的每個客戶端物件中重複出現

實務上為何需要它：

- **一個客戶端橫跨多個 inbound/節點**。如果某個客戶端被「售出」為同時存取多個 inbound 的權限 (例如不同協定或不同節點)，群組可讓您將其視為一個整體來管理：重設流量、刪除、重新命名標籤——只需對其所有 inbound 執行一次操作即可。
- **批次操作與篩選**。在 **Клиенты** (客戶端) 頁面，清單可依群組篩選；在 **Группы** (群組) 頁面，可對群組所有成員執行批次動作。
- **管理大量客戶端**。諸如 vip、trial、team-A 之類的標籤有助於將成千上萬的客戶端歸入邏輯分類中，而不必為此衍生出獨立的 inbound。

9.2. 群組與客戶端、inbound、節點及協定的關聯

這是對理解至關重要的一節，因為標籤的同步並不簡單。

群組描述的是客戶端，而非 inbound。標籤存活於客戶端記錄中 (clients.group_name)。當客戶端隸屬於多個 inbound 時，每次群組變更面板都會遍歷該客戶端所屬的所有 inbound，並在它們的 Xray 設定

(`settings.clients[]`) 內設定/移除 `group` 欄位。技術上的做法是：依客戶端的 email 找出其所屬的所有 inbound，然後在每個這類 inbound 的 JSON 設定中，修改帶有該 email 的客戶端物件。因此：

- **群組與協定無關**。同一個 email 可以在某個 inbound 中是 VLESS 客戶端，在另一個 inbound 中是 Hysteria 客戶端——它的群組標籤仍然只有一個，並會套用到兩者（同時每個協定有各自的憑證，分別獨立儲存）。
- **群組橫跨節點**。隸屬於節點的 inbound 與主面板的 inbound 之間，差別在於 `nodeId` 欄位（主面板 inbound 的該欄位為 `null/0`）。群組標籤會套用到 inbound 中的客戶端物件，無論該 inbound 是主面板的還是節點的——只要該 email 的客戶端存在於其中即可。

群組標籤對來自節點的同步以及設定重建具有持續性。這是刻意設計的行為：

- 當節點送來流量快照時，其資料**不會**覆寫面板資料庫中客戶端的本地 `group_name` 與 `comment`。群組與註解被視為「面板本地」欄位——節點不負責管理它們。
- 在重建 inbound 設定時，傳入資料中的空 `group` 值**不會**重設已儲存的標籤。群組正是透過 **Группы**（群組）頁面來管理（而非透過編輯 inbound 的 Xray 設定），因此在一般的設定重建中，「空群組」被解讀為「不要動」，而非「清空」。

實務結論：唯一會**刻意清除**標籤的操作，是刪除群組以及明確地將客戶端從群組中移除（見下文）。一般的 inbound 編輯或與節點的背景同步都不會遺失群組。

9.3. 群組目錄與「空」群組

頁面上的群組清單由兩個來源合併而成：

1. **衍生群組（derived）**——所有實際出現在客戶端身上的非空 `group_name` 值，並計算客戶端數量。
2. **已儲存群組（stored）**——`client_groups` 資料表中的記錄。

這種合併帶來一個重要效果：群組可以在**沒有任何客戶端**的情況下存在。這類群組是透過明確的「Добавить группу」（新增群組）按鈕建立的（在 `client_groups` 中產生一筆記錄），並在清單中以計數 **0** 顯示。這些記錄就被視為**空群組**。清單始終以不分大小寫的名稱排序。

頁面上的彙總計數：

欄位 (RU)	顯示內容
Всего групп	群組總數（已儲存的與衍生的合計）。
Клиенты с группой	有多少客戶端帶有非空群組標籤。
Пустые группы	有多少群組是沒有客戶端的（計數為 0 ）。
Клиентов в группе	特定群組中的客戶端數量（資料表的一欄）。

9.4. 群組的欄位與資料表欄位

`client_groups` 資料表中的群組記錄包含：

欄位	類型	預設值	說明
<code>Id</code>	int	自動遞增	群組記錄的主鍵。
<code>Name</code>	string	–（必填）	群組名稱。唯一索引，不可為空。在 UI 中—Имя（名稱）欄。
<code>CreatedAt</code>	int64（毫秒）	建立時間	群組記錄的建立時刻。
<code>UpdatedAt</code>	int64（毫秒）	修改時間	最後一次修改的時刻。

頁面上的資料表至少會顯示 **Имя**（名稱）與 **Клиентов в группе**（群組中的客戶端數）兩欄，以及操作按鈕（見下文）。

9.5. 建立群組

按鈕 **Добавить группу**（新增群組）。

步驟：

1. 點擊 **Добавить группу**。
2. 輸入群組名稱。
3. 確認。

後端行為（`POST /panel/api/clients/groups/create`，請求主體 `{"name": "..."}` ）：

- 名稱會去除前後空白。空名稱會被拒絕，回傳錯誤「group name is required」。
- 如果已存在同名群組—錯誤「group already exists」。
- 成功時會在 `client_groups` 中建立一筆記錄（初始沒有任何客戶端—即空群組）。

成功訊息：「群組「`{name}`」已建立。」

範例：透過 API 建立空群組。在尚未填入客戶端之前就先準備好一組標籤：

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"name": "vip"}
```

成功時的回應：

```
{ "success": true, "msg": "Группа «vip» создана.", "obj": null }
```

以相同名稱再次呼叫會回傳 `"success": false` 以及訊息 `group already exists`。

事先建立空群組在您想要先準備好一組標籤、之後再透過「Добавить клиентов...」（新增客戶端...）將客戶端填入時很方便。

9.6. 重新命名群組

按鈕 **Переименовать**（重新命名），對話框標題為「**Переименовать {name}**」。

行為（POST /panel/api/clients/groups/rename，請求主體 `{"oldName": "...", "newName": "..."}` ）：

- 兩個名稱都會去除空白。空的舊名稱——錯誤「old group name is required」，空的新名稱——「new group name is required」。
- 如果新名稱與舊名稱相同——不做任何事（影響 0 個客戶端）。
- 否則會以原子方式執行重新命名：
 - `client_groups` 中的記錄被重新命名；
 - 所有 `group_name = oldName` 的客戶端，該欄位都被更新為 `newName`；
 - 在受影響客戶端所屬的所有 **inbound**（包含節點的）中，Xray 設定裡的 `group` 值由舊名改為新名。
- 重新命名後，面板會將 Xray 標記為需要重新啟動，並發送客戶端變更的通知。

訊息：

- 成功：「已為 {count} 個客戶端重新命名群組。」
- UI 中的名稱衝突：「名為「{name}」的群組已存在。」

9.7. 將客戶端新增到群組

按鈕 **Добавить клиентов...**（新增客戶端...），標題為「**將客戶端新增到群組「{name}」**」。

對話框中的逐字提示：

「選擇要新增到此群組的客戶端。現有的 inbound 綁定會保留；只會變更群組標籤。已在此群組中的客戶端不會顯示。」

如果沒有可新增的對象，會顯示「沒有其他可新增的客戶端。」

行為（POST /panel/api/clients/groups/bulkAdd，請求主體 `{"emails": [...], "group": "..."}` ）：

- 群組名稱為必填（否則錯誤「group name is required」）；空的 email 清單——操作不做任何事。
- 如果該群組在 `client_groups` 中和衍生群組中都尚未存在——它會被自動建立。
- 對所選 email 的客戶端設定 `group_name = group`；客戶端與 **inbound** 的綁定不變——只影響標籤。接著在這些客戶端的所有 inbound 中設定 `group` 欄位。
- 回傳受影響的客戶端記錄數量；Xray 被標記為需重新啟動。

成功訊息：「已將 {count} 個客戶端新增到 {name}。」

範例：以單一請求為多個客戶端標記群組。客戶端以 email 指定，而與 inbound 的綁定不會因此變更：


```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

如果群組 `vip` 尚不存在，它會被自動建立。請求之後，這些客戶端的記錄中會設定 `group_name = "vip"`，而其每個 inbound 的 Xray 設定中，客戶端物件會得到 `"group": "vip"` 欄位：

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. 將客戶端從群組移除（不刪除客戶端本身）

按鈕 **Удалить клиентов...**（移除客戶端...），標題為「將客戶端從群組「`{name}`」移除」。

逐字提示：

「選擇要從此群組移除的成員。客戶端本身會保留（若要完全刪除，請使用「Удалить клиентов группы」（刪除群組客戶端））。」

行為（`POST /panel/api/clients/groups/bulkRemove`，請求主體 `{"emails": [...]}`）：技術上這與「以空群組進行新增到群組」相同。所選客戶端的 `group_name` 被清空，而其 inbound 中的 `group` 欄位會從 Xray 設定中移除。客戶端本身及其與 inbound 的綁定保持不變。

成功訊息：「已將 `{count}` 個客戶端從 `{name}` 移除。」

9.9. 重設群組流量

按鈕 **Сбросить трафик**（重設流量）。

確認對話框：

- 標題：「重設群組 `{name}` 的流量？」
- 內文：「僅重設群組的流量計數器。個別客戶端的計數器不受影響。」

行為：僅將群組本身顯示的計數器歸零——面板會記住群組當前的 `up/down` 總和作為基準點，之後在群組清單中顯示 `max(0, [?] - [?])`。個別客戶端的 `up/down` 計數器、配額與 `enable` 欄位不會變更，也無需重啟 Xray。此為相對於先前版本的行為變更，先前重設會將群組中所有客戶端的流量歸零。

請求：`POST /panel/api/clients/groups/resetTraffic`，主體為 `{"name": "<[?][?][?]>"}`。

成功訊息：「群組 `{name}` 的流量已重設。」

9.10. 刪除群組與刪除群組客戶端

頁面上有兩種本質上截然不同的刪除操作——它們很容易混淆，因此區別至關重要。

9.10.1. 刪除群組（保留客戶端）

按鈕「Удалить группу (сохранить клиентов)」（刪除群組（保留客戶端））。

對話框：

- 標題：「刪除群組 {name}？」
- 內文：「這會刪除群組並清除其在 {count} 個客戶端上的標籤。客戶端本身不會被刪除。」

行為（POST /panel/api/clients/groups/delete，請求主體 {"name": "..."}）：群組記錄會從 client_groups 中刪除，其所有客戶端的 group_name 被清空，並從它們的 inbound 中移除 group 欄位。客戶端、其連線與流量都會保留。Xray 被標記為需重新啟動。

成功訊息：「已清除 {count} 個客戶端的群組。」

9.10.2. 刪除群組客戶端（完全刪除）

按鈕「Удалить клиентов группы」（刪除群組客戶端）。

對話框：

- 標題：「刪除 {name} 中的所有客戶端？」
- 內文：「這會連同流量記錄一起不可逆地刪除 {count} 個客戶端。群組標籤也會被清除。此操作無法復原。」

這是破壞性操作：它會刪除客戶端本身（透過依 email 批次刪除，端點 POST /panel/api/clients/bulkDel），包含其流量記錄，並藉此將它們從所有 inbound 中移除。

訊息：

- 成功：「已刪除 {count} 個客戶端。」
- 部分結果：「{ok} 已刪除，{failed} 已跳過」

如果群組是空的，則無法對其成員執行動作——會顯示「此群組目前還沒有客戶端。」

9.11. 與「客戶端」頁面的關聯

群組標籤在 Группы（群組）頁面之外也可見並被使用：

- 客戶端的精簡記錄中有 group 欄位，因此在客戶端清單中會顯示其所屬群組。
- 客戶端清單（/panel/api/clients/list/paged）接受篩選參數 group：可以傳入單一名稱，或以逗號分隔的多個名稱。比對在該欄位內以「或」的原則進行，且不分大小寫。特殊情況：篩選的群組清單中若有空元素，表示「沒有群組的客戶端」（其 group 為空）。
- 在客戶端頁面的回應中會回傳一個 groups 陣列——所有現有群組名稱的完整清單，以便 UI 建立篩選的下拉選單。

範例：依群組篩選客戶端。此請求只會回傳帶有 vip 或 trial 標籤的客戶端（多個名稱——以逗號分隔，「或」）：

```
GET /panel/api/clients/list/paged?group=vip,trial
```

若要取得沒有群組的客戶端，請在清單中傳入一個空元素——例如，篩選值 `group=`（空字串）或 `group=vip`，（`vip` 標籤外加沒有群組的客戶端）。

9.12. API 端點彙總

所有群組路由都掛載在 `/panel/api/clients` 之下：

方法與路徑	用途	請求主體
GET <code>/panel/api/clients/groups</code>	列出群組及客戶端計數	—
GET <code>/panel/api/clients/groups/:name/emails</code>	群組所有成員的 email（依 email 排序）	—
POST <code>/panel/api/clients/groups/create</code>	建立空群組	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	重新命名群組	<code>{"oldName", "newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	刪除群組並保留客戶端（清除標籤）	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	將客戶端新增到群組（依 email）	<code>{"emails": [...], "group"}</code>
POST <code>/panel/api/clients/groups/bulkRemove</code>	將客戶端從群組移除（清除標籤）	<code>{"emails": [...]}</code>
POST <code>/panel/api/clients/bulkDel</code>	完全刪除客戶端（供「刪除群組客戶端」使用）	<code>{"emails": [...], "keepTraffic"}</code>

範例：透過 API 的群組典型生命週期情境。

```
# 1. 3.1.0 trial 3.1.0
curl -s ../panel/api/clients/groups/create -d '{"name": "trial"}'

# 2. 3.1.0 3.1.0 3.1.0 3.1.0
curl -s ../panel/api/clients/groups/bulkAdd -d '{"emails": ["u1@example.com", "u2@example.com"], "group": "trial"}'

# 3. 3.1.0 3.1.0 3.1.0 3.1.0 email 3.1.0 /groups/trial/emails3.1.0
curl -s ../panel/api/clients/groups/bulkRemove -d '{"emails": ["u2@example.com"]}'

# 4. 3.1.0 3.1.0 3.1.0 3.1.0 3.1.0 3.1.0 3.1.0 3.1.0
curl -s ../panel/api/clients/groups/delete -d '{"name": "trial"}'
```

第 4 步會刪除群組記錄並清除其客戶端的 `group_name`，但客戶端本身、其連線與流量都會保留。若要不可逆地刪除客戶端本身，則應改用 `bulkDel`。

會變更客戶端標籤的操作（`rename`、`delete`、`bulkAdd`、`bulkRemove`）會將 Xray 標記為需要重新啟動，並發送客戶端變更的通知。

9.13. 依群組的流量

3.3.0 版本的新功能：在 **Группы**（群組）區段（「客戶端」頁面的群組管理分頁）中，群組資料表現在不僅顯示每個群組的客戶端數量，還顯示該群組的累計已用流量。該欄位標示為 **«Использованный трафик»**（已用流量）。

該欄位顯示的內容

對每一列群組，會輸出該群組所有客戶端的流量總和——也就是其所有成員的 `up + down`（送出 + 接收流量）相加。這能快速回答「整個群組總共下載/分享了」多少的問題，而不必逐一打開客戶端再手動相加。

群組資料表中相鄰會輸出：

欄位	含義
Имя	群組名稱
Клиенты	有多少客戶端被標記為此群組（先前該欄稱為「Клиентов в группе」（群組中的客戶端數））
Отправлено	群組所有客戶端的 <code>up</code> （送出流量）總和
Получено	群組所有客戶端的 <code>down</code> （接收流量）總和
Использованный трафик	群組所有客戶端的 <code>up + down</code> 總和

送出與接收流量分別以 **Отправлено**（已送出）與 **Получено**（已接收）兩欄輸出，而 **Использованный трафик**（已用流量）欄則顯示它們的總和。客戶端數量的欄位就簡單稱為 **Клиенты**（客戶端）。

資料表上方的彙總另外會顯示所有群組的聚合數值——**«Всего групп»**（群組總數）與 **«Клиенты с группой»**（有群組的客戶端），而累計流量則分為兩張卡片：**«Всего отправлено / получено»**（總計送出 / 接收）（帶有上/下箭頭——分別為所有群組的送出與接收流量）以及 **«Общий трафик»**（總流量）（帶有圖表圖示——它們的合計總和）。

如何計算

計算透過對客戶端資料表執行一次 SQL 查詢、並聯結（`LEFT JOIN`）流量記帳資料表來完成：

- 依群組標籤欄位（`group_name`）將客戶端分組並計算其數量——這就是「群組中的客戶端數」；
- 流量取自所聯結的 `client_traffics` 資料表中 `up + down` 的總和。也就是將每個客戶端送出（`up`）與接收（`down`）的位元組相加；
- 由於 `email` 在客戶端資料表與流量資料表中都是唯一的，因此聯結不會使單一客戶端的流量重複計算。

數值的特性：

- **沒有流量記錄的客戶端**會被計入成員計數，但對總和貢獻 0，因此剛建立的群組顯示流量為 0。
- **空群組**（已建立但沒有客戶端）同樣會以零計數與零流量出現在清單中：除了由客戶端標籤「衍生」出的群組外，明確儲存的群組也會被併入結果，之後清單會以不分大小寫的名稱排序。

- 沒有群組標籤的客戶端（group_name 為空）不會被納入計算。

相關動作

在群組資料表中，針對整個群組的動作仍然可用，包括「Сбросить трафик」（重設流量）——僅將群組本身的計數器（基準點）歸零，不影響個別客戶端的流量（詳見 9.9）。在這樣重設之後，該群組的「已用流量」欄會顯示 0。

10. 訂閱 (Subscription)

訂閱 (subscription) 是一種機制，可讓您為用戶端提供一個固定連結 (URL)，VPN 用戶端透過該連結自動下載並定期更新完整的配置集合。無需手動將每個 inbound 的獨立連結逐一傳送給使用者，只需給予他們一個格式為 `https://[?]:[?]/sub/<subId>` 的統一地址。透過該地址，面板即時彙整與該用戶端綁定的所有配置，並以用戶端所需的格式傳回。當伺服器設定變更時 (新地址、Reality 金鑰輪替、新增 inbound)，用戶端在下次自動更新時即可取得最新配置，無需使用者採取任何操作。

訂閱由面板內部的獨立 HTTP/HTTPS 伺服器提供服務，該伺服器獨立於 Web 面板運行，並監聽自己的埠。這樣設計出於安全考量：可以對外開放訂閱埠，而無需開放面板本身的埠。

10.1. 什麼是 subId 以及連結的組成方式

inbound 中每個用戶端均有 `subId` 欄位 (介面中稱為「訂閱 ID」)。該值即為訂閱的金鑰：面板在所有 inbound 中搜尋 `subId` 與請求值相符的用戶端，並將其配置合併為一個回應。

- 若多個用戶端 (位於同一或不同 inbound 中) 設定了相同的 `subId`，其配置將合併至同一訂閱。這是透過一個連結為單一使用者提供多個伺服器 / 協議的標準方式。

範例：一位使用者透過一個連結取得兩台伺服器的配置。假設有兩個 inbound (伺服器 A 上的 VLESS 和伺服器 B 上的 Trojan)。若要透過一個連結為使用者提供兩個配置，只需將兩個用戶端的 `subId` 設為相同值：

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

這樣，透過 `https://sub.example.com:2096/sub/ivan2025` 這個地址，面板將同時傳回兩個配置。若之後新增第三個具有相同 `subId` 的 inbound，使用者在下次自動更新訂閱時即可取得，無需傳送新連結。

- 若用戶端的 `subId` 欄位為空，則無法分享公開連結。介面中會顯示提示：「此用戶端沒有 subId，無法使用公開分享連結。」

用戶端外部連結與訂閱 (「Links」分頁)

用戶端表單中有一個「Links」分頁，可為個別用戶端附加額外的配置來源，這些配置將混入其訂閱中 (支援 RAW、JSON 和 Clash 格式)：

- **Add External Link** – 第三方分享連結 (`vless://`、`trojan://`、`ss://` 等)。直接加入輸出，JSON/Clash 格式還會額外解析為配置。
- **Add External Subscription** – 外部訂閱地址。面板會自動下載 (含快取及短暫逾時)，並將取得的配置合併至用戶端的清單中。

此功能可方便地透過同一個統一連結，在您的 inbound 之上為用戶端提供額外的伺服器。若遠端訂閱的回應過大，面板不再靜默截斷，而是傳回錯誤並繼續使用最後一次成功快取的值。

- `subId` 的值不能任意設定：儲存時會驗證其中不含空格、`/`、`\` 及控制字元。對應的驗證提示為：「訂閱 ID 不能包含空格、`/`、`"` 或控制字元」。

最終連結的格式為 `<subPath>/<subId>`（請參閱訂閱伺服器設定與「反向代理 URI」欄位的相關章節）。若根據 `subId` 找不到任何用戶端（用戶端已刪除、`subId` 不存在），伺服器將傳回不含正文的 HTTP 404。若發生內部錯誤，則傳回 HTTP 500。VPN 用戶端僅依賴回應碼，因此錯誤正文被刻意設計為空白。

訂閱中 inbound 連結的排序

每個 inbound 都有一個「訂閱中的順序」欄位（`subSortIndex`）——從 1 開始的數字，決定該 inbound 的連結在訂閱輸出中的位置。數值較小者排在前面；數值相同時按原始建立順序（依 id）排列。排序適用於所有輸出格式——純文字、訂閱頁面、JSON 和 Clash。此欄位不影響面板本身中 inbound 的排列順序。

此欄位可在 inbound 表單中與分享地址設定（share address）相鄰的位置進行編輯，並依照一般規則同步至節點。若至少有一個 inbound 的順序不為 1，「inbound 清單」中將顯示一個精簡的「順序」欄。

10.2. 訂閱伺服器設定

所有訂閱參數位於面板設定的「訂閱」分頁中。以下說明各項參數；括號內為內部設定鍵名及預設值。

該分頁本身劃分為數個子分頁：「面板設定」、「資訊」、「個人資料」、「憑證」、「Happ」和「Clash / Mihomo」。訂閱標題、支援 URL、個人資料頁面、公告及佈景目錄等欄位位於「個人資料」分頁；Happ 及 Clash/Mihomo 的路由規則位於同名分頁；訂閱更新間隔位於「資訊」分頁。

基本參數

欄位 (UI)	鍵名	預設值	說明
啟用訂閱	<code>subEnable</code>	<code>true</code> (啟用)	啟動獨立的訂閱伺服器。提示：「具有獨立配置的訂閱功能」。若關閉，訂閱伺服器不會啟動，所有連結均無法使用。
監聽 IP	<code>subListen</code>	空白	訂閱伺服器接受連線的 IP 地址。提示：「預設留空以監聽所有 IP 地址」。
訂閱埠	<code>subPort</code>	<code>2096</code>	訂閱伺服器的 TCP 埠。提示：「用於訂閱服務的埠號不應在伺服器上已被佔用」——埠須閒置，且不與面板或 Xray 衝突。
URI 路徑	<code>subPath</code>	<code>/sub/</code>	提供一般訂閱的路徑。提示：「必須以 <code>/</code> 開頭並以 <code>/</code> 結尾」。
監聽域名	<code>subDomain</code>	空白	允許訪問訂閱的域名（Host 驗證）。提示：「預設留空以監聽所有域名和 IP 地址」。若已設定，則帶有其他 Host 的請求將被拒絕。

安全性注意事項：預設路徑 `/sub/`（以及 JSON 的 `/json/`）廣為人知，容易被猜測。面板會顯示警告：「預設訂閱路徑 `/sub/` 廣為人知——請修改它。」以及類似的 JSON 警告。建議設定自訂的不易猜測路徑。

TLS / 憑證

欄位 (UI)	鍵名	預設值	說明
訂閱憑證公鑰檔案路徑	<code>subCertFile</code>	空白	憑證檔案（ <code>.crt / fullchain</code> ）的完整路徑。提示：「輸入以 <code>/</code> 開頭的完整路徑」。

欄位 (UI)	鍵名	預設值	說明
訂閱憑證私鑰檔案路徑	subKeyFile	空白	私鑰檔案的完整路徑。提示：「輸入以 '/' 開頭的完整路徑」。

若兩個路徑均已填寫且憑證成功載入，訂閱伺服器將以 **HTTPS** 運行。若欄位為空或憑證讀取失敗，伺服器將退回至 **HTTP**（錯誤記錄至日誌）。有效的 TLS 也會影響基礎 URL 的生成：當埠為 443 且有 TLS，或埠為 80 且無 TLS 時，連結中的埠號將被省略。

更新間隔

欄位 (UI)	鍵名	預設值	說明
訂閱更新間隔	subUpdates	12	用戶端應用程式重新請求訂閱的頻率（以小時計）。提示：「用戶端應用程式更新間隔（以小時計）」。自 3.5.0 起，欄位接受 0-525600 並顯示範圍提示（先前的介面上限 168 與伺服器不一致，從 2.x 升級後會阻擋任何設定的儲存）。

該值透過 HTTP 標頭 `Profile-Update-Interval` 傳遞給用戶端；現代用戶端將其用作配置自動更新的週期。

回應格式與資訊

欄位 (UI)	鍵名	預設值	說明
加密	subEncrypt	true	提示：「加密訂閱中傳回的配置」。技術上這並非加密，而是對一般訂閱整個正文進行 Base64 編碼 （大多數用戶端期望的格式）。關閉後，連結以明文傳回，每行一個。
顯示使用資訊	subShowInfo	true	提示：「在配置名稱後顯示剩餘流量和到期日期」。啟用後，每個配置的名稱（remark）後方會附加剩餘流量標記（  ）及有效期（例如 5D, 3H  ）；若用戶端已過期或不可用，則顯示 N/A 。
在名稱中包含 Email	subEmailInRemark	true	提示：「在訂閱個人資料名稱中包含用戶端 email。」。將用戶端 email 加入個人資料的 remark 中。

備註範本 (Remark Template)

訂閱中每個配置的顯示名稱（remark）是根據 **備註範本** 生成的——即訂閱設定「**資訊**」分頁中的「**備註範本**」欄位（`remarkTemplate`）。先前的備註模式建構器（分別選取 inbound / email / 外部代理和分隔符號）已從介面中移除；現在您可以自由撰寫名稱格式並插入變數。預設值為 `{{INBOUND}}-{{EMAIL}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D`（即預設情況下個人資料名稱包含用戶端 email）。若欄位留空，則使用舊有的（無法透過介面配置的）備註模式。

變數按 **Client**、**Traffic** 和 **Time & status** 分類，並以可點擊的 `{{VAR}}` 標籤顯示在欄位旁邊（懸停時顯示提示）；點擊可將標記插入範本，並提供即時預覽。每個變數在生成訂閱時針對特定用戶端個別替換。也支援

使用單括號的簡化寫法（`{DATA_LEFT}`、`{EXPIRE_DATE}`、`{PROTOCOL}`、`{TRANSPORT}` 等）——面板會自動將其轉換為內部 `{{...}}` 格式。

可用變數：

- **用戶端識別**：`{{EMAIL}}`（同義詞——`{{USERNAME}}`）、`{{INBOUND}}`（inbound 本身的 remark）、`{{HOST}}`（主機的 remark）、`{{ID}}`（UUID）、`{{SHORT_ID}}`（UUID 前 8 個字元）、`{{SUB_ID}}`、`{{COMMENT}}`、`{{TELEGRAM_ID}}`、`{{PROTOCOL}}`、`{{TRANSPORT}}`。
- **流量**：`{{TRAFFIC_USED}}`、`{{TRAFFIC_LEFT}}`、`{{TRAFFIC_TOTAL}}`（及其 `*_BYTES` 精確位元組變體）、`{{UP}}`、`{{DOWN}}`、`{{USAGE_PERCENTAGE}}`。
- **有效期與狀態**：`{{DAYS_LEFT}}`、`{{TIME_LEFT}}`、`{{EXPIRE_DATE}}`（YYYY-MM-DD）、`{{JALALI_EXPIRE_DATE}}`（波斯曆日期）、`{{EXPIRE_UNIX}}`、`{{CREATED_UNIX}}`、`{{RESET_DAYS}}`、`{{STATUS}}`（active / expired / disabled / depleted）、`{{STATUS_EMOJI}}`。
- **連線（Connection）**：`{{PROTOCOL}}` — 協議（VLESS、VMess、Trojan 等），`{{TRANSPORT}}` — 傳輸網路（tcp、ws、grpc 等），`{{SECURITY}}` — 傳輸安全性（TLS、REALITY、NONE；以大寫顯示）。與流量和有效期變數一樣，這三個變數僅在訂閱正文中有效，並自動從面板顯示連結（QR / 「資訊」）及訂閱資訊頁面的 remark 中移除。

範本可使用垂直線 | 分隔為多個區段。若某變數在某個區段中產生「無限制」值（ ∞ ）——例如無限制用戶端的 `{{TRAFFIC_LEFT}}` 或 `{{DAYS_LEFT}}` ——該區段將自動隱藏。此外，流量和有效期資訊區塊僅在用戶端的第一個連結中顯示一次，避免在每個配置中重複出現。

範例。 範本 `{{EMAIL}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` 對於剩餘 42 GB 和 7 天的用戶端，將產生類似 `ivan@vpn 42.00GB 7D` 的名稱；對於無限制用戶端，則僅顯示 `ivan@vpn`（含 ∞ 的區段被省略）。

自 3.4.2 起，`{{EMAIL}}` 變數（及同義詞 `{{USERNAME}}`）僅在訂閱正文中用戶端的第一個連結上輸出——與流量 / 時限剩餘量區塊相同；同一用戶端的其餘連結會省略它。

在面板顯示的連結（「用戶端」頁面上的 QR 碼和「資訊」視窗）以及訂閱資訊頁面上，用戶端 email 會出現在個人資料名稱中：設定了主機時顯示為「inbound-主機-email」，未設定主機時顯示為「inbound-email」。流量和有效期資訊（以及「連線」分類的變數）不會替換至這些顯示名稱中——它們僅在 VPN 用戶端接收的訂閱正文中有效。

若用戶端的流量統計列在刪除並重新建立 inbound 後「孤立」，`{{TRAFFIC_USED}}`（及其他使用量指標）不再顯示 `0.00B`：面板會額外依用戶端 email 搜尋統計資料，並填入正確的已使用流量。| 備註範本 | `remarkTemplate | {{INBOUND}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` | 使用 `{{VAR}}` 變數替換的自由格式顯示名稱（remark）範本，在生成訂閱時針對每個用戶端個別替換。舊有的「備註模式」建構器（選取 inbound/email/外部代理和分隔符號）已從介面中移除，僅作為欄位留空時的備用方案。詳情請參閱下方「備註範本（Remark Template）」章節。|

個人資料元資料（回應標頭）

這些字串透過 HTTP 回應標頭傳遞給用戶端，並在 VPN 用戶端中顯示為個人資料元資料。預設均為空白。

欄位 (UI)	鍵名	標頭	說明
訂閱標題	subTitle	Profile-Title (Base64 編碼)	「VPN 用戶端中顯示給用戶端的訂閱名稱」。對於 Clash, 也透過 Content-Disposition 作為匯入個人資料的名稱使用。
支援 URL	subSupportUrl	Support-Url	「VPN 用戶端中顯示的技術支援連結」。
個人資料 URL	subProfileUrl	Profile-Web-Page-Url	「VPN 用戶端中顯示的您的網站連結」。若未設定，則替換為訂閱請求的實際 URL。
公告	subAnnounce	Announce (Base64 編碼)	「VPN 用戶端中顯示的公告文字」。自 3.5.0 起，該文字（非空時）也會以資訊橫幅顯示在訂閱資訊頁面頂部，且自訂範本可使用 announce 變數。

此外，每個回應都會傳送 Subscription-Userinfo 標頭，其中包含用戶端的彙整流量資料：upload、download、total 和 expire（以秒為單位的到期時間戳）。用戶端根據此標頭顯示剩餘流量和有效期。

路由（僅限 Happ 用戶端）

欄位 (UI)	鍵名	預設值	說明
啟用路由	subEnableRouting	false	「用於在 VPN 用戶端中啟用路由的全局設定（僅限 Happ）」。透過 Routing-Enable 標頭傳遞。
路由規則	subRoutingRules	空白	「VPN 用戶端的全局路由規則（僅限 Happ）」。透過 Routing 標頭傳遞。

| 隱藏伺服器設定 | subHideSettings | false | 「隱藏訂閱中的伺服器設定（僅限 Happ）」。啟用後，Happ 用戶端將隱藏查看和修改伺服器設定的功能。此選項僅對 Happ 用戶端有效。|

Incy 路由（僅限 Incy 用戶端）

針對 Incy VPN 用戶端，訂閱設定中有一個獨立的「Incy」分頁，包含兩個欄位：「啟用路由」切換器（subIncyEnableRouting，預設關閉）和格式為 incy://routing/onadd/<base64> 的「路由規則」文字欄位（subIncyRoutingRules）。啟用路由且欄位已填寫時，此字串將作為獨立行附加至訂閱正文（raw 格式）——這樣路由個人資料就能傳遞給 Incy 用戶端，而不會與 Happ 用戶端的 Routing 標頭衝突。這些設定僅對 Incy 用戶端有效。

反向代理 URI

欄位 (UI)	鍵名	預設值	說明
反向代理 URI	subURI	空白	「修改訂閱 URL 的基礎 URI，以便在代理伺服器後方使用」。

若欄位為空，面板將根據訂閱域名和埠（考慮 TLS）自動建構連結的基礎地址。若訂閱透過不同域名或路徑的外部反向代理 / CDN 提供，則在此欄位填入最終的基礎 URI，所有連結將以此為基礎建構。JSON（`subJsonURI`）和 Clash（`subClashURI`）各有獨立的對應欄位。

若僅設定了通用的 `subURI`，而 JSON 和 Clash 的獨立欄位留空，這些格式的連結在訂閱頁面上將繼承 `subURI` 的方案和主機（而非訂閱伺服器的埠和 `http`）——使其與反向代理的地址保持一致。

範例：透過反向代理提供訂閱。 訂閱本身監聽在 2096，但透過 nginx/CDN 從外部以 `https://cfg.example.com/u/` 提供。若要讓回應中的連結以外部地址而非內部的 `127.0.0.1:2096` 為基礎建構，在「Reverse proxy URI」欄位填入最終的基礎 URI：

```
Reverse proxy URI: https://cfg.example.com/u
```

這樣最終連結將為 `https://cfg.example.com/u/ivan2025`。如有需要，JSON 和 Clash 格式可以相同方式填寫各自的 `subJsonURI` 和 `subClashURI` 欄位。

10.3. 輸出格式

訂閱可以三種獨立格式提供，每種格式有各自的端點，可分別啟用或停用。

輸出中的伺服器地址與節點

訂閱連結中的伺服器地址採用與面板中一般連結和 QR 碼相同的地址策略：「listen」——可路由的綁定地址，「custom」——使用者設定的自訂地址（`shareAddr`），「node」（預設）——節點地址。未明確設定策略的 inbound，其訂閱輸出不會改變。這讓綁定到特定公共 IP 的節點 inbound 能向用戶端提供可到達的地址。該策略適用於 raw、JSON 和 Clash 格式。

節點名稱（Node）不會附加至訂閱中個人資料的名稱（remark）：用戶端應用程式中僅顯示管理員設定的 inbound remark，不帶 `@127.0.0.1` 格式的內部後綴。若要在多節點訂閱中區分同名條目，請手動為其設定不同的 remark，或使用具有自訂 Remark 的受管主機（Hosts）。

若因節點間不同步而導致同一用戶端在服務 JSON inbound 中出現兩次，訂閱輸出會在三種格式中自動依 email 去除重複項，因此輸出中不會出現重複的個人資料。

受管主機（Hosts）

Hosts 章節（側邊選單項目；顯示 Total/Enabled/Disabled 數量及清單的彙整頁面）為訂閱連結定義地址覆寫規則。每個 inbound 可新增一個或多個**主機**——端點，這些端點將在傳遞給用戶端的訂閱連結中**替代 inbound 本身的地址、埠和 TLS 參數**。這讓您能透過 CDN 或中繼分發流量，而無需修改 inbound 本身。

自 3.5.0 起，主機是一個「群組」：一筆記錄可同時涵蓋多個 inbound 與多個地址。清單操作（啟用/停用/刪除/排序）與 API 以群組為單位運作（字串型 `groupId`）；新增了批量端點 `POST /panel/api/hosts/bulk/add`（`{"inboundIds": [...], "hosts": [...], ...}`），而 `list/get/update/del/setEnable` 均以群組物件操作。

每個主機（群組）可設定：

- **Remark** 和說明（Description）、綁定至 **Inbounds**（可搜尋的多重選取；至少一個）、**Enable** 切換器及節點指派（**Nodes**）。
- **Address**——自 3.5.0 起為**地址清單**（標籤式輸入；分隔符為逗號、分號、空格；輸入提示 `cdn.example.com, cdn2.example.com:443`）。每筆記錄可內嵌自己的 `[:?]??`（包括帶方括號的 IPv6 `:::1]:443`）；下拉建議會提示其他主機已使用過的地址。清單留空——繼承 inbound 本身的地址（清單中顯示橘色 **Inherits** 標籤）。**Port**（`0`——繼承 inbound 埠）作為未內嵌連接埠之記錄的預設連接埠；**Tags**（僅在 RAW 訂閱中生效）。
- **Security** 分頁——`same / tls / none / reality`，含 SNI、指紋（fingerprint）、ALPN、憑證固定（pinned-cert）、`allowInsecure` 和 ECH。
- **Advanced** 分頁——Host 標頭、Path、VLESS 路由、Mux、Sockopt、Final Mask，以及從個別訂閱格式（raw / json / clash）中排除主機的選項。自 3.5.0 起，主機的 **Final Mask** 也會進入 **raw 連結**（`fm=`；先前僅 JSON/Clash）：主機的 TCP/UDP 遮罩會附加至 inbound 的遮罩，主機的 QUIC 參數僅在 inbound 未設定時採用。**Allow insecure** 切換器現在也對 **Hysteria/Hysteria2** 生效（連結中 `insecure=1`，Clash 中 `skip-cert-verify: true`）。
- **Clash (mihomo)** 分頁——IP 版本、Mihomo X25519、主機隨機排序（Shuffle host）。

清單中的 **Endpoint** 欄位以標籤形式顯示地址（第一個直接可見，其餘收合於「+N」彈出視窗），**Inbounds** 欄位則顯示依協議著色的 inbound 標籤。自 3.5.0 起，主機採**全域**排序（先依排序順序，再依備註），不再侷限於各自的 inbound；批次啟用、停用和刪除功能保留。受管主機取代了先前的 External Proxy 陣列。

VLESS 路由（VLESS route）。自 3.4.2 起，這是單一數值 `0-65535`（不再是連接埠清單；提示——「одно значение VLESS route (0-65535), зашиваемое в UUID, например 443; пусто – без него」，輸入提示 `443`）。設定的值會實際「嵌入」每份生成訂閱的 UUID 中（raw / JSON / Clash）：Xray 讀取 UUID 的第 6-7 位元組並在驗證前將其遮罩，因此用戶端仍可比對成功。空值或不正確的值不會更改 UUID。

一般連結（SUB）——Base64 / 純文字

基本格式，端點為 `subPath`（預設 `/sub/`）。在整體訂閱啟用時始終開啟。傳回 Xray 連結清單（`vless://`、`vmess://`、`trojan://`、`ss://` 等），每行一個。啟用「加密」選項（`subEncrypt`）時，整個清單以 Base64 編碼；停用時以明文傳回。幾乎所有用戶端均支援此格式（v2rayNG、V2RayTun、Sing-box、NekoBox、Streisand、Shadowrocket、Happ 等）。

範例：停用「加密」時的回應正文。當 `subEncrypt = false` 時，`/sub/` 端點傳回明文——每行一個連結：

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```

當 `subEncrypt = true`（預設）時，相同清單整體以 Base64 編碼後以單行傳回——大多數用戶端期望的正是此格式。

JSON 訂閱（sing-box 及相容用戶端）

端點為 `subJsonPath`（預設 `/json/`），可透過獨立核取方塊啟用。

欄位 (UI)	鍵名	預設值	說明
JSON 訂閱	subJsonEnable	false	「獨立啟用/停用 JSON 訂閱端點。」

傳回完整的 JSON 配置 (sing-box 及衍生用戶端可理解的格式——Podkop、OpenWRT sing-box、Karing、NekoBox)。自 3.5.0 起，**原生 WireGuard inbound** 的用戶端也會納入 JSON 訂閱 (secretKey、address、含 publicKey / endpoint / preSharedKey / keepAlive / allowedIPs 的 peers[]、mtu) ——先前它們會被靜默跳過。此格式提供額外參數 (subFormats 分頁)：

- **Mux** (subJsonMux, 預設空白) ——注入至 JSON 訂閱每個串流 outbound 的多工 (Mux) JSON 設定。
「在單一連線中傳輸多個獨立資料串流。」
- **Final Mask** (subJsonFinalMask, 預設空白) ——「新增至 JSON 訂閱每個串流的 xray finalmask (TCP/UDP) 設定和 QUIC 設定。需要用戶端上的較新版本 xray。」透過子欄位進行配置：「封包」(packets)、「長度」(length)、「間隔」(interval)、「最大分割」(maxSplit)、「雜訊」(noises：「類型」/type、「封包」/packet、「延遲 (毫秒)」/delayMs、「套用至」/applyTo、「+ 雜訊」按鈕)，以及「並行數」(concurrency)、「xudp 並行數」(xudpConcurrency) 和「xudp UDP 443」(xudpUdp443)。
- **路由規則** (subJsonRules, 預設空白) ——新增至 JSON 配置的全局規則。

Clash / Mihomo 訂閱 (YAML)

端點為 subClashPath (預設 /clash/)，可透過獨立核取方塊啟用。

欄位 (UI)	鍵名	預設值	說明
Clash / Mihomo 訂閱	subClashEnable	false	為 Clash 和 Mihomo 用戶端啟用 YAML 配置生成。
啟用路由	subClashEnableRouting	false	「將全局 Clash/Mihomo 路由規則新增至生成的 YAML 訂閱中。」
全局路由規則	subClashRules	空白	「在 MATCH,PROXY 之前新增至每個 YAML 訂閱開頭的 Clash/Mihomo 規則。」

回應以 application/yaml; charset=utf-8 類型傳回。若已設定「訂閱標題」(subTitle)，也會透過 Content-Disposition 標頭傳遞 (attachment; filename*=UTF-8'<[?][?]>)，使 Clash 用戶端以此名稱命名匯入的個人資料。

生成的連結和 YAML 格式針對現代用戶端保持最新狀態：Shadowsocks-2022 (SS2022) 不再對 userinfo 進行 Base64 編碼；帶有 http 混淆的 Shadowsocks 連結以 SIP002 格式配合 obfs-local 插件傳回；Clash/Mihomo 訂閱實現了完整的 XHTTP 欄位集。自 3.5.0 起，**原生 WireGuard inbound** 的用戶端也會納入 Clash/Mihomo 訂閱 (private-key、public-key、pre-shared-key、persistent-keepalive、ip / ipv6、mtu、dns 欄位) ——先前它們會被靜默跳過。這不需要額外設定——連結只是被用戶端更正確地識別。

備註：此版本支援三種格式——一般連結 (Base64/文字)、JSON (sing-box 相容) 和 Clash/Mihomo (YAML)。訂閱伺服器中沒有獨立的 Outline 格式。

10.4. 訂閱資訊頁面與 QR 碼

若在瀏覽器中開啟三種訂閱連結中的任何一種——raw、JSON 或 Clash（或在 URL 後明確加上 `?html=1` 或 `?view=html` 參數，或傳送 `Accept: text/html` 標頭），伺服器將傳回視覺化的訂閱資訊頁面（「訂閱資訊」），而非「原始」回應。3.5.0 之前只有主連結 `/sub/` 如此運作，`/json/` 與 `/clash/` 在瀏覽器中會傳回原始 JSON/YAML。VPN 用戶端仍會收到機器可讀回應，因為它們不請求 HTML。

該頁面（由 Vite 構建的單頁應用程式）顯示：

- **訂閱資訊**（Descriptions 區塊）：
 - 「訂閱 ID」—— `subId` 的值；
 - 「狀態」——「有效」、「無效」或「無限制」。若用戶端被停用、已耗盡流量限制或已過期，則顯示「無效」狀態；
 - 「下載」和「上傳」——流量用量；
 - 「總限制」——流量限制，若無限制則顯示 ∞ ；
 - 「有效期」——到期日期或「永久」；
 - 剩餘流量和最後上線時間。
 - 日期根據面板的「Calendar Type」設定（`datepicker`，預設為 `gregorian`）以公曆或波斯曆顯示。
- **訂閱連結**：每種已啟用格式各有一行，帶有彩色標籤（綠色 **SUB**、紫色 **JSON**、金色 **CLASH**）、複製按鈕和 QR 碼按鈕（彈出視窗，尺寸 240 px）。JSON 和 CLASH 行僅在設定中啟用對應格式時才會顯示。
- **個別連結**（「複製連結」）：訂閱中所有單獨配置的完整清單，每個均帶有協議標籤、複製按鈕和 QR 碼（後量子連結不生成 QR 碼）。
- **「複製所有配置」按鈕**（在個別連結清單上方）：一鍵將所有配置連結複製到剪貼簿（每個換行），無需逐一複製；完成後顯示「所有配置已複製」通知。
- **快速匯入應用程式按鈕**（按平台分類的下拉選單）：Android——`v2box`、`v2rayNG`（深層連結 `v2rayng://install-config?url=...`）、`Sing-box`、`V2RayTun`、`NPV Tunnel`、`Happ`（`happ://add/...`）、`Incy`（`incy://add/...`）；iOS——`Shadowrocket`（透過 `flag=shadowrocket` 參數）、`v2box`（`v2box://install-sub?url=...&name=...`）、`Streisand`（`streisand://import/...`）、`V2RayTun`、`NPV Tunnel`、`Happ`、`Incy`。這些按鈕可開啟目標應用程式的深層連結（已預填訂閱地址），或將連結複製到剪貼簿。

資訊頁面傳送時帶有禁止快取標頭（`Cache-Control: no-cache`），確保用戶端始終看到最新的流量和有效期資料。

10.5. 自訂訂閱頁面範本

從 3.3.0 版起，可以使用自訂 HTML 範本替換標準訂閱登陸頁面。預設情況下，訂閱地址提供的是內建頁面，但若指定包含自訂範本的目錄，面板將渲染該範本並填入用戶端的最新資料（流量、有效期、連結等）。

重要：面板不提供現成範本 — 您需要自行建立佈景。建立說明與可用變數清單請見 [docs/custom-subscription-templates.md](https://docs.x-ui.com/custom-subscription-templates.md)。

啟用位置

佈景目錄在面板設定中指定：

設定 → 訂閱 → 「訂閱資訊」章節，「訂閱佈景目錄」欄位（subThemeDir）。

介面中的欄位說明：「包含訂閱頁面自訂範本（index.html/sub.html）的資料夾絕對路徑（例如 /etc/3x-ui/sub_templates/my-theme/）。留空則使用預設頁面。」

同一章節中還有相關設定，其值可在範本中使用：

「訂閱佈景目錄」欄位說明中有一個「範本指南 ↗」連結，指向建立自訂訂閱頁面佈景的文件。

- 「訂閱標題」（subTitle）——用戶端可見的名稱；
- 「支援 URL」（subSupportUrl）——技術支援連結。

設定參數

參數	預設值	用途
subThemeDir	""（空白）	HTML 範本所在目錄的絕對路徑。空白 = 使用內建預設頁面。

如何套用自訂範本

1. 在伺服器上建立佈景資料夾（任意位置），例如 /etc/3x-ui/sub_templates/my-theme/。
2. 在其中放入名為 index.html 或 sub.html 的 HTML 檔案。

範例：佈景路徑。 伺服器上的最終目錄結構及設定欄位中的值：

```
/etc/3x-ui/sub_templates/my-theme/
└─ index.html      ([?]sub.html—[?] [?] [?] [?] [?])
```

```
[?] → [?] → [?] [?] [?] [?] [?] [?]
/etc/3x-ui/sub_templates/my-theme/
```

路徑必須為絕對路徑（以 / 開頭）。若資料夾中既無 index.html 也無 sub.html，面板將提供內建頁面。3. 在面板中開啟 設定 → 訂閱，在「訂閱佈景目錄」欄位填入該資料夾的絕對路徑。4. 儲存設定。

檔案選擇與渲染行為：

- 若目錄中存在 sub.html，則使用該檔案；否則使用 index.html。即 sub.html 的優先順序高於 index.html。
- 範本使用 Go 標準引擎 html/template 渲染。
- 已解析的範本會被快取，僅在檔案修改時間變更時才重新從磁碟讀取。因此範本的修改無需重新啟動面板即可生效，但也不會在每次請求時產生讀取/解析的額外開銷。
- 回應先完整緩衝於記憶體中，再傳送給用戶端：若範本在執行過程中發生錯誤，部分生成的（損壞的）頁面不會傳送給使用者。

預設行為與回退 (fallback)

- 欄位為空 → 提供內建 SPA 頁面（資料注入至 `window.__SUB_PAGE_DATA__`）。
- 路徑不存在或不是目錄 → 使用預設頁面。
- 目錄中既無 `index.html` 也無 `sub.html` → 日誌記錄警告「subThemeDir set but no usable template found」，提供預設頁面。
- 範本檔案存在但解析失敗 → 日誌記錄錯誤「custom template parse failed」，提供預設頁面。
- 範本執行時發生錯誤 → 日誌記錄「custom template execution failed」，提供預設頁面。

即任何自訂範本問題都不會「損壞」訂閱——面板始終退回至內建頁面。所有訂閱頁面（自訂和標準）均傳送時帶有禁止快取標頭（`Cache-Control: no-cache, no-store, must-revalidate`），確保用戶端始終獲取最新的流量和有效期資料。

可用的範本變數

傳遞給範本上下文的是一組訂閱用戶端資料。透過 `{{ .? }}` 存取：

變數	類型	說明
<code>{{ .sId }}</code>	string	訂閱 ID (UUID)。
<code>{{ .enabled }}</code>	bool	用戶端/訂閱是否已啟用。
<code>{{ .download }}</code>	string	格式化的下載量（例如「2.5 GB」）。
<code>{{ .upload }}</code>	string	格式化的上傳量。
<code>{{ .total }}</code>	string	格式化的流量總限制。
<code>{{ .used }}</code>	string	格式化的已使用流量（下載 + 上傳）。
<code>{{ .remained }}</code>	string	格式化的剩餘流量。
<code>{{ .expire }}</code>	int64	有效期——Unix 時間（秒）（ <code>0</code> = 永久）。用於 JS <code>Date</code> 時乘以 1000。
<code>{{ .lastOnline }}</code>	int64	最後上線時間——Unix 時間（毫秒）（ <code>0</code> = 從未上線）。
<code>{{ .downloadByte }}</code>	int64	精確位元組下載量。
<code>{{ .uploadByte }}</code>	int64	精確位元組上傳量。
<code>{{ .totalByte }}</code>	int64	精確位元組總限制。
<code>{{ .subUrl }}</code>	string	訂閱頁面 URL。
<code>{{ .subJsonUrl }}</code>	string	JSON 訂閱配置 URL。
<code>{{ .subClashUrl }}</code>	string	Clash/Mihomo 配置 URL。
<code>{{ .subTitle }}</code>	string	設定中的訂閱標題（可能為空）。
<code>{{ .subSupportUrl }}</code>	string	設定中的支援 URL（可能為空）。
<code>{{ .links }}</code>	[]string	配置字串清單（VMess、VLESS 等）。遍歷： <code>{{ range .links }} ... {{ end }}</code> 。
<code>{{ .emails }}</code>	[]string	訂閱相關的 email 清單。

變數	類型	說明
{{ .datepicker }}	string	面板目前的日曆格式：gregorian 或 jalali（取自「日曆類型」設定；若為空則為 gregorian）。

使用部分變數的最小範本正文範例：

```
<h1>{{ .subTitle }}</h1>
<p>{{ .used }}{{ .total }}{{ .remained }}</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}
```

範例：從 `expire` 獲取到期日期。 `{{ .expire }}` 欄位是 Unix 時間（秒），因此在 JavaScript 中需乘以 1000；值為 0 表示「永久」：

```
<script>
  var exp = {{ .expire }};
  document.write(exp === 0
    ? '??'
    : '??? ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

請注意：`{{ .lastOnline }}` 已以毫秒為單位——無需再乘以 1000。

11. Xray：路由、outbounds、DNS 與擴充功能

「Xray 設定」部分是 Xray-core 設定範本的編輯器，面板依此範本生成最終的 `config.json` 以啟動核心。該部分的提示：「依據範本建立 Xray 設定檔。」與 inbounds（單獨存儲在資料庫中，並在建構設定時代入範本）不同，其他所有內容——日誌、路由、outbounds、DNS、策略、統計——均在此處設定。

重要：範本的值以 `xrayTemplateConfig` 鍵儲存在資料庫中。儲存時，面板會對其進行一系列自動轉換（詳見 11.11）。任何語法不正確的 JSON 都會被拒絕，並顯示錯誤「*xray template config invalid*」。

選單位置：「Outbounds」與「路由」

「Outbounds」（Outbounds）和「路由」（Routing）是側邊選單中獨立的项目（位於「Hosts」下方、「面板設定」上方），各自有對應的網址：`/outbound` 和 `/routing`。直接連結至這些頁面以及重新整理頁面均可正常使用。「Xray 設定」子選單則只保留：基本、負載均衡器、DNS 和進階範本。在下方說明中，11.3 和 11.4 分別對應「路由」和「Outbounds」頁面。

11.1. 編輯器結構：分頁/模式

編輯器提供多種範本顯示模式（按 JSON 部分篩選）：

模式	顯示內容
基本	基本部分（日誌、基本路由、主要設定）
進階範本	完整的 Xray JSON 範本
全部	同時顯示所有部分

編輯器內的邏輯設定群組：

- 主要設定（提示：「這些參數描述通用設定」）。
- 日誌（詳見 11.10）。
- 基本連線：封鎖與直連路由。
- Inbounds（提示：「修改設定範本以連接特定客戶端」）。
- Outbounds（詳見 11.4）。
- 負載均衡器（詳見 11.5）。
- 路由（提示：「每條規則的優先順序很重要！」，詳見 11.3）。
- DNS / Fake DNS（詳見 11.6）。

11.2. 主要設定 (General)

Freedom Protocol Strategy

欄位	標籤	說明	預設值
FreedomStrategy	Freedom 協議策略設定	直連 (freedom) outbound 的網路輸出策略。提示：「設定 Freedom 協議中的網路輸出策略」。控制協議為 freedom 的 outbound 中 settings 的 domainStrategy 欄位。	在參考範本中，direct freedom outbound 的 domainStrategy 為 AsIs （位址不解析，以原始形式傳遞）。

freedom 的 domainStrategy (Xray-core 值)：AsIs（不在伺服器端解析網域），以及 UseIP / UseIPv4 / UseIPv6 系列及其「強制」變體 ForceIP*，這些值會強制輸出伺服器解析網域並透過取得的 IP 連線。如果輸出伺服器沒有 IPv6 或需要強制僅走 IPv4，請改為 UseIPv4。

Freedom Happy Eyeballs (IPv4/IPv6)

欄位	標籤	說明
FreedomHappyEyeballs	Freedom Happy Eyeballs (IPv4/IPv6)	提示：「適用於直連 (freedom) 輸出的雙協議棧連線——在同時具有 IPv4 和 IPv6 的輸出伺服器上非常實用。」為 freedom outbound 啟用 Happy Eyeballs 演算法（同時嘗試兩個位址族）。

個別 outbound 的 dial 設定 (sockopt) **中也有獨立的 Happy Eyeballs：自 3.5.0 起它會確實啟用（先前設定會被序列化為停用狀態，開關會「彈回」）。「Try delay (ms)」預設值現在為 **250；明確設定的 **0**（= 停用）會被保留；另外還提供 Prioritize IPv6、interleave (1) 與 maxConcurrentTry (4)。| try delay |（提示）|「嘗試其他位址族前等待的毫秒數。150–250 毫秒是一個不錯的起點。」 切換至替代位址族前的延遲。建議範圍為 150–250 毫秒。|

Overall Routing Strategy

欄位	標籤	說明	預設值
RoutingStrategy	網域路由設定	路由的整體 DNS 解析策略。提示：「設定 DNS 解析的整體路由策略」。控制 routing.domainStrategy 欄位。	在參考範本中， routing.domainStrategy = AsIs 。

routing.domainStrategy 決定路由如何將 IP 規則與網域請求進行匹配：AsIs（僅依網域規則，不解析）、IPIfNonMatch（若網域未匹配任何規則，則解析並檢查 IP 規則）、IPOnDemand（遇到 IP 規則時立即解析）。若要讓 IP 規則（例如 geoip:*）對網域請求生效，通常需要設定 IPIfNonMatch。

Outbound Test URL

欄位	標籤	說明	預設值
<code>outboundTestUrl</code>	Outbound 測試 URL	測試 outbound 連通性時所使用的 URL。提示：「用於檢查 <i>outbound</i> 連線的 URL」。以 <code>xrayOutboundTestUrl</code> 鍵單獨儲存，不存於範本。	<code>https://www.google.com/generate_204</code>

該值會進行清理。在實際測試 outbound 時，它會額外被驗證為公開 URL——此為防範 SSRF 的保護措施：使用者無法透過客戶端傳入任意（包括內部）URL，測試 URL 始終取自伺服器設定。儲存或測試時，若為空值則以預設的 `generate_204` 取代。

Default Outbound（自 3.5.0 起）

「基本路由」分頁的第一個項目是 **Default Outbound 選擇器**：「未匹配任何路由規則的流量會使用此 outbound（Xray 取清單中的第一個 outbound）」。清單包含 `direct`、`blocked` 及所有現有標籤；預設為 `direct`。選擇後會將指定的 outbound 移至範本的首位；不存在的 `direct/blocked` 會即時建立，切換時不會被刪除。

Block BitTorrent

欄位	標籤	說明
<code>Torrent</code>	封鎖 BitTorrent	在 <code>routing.rules</code> 中新增規則，將 <code>protocol: ["bittorrent"]</code> 的流量傳送至 <code>outbound blocked</code> 。此規則在參考範本中預設存在。

連線限制（Connection Limits）

提示：「使用者等級 0 的連線層級策略。留空則使用 Xray 預設值。」這些參數寫入 `policy.levels.0`。

欄位	標籤	說明	預設值
<code>connIdle</code>	閒置逾時（秒）	「在指定秒數閒置後關閉連線。降低此值可在高負載伺服器上更快釋放記憶體和檔案描述符（Xray 預設值：300）。」	空白 → Xray 預設 300
<code>bufferSize</code>	緩衝區大小（KB）	「每個連線的內部緩衝區大小（KB）。設為 0 可在低記憶體伺服器上最小化記憶體使用（Xray 預設值因平台而異）。」預留位置：「自動」。	空白 → 取決於平台；0 → 最小化

範例（`policy.levels.0`）。此群組的欄位寫入等級 0 策略。在低記憶體的高負載伺服器上，可以這樣加速資源釋放：

```
"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}
```

此處連線在閒置 120 秒後關閉（而非預設的 300 秒），而 `bufferSize: 0` 則最小化緩衝區的記憶體佔用。在表單中留空的欄位不會進入 JSON—Xray 將套用其預設值。

11.3. 路由規則（routing）

路由規則列表 `routing.rules`。順序至關重要（「每條規則的優先順序很重要！」）：規則從上到下依序評估，第一個匹配的規則生效。提示：「拖曳以變更順序」。順序控制按鈕：第一個、最後一個、上移、下移。

每條規則具有 `type: "field"`。按鈕：建立規則、編輯規則。列表欄位提示：「以逗號分隔的項目」。

在「路由」頁面上，**「匯入規則」和「匯出規則」按鈕集中在「更多」（more）下拉選單中——

「Outbounds」頁面亦同。「匯出規則」按鈕不會立即下載檔案，而是開啟帶有 JSON 預覽的模態視窗，以及「複製」和「下載」**按鈕：可在儲存前先預覽內容。「Outbounds」頁面的 outbound 匯出功能也以相同方式運作。

Route Tester（路由測試器）

在路由分頁上有一個 **Route Tester** 子分頁——它向正在執行的 Xray 查詢特定連線會由哪個 outbound 處理，而不傳送真實流量。輸入網域或 IP、連接埠、網路（TCP/UDP），以及（可選）inbound 和偵測到的協議（`http/tls/quic/bittorrent`），然後點擊 **Test Route**。結果直接取自即時路由引擎。

回應中會顯示匹配的 outbound；若使用負載均衡器，還會顯示負載均衡器標籤。若沒有任何規則匹配，測試器會顯示流量將進入預設 outbound（outbounds 列表中的第一個）。這便於在依賴規則之前驗證其順序。

啟用與停用個別規則

個別路由規則可透過開關暫時**停用**，而無需刪除。規則表格中有一個**「啟用」欄，其中包含開關

（Switch），規則表單中也有「啟用」**欄位——同樣是開關。停用的規則不會進入 Xray 的最終設定，但會保存在範本中，可隨時重新啟用。

統計服務規則（`inboundTag: ["api"]` → `outboundTag: "api"`）無法停用——其開關被鎖定，以免破壞面板的流量統計（詳見 11.11）。

規則表單欄位

表單欄位	標籤	JSON 欄位	說明
來源	來源	<code>source</code>	來源 IP 位址/子網路。以逗號分隔的列表。

表單欄位	標籤	JSON 欄位	說明
來源連接埠	來源連接埠	sourcePort	來源連接埠。
目的地	目的地	domain + ip + port	目標網域、IP 和連接埠。網域支援前綴 domain:、full:、regexp:、keyword:，以及 geosite:*；IP 支援 geoip:* 和 CIDR。
網路	—	network	tcp、udp 或 tcp,udp。自 3.4.2 起，在設定中以小寫寫入（tcp / udp），在路由表中以大寫顯示（TCP / UDP）；留空為任意網路。
協議	—	protocol	http、tls、bittorrent（透過 sniffing 識別）。
使用者	使用者	user	依使用者電子郵件/識別碼篩選。
屬性 / 值	屬性 / 值	attrs	用於匹配的 HTTP 標頭屬性。
VLESS route	VLESS route	—	依 VLESS 的 route 欄位路由。
Inbound 標籤	Inbound 標籤	inboundTag	規則適用的一個或多個 inbound 標籤（包括內建的 api 和 DNS 設定中的 DNS 標籤）。在 inbound 列表中，若 inbound 設定了獨立備注則顯示為「標籤（備注）」，否則僅顯示標籤；儲存的規則中仍僅儲存標籤。
Outbound 標籤	Outbound 標籤 / Outbound 連線	outboundTag	匹配流量的轉送目的地。
負載均衡器標籤	負載均衡器標籤 / 負載均衡器	balancerTag	提示：「透過其中一個已設定的負載均衡器路由流量」。

outboundTag 和 balancerTag 互斥：「無法同時使用 balancerTag 和 outboundTag。若同時使用，只有 outboundTag 會生效。」在同一條規則中，只能設定 outbound 標籤或負載均衡器標籤其中之一。

參考範本的内建規則

標準 config.json 的 routing 部分包含三條規則（按此順序）：

1. inboundTag: ["api"] → outboundTag: "api" — 面板統計 gRPC API 的服務規則。
2. ip: ["geoip:private"] → outboundTag: "blocked" — 封鎖私有位址範圍。
3. protocol: ["bittorrent"] → outboundTag: "blocked" — 封鎖 BitTorrent。

api → api 規則在儲存時始終自動移到位置 0（詳見 11.11），以防統計請求被上層的 catch-all 規則攔截。

規則範例。 將所有俄羅斯網站和私有網路的流量直接轉發（繞過代理），其餘流量傳送至負載均衡器。順序至關重要：「直連」規則必須置於 catch-all 之前。在 routing.rules 中：


```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

若要讓 IP 規則（`geoip:ru`）對網域請求也能生效，通常需要在頂層路由設定 `routing.domainStrategy: "IPIfNonMatch"`（詳見 11.2）。

預設路由群組（基本連線）

在「基本連線」模式中，面板可幫助從現成列表組合典型規則：

群組	欄位	提示
依協議/網站封鎖	—	「設定以防止客戶端存取特定協議」
依國家封鎖	封鎖的 IP 位址、封鎖的網域	「這些設定將根據目的地國家封鎖流量。」
直連	直連 IP 位址、直連網域	「直連意味著特定流量不會透過其他伺服器轉發。」
IPv4 規則	—	「這些設定將讓客戶端僅透過 IPv4 路由至目標網域」
WARP 規則	—	「這些選項將根據特定目的地透過 WARP 路由流量。」
NordVPN 路由	—	「這些選項將根據特定目的地透過 NordVPN 路由流量。」

MTProto inbound：透過 Xray 路由 Telegram 流量

MTProto inbound 有一個**「透過 Xray 路由」**開關（預設關閉）和一個可選的 **Outbound** 選擇器。啟用後，面板會在 Xray 設定中新增一個帶有 inbound 標籤的回環 SOCKS 橋接器，mtg 會將 Telegram 流量透過其傳輸。此後，Telegram 的外出流量由路由器管理：可在路由分頁上依 inbound 標籤用一般規則進行匹配，或透過 **Outbound** 欄位強制轉送至特定 outbound 或負載均衡器。若將 **Outbound** 留空，則由路由規則決定。

11.4. Outbounds（外出連線）

`outbounds` 列表。按鈕：**建立 Outbound**、**編輯 Outbound**。提示：「修改設定範本以定義此伺服器的外出連線」。

參考範本中包含兩個必要的 outbound：

- `protocol: "freedom", tag: "direct"` — 直連網際網路（帶有 `domainStrategy: "AsIs"` 和 `finalRules: [{action: "allow"}]`）；
- `protocol: "blackhole", tag: "blocked"` — 用於封鎖流量的「黑洞」。

Outbound 表單通用欄位

欄位	標籤	說明
標籤	標籤（提示：「唯一標籤」）	Outbound 的唯一識別碼。預留位置：「唯一標籤」。驗證：「標籤為必填項」、「標籤已被其他 outbound 使用」。
協議	—	Outbound 的類型（詳見下文）。
位址 / 連接埠	位址 / 連接埠	連線目標。位址和連接埠為必填。
傳送介面	傳送介面	外出介面的本地 IP 位址（ <code>sendThrough</code> ）。預留位置：「本地 IP」。
Target Strategy	Target Strategy	3.5.0 新增 。連線前如何解析目標域名。11 個值： <code>AsIs</code> （預設，不解析）、 <code>UseIP</code> 、 <code>UseIPv4</code> 、 <code>UseIPv6</code> 、 <code>UseIPv6v4</code> 、 <code>UseIPv4v6</code> （解析並帶 fallback）、 <code>ForceIP</code> 、 <code>ForceIPv6v4</code> 、 <code>ForceIPv6</code> 、 <code>ForceIPv4v6</code> 、 <code>ForceIPv4</code> （要求解析成功）。空白 = <code>AsIs</code> （不寫入設定）；對 <code>freedom</code> 而言，讀取該值時會回退至舊有的 <code>domainStrategy</code> （為舊版核心仍會寫入此舊鍵）。
Dialer proxy（鏈式代理）	—	提示：「透過另一個 outbound（依標籤）連接此 outbound，以建立代理鏈。留空則直連。」預留位置：「選擇要鏈接的 outbound」。透過 <code>streamSettings.sockopt.dialerProxy</code> 實現。

Dialer Proxy 下拉列表不僅顯示本地 outbound，還顯示訂閱的 outbound 標籤——因此也可以透過訂閱取得的出口建立鏈式代理。列表中仍會排除 blackhole outbound 和正在編輯的 outbound 本身。留空則直連。

支援的 outbound 協議

表單支援的協議：

- **freedom** — 直連。欄位 `settings.domainStrategy`、`finalRules`（詳見下文）、Happy Eyeballs。無法測試（「Outbound has no testable endpoint」）。
- **blackhole** — 丟棄流量。欄位**回應類型**。無法測試。
- **socks**、**http** — `settings.servers[]` 列表，包含 `address / port`；欄位**授權密碼**。對於 **http** 協議，在 **Username/Password** 欄位下方有 **Headers**（標頭）編輯器——用於發送至上游 HTTP 代理 CONNECT 請求的鍵值對。這些標頭在重新開啟和儲存 outbound 時會保留（之前會遺失）。請注意：僅套用設定層級的標頭（`settings.headers`）；xray-core 會忽略個別伺服器層級的標頭。
- **vmess** — `settings.vnext[]`（`address / port`）。
- **vless** — `settings.address / settings.port`。
- **trojan**、**shadowsocks** — `settings.servers[]`。
- **wireguard** — `settings.peers[]`，包含 `endpoint`，以及金鑰（詳見 11.8）。
- **hysteria** — `settings.address / settings.port`（UDP 傳輸）。

對於 **loopback** 類型的 outbound，可使用 **Sniffing** 區塊，其參數與 inbound 相同：啟用、**destOverride**、**Metadata Only**、**Route Only** 以及排除網域列表。

在 **Hysteria2** 的 **UDP 遮罩**（`FinalMask`）中，可使用額外模式。**Salamander** 遮罩有一個 **Mode** 選擇器，值為 **Salamander** 和 **Gecko**：Gecko 模式會為封包添加隨機填充，帶有 **Min/Max** 大小欄位（`packetSize`，範圍

1–2048，預設 512–1200）——這可防止基於封包長度的指紋識別。**Realm** 遮罩（UDP hole-punching）新增了一個可選的 **TLS Config** 區塊，包含 **Server Name**（SNI）、**ALPN**（h3/h2/http/1.1）、**Fingerprint**（uTLS）和允許不安全開關欄位。

自 3.5.0 起新增一種 **TCP 遮罩類型**——**XMC (Minecraft)**（xmc）：將流量偽裝成 Minecraft 協議。欄位：**Hostname**（選填；「握手中模擬的伺服器位址」）、**Username**（選填清單；「向探測者展示的玩家名稱；核心預設使用 Dream」）和必填的 **Password**（16 個字元， 按鈕自動生成；「混淆密碼」）。重要：**TCP Finalmask** 與 **REALITY** 的組合在儲存時會被拒絕——它會在首次連線時使 Xray-core 崩潰（錯誤：「Finalmask is not supported with REALITY security...」；見 Xray-core#6453）。UDP 遮罩可與 REALITY 並用；先前儲存的錯誤組合會在生成設定時自動「修復」（捨棄遮罩）。

範例：透過上游 SOCKS 建立代理鏈。 Outbound upstream 連接至外部 SOCKS5 代理，而 chained 透過其傳送流量（dialerProxy），形成代理鏈。在 outbounds 中：

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

現在，帶有 outboundTag: "chained" 的路由規則將會透過 upstream 把流量輸出至網際網路。

從分享連結匯入 outbound

Outbound 可從分享連結（vless://、vmess:// 等）匯入。匯入時，透過連結的 extra= 區塊傳遞的 xmux（XHTTP）多路複用器設定也會一併保留：匯入後，其值會填入所建立 outbound 的 XMUX 子表單。

Mux（多路複用）欄位

最大並行數、最大連線數、最大重複使用次數、最大請求數、最大重複使用秒數、保活週期。這些參數設定 outbound 的 mux/XUDP 行為。

Sockopts（Socket 設定）

Sockopts 群組：保活間隔、Mark（fwmark）、介面、僅 IPv6、接受 proxy protocol、Proxy protocol、TCP 使用者逾時（毫秒）、TCP 保活閒置（秒）。dialer-proxy 鏈式設定也在此處。

Freedom finalRules（覆寫私有 IP 封鎖）

對於 freedom outbound，可使用最終規則群組：

欄位	標籤	說明
overrideXrayPrivateIp	覆寫 Xray 預設的私有 IP 封鎖	解除 Xray 內建的私有 IP 外出限制。
action	動作	allow（如參考範本： <code>finalRules: [{action: "allow"}]</code> ）、redirect（Redirect）或其他。
blockDelay	封鎖延遲（毫秒）	丟棄連線前的延遲。
redirect / fragment	Redirect / Fragment	流量重定向和分片動作。

fragment 遮罩：分段 Lengths 和 Delays

在 fragment 遮罩（TCP 的 FinalMask 中的 fragment 類型）中，單一的 Length 和 Delay 欄位被替換為 Lengths 和 Delays 列表：可為每個片段設定獨立的長度範圍（例如 100–200）和毫秒延遲（例如 10–20 或 0）。列表行可以新增和刪除；之前儲存的單一值會自動遷移為單元素陣列。

其他表單欄位

- UDP over TCP 和 UoT 版本 – 用於類 shadowsocks 協議。
- 無 gRPC 標頭、Uplink Chunk 大小 – gRPC 傳輸參數。
- TLS/uTLS 欄位：驗證 peer 名稱、Pinned SHA256、Short ID、Vision testpre，預留位置「伺服器名稱」。

測試 outbound

按鈕：測試、全部測試。狀態：正在測試連線...、測試成功、測試失敗、無法測試 outbound 連線。結果：測試結果，延遲以毫秒為單位。

兩種模式（提示：「TCP: 快速僅撥號探測。HTTP: 透過 xray 進行完整請求。」）：

- TCP（mode=tcp）– 對 host:port 進行簡單撥號，對所有端點並行執行，逾時約 5 秒。僅驗證 TCP 可達性，不驗證代理協議。對 freedom / blackhole / blocked 標籤將返回「Outbound has no testable endpoint」。
- HTTP（mode=http 或空白）– 啟動一個臨時 Xray 實例，執行真實的 HTTP 請求（探測 URL = 伺服器的 outboundTestUrl），測量真實延遲。此為可靠但成本較高的模式：透過全域鎖定序列化（「Another outbound test is already running, please wait」）。單次嘗試逾時為 10 秒，結果等待視窗為 15 秒（已增加，以防止在緩慢或隧道連線上的健康 outbound 被標記為「Failed」）。失敗時，實際原因（DNS 錯誤、連線被拒、截止日期超時、TLS 錯誤等）會寫入面板/Xray 日誌，逾時的一般訊息會指向該日誌。
- Real delay（3.5.0 新增）– 切換器的第三種模式。使用同一個臨時 Xray 實例，但報告**「冷」請求的完整時間（含隧道建立）**– 這一數字接近客戶端應用程式顯示的數值。切換器提示：「TCP: 快速僅

撥號探測。HTTP：透過 xray 進行完整請求。Real delay：包含連線建立的完整時間」。自 3.5.0 起，一般 HTTP 模式改以「暖」（keep-alive）連線測量，因此數字較低，更接近單次請求的延遲。

UDP 協議（wireguard、hysteria）和 UDP 傳輸（kcp、quic、hysteria）始終以 HTTP 模式測試，即使請求的是 TCP——裸 UDP 撥號無法區分「活躍」和「死亡」端點。對於 wireguard，測試設定中強制設定 noKernelTun: true。

Egress 與 Country 欄位（自 3.5.0 起）

HTTP 或 Real delay 測試成功後，面板會透過同一臨時路由請求 Cloudflare trace 中繼資料，並在 outbound 清單中顯示兩個欄位：**Egress**——出口 IP（IPv4/IPv6，各自隱藏在「眼睛」圖示後；尚未測試時顯示破折號並附提示「執行 HTTP 測試以顯示出口 IP 與國家」）與 **Country**——出口國家的旗幟與名稱；若出口經由 Cloudflare WARP，會加上橘色 WARP 標記。TCP 測試不會填入這些欄位；在行動裝置上，資料顯示在 outbound 卡片中。

批量檢測與階段分解

HTTP 模式下的測試和**全部測試**會為一批 outbound 啟動一個共用的臨時 Xray 實例，為每個 outbound 建立一個回環 SOCKS inbound 及規則，並透過其並行發送真實 HTTP 請求；**全部測試**以批次方式檢測 outbound。**全部測試**也會檢測從訂閱獲取的 outbound（「來自訂閱」表格，唯讀）——其行也會以測試結果高亮顯示。此時，freedom（「direct」）和 dns outbound 在任何模式下都不會被測試（它們不是代理）：其測試按鈕不可用，**全部測試**會跳過它們，而且即使直接呼叫 API，伺服器端保護也禁止對其進行 HTTP 測試。除成功/錯誤外，彈出式結果還會顯示 HTTP 回應狀態以及各階段的時間分解：**Proxy connect**（代理連線）、**TLS via outbound**（透過 outbound 的 TLS）和 **First byte**（首位元組時間）——這有助於了解延遲或故障發生在哪個步驟。

Outbound 流量統計

面板依標籤維護流量計數器（up/down/total）。重置按鈕可重置特定標籤或所有標籤（tag = "alltags"）的計數器。帳戶資訊和 **Outbound 連線狀態**欄位顯示摘要。

11.5. 負載均衡器（Balancers）

routing.balancers 列表。按鈕：**建立負載均衡器**、**編輯負載均衡器**。

在「負載均衡器」分頁上有即時狀態欄：**Live Target** 顯示正在執行的 Xray 中負載均衡器的當前活躍目標，**Override** 允許手動覆寫目標選擇（**Auto (strategy)** 值恢復依策略選擇）。狀態透過單獨的按鈕更新。若負載均衡器尚未在正在執行的 Xray 中啟用，面板會提示先儲存變更或啟動 Xray。

欄位	標籤	說明
標籤	標籤（提示：「唯一標籤」）	唯一識別碼。預留位置：「唯一負載均衡器標籤」。驗證：「標籤為必填項」、「標籤已被其他負載均衡器使用」。

欄位	標籤	說明
選擇器	選擇器	Outbound 標籤列表（依子字串），負載均衡器在其中選擇出口。至少需選擇一個：「請至少選擇一個 <i>outbound</i> 」。
Fallback	Fallback	若沒有選擇器匹配時的備用標籤。自 3.5.0 起也可選擇另一個負載均衡器（清單中此類選項標有藍色「Balancer」標籤，提示「選擇另一個負載均衡器作為 fallback」）。Xray 原生不支援此功能，因此面板會自行建立隱藏的 loopback outbound <code>_bl_<[?]></code> 及路由規則（路徑：負載均衡器 → loopback → 伺服器 → 目標負載均衡器 → outbound；表單中會顯示多一個「跳點」的警告）。具備循環防護（錯誤「無法選擇——會產生循環依賴」，清單中的選項會被停用並提示循環路徑），也不允許刪除被用作 fallback 的負載均衡器（「無法刪除——正被以下負載均衡器用作 fallback: ...」）。 <code>_bl_</code> 標籤前綴為保留字（標籤中禁止使用）；服務性的 <code>_bl_</code> 物件會從 Outbounds/Routing 清單中隱藏，而 Fallback/Live Target/Override 欄位顯示的是目標負載均衡器的名稱，而非 loopback 標籤。
策略	策略	選擇演算法（詳見下文）。

策略與監測參數

策略（`strategy.type`）決定負載均衡器如何從選擇器中選擇 outbound。Xray-core 的值：`random`（隨機）、`roundRobin`（輪詢）、`leastPing`（依 observatory 結果選最低延遲）、`leastLoad`（最低負載）。`leastLoad/leastPing` 使用 `strategy.settings` 中的參數：

欄位	標籤	說明
<code>expected</code>	期望值	預留位置：「最佳節點數量」——目標活躍節點數。
<code>maxRtt</code>	最大 RTT	選擇候選節點時可接受的 RTT 上限。
<code>tolerance</code>	容差	比較延遲/負載時的容差。
<code>baselines</code>	Baselines	用於節點分組的延遲閾值。
<code>costs</code>	Costs	個別標籤的權重係數。

策略範例。`strategy` 區塊位於負載均衡器內部（在 JSON 中與 `tag` 和 `selector` 並列）：

```
"strategy": { "type": "random" }           // [?] [?] [?] [?] [?] [?] [?] [?]
"strategy": { "type": "roundRobin" }       // [?] [?] [?] [?] [?] [?] [?]
"strategy": { "type": "leastPing" }        // [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
```

`leastLoad` 的參數在 `settings` 中設定：


```

"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regex": false, "match": "proxy-premium", "value": 0.1 },
      { "regex": true, "match": "^proxy-cheap-.+$", "value": 5 }
    ]
  }
}

```

運作方式（範例）。 假設監測器測得各出口的延遲：A = 250 ms、B = 280 ms、C = 700 ms、D = 1500 ms。使用上述設定，選擇過程如下：

1. **maxRTT: "1s"** – 延遲超過 1 秒的出口被淘汰：D（1500 毫秒）出局。剩餘 A、B、C。
2. **baselines + expected** – 出口依延遲閾值分組，選取包含至少 expected 個出口的最小閾值。閾值 500ms 已包含 A 和 B —— 共 2 個 (= expected)，因此選擇 {A、B} 組。C（700 毫秒）不進入選擇，只要有足夠的快速出口（它是「熱備份」）。
3. **tolerance: 0.05** – 在所選群組內，延遲差異不超過 5% 的出口被視為等價，負載平均分配。A（250）和 B（280）相差約 12% (> 5%)，因此在其他條件相同時優先選擇較快的 A；若差異在 5% 以內，流量才會同時走 A 和 B。
4. **costs** – 在比較前調整個別出口的「成本」：較小的 value 使出口更具吸引力，較大的則相反。在此範例中，proxy-premium 獲得 0.1（變得「更便宜」，更優先被選擇），而所有 proxy-cheap-*（透過正規表達式，regex: true）獲得 5（變得「更貴」，最後才使用）。這樣可以在不強制排除出口的情況下，對其進行軟優先排序。

結果：流量主要走 A（若延遲接近則與 B 平均分配），C 保持為備用，D 被排除，直到其 RTT 降至 maxRTT 以下。

監測器：observatory 和 burstObservatory（leastPing / leastLoad 的測量數據）

leastPing 和 leastLoad 策略本身不測量任何東西——它們需要每個 outbound 的延遲和可用性數據。這些數據由**監測器**（observatory）收集：它定期「ping」每個被監測的 outbound 並記錄回應時間和可用性。相同數據顯示在**「Observatory」分頁上（狀態活躍 / 不可用**、「最後活躍」、「最後嘗試」）。

自 3.4.2 起，「負載均衡器」頁面拆分為「負載均衡器設定」（表格與新增）與「**Observatory**」分頁，且監測器改以**結構化表單編輯**（先前的原始 JSON 編輯器已移除）。面板會視需要自動建立與刪除監測器：一般 observatory —— 用於 **Least Ping** 策略；進階 burstObservatory —— 用於 **Least Load**，以及帶有指定 fallbackTag 的 **Random/Round-robin**（xray-core 對此有要求；當清空最後一個 fallbackTag 時，監測器會被移除）。建立或刪除監測器需要完整重啟 Xray。

有兩種變體：

- **observatory** – 簡單型：subjectSelector + probeURL + probeInterval。
- **burstObservatory** – 進階型，透過 pingConfig 進行精細的 ping 設定；適用於多個出口。

burstObservatory 區塊範例：

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

欄位說明：

欄位	功能
subjectSelector	要監測的 outbound 標籤前綴列表。Xray 會取所有標籤以指定字串開頭的 outbound。在此範例中，監測 WS-SE...、WS-FR...、WS-PL... 出口。這些標籤必須與負載均衡器的選擇器中選擇的標籤一致。
pingConfig.destination	透過每個 outbound 請求以測量延遲的 URL。使用回應 204 且無正文的「輕量級」頁面——例如 https://www.google.com/generate_204 。從請求到收到回應的時間即為測量到的延遲。
pingConfig.interval	每個 outbound 的 ping 頻率。持續時間字串："1m" 表示每分鐘，也可用 "30s"、"5m" 等。頻率越高，數據越新鮮，但後台流量也越多。
pingConfig.connectivity	(可選) 用於檢測伺服器本身基本連通性的 URL。若此 URL 不可達——說明問題出在伺服器網路上，監測器不會將 outbound 標記為不可用（防止本地故障時的誤報）。通常也是回應 204 的端點。
pingConfig.timeout	等待單次 ping 回應的時間，超過後視為嘗試失敗（例如 "5s"）。
pingConfig.sampling	每個 outbound 保留並取平均的最近測量次數。2 表示考慮最近兩次 ping（平滑偶發抖動）。
pingConfig.httpMethod	3.4.2 新增。探測的 HTTP 方法：HEAD（預設）或 GET。

在「**Observatory**」分頁中，相同參數以表單（而非 JSON）設定。一般 observatory 為 **Watched Outbounds**（subjectSelector，唯讀）、**Probe URL**（probeURL，預設 https://www.google.com/generate_204）、**Probe Interval**（probeInterval，1m）與 **Enable Concurrency**（enableConcurrency，預設啟用）。burstObservatory 則為上述欄位再加上新的「**HTTP 方法**」。若負載均衡器同時有兩個監測器，則以分段切換器在 Observatory 與 Burst 表單之間切換。

如何將所有設定關聯起來：

1. 在「**Observatory**」分頁中設定監測器參數（會依所選策略自動建立；subjectSelector 跟隨負載均衡器的選擇器）。
2. 建立負載均衡器：策略 = leastPing，在選擇器中指定相同的 outbound 標籤（WS-SE、WS-FR、WS-PL）。
3. 透過路由規則將流量指向它（負載均衡器標籤欄位，詳見 11.3）。
4. 重啟 Xray。*「Observatory」*分頁上將顯示出口狀態，負載均衡器開始從活躍節點中選擇最快的。

在同一條規則中不能同時設定 `balancerTag` 和 `outboundTag` ——只有 `outboundTag` 會生效。

刪除 `outbound` 或負載均衡器時會發生什麼（自 3.4.2 起）

刪除 `outbound` 或負載均衡器時，面板會在同一動作中清除路由中對它的引用，並在確認對話框中顯示影響預覽（標題「При удалении также обновится ваша маршрутизация:」，接著逐條規則顯示——「удалено (не осталось назначения)」或「оставлено (теперь использует ...)」，以及「Балансировщик ... удалён (не осталось целей)」）。如此可避免「懸空」引用（`balancerTag` / `outboundTag` / `dialerProxy`），先前這類引用可能導致 `xray-core` 在下次重啟時無法啟動。刪除負載均衡器：規則保留其 `outboundTag`，否則被刪除。刪除 `outbound`：其標籤會從負載均衡器的選擇器與 `fallbackTag` 中移除，清除其他 `outbound` 的 `dialerProxy`，清空的負載均衡器連同已失去目標的規則一併刪除，並重建監測器。

11.6. DNS

`dns` 部分。啟用：啟用 **DNS**（提示：「啟用內建 `DNS` 伺服器」）。

自 3.5.0 起，**私有 IP** 上的 `DNS` 伺服器（例如同一 `docker` 網路中自建的 `AdGuard/Pi-hole`）可「開箱即用」：面板會自動插入並維護一條受管的放行規則（`direct`，且限定於該 `DNS` 伺服器的連接埠），置於封鎖規則 `geoip:private` 之前，並在 `dns.servers` 變更時同步。先前 `Xray` 自身的 `DNS` 查詢會被該規則靜默封鎖，每個新域名增加約 4 秒延遲；不再需要手動規則（手動建立的規則不受影響）。

DNS 通用參數

欄位	標籤	JSON	說明 / 提示
<code>tag</code>	DNS 標籤名稱	<code>dns.tag</code>	「此標籤將在路由規則中作為 <code>inbound</code> 標籤使用。」允許透過 <code>inboundTag</code> 路由 <code>DNS</code> 請求本身。
<code>clientIp</code>	客戶端 IP	<code>dns.clientIp</code>	「用於在 <code>DNS</code> 請求期間通知伺服器指定的 <code>IP</code> 位置」（ <code>EDNS Client Subnet</code> ）。
<code>strategy</code>	請求策略	<code>dns.queryStrategy</code>	「域名解析的整體策略」。值： <code>UseIP</code> 、 <code>UseIPv4</code> 、 <code>UseIPv6</code> 。
<code>disableCache</code>	停用快取	<code>dns.disableCache</code>	「停用 <code>DNS</code> 快取」。
<code>disableFallback</code>	停用備用 DNS	<code>dns.disableFallback</code>	「停用備用 <code>DNS</code> 請求」。
<code>disableFallbackIfMatch</code>	匹配時停用備用 DNS	<code>dns.disableFallbackIfMatch</code>	「當 <code>DNS</code> 伺服器的網域列表匹配時停用備用 <code>DNS</code> 請求」。

欄位	標籤	JSON	說明 / 提示
enableParallelQuery	啟用並行查詢	—	「啟用向多個伺服器並行發送 DNS 查詢以加快解析速度」。
useSystemHosts	使用系統 Hosts	dns.useSystemHosts	「使用已安裝系統的 hosts 檔案」。

dns 區塊範例。 Google 網域的查詢透過 Cloudflare DoH 伺服器解析，其餘透過 1.1.1.1；Google 查詢期望的是非私有 IP。在設定頂層：

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

字串形式的伺服器（"1.1.1.1"）不含欄位，是所有其他網域的預設伺服器。標籤 dns-inbound 之後可作為路由規則中的 inboundTag，將 DNS 請求本身透過所需的 outbound 路由。

過期記錄快取

欄位	標籤	說明
serveStale	使用過期記錄	「在後台更新時從快取返回過期結果」。
serveExpiredTTL	過期 TTL	「快取過期記錄的有效期（秒）；0 = 永久」。

DNS 伺服器（dns.servers 列表）

按鈕：建立 DNS、編輯 DNS、全部刪除（確認：「所有 DNS 伺服器將從列表中刪除。此操作無法撤銷。」）。範本：使用範本，DNS 範本視窗，包括家庭預設值。

點擊 DNS 伺服器記錄（以及 Fake DNS 記錄）上的編輯 DNS 時，編輯視窗會填入已儲存的伺服器值，而非預設值。

DNS 伺服器欄位：

欄位	標籤	說明
address	—	DNS 位址（IP、DoH URL、localhost、fakedns 等）。
domains	網域	使用此伺服器的網域列表。
expectIPs	期望 IP	僅在 IP 屬於列表時才接受回應。

欄位	標籤	說明
unexpectedIPs	非期望 IP	丟棄含有指定 IP 的回應。
skipFallback	跳過 Fallback	不將此伺服器用作 fallback。
finalQuery	最終查詢	將此伺服器標記為鏈中的最終查詢。
timeoutMs	逾時（毫秒）	伺服器請求逾時時間。

Hosts（靜態記錄）

Hosts 群組（`dns.hosts`）。新增 **Host** 按鈕；空狀態**未定義 Host**。欄位：網域（預留位置：「網域（例如 `domain:example.com`）」）和值（預留位置：「IP 或網域——輸入後按 `Enter`」）。

DNS 日誌

詳見 11.10：日誌部分的 **DNS 日誌** 旗標（`dnsLog`）。

11.7. Fake DNS

`fakedns` 部分。按鈕：建立 **Fake DNS**、編輯 **Fake DNS**。

欄位	標籤	說明
ipPool	IP 池子網路	發放虛擬 IP 的 CIDR 範圍（例如 <code>198.18.0.0/15</code> ）。
poolSize	池大小	循環池中保留的位址數量。

Fake DNS 與 inbound 上的 sniffing 搭配使用：核心向客戶端發放虛擬 IP，記憶網域↔IP 的對應關係，並在路由時還原網域。若要讓 Fake DNS 生效，位址為 `fakedns` 的 DNS 伺服器必須新增至 DNS 伺服器列表中。

範例：Fake DNS + DNS 伺服器的組合。 首先定義虛擬位址池，然後新增 DNS 伺服器 `fakedns`，使網域查詢從該池中獲得 IP：

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

此外，還需在 inbound 上啟用帶有 `destOverride: ["fakedns"]` 的 sniffing，否則核心無法取得還原真實網域所需的資訊。

11.8. WireGuard / WARP / NordVPN

WireGuard 欄位 (wireguard)

欄位	標籤	說明
secretKey	私密金鑰	本地介面的私鑰。
publicKey	公開金鑰	peer 的公鑰。
psk	預共用金鑰	PreShared Key (可選)。
allowedIPs	允許的 IP 位址	路由至隧道的 IP 範圍。
endpoint	端點	peer 的 host:port。
domainStrategy	網域策略	WireGuard outbound 的解析策略。

Cloudflare WARP (warp)

整合使用 API <https://api.cloudflareclient.com/v0a4005> (client-version a-6.30-3596)。控制器動作 (/xray/warp/:action)：config、reg、license、data、del。

步驟：

1. **建立 WARP 帳戶** → reg：面板生成/接受私鑰 (privateKey) 和公鑰 (publicKey)，在 Cloudflare 上註冊裝置，並將 access_token、device_id、license_key、private_key (以及 client_id) 儲存在 warp 設定中。
2. **WARP / WARP+ 授權金鑰** → license：設定 26 字元的 WARP+ 金鑰 (預留位置：「26 字元 WARP+ 金鑰」)。失敗時：「無法設定 WARP 授權。」若尚未獲取設定：「請先取得 WARP 設定。」
3. **帳戶資訊**：裝置名稱、裝置型號、裝置已啟用、帳戶類型、角色、WARP+ 數據、配額、使用量。
4. **新增 Outbound** – 使用獲取的金鑰和 Cloudflare 端點建立 WireGuard outbound。
5. **刪除帳戶** → del：清除已儲存的 WARP 數據。

NordVPN (nord / nordvpn)

整合使用 NordLynx (= WireGuard)。控制器動作 (/xray/nord/:action)：countries、servers、reg、setKey、data、del。

步驟：

1. **存取憑證** → reg：面板向 api.nordvpn.com 請求 NordLynx 憑證並提取 nordlynx_private_key。將 private_key 和 token 儲存在 nord 設定中。替代方案——setKey：直接輸入私鑰 (不能為空)。
2. **國家** → countries 載入國家列表；城市 (或所有城市)。
3. **伺服器** → servers 載入所選國家的伺服器 (countryId 被驗證為數字——防止注入攻擊)。篩選器：僅顯示負載 > 7% 的伺服器。若沒有伺服器：「未找到所選國家的伺服器」。若伺服器沒有 NordLynx 公鑰：「所選伺服器未報告 NordLynx 公鑰。」
4. **建立/更新 outbound**：提示「NordVPN outbound 已新增」 / 「NordVPN outbound 已更新」。

IPv4 優先與使用者空間 TUN

WARP 和 NordVPN 精靈生成的 WireGuard outbound 使用 `domainStrategy: "ForceIPv4v6"`（IPv4 優先，在僅 IPv6 主機上回退至 IPv6），而非 `ForceIP`——這消除了在半設定 IPv6 的主機上選擇 Cloudflare 端點 AAAA 記錄時握手「掛起」的問題。此外，它們啟用使用者空間 TUN（`noKernelTun: true`）而非核心 TUN：後者需要許可權和 fwmark 路由，在許多 VPS 上會悄然失敗，而面板的內建連線檢查始終透過使用者空間 TUN 進行測試——現在實際流量和檢查走相同路徑。此變更僅對新增或重置的 outbound 生效；已儲存的範本保留其設定。

11.9. 反向代理與 TUN

Reverse（反向代理）

Xray 設定的 `reverse` 部分。Outbound 表單中有切換至反向代理類型的開關。按鈕：[建立反向代理](#)、[編輯反向代理](#)。

欄位	標籤	說明
類型	類型	Bridge 或 Portal – Xray 反向代理的兩種角色。
網域	網域	bridge↔portal 配對的服務網域標識。
標籤 / 連線	標籤 / 連線	連結 bridge 和 portal 的標籤。
Reverse Tag	反向代理標籤	提示：「用於簡易 VLESS 反向代理的 <i>outbound</i> 標籤。留空則停用。」預留位置：「 <i>outbound</i> 標籤（空白 = 停用）」。實現簡化版 VLESS 反向代理。

Outbound 表單中也有反向流欄位：**反向 sniffing**、**Workers**、**Reserved**、**最小上傳間隔（毫秒）**、**最大上傳大小（位元組）**。

TUN（tun）

欄位	標籤	說明	預設值
name	—	「TUN 介面名稱。」	xray0
mtu	—	「最大傳輸單元。數據封包的最大大小。」	1500
userLevel	使用者等級	「透過此 <i>inbound</i> 建立的所有連線都將使用此使用者等級。」	0

11.10. 日誌與統計（Stats, metrics）

日誌（log）

提示：「日誌可能會降低伺服器速度。僅在需要時啟用所需的日誌類型！」參考範本的 `log` 部分：
`access: "none"`、`error: ""`、`loglevel: "warning"`、`dnsLog: false`、`maskAddress: ""`。

欄位	標籤	JSON	說明	預設值
logLevel	日誌等級	loglevel	「錯誤日誌的日誌等級...」值：debug、info、warning、error、none。	warning
accessLog	存取日誌	access	「存取日誌檔案的路徑。特殊值「none」停用存取日誌。」	none
errorLog	錯誤日誌	error	「錯誤日誌檔案的路徑。特殊值「none」停用錯誤日誌。」	"" (預設)
dnsLog	DNS 日誌	dnsLog	「啟用 DNS 查詢日誌」	false
maskAddress	位址遮罩	maskAddress	「啟用後，日誌中的真實 IP 位址將被替換為遮罩位址。」	"" (停用)

統計 (stats / policy)

統計群組。在 policy.system 和 policy.levels 中啟用計數器。參考範本中：statsInboundUplink: true、statsInboundDownlink: true、statsOutboundUplink: false、statsOutboundDownlink: false；等級 0 為 statsUserUplink: true、statsUserDownlink: true。

欄位	標籤	說明	預設值
statsInboundUplink	Inbound 上行統計	「啟用所有 inbound 代理的外出流量統計收集。」	true
statsInboundDownlink	Inbound 下行統計	「啟用所有 inbound 代理的進入流量統計收集。」	true
statsOutboundUplink	Outbound 上行統計	「啟用所有 outbound 代理的外出流量統計收集。」	false
statsOutboundDownlink	Outbound 下行統計	「啟用所有 outbound 代理的進入流量統計收集。」	false

客戶端和 inbound 的統計（上行/下行）是儀表板和客戶端顯示流量的基礎；不建議停用。Outbound 統計預設關閉，僅在需要依 outbound 標籤追蹤流量時才需要啟用。

Metrics

參考範本中存在 metrics 部分（listen: "127.0.0.1:11111"、tag: "metrics_out"）以及對應的 API metrics_out。面板使用此 listener 收集指標和 observatory 快照：它從範本中解析 metrics.listen，查詢 /debug/vars 並依標籤彙整延遲歷史。若您修改 metrics.listen 的位址/連接埠，面板將連接至新位址；刪除 metrics 部分將停用 observatory 圖表的收集。

HTTP 模式的 outbound 測試會啟動一個獨立的臨時 Xray 實例，其 metrics listener 使用隨機連接埠——這與主設定中的 listener 不同。

11.11. 儲存、重啟與自動轉換

按鈕

按鈕	動作
儲存	POST /xray/update : 驗證並儲存範本 + <code>outboundTestUrl</code> 。
重啟 Xray	以儲存的設定重新載入服務。確認：「重啟 xray?」 / 「以儲存的設定重新載入 xray 服務。」

提示：成功——「Xray 已成功重啟」、「Xray 已成功停止」；錯誤——「重啟 Xray 時發生錯誤。」、「停止 Xray 時發生錯誤。」 **Xray 重啟輸出視窗**顯示核心的診斷輸出。

熱套用變更（無需完整重啟）

Inbound、outbound 和路由規則的變更可「即時」套用：點擊**儲存**時，面板計算新舊設定之間的差異，並僅透過 Xray 的 gRPC API (HandlerService/RoutingService) 套用變更的部分，而無需重啟程序。只有當沒有熱重載 API 的部分（`log`、`dns`、`policy`、`observatory` 等）發生變更時，才會自動執行完整重啟。因此，Xray 頁面上不需要單獨的「重啟」按鈕——**儲存**本身即會套用變更。在必要時，核心仍會自動重啟（另見訂閱更新和 WARP 輪換時的自動重載）。

恢復預設範本

端點 GET /xray/getDefaultJsonConfig 返回參考範本（二進位檔中內建的 `config.json`）。可用於將設定重置為原廠設定。

儲存時的自動轉換

儲存 Xray 設定時，面板按此順序執行：

- 去除包裝** – 移除形如 `{ "xraySetting": <JSON>, "inboundTags": ..., "outboundTestUrl": ... }` 的包裝（若它們意外進入值中，否則每次儲存都會累積層數）。最多移除 8 層。
- 設定驗證** – 將 JSON 解析為 Xray 設定結構；若出錯——以「*xray template config invalid*」拒絕。
- 保證統計規則** – 規則 `inboundTag: ["api"]` → `outboundTag: "api"` 強制移至 `routing.rules` 的位置 0（若不存在則新增）。這確保面板的 gRPC 統計請求不會被上層的 catch-all 規則攔截（否則在代理正常運行時，客戶端可能顯示離線且流量為零）。

由於第 3 點，請勿嘗試刪除或移動 `api` → `api` 規則——面板在下次儲存時無論如何都會將其還原。這是統計基礎設施的服務設施，而非使用者路由。

11.12. 訂閱的 Outbound（含自動更新）

從 3.3.0 版開始，面板能夠直接從訂閱 URL 匯入 `outbound` ——與 VPN 提供商為客戶端應用提供的格式相同。訂閱在後台定期重新讀取，因此伺服器上的 `outbound` 集合無需手動編輯設定範本即可保持最新。自

3.5.0 起，帶 query 參數（`?plugin=`、`?type=`）或結尾 `/` 的 `ss://` 連結（SIP002）能正確解析連接埠（先前會靜默變為 `0`）；連接埠確實無效的連結會被跳過，而不會以損壞的形式匯入。

此部分的說明：「從遠端訂閱 URL（`vmess/vless/trojan/ss/...`）匯入 `outbound`。標籤保持不變以供負載均衡器和路由規則使用。自動更新。」此部分位於 Xray 頁面上，在 `outbound` 設定面板上方。

運作機制

訂閱與 Xray 設定範本分開儲存。範本**永遠不會被覆寫**：從訂閱取得的 `outbound` 在每次生成 Xray 設定時即時新增至最終設定中。

新增訂閱

「新增訂閱」表單包含以下欄位：

欄位	鍵	預設值	說明
訂閱 URL	<code>url</code>	—（必填）	訂閱位址。預留位置：「 <code>https://...</code> （base64 連結列表）」。 僅接受 HTTP(S)；位址會進行安全性驗證。
備注	<code>remark</code>	空白	任意標籤（預留位置「例如 HK 節點」）。
標籤前綴	<code>tagPrefix</code>	<code>subN-</code>	匯入的 <code>outbound</code> 標籤的起始前綴。若留空，面板會自動選擇最小的可用編號，格式為 <code>sub1-</code> 、 <code>sub2-</code> 等。
更新間隔	<code>updateInterval</code>	600 秒（10 分鐘）	訂閱重新讀取的頻率。UI 中以小時/分鐘設定。
已啟用	<code>enabled</code>	是（ <code>true</code> ）	只有已啟用的訂閱才會進入設定並自動更新。
允許私有位址	<code>allowPrivate</code>	否（ <code>false</code> ）	允許 <code>localhost</code> 、LAN 和私有 IP 的 URL。預設關閉以防範 SSRF——僅對受信任的本地來源啟用。
置於手動 <code>outbound</code> 之前	<code>prepend</code>	否（ <code>false</code> ）	若啟用，此訂閱的 <code>outbound</code> 將放置在範本手動 <code>outbound</code> 之前，其中一個可成為預設 <code>outbound</code> 。否則，它們添加至之後。

「預覽」按鈕（`POST /outbound-subs/parse`）允許在儲存前下載並解析 URL，預覽將獲得哪些 `outbound` 和標籤；此操作不寫入資料庫。若 URL 未識別到任何內容，顯示「此 URL 未找到 `outbound`。」

多個訂閱在 `outbound` 整體列表中的順序由優先順序（`priority`）決定，可透過上移/下移箭頭更改（`POST /outbound-subs/:id/move`）。

接受的訂閱格式

URL 回應正文的處理方式：

- 內容首先嘗試作為 **base64** 解析（標準和 URL-safe 變體，自動補全填充和去除空格/換行）。若是 base64 則解碼；否則原樣使用。
- 然後正文按行分割。每個不以 `#` 開頭的非空行被解析為連結。無法識別的行（注釋、不支援的協議）靜默跳過。

- 支援的連結方案：`vmess://`、`vless://`、`trojan://`、`ss://`（Shadowsocks）、`hysteria2://` / `hy2://`、`wireguard://` / `wg://`。

也就是說，接受大多數提供商使用的「base64 編碼的連結列表」格式的正常訂閱。

穩定標籤

每個連結都會計算一個穩定的「身份」（不含備註片段的核心 URI；vmess 的情況下為不含 `ps` 欄位的內部 JSON）。「身份 → 標籤」的對應關係會被保留，在下次更新時，相同的伺服器會獲得相同的標籤，即使備註或次要參數發生了變化。這是為了確保負載均衡器和路由規則在更新後繼續正常運作：

- 負載均衡器/規則中的精確標籤將繼續指向相同的伺服器。
- 前綴/萬用字元選擇器（例如 `hk-*`）會自動接收訂閱之後返回的新伺服器——這是「訂閱節點池」的推薦方式。
- 若伺服器從訂閱中消失，其標籤只會從最終的 `outbound` 陣列中消失；若負載均衡器有 `fallbackTag`，Xray 將使用它。
- 若提供商更改了伺服器的 UUID/主機/憑證，身份就會改變——這被視為具有新標籤的新 `outbound`。

同一批次內，標籤透過 `-N` 後綴去重。訂閱的標籤保留非 ASCII 字元（例如西里爾文）並保持可讀性：Unicode 字母和數字在 slug 中保留，標點符號替換為連字號——來自西里爾文名稱的標籤不再只剩數字。

自動更新的運作方式

- 訂閱更新的後台任務**每 5 分鐘**按計畫執行一次。
- 每次執行時，它遍歷所有已啟用的訂閱，只更新那些自身間隔已到期的訂閱：若訂閱從未更新過，或自上次更新以來已過去至少其 `updateInterval` 時間，則更新該訂閱。因此，任務頻繁檢查訂閱，但每個特定訂閱的重新讀取頻率不超過其 `updateInterval`（預設 10 分鐘）。UI 中有對應的提示說明這一點。
- 更新過程：URL 重新進行公開安全性驗證（若訂閱未設定 `allowPrivate`，則阻止私有位址），請求透過面板的代理客戶端發送，帶有標頭 `User-Agent: 3x-ui-outbound-sub/1.0`。重定向鏈限制在 10 跳，每一跳也會進行私有性檢查（SSRF 保護）。期望 HTTP 200；否則記錄錯誤。
- 成功解析後，儲存結果，設定最後更新時間，清除錯誤。發生錯誤時，錯誤文字在 UI 中顯示為「最後錯誤」，之前取得的 `outbound` 繼續有效。
- 若至少有一個訂閱實際更新，任務會將 Xray 標記為需要重啟，並傳送 UI 失效訊號，使介面拉取新的 `outbound`。Xray 的實際重載發生在管理器的下一個 30 秒週期。

手動更新單個訂閱——**「立即更新」**按鈕（`POST /outbound-subs/:id/refresh`）；它也會將 Xray 標記為需要重啟。新增、修改、刪除訂閱同樣會觸發 Xray 重啟旗標（刪除時，其 `outbound` 會在下次重載時從設定中消失）。UI 提示：「新增或更新後，請重啟 Xray（或等待下次自動重載），使 `outbound` 生效。」

如何進入 Xray 設定

每次生成 Xray 設定時，活躍的訂閱 `outbound` 分為兩組——`prepend`（「置於手動 `outbound` 之前」旗標）和其餘——並與範本拼接：`[?] [?] [?] [?] prepend [?]` + `[?] [?] [?] [?] outbound` + `[?] [?] [?] [?]`。每組內訂閱按優先順序排列。範本中的手動 `outbound` 不受影響；若範本的 `outbound` 陣列因故無法解析，訂閱的 `outbound` 不會被混入（以免遺失手動設定的 `outbound`）。

匯入的 `outbound` 還會在 `outbound` 面板中以獨立區塊**「來自 Outbound 訂閱（唯讀）」**顯示——無法在那裡編輯，管理僅透過「Outbound 訂閱」部分進行。

11.13.WARP 的 IP 輪換

在 3X-UI 中，可以建立 WARP outbound——連接至 Cloudflare WARP 的 WireGuard 外出連線（Xray 設定中的標籤 `warp`）。面板會自動在 Cloudflare 伺服器上註冊裝置帳戶，取得 WireGuard 金鑰和位址，並將其填入標籤為 `warp` 的 `outbound` 中。透過此 `outbound`，流量以 Cloudflare WARP 的 IP 位址輸出至網際網路。3.3.0 版的新功能——可以手動或按計畫更換此輸出 IP，而無需手動重新建立 WARP 帳戶。

管理位於 **Xray** 部分的 WARP 卡片中（點擊「建立 WARP 帳戶」並取得設定後；在此之前無法使用——面板會提示「請先取得 WARP 設定」）。

更換 IP 時發生的事情

「更換 IP」按鈕啟動 IP 更換。邏輯：

1. 生成新的 WireGuard 金鑰對。
2. 使用新金鑰在 Cloudflare 伺服器上重新註冊 WARP 裝置（新的 `device_id`、`access_token`、位址和 `peer` 數據）。
3. 新數據寫入 Xray 設定的 WARP `outbound`：更新 `secretKey`、`address`（`v4 /32` 和 `v6 /128`）、`reserved`（來自 `client_id`），以及 `peer` 的 `publicKey` 和 `endpoint`。
4. 若之前已設定 WARP+ 授權金鑰（長度不少於 26 個字元），它會自動重新安裝至新帳戶。若失敗，這只是日誌中的警告——IP 更換不會取消。
5. 成功更換後，Xray 被標記為需要重啟，以使新的 `outbound` 生效。

成功時，介面顯示「WARP IP 位址已成功更換！」。

按計畫自動輪換

WARP 卡片中有**「自動更新 IP 位址」開關和「間隔（天）」**欄位。提示：「0 – 停用。自動更換 IP 位址。」

參數	值
資料庫設定	<code>warpUpdateInterval</code> （整數， ≥ 0 ）
預設值	0（自動輪換已停用）
計量單位	天
0	停用自動更換
> 0	每 N 天更換一次 IP

儲存間隔時會保存 `warpUpdateInterval`，若值大於 0，則將「最後更新時間」重置為當前時刻——否則計畫器會在最近一次 tick 時就更換 IP。

計畫由每小時執行一次的後台任務執行——也就是說，面板每小時檢查一次是否需要輪換。檢查演算法：

- 若間隔 ≤ 0 ——不執行任何操作；
- 若「最後更新時間」等於 0（例如，間隔是透過直接編輯資料庫設定的）——這是首次執行：任務只記錄基準時間戳，不立即更換 IP；
- 若自上次更新起已過去不少於 24×3600 秒——執行相同的 IP 更換，更新時間戳，並計畫重啟 Xray。

重要細節：手動點擊「更換 IP」按鈕也會重置最後更新時間的時間戳。因此，手動輪換後，自動間隔的計數重新開始，計畫的更換不會緊隨其後立即執行。

「透過面板代理」

3.3.1 版變更。 獨立的「面板網路代理」設定（panelProxy）已移除。面板本身的外出流量（包括對 WARP API 的請求）現在透過選定的**面板流量 outbound**路由——Xray outbound 或負載均衡器（詳見 13）。下文說明適用於 3.3.1 之前的版本。

對 Cloudflare WARP API 的所有請求（註冊、取得設定、設定授權、更換 IP）不直接發送，而是透過面板的 HTTP 客戶端發送，逾時為 15 秒。此客戶端遵守面板設定中的**「面板網路代理」**（panelProxy）設定。

根據設定說明：代理路由面板本身的外出請求（geo 資料庫更新、Xray/面板版本檢查、Telegram，現在還包括 WARP 請求）——以繞過伺服器端篩選。接受 socks5:// 或 http(s):// 格式的位址，例如 Xray 本身的本地 SOCKS inbound。若欄位為空或代理設定不正確——使用直連（行為不會中斷）。

對 WARP 的好處：若伺服器無法直接連接 `api.cloudflareclient.com`，之前的註冊和輪換會失敗。現在，在 panelProxy 中指定一個可用的代理（包括 Xray 本身的 inbound），可以保證 WARP API 的可達性，以及手動按鈕和計畫輪換的正常運作。

使用場景

- 定期更換走 WARP 的 outbound 的外出 IP——降低因使用同一位址而被封鎖和追蹤的風險。
- 若當前 Cloudflare 位址被列入黑名單或速度緩慢，手動「刷新」IP。
- 無法直接連接 Cloudflare WARP API 的伺服器：透過 panelProxy 路由請求可使註冊和輪換正常運作。

12. 節點（多面板，master/slave）

節點區段將普通的 3X-UI 安裝轉變為**中央（主）面板**，可遠端監控並管理其他（子）3X-UI 面板。每個節點都是在自己伺服器上獨立安裝的 3X-UI；主面板透過其 HTTP API 連接到該節點，輪詢其狀態，並將指定的 inbound 和客戶端同步到它。這就是**多面板**功能：不必逐一登入每個面板，您可以在一個清單中看到所有伺服器，並集中管理它們。

重要原則：**節點不是代理程式，而是完整的 3X-UI 面板**。主面板不會在其上「安裝」任何東西——它只是透過令牌連接到其 API。從清單中刪除節點只會停止監控；遠端面板本身不會受到影響（提示：「這將停止節點監控。遠端面板本身不會受到影響」）。

12.1. 清單頂部摘要

節點表格上方顯示聚合計數器：

欄位	描述
節點總數	清單中的節點總數。
線上	狀態為 <code>online</code> 的節點數量。
離線	狀態為 <code>offline</code> 的節點數量。
平均延遲	到節點的平均延遲（ping），以毫秒為單位。

12.2. 新增和編輯節點

新增節點和編輯節點按鈕會開啟包含節點欄位的表單。

必填欄位（提示：「名稱、地址、連接埠和 API 令牌為必填」）為**名稱**、**地址**、**連接埠**和 **API 令牌**。

點擊「儲存」時（無論是新增還是編輯），面板都會先以 **6 秒逾時驗證節點是否可達**。如果節點未回應，記錄不會被儲存且會顯示錯誤。也就是說，無法新增明顯無法存取的節點。

表單欄位

欄位	預設值	允許的值	描述
名稱	—（必填）	非空字串， 唯一	節點的內部名稱。名稱欄位有唯一性限制——無法建立兩個同名節點。佔位提示： <code>de-frankfurt-1</code> 。儲存時會去除首尾空格。
備註	空	任意字串	節點的選填附註/描述。不影響運作。
通訊協定	<code>https</code>	<code>http</code> / <code>https</code>	連接到遠端面板的協定。如果留空或指定無效值，正規化後會設為 <code>https</code> 。如果節點透過普通 HTTP 回應，但通訊協定設為 <code>https</code> ，面板將返回清楚的提示：「the server speaks HTTP, not HTTPS; set the node scheme to http」。
地址	—（必填）	主機名稱或 IP	遠端面板的地址。佔位提示： <code>panel.example.com</code> <code>1.2.3.4</code> 。地址會被正規化；預設情況下，私有/本地地址會被禁止以防止 SSRF 攻擊——請參閱「允許私有地址」。

欄位	預設值	允許的值	描述
連接埠	– (必填)	整數 1–65535	遠端節點網頁面板的連接埠。超出範圍的值將被拒絕（「node port must be 1-65535」）。
基礎路徑	/	路徑字串	遠端面板的基礎路徑（web base path），如果已設定。會被正規化：保證以 / 開頭和結尾（空值 → / ）。面板在查詢時會在其後附加 panel/api/server/status。
API 令牌	– (必填)	遠端面板的令牌	存取節點 API 的 Bearer 令牌。在 Authorization: Bearer <token> 標頭中傳送。佔位提示：「遠端面板設定頁面上的令牌」。提示：「遠端面板在設定 → API 令牌部分顯示其 API 令牌」。也就是說，令牌必須在節點本身（設定 → API 令牌）上建立，然後貼到此處。
已啟用	true	是/否	啟用節點的監控和同步。已停用的節點 不會被 背景任務輪詢（心跳和流量同步會跳過它們），也不參與面板批次更新。
允許私有地址	false	是/否	解除 SSRF 保護，允許透過私有/本地地址連接到節點。提示：「僅對私有網路或 VPN 中的節點啟用」。僅在節點確實位於私有網路或可透過 VPN 存取時才啟用。

在節點端取得和重新產生令牌

令牌在遠端面板的設定 → **API 令牌** 部分取得。也可以在那裡重新發放：**重新產生令牌** 按鈕附有警告：「重新產生將使目前的令牌失效。任何使用它的中央面板在更新之前將失去存取權限。是否繼續？」重新產生後，主面板中的舊令牌將停止運作——需要在節點表單中更新。

連線 outbound (Connection outbound)

Connection outbound（連線 outbound，outboundTag）欄位指定主面板對此節點 API 的流量如何離開伺服器。如果選擇 Xray outbound 的標籤，面板對節點的請求將不直接發送，而是透過指定的 outbound 路由；面板本身會在回環位址上向執行中的設定新增橋接 inbound 並即時套用變更，無需重新啟動。提示：

「Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection」。

選擇器的結構類似面板的 outbound 選擇：標籤分組為 **Outbounds**（一般 outbound）和 **Balancers**（負載均衡器），黑洞 outbound 在清單中隱藏。空值（佔位提示「Direct connection」）= 直接連接到節點。

匯入 inbound (選擇要同步的 inbound)

節點表單中有 **匯入 inbound**（inboundSyncMode）設定，包含兩種模式：**所有 inbound**（all，預設）和**選取**（selected）。預設情況下，主面板會將選擇了此節點的所有 inbound 同步到節點；現有節點繼續在「所有 inbound」模式下工作。

在已選取模式下，欄位下方會出現 inbound 標籤的多選框。點擊**載入 inbound**——主面板將使用輸入的（尚未儲存的）連線參數向節點請求其 inbound 清單（端點 POST /panel/api/nodes/inbounds）並顯示其標籤；勾選所需的標籤。面板將只同步並部署到節點帶有選取標籤的 inbound，而節點上直接存在的其他 inbound 將保持不變——主面板不會刪除它們，也不管理它們。

範例：請求節點的 inbound 清單以進行選擇性匯入。 請求主體傳送尚未儲存的連線參數；回應包含節點上可用 inbound 的標籤：


```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. TLS 驗證（適用於 https 節點）

此欄位群組指定主面板如何驗證節點的 HTTPS 憑證。這些設定僅對 **https** 通訊協定有效；對於 **http** 節點會被忽略。

TLS 驗證——下拉清單，提示：「面板如何驗證節點的 HTTPS 憑證。釘選或略過——適用於自簽憑證（僅 https 節點）」。

模式	值	預設	描述
驗證（標準 CA）	verify	是（預設）	透過受信任的 CA 進行標準憑證鏈驗證。適用於具有公開/Let's Encrypt 憑證的節點。也用於所有 http 節點。
釘選憑證（SHA-256）	pin	—	不驗證 CA 鏈，但節點葉憑證的 SHA-256 會與儲存的指紋進行比對（常數時間比較）。為自簽憑證保留中間人攻擊防護。需要填寫指紋欄位。
略過驗證	skip	—	完全停用憑證驗證。警告：「略過驗證會移除對中間人攻擊的防護——API 令牌可能被攔截。最好釘選憑證」。

在 3.4.0 版本中，在上述三種模式之外新增了第四種——**Mutual TLS（用戶端憑證）**（mtls），與其他模式一樣，僅適用於 https 通訊協定。

模式	值	預設	描述
Mutual TLS（用戶端憑證）	mtls	—	除了驗證節點的憑證外，主面板還會用由其自身 CA 簽發的用戶端憑證向節點額外驗證自身。在此模式下，API 令牌對節點而言變為可選——節點透過憑證識別主面板。選擇此模式時會顯示提示：「This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it」。

若要為節點啟用雙向 TLS：在節點端設定 **Mutual TLS** 模式，從 **Node mTLS** 部分（見下文）複製管理面板的 CA，在節點上將其設為受信任的父 CA，然後重新啟動節點。

如果選擇 skip、pin 或 mtls 以外的任何值，正規化會強制設為 verify。

憑證釘選

選擇釘選憑證後，會出現：

- 釘選憑證的 **SHA-256**——輸入欄位。接受 **base64** 格式的指紋（Xray 的 pinnedPeerCertSha256 格式）或帶冒號或不帶冒號的 **hex** 格式（openssl -fingerprint 風格）。提示：「節點憑證的 SHA-256（base64 或 hex）。點擊「取得」立即從節點讀取」。佔位提示：「SHA-256（base64 或 hex）」。選擇 pin 時，若指紋為空或格式不正確，儲存時將觸發驗證錯誤。

範例：兩種格式的相同指紋。 欄位接受任何一種格式——兩者都表示同一個憑證：

```
# base64 (Xray [?][?]pinnedPeerCertSha256 [?][?])
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# [?][?][?][?][?][?]hex (openssl x509 -fingerprint -sha256 [?][?])
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

如果指紋尚不清楚，點擊**取得**——主面板會透過 HTTPS 自動從節點讀取並填入欄位。

- **取得按鈕**——不驗證憑證地透過 HTTPS 連接到節點並讀取目前葉憑證的 SHA-256（端點 `POST / certFingerprint`），將其填入欄位。成功後——「已取得節點的目前憑證」；失敗時——「無法取得憑證」。僅適用於 https 節點。

Node mTLS（面板間的雙向 TLS 驗證）

節點頁面有一個獨立的 **Node mTLS** 部分——雙向 TLS 驗證設定，為「面板 → 節點」呼叫在 API 令牌之上新增第二個因素（用戶端憑證）。雙向 TLS 為選用功能；如果該部分的欄位為空，節點將按以前的方案運作——**僅使用 API 令牌**（提示：「Mutual TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth」）。該部分包含兩個操作：

- **複製此面板的 CA**（`POST /panel/api/nodes/mtls/ca`）——將此面板的根憑證（CA）複製到剪貼簿。此 CA 需要傳給受管節點，以便它們信任面板的用戶端憑證；在節點本身上，之後需設定 TLS 驗證模式為 **Mutual TLS**（提示：「Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS」）。複製後——「CA 憑證已複製到剪貼簿」。
- **受信任的父 CA**（`Trusted parent CA`，`POST /panel/api/nodes/mtls/trustCA`）——此欄位在此面板本身作為上級（管理）面板的節點時使用。在此貼上管理面板的 CA 以要求其用戶端憑證，然後點擊 **Save trust CA**。此變更需要**重新啟動面板**（提示：「When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply」）。

12.4. 每個節點顯示的內容

表格欄和節點卡片的欄位（觀測狀態，在每次心跳輪詢時填充）：

欄位	描述
狀態	<code>online</code> / <code>offline</code> / <code>unknown</code> ——見下文。
CPU	遠端伺服器的處理器負載百分比。
記憶體	RAM 使用百分比（計算方式為 <code>current/total*100</code> ）。
正常執行時間	伺服器持續執行的時間（以秒為單位）。
延遲	節點對最後一次輪詢的回應時間（毫秒）。
最後 ping	最後一次成功心跳的時間（Unix 秒； <code>0</code> = 「從未」；近期值顯示為「剛才」）。
Xray 版本	節點上執行的 Xray-core 版本。
面板版本	節點上的 3X-UI 版本——與最新版本比較以顯示更新指示器。

欄位	描述
(inbound)	實際部署在此節點上的 inbound 數量。
(用戶端)	節點 inbound 上的用戶端數量。
(線上)	目前在線的節點用戶端數量。
(已耗盡)	已過期或已達流量限制的節點用戶端數量。手動停用的用戶端不計入此計數器。
(速度)	部署在節點上的 inbound 目前（即時）傳輸速度。

inbound/用戶端/線上計數器透過節點的穩定 GUID（panelGuid）而非本地 id 與節點關聯——這樣子節點上的用戶端就能精確歸屬於子節點，而不是歸屬於同步它的中間節點。

對於部署在節點上的 inbound，頁面會顯示線上用戶端、計數器和**目前傳輸速度**。透過穩定 GUID 的關聯可以正確區分具有相同 panelGuid 的「克隆」節點。

節點狀態

狀態	設定時機
online	節點對 panel/api/server/status 輪詢回應了 success=true。
offline	節點未回應、返回 HTTP 錯誤、success=false 或無法識別的回應。
unknown	初始值，節點尚未被輪詢過。

輪詢失敗時，錯誤文字會被儲存並以清楚的措辭顯示，有助於診斷「offline」的原因。

12.5. 節點操作

- **測試連線**（POST /test）——在節點表單中使用輸入的（尚未儲存的）參數測試連線，逾時 6 秒。結果：「連線正常（{ms} 毫秒）」或「無法連線」。便於在儲存之前除錯地址/連接埠/令牌/TLS。
- **立即探測**（「立即探測」按鈕，POST /probe/:id）——對已儲存節點進行臨時輪詢；立即更新狀態和指標（CPU/記憶體/正常執行時間/延遲/版本）並記錄心跳。失敗時——「探測失敗」。

範例：透過主面板 API 測試和探測節點。「測試連線」使用表單中尚未儲存的參數進行測試：

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

對 id 為 7 的已儲存節點進行臨時輪詢：

```
POST /panel/api/nodes/probe/7
```

- **更新面板**（`POST /updatePanel`，請求主體 `{ids:[...]}`）——在節點上啟動其標準自更新器：節點下載最新的 3X-UI 版本並在其上重新啟動。**更新已選取（{count}）按鈕一次為多個選取的節點執行此操作。**
節點旁邊會顯示指示器：有可用更新或已是最新，根據節點面板版本與最新版本的比較結果而定。

範例：透過一個請求更新多個節點。 請求主體傳送選取節點的 id；只有已啟用且 `online` 的節點會被更新，其他節點作為跳過項目返回。

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

回應形式為「已在 2 個節點上啟動更新，1 個失敗」：例如節點 12 可能是 `offline` 因此被跳過。

- 確認：「將 {count} 個節點更新到最新版本？每個選取的節點將下載最新版本並重新啟動。只有線上的已啟用節點會被更新」。
- 只有處於 `online` 狀態的已啟用節點才會被更新。已停用的節點在結果中標記為「node is disabled」，離線節點標記為「node is offline」。結果：「已在 {ok} 個節點上啟動更新，{failed} 個失敗」。如果沒有選取任何合適的節點——「請至少選取一個線上的已啟用節點」。

在更新確認對話方塊中（無論是單個節點還是批次更新），都有**更新到開發版本頻道（最新提交）**核取方塊。如果勾選，選取的節點將安裝 `dev-latest` 滾動版本（`main` 分支的最新提交）而不是穩定版本；未勾選時，節點按其正常頻道更新。啟用核取方塊時，其下方會顯示警告：「開發版本追蹤 `main` 分支的每次提交，不是穩定版本——沒有自動回滾」。dev 標誌透過 `POST /panel/api/nodes/updatePanel` 傳遞到節點，節點即按 dev 頻道啟動更新。

- **Set Cert from Panel**（輔助操作，`GET /webCert/:id`）——在節點上建立 inbound 時，允許填入節點自身 web-TLS 憑證的路徑（而非中央面板的憑證），以確保檔案確實存在於節點上。需要節點已啟用且可達。
- **刪除節點**（`POST /del/:id`）——確認：「刪除節點「{name}」？這將停止節點監控。遠端面板本身不會受到影響」。刪除節點記錄及其累積的流量統計；遠端面板繼續正常工作。**只有在從節點移除所有 inbound 後才能刪除節點。** 如果節點仍有至少一個 inbound 附加（透過 `node_id`），面板將拒絕刪除並返回錯誤，例如「cannot delete node: N inbound(s) still attached to it; detach or delete them first」——請先分離或刪除這些 inbound，然後再刪除節點。這樣可以避免出現帶有懸空參考指向已刪除節點的「孤兒」inbound。

12.6. 指標歷史

歷史按鈕/圖表訪問 `GET /history/:id/:metric/:bucket`。可用的指標：`cpu` 和 `mem`——在每次成功的心跳時累積。聚合間隔大小（`bucket`，以秒為單位）受白名單限制：

範例：請求歷史記錄。 節點 7 的 CPU 負載歷史，按 60 秒間隔聚合（返回最多 60 個數據點）：

```
GET /panel/api/nodes/history/7/cpu/60
```

對於記憶體和「即時」模式（2 秒）——分別使用 `.../7/mem/60` 和 `.../7/cpu/2`。白名單之外的值將被拒絕（「invalid metric」/「invalid bucket」）。

Bucket（秒）	用途
2	即時模式
30	30 秒間隔
60	1 分鐘間隔
120	2 分鐘間隔
180	3 分鐘間隔
300	5 分鐘間隔

返回最多 60 個數據點。無效的指標或 bucket 將被拒絕（「invalid metric」/「invalid bucket」）。

12.7. inbound 和用戶端如何同步

inbound 透過 `node_id` 欄位「屬於」某個節點（在 inbound 編輯器中選擇節點）：

範例：節點表單中的令牌。 令牌從子面板（設定 → API 令牌）取得並貼到主面板的 **API 令牌** 欄位。每次輪詢時，主面板在標頭中傳送它：

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

如果子面板設定了**基礎路徑**（web base path），例如 `/secret/`，主面板會自動在 `panel/api/server/status` 前添加它 → `https://panel.example.com:2053/secret/panel/api/server/status`。

- 設定部署（調諧）。** 當連結到節點的 inbound/用戶端發生任何更改時，節點被標記為「臟」。背景任務為每個處於 `online` 狀態的已啟用節點在有更改時部署其 inbound（按 `node_id`），然後重設「臟」狀態標誌。已停用、離線或「臟」的節點被認為是「待處理」——對其的部署會延遲到連線恢復。
- 流量收集。** 同一任務從節點請求流量快照並將其合併到本地統計資訊中。根據合併的流量執行限制/期限耗盡檢查，並在必要時停用用戶端；節點的「已耗盡」計數器正是反映這一點。如果節點不可達，其線上用戶端將被清除。

對於同時連結到多個面板的用戶端，主面板在同一任務中還會向節點分發該用戶端**跨所有面板的總計**流量使用量（在節點上以單獨的表格儲存，鍵為主面板的 GUID；每次傳送時覆寫，因此主面板端的重設也會傳播）。在節點上，用戶端的流量顯示本地值和傳送值中較大的那個，當超過總配額時，用戶端**在節點本身本地停用**（透過與自動停用時相同的 Xray 重新啟動機制，這也會中斷已建立的連線）。這消除了節點只看到其自己的流量份額、低估使用量並繼續為已耗盡總限制的用戶端提供服務的情況。當重設流量、自動續約或刪除用戶端時，傳送的計數器會被清除。

在節點上部署 inbound 的**第一次**同步時（新增新節點或重新匯入 inbound），主面板會用節點的實際值初始化用戶端流量計數器。以前在這種情況下，inbound 的總計數器傳輸正確，但用戶端的個別計數器會被清零，主面板會對在節點連接之前累積的所有歷史記錄少計用戶端使用量。現在，如果 inbound 在同一同步中建立，新的 `client_traffics` 行會繼承節點的計數器值（基準設為該值，所以下一個差值

為零，流量不會被計算兩次）。計數器種植只適用於在這次傳遞中建立的 inbound：在已有 inbound 下重新出現的用戶端仍從零開始（防止「幽靈」流量），而剛被刪除的用戶端在重建其 inbound 時不會「復活」。

- 心跳。一個單獨的背景任務定期透過 `panel/api/server/status` 輪詢所有已啟用節點（並發限制），更新狀態/指標/版本，並在有 Web 用戶端時透過 WebSocket 廣播更新的節點樹。

12.8. 節點鏈（子節點/傳遞節點）

拓撲可以是非扁平的：節點本身可以成為其自身節點的主面板。這類下級面板在您處顯示為子節點——這些是從直接節點接收的唯讀投影。

- 提示：「唯讀：透過 {parent} 可達的下屬節點。從 {parent} 自己的面板管理它」。也就是說，子節點在這裡無法編輯、刪除或更新——所有操作都從其直接父節點的面板執行。
- 子節點的身份由其 GUID 決定；因此，線上用戶端和 inbound 被精確歸屬於實際託管它們的物理節點，即使在 `Node1 → Node2 → Node3` 鏈中（主面板通過每個直接節點向下一層「穿透」）。
- 如果直接節點變得不可達，其子節點快取將被清除，子節點在連線恢復之前會從樹中消失。

12.9.3.5.0 修正（節點穩定性）

3.5.0 中節點操作者可感知的修正合集：

- 未變更的用戶端儲存不再中斷節點流量。先前任何用戶端儲存（即使「開啟後直接儲存」）都會向節點發送完整的 inbound 更新並重建 Xray 處理器——節點顯示在線，但在手動重啟前不再傳輸流量。現在無實質變更（no-op）的儲存不做任何事，實際變更則以一次輕量更新送出，不重建處理器。
- 節點的 Host 覆寫會匯入主面板：首次接受節點的 inbound 時，其 Hosts 記錄（SNI、指紋、ALPN 等）會被匯入，訂閱因此帶有正確的 TLS 參數。
- 節點上的自動續期會開啟新的配額週期（「每 30 天 100 GB」之類的方案會再次發放完整額度）。
- 在主面板刪除用戶端會在節點上完整刪除（記錄 + 流量），而不只是解除繫結；先前批量刪除留下的殘餘可在節點上透過「刪除未綁定客戶端」操作清除。
- 節點的 inbound 在首次接受前不會被清除：儲存剛新增的節點不再可能刪除其正在運作的 inbound。
- 單一異常 inbound 不會中止節點流量同步（同時，節點上的舊式 socks inbound 會改名為 mixed）。
- 連接埠衝突檢查僅限於自身節點——編輯節點 inbound 的監聽地址不再因其他節點使用相同連接埠而被拒絕（「port ... already used by inbound ...」）。

12.10. 節點：3.3.0 版本的新功能

在 3.3.0 版本中，節點部分獲得了三項重大改進：多跳拓撲中流量和線上用戶端的正確歸屬、節點間用戶端 IP 的同步，以及在節點面板正常運行但其上的 Xray 核心崩潰時的獨立狀態指示器。以下 inbound/outbound 保持原文不譯。

1. 多跳：沿子節點鏈的正確流量歸屬

以前，計數器（inbound 數量、線上用戶端、已耗盡）在「直接」節點層面計算。如果您有 `[?] [?] [?] → [?] [?] 1 → [?] [?] 2 → [?] [?] 3` 這樣的鏈，所有實際位於 `[?] [?] 2 / [?] [?] 3` 上的內容都被錯誤地歸屬於 `[?] [?] 1`（流量通過它到達主面板）。在 3.3.0 中，歸屬按實際來源進行。

工作原理：

- **子節點作為獨立行可見。** 每個面板發布其直接節點的清單；只包含具有已知 `Guid` 的節點——穩定的身份對於將節點歸屬到上一「跳」是必要的。主面板定期（從心跳任務）提取這些清單並快取它們，然後將「傳遞」子節點添加到直接節點中。
- **傳遞節點為唯讀。** 在 UI 中，它們標記為**「子節點」**，並帶有提示：「唯讀：透過 {父節點} 可達的下屬節點。從 {父節點} 自己的面板管理它。」此類行沒有操作按鈕——節點從其直接父節點的面板管理。
- **透過 GUID 的層次結構。** 直接節點的 `ParentGuid` 是主面板本身的 GUID；傳遞節點的是其父節點的 GUID。這樣就建立了樹狀結構。
- **計數器的真實來源——inbound 上的 `origin_node_guid`。** 這是實際持有此 inbound 的節點的 `panelGuid`。它在 inbound 從節點同步時設置，並且在後續跳躍中保持原樣，因此深度嵌套的 inbound 被歸屬於真實節點而非中間節點。inbound 數量、線上用戶端和已耗盡用戶端的計數器都根據此 GUID 重新計算。鍵選擇邏輯：

inbound 狀態	歸屬於
<code>origin_node_guid</code> 已設置	此 GUID（實際源節點）
為空，但 <code>node_id</code> 已設置	節點的合成 GUID（舊版本，尚未報告其 <code>panelGuid</code> ）
為空且 <code>node_id</code> 為空	主面板自身的 GUID（本地 Xray 上的 inbound）

線上用戶端同樣按 GUID 分組，因此每個節點行只顯示實際連接到它的用戶端。

使用者看到的效果： 在扁平拓撲（節點直接位於主面板下方）中，沒有任何變化——按 GUID 和按 `id` 的計數器一致。但一旦出現節點鏈，清單中就會出現「子節點」行，每個節點的 inbound/線上/已耗盡數字現在反映的是其自身的負載，而非所有通過它傳遞的總和。

2. 節點間 access.log 用戶端 IP 同步

IP 限制（用戶端的 `limitIp`）依賴 Xray 寫入 `access.log` 的地址。以前每個節點只看到到自己的連線，因此「每個用戶端最多 N 個 IP」的限制在叢集中不起作用：用戶端可以連接到不同的節點並繞過限制。在 3.3.0 中，觀察到的 IP 在整個叢集中同步。

工作原理：

- 在每個節點上，一個背景任務解析 `access.log`，為每行提取 IP、用戶端 email 和時間戳，並將它們儲存在本地表格中（每個 email 一條記錄，IP 儲存為 JSON 陣列 `{ip, timestamp}`）。本地地址 `127.0.0.1` 和 `::1` 會被丟棄。
- **每 10 秒一次**的同步對每個在線的已啟用節點執行雙向交換：從節點提取 IP 並將其合併到本地表格，然後將主面板的摘要視圖傳送給節點。
- 合併操作**不重複計算**在多個節點上觀察到的同一 IP，**也不復活過時的記錄**：應用與本地任務相同的時效閾值——**30 分鐘**。每個 IP 儲存最新的時間戳。來自其他節點的記錄獲得新的本地 id（節點的 id 空間是獨立的）；對同一 email 的並發插入受到保護以防重複。
- 計算限制時，「活躍」IP 是指在當前本地掃描中觀察到的，或具有非常新鮮的同步資料庫時間戳的 IP（**2 分鐘內**）。這正是讓限制在整個叢集範圍內有效的原因，即使地址是在另一個節點上觀察到的。超

過限制時，最舊的「活躍」IP 會被傳送到 fail2ban 日誌，連線被強制中斷（透過 Xray API 移除/重新添加用戶端）。

使用者看到的效果：IP 數量限制現在對整個叢集有效，而不是對每個節點單獨有效；面板中每個用戶端可見在任何節點上觀察到的 IP（在 30 分鐘視窗內）。沒有單獨的按鈕/設定——同步在後台自動進行，前提是節點啟用了 access.log 且可達（對於限制本身，節點上也需要 Fail2Ban）。

3. 獨立狀態指示器：節點面板線上但 Xray 崩潰

以前，節點狀態本質上是「線上/離線」。如果節點面板回應，節點就被認為是線上——即使其上的 Xray 核心未在工作，用戶端實際上無法連線。在 3.3.0 中，面板健康狀況和 Xray 核心健康狀況被分離。

工作原理：

- 輪詢節點時，主面板從遠端 `/panel/api/server/status` 回應中取得 `xray.state` 和 `xray.errorMsg` 欄位並儲存在節點中。即使在面板 ping 成功但核心不健康時，這些欄位也會被填充——正是為了區分面板的可達性和 Xray 的狀態。
- `xray.state` 的值：`"running"`（執行中）、`"stop"`（已停止）、`"error"`（錯誤）。
- 這些值被轉換為節點狀態。除了常見狀態外，還新增了新狀態：

狀態鍵	說明	顯示時機
<code>online</code>	線上	面板回應，Xray 執行中（ <code>running</code> ）
<code>offline</code>	離線	面板不可達/ping 失敗
<code>unknown</code>	未知	狀態尚未確定
<code>xrayError</code>	Xray 錯誤	面板線上，但 Xray 核心處於 <code>error</code> 狀態（有 <code>errorMsg</code> ）
<code>xrayStopped</code>	已停止	面板線上，但 Xray 已停止（ <code>stop</code> ）

- 對於此類狀態，UI 使用**獨立的紫色指示器**（顏色與線上的綠色和離線的紅色不同）。紫色直接表示：可以連接到節點，問題在 Xray 核心本身，而非網路或面板。

使用者看到的效果：核心崩潰時，不再顯示誤導性的「綠色」，節點以**紫色**高亮顯示，狀態為「**Xray 錯誤**」或「**已停止**」。這立即表明需要修復節點上的 Xray（重新啟動核心、查看 `errorMsg`），而不是排查節點本身的可達性問題。同樣的 `xrayState` / `xrayError` 也傳遞到傳遞子節點（見第 1 點），因此核心的異常狀態在整個鏈中都可見。

13. 面板設定

「設定」區段（頁面標題為 **設定**，英文 *Panel Settings*）管理 3X-UI 網頁面板本身的行為：它在哪個位址與連接埠上監聽、如何受到保護、如何與 Telegram 機器人及外部服務互動、以哪個時區執行排程任務。每個參數都以「鍵-值」配對的形式儲存在資料庫的 `settings` 表中；若資料庫中沒有相應的值，則套用預設值。

重要——套用變更。 此頁面上的任何變更都需要用 **儲存 (Save)** 按鈕儲存，然後重新啟動面板，變更才會生效。原文提示為：「儲存變更並重新啟動面板以套用變更。」儲存時會顯示「設定已變更」的通知。

13.1. 儲存與重新啟動面板

元素	用途
儲存 (Save)	將表單中所有欄位寫入資料庫（POST <code>/panel/setting/update</code> ）。寫入前，值會經過驗證——不正確的位址、連接埠或路徑將被拒絕，面板會回傳錯誤。
重新啟動面板 (Restart Panel)	重新啟動面板的網頁伺服器（POST <code>/panel/setting/restartPanel</code> ）。重新啟動會延遲 3 秒進行。提示：「您確定要重新啟動面板嗎？確認後，將在 3 秒後重新啟動。若面板無法存取，請檢查伺服器日誌」。成功時——「面板已成功重新啟動」。
重設為預設設定 (Reset to Default)	刪除資料庫中所有已儲存的設定，之後面板將使用預設值。此操作不會重設管理員的登入憑證。

重新啟動是透過向面板處理程序傳送 `SIGHUP` 訊號（或透過已註冊的重啟掛鉤）來執行。在 Windows 上不支援透過訊號自動重新啟動。**監聽參數 (IP、連接埠、路徑、網域、憑證、時區)** 的變更只有在重新啟動面板後才會套用。

13.2. 一般設定（「面板」分頁 / *General*）

介面語言 (*Language*)

面板網頁介面的語言。可用語言：`en-US`（英語）、`ru-RU`（俄語）、`zh-CN`、`zh-TW`、`fa-IR`、`ar-EG`、`es-ES`、`id-ID`、`ja-JP`、`pt-BR`、`tr-TR`、`uk-UA`、`vi-VN`。這是顯示設定，不影響 Xray 的運作。

日曆類型 (*Calendar Type*)

- **鍵：** `datepicker`
- **預設值：** `gregorian`（公曆）。
- **用途：** 在選擇日期時所使用的日曆類型（例如，設定客戶的有效期限時）。提示：「排程任務將依照此日曆執行。」替代值為波斯（賈拉里）曆，這在面板的伊朗使用族群中很有需求。

分頁大小 (Pagination Size)

- 鍵： `pageSize`
- 預設值： 25
- 允許值： 從 0 到 1000 的整數。
- 用途： 表格（連線/inbound 清單）中每頁的列數。提示：「設定連線表格的分頁大小。設為 0 以停用」——當為 0 時，分頁輸出會被停用，所有記錄會顯示為單一清單。
- 無需重新啟動面板（顯示設定）。

自動停用後重新啟動 Xray (Restart Xray After Auto Disable)

- 鍵： `restartXrayOnClientDisable`
- 預設值： `true`
- 用途： 當客戶被自動停用（因有效期限到期或達到流量上限）時，重新啟動 Xray，以中斷該客戶已建立的連線。提示：「當客戶因有效期限到期或流量上限而自動停用時，重新啟動 Xray。」功能本身未變更——此切換開關只是放在「面板」分頁 (General) 上，與其他一般設定並列。

備註模型與分隔字元 (Remark Model & Separation Character)

- 鍵： `remarkModel`
- 預設值： `-ieo`
- 用途： 設定訂閱中組態名稱 (remark) 的產生方式。該字串由第一個字元——分隔符，以及後續的順序字母序列組成：
 - `i` — inbound 備註 (inbound remark) ；
 - `e` — 客戶的 email；
 - `o` — 額外標籤 (extra) 。

在預設值 `-ieo` 下，分隔符為 `-`，而各部分的順序為：inbound → email → extra（例如 `MyInbound-user@mail-extra`）。空白部分會被略過。介面中的「備註範例」 (Sample Remark) 欄位會顯示所產生名稱的預覽。是否將 email 納入名稱，還取決於訂閱設定中的「在名稱中包含 Email」參數（訂閱相關區段）。

範例： `remarkModel` 的值如何影響組態名稱。假設 inbound 名為 `VLESS-Reality`，客戶的 email 為 `alex@vpn`，而額外標籤為 `RU`。則：

欄位值	最終名稱 (remark)
<code>-ieo</code> (預設)	<code>VLESS-Reality-alex@vpn-RU</code>
<code>_ie</code>	<code>VLESS-Reality_alex@vpn</code>
<code>-ei</code>	<code>alex@vpn-VLESS-Reality</code>
<code>o</code> (空格分隔符，僅標籤)	<code>RU</code>

字串的第一個字元——始終是分隔符；其餘字母決定哪些部分以何種順序納入名稱。

13.3. 面板存取：IP、連接埠、路徑、網域、憑證

這組設定定義了面板的網路進入點。此處所有變更只有在重新啟動面板後才會套用。

欄位	鍵	預設值	說明
面板管理用 IP 位址 (<i>Listen IP</i>)	<code>webListen</code>	"" (空)	網頁面板監聽的 IP。空 = 在所有 IP 上監聽。提示：「留空以允許從任何 IP 連線」。若已設定，必須是有效的 IP 位址（否則儲存會被拒絕）。
面板網域 (<i>Listen Domain</i>)	<code>webDomain</code>	"" (空)	用於依網域檢查請求的面板網域名稱。空 = 接受來自任何網域和 IP 的連線。提示：「留空以允許從任何網域和 IP 連線。」
面板連接埠 (<i>Listen Port</i>)	<code>webPort</code>	2053	面板運作所在的連接埠。提示：「面板運作所在的連接埠」。允許 1–65535。連接埠必須是空閒的；面板與訂閱服務不能同時使用相同的 IP: [?] [?] [?] 配對。
URI 路徑 (<i>URI Path</i>)	<code>webBasePath</code>	/	面板 URL 的基本路徑 (<i>basePath</i>)。提示：「必須以 '/' 開頭並以 '/' 結尾」。儲存時，若缺少前導和結尾的 /，面板會自動加上。路徑中的禁用字元會被拒絕。

面板憑證 (TLS / HTTPS)

欄位	鍵	預設值	說明
面板憑證公鑰檔案路徑 (<i>Public Key Path</i>)	<code>webCertFile</code>	""	憑證（憑證鏈）檔案的完整路徑。提示：「輸入以 '/' 開頭的完整路徑」。
面板憑證私鑰檔案路徑 (<i>Private Key Path</i>)	<code>webKeyFile</code>	""	私鑰檔案的完整路徑。提示：「輸入以 '/' 開頭的完整路徑」。

若設定了憑證/金鑰路徑中的**至少一個**，面板在儲存時會嘗試載入「憑證 + 金鑰」配對；若發生錯誤（檔案不存在、金鑰與憑證不符），儲存會被拒絕。當兩個正確的路徑都已設定時，面板會切換為 HTTPS。兩個欄位都為空 = 面板以一般 HTTP 運作。

安全警告 (Security warnings)。若偵測到不安全的組態，面板會顯示「您的面板可能對外開放：」的區塊並附上警告：

- 以一般 HTTP 運作——「正式環境請設定 TLS」；
- 標準連接埠 2053——「將其改為隨機值」；
- 預設基本路徑 / ——「將其改為隨機值」；
- 標準訂閱路徑 /sub/ 與 JSON 訂閱 /json/ ——「將其變更」。這些是建議，而非封鎖。

13.4. 工作階段、面板代理與受信任代理（「代理與伺服器」分頁 / *Proxy and Server*）

工作階段持續時間 (*Session Duration*)

- 鍵：`sessionMaxAge`

- **預設值：** 360（分鐘，即 6 小時）。
- **允許值：** 從 1 到 525600 分鐘（1 年）。
- **用途：** 管理員在不重新登入的情況下保持已授權的時間。單位為**分鐘**。提示：「系統中的工作階段持續時間（值：分鐘）」。

面板流量的 outbound (*Panel Traffic Outbound*)

- **鍵：** panelOutbound
- **預設值：** ""（空 = 直接連線）。
- **用途：** 設定一個 Xray **outbound**，面板透過它傳送**自身的請求**——版本檢查與面板/Xray 的下載、對 Telegram 的連線、一般的 geo 檔案更新——以繞過伺服器端對 GitHub/Telegram 的過濾。此欄位為**下拉清單**：其中列出了來自 Xray 組態範本的 outbound、來自 outbound 訂閱的 outbound，以及路由**負載平衡器**（單獨一組）。清單中已排除 blackhole 類型的 outbound——將下載路由到「黑洞」毫無意義。原文提示：「將面板自身的請求——版本檢查與面板/Xray 的下載、Telegram 以及一般的 geo 檔案更新——透過此 Xray outbound 路由，以繞過伺服器端對 GitHub/Telegram 的過濾。本機橋接 inbound 會自動加入運作中的組態並即時套用。Xray 內建的 Geodata 自動更新不受影響；它有自己的下載 outbound。留空以直接連線。」

運作方式。 選擇某個 outbound 時，面板會自行在運作中的組態裡加入一個服務用的 loopback-inbound（標籤為 panel-egress 的 SOCKS 橋接）以及一條路由規則，將面板自身的 HTTP 流量導向所選的 outbound。若選擇了負載平衡器，規則中會代入 balancerTag，面板流量會在其成員之間分配。橋接與規則會**即時**套用，無需完整重新啟動面板。留空此欄位以直接連線。Xray 內建的 geo 資料自動更新不受此設定影響——它在 Xray 路由內部有自己的 outbound。

- **格式：** socks5://（或 socks5h://）或 http(s)://，必要時可附帶 socks5:// user:pass@host:port 形式的授權。嚴格支援的協定為：socks5、socks5h、http、https——其他協定視為不允許，此時面板會回退至直接連線。典型範例是 Xray 自身的本機 SOCKS inbound。
- 原文提示：「將面板自身的 outbound 請求（geo 更新、Xray/面板版本檢查、Telegram）透過此代理路由，以繞過伺服器端對 GitHub/Telegram 的過濾。接受 socks5:// 或 http(s)://，例如 Xray 本機 SOCKS inbound。留空以直接連線。」
- 無效的代理不會導致儲存錯誤——面板只是改用直接連線，並在日誌中寫入警告。

欄位值範例。 若伺服器上已啟動了一個位於連接埠 10808 的本機 Xray SOCKS inbound，請將面板自身的請求透過它導向：

```
socks5://127.0.0.1:10808
```

對於帶授權的外部 HTTP 代理：

```
http://user:pass@proxy.example.com:8080
```

儲存並重新啟動後，面板將透過所指定的代理來拉取 geo 資料庫更新、檢查版本並連線 Telegram。

受信任的代理 CIDR (*Trusted proxy CIDRs*)

- 鍵： `trustedProxyCIDRs`
- 預設值： `127.0.0.1/32,::1/128`（僅本機主機）。
- 格式：以逗號分隔的 IP 位址或 CIDR 子網路清單（例如 `10.0.0.0/8`, `192.168.1.5`）。每個項目都會驗證為 IP 或 CIDR——不正確的值會在儲存時被拒絕。
- 用途：列出允許設定 `X-Forwarded-Host`、`X-Forwarded-Proto` 標頭以及真實客戶端 IP 標頭的來源。原文提示：「以逗號分隔的 IP/CIDR，允許其設定 forwarded host、proto 與 client IP 標頭。」若面板運作在反向代理（nginx、Caddy 等）之後，需要進行設定，以便正確判定客戶端 IP 與協定。

範例：面板位於反向代理之後。若 nginx 位於同一主機並將請求代理至面板，請僅信任本機主機（預設值）：

```
127.0.0.1/32,::1/128
```

若代理位於內部網路 `10.0.0.0/8` 中的另一台獨立伺服器上，請加入其子網路，否則面板會忽略它所傳遞的標頭，並會看到代理的 IP 而非真實客戶端的 IP：

```
127.0.0.1/32,::1/128,10.0.0.0/8
```

傳遞真實 IP 與協定的對應 nginx 區塊範例：

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Telegram 機器人（「Telegram 機器人」分頁 / *Telegram Bot*）

啟用 Telegram 機器人 (*Enable Telegram Bot*)

- 鍵： `tgBotEnable`
- 類型/預設：布林值， `false`。
- 用途：啟用 Telegram 機器人的運作。提示：「透過 Telegram 機器人存取面板功能」。

Telegram 權杖 (*Telegram Token*)

- 鍵： `tgBotToken`
- 預設： `""`。
- 用途：機器人的權杖。提示：「需向 Telegram 機器人管理員 @botfather 取得權杖」。
- 安全特性：權杖屬於機密值。在面板讀取設定的回應中不會回傳此值（欄位會被清空，僅回傳「已設定 / 未設定」的旗標）。若儲存時將此欄位留空，則先前儲存的權杖會**被保留**（不被覆寫）。

Telegram 機器人語言 (*Telegram Bot Language*)

- 鍵： `tgLang`
- 預設： `en-US`。
- 用途： 機器人訊息的語言（與網頁介面語言無關）。可用語言清單與面板語言相同。

機器人管理員的 User ID (*Admin Chat ID*)

- 鍵： `tgBotChatId`
- 預設： `""`。
- 格式： 一個或多個數字型 Telegram User ID，以逗號分隔。
- 用途： 通知的接收者，以及獲准透過機器人管理面板的管理員。提示：「一個或多個 Telegram 機器人管理員的 User ID。要取得 User ID，請使用 `@userinfobot` 或在機器人中使用 `/id` 指令。」

通知頻率 (*Notification Time*)

- 鍵： `tgRunTime`
- 預設： `@daily`（每天一次）。
- 格式： **Crontab** 格式的字串（支援標準 cron 運算式，也支援 `@daily`、`@hourly`、`@every 1h` 之類的簡寫）。提示：「以 Crontab 格式指定通知間隔」。控制機器人的週期性報告。

欄位值範例。

值	機器人何時傳送報告
<code>@daily</code>	每天午夜一次（預設）
<code>@hourly</code>	每小時
<code>@every 6h</code>	每 6 小時
<code>0 9 * * *</code>	每天 09:00
<code>30 8 * * 1</code>	每週一 08:30

時間以「時區」設定（第 13.6 節）中的時區計算。

SOCKS 代理 (*SOCKS Proxy*)

- 鍵： `tgBotProxy`
- 預設： `""`。
- 用途： 專供機器人連線 Telegram 使用的 SOCKS5 代理。提示：「若您需要代理 Socks5 才能連線 Telegram，請依指南設定其參數。」此設定恰好套用於機器人的流量（與第 13.4 節的一般「面板網路代理」不同）。

Telegram API Server (*Telegram API Server*)

- 鍵： `tgBotAPIServer`

- **預設：** "" (使用標準伺服器 `api.telegram.org`)。
- **格式：** URL `http(s)://...`；儲存時會通過 URL 正確性檢查——無效位址會被拒絕。提示：「所使用的 Telegram API 伺服器。留空以使用預設伺服器。」用於自行部署的 Telegram Bot API server。

機器人通知（「通知」群組 / *Notifications*）

欄位	鍵	預設	說明
資料庫備份 (<i>Database Backup</i>)	<code>tgBotBackup</code>	<code>false</code>	將資料庫備份檔案連同報告一起傳送至 Telegram。提示：「傳送帶有資料庫備份檔案的通知」。
登入通知 (<i>Login Notification</i>)	<code>tgBotLoginNotify</code>	<code>true</code>	嘗試登入面板時發出通知。提示：「顯示有人嘗試登入您的面板時的使用者名稱、IP 位址與時間。」
工作階段到期通知延遲 (<i>Expiration Date Notification</i>)	<code>expireDiff</code>	<code>0</code>	在客戶有效期限到期前幾天傳送通知。 <code>0</code> – 停用。允許 <code>>= 0</code> 。提示：「在達到閾值前接收工作階段有效期限到期的通知（值：天）」。
通知用流量閾值 (<i>Traffic Cap Notification</i>)	<code>trafficDiff</code>	<code>0</code>	通知用的剩餘流量閾值。提示：「在達到閾值前接收流量耗盡的通知（值：GB）」。允許 <code>0-100</code> 。
CPU 負載閾值 (<i>CPU Load Notification</i>)	<code>tgCpu</code>	<code>80</code>	若 CPU 負載超過閾值（以**%計），通知管理員。允許 <code>0-100</code> 。提示：「若 CPU 負載超過此閾值，在 Telegram 中通知管理員（值：%）」。

13.6. 日期與時間（「日期與時間」分頁 / *Date and Time*）

時區 (*Time Zone*)

- **鍵：** `timeLocation`
- **預設值：** `Local` (伺服器的系統時區)。
- **格式：** IANA tz 資料庫中的時區名稱 (例如 `Europe/Moscow`、`UTC`、`Asia/Tehran`)。
- **用途：** 面板執行排程任務 (機器人報告、流量重設/檢查、期限到期) 所依據的時區。提示：「排程任務依照此時區的時間執行」。
- **驗證：** 儲存時會檢查時區——不存在的時區會被拒絕。若之後資料庫中出現不正確的值，面板在執行階段會回退至 `Local`，若連 `Local` 也不可用——則回退至 `UTC`。

13.7. 外部流量與 Xray 行為（「外部流量」分頁 / *External Traffic*）

欄位	鍵	預設	說明
外部流量通知 (<i>External Traffic Inform</i>)	<code>externalTrafficInformEnable</code>	<code>false</code>	每次流量更新時通知外部 API。提示：「每次流量更新時通知外部 API。」
外部流量通知 URI (<i>External Traffic Inform URI</i>)	<code>externalTrafficInformURI</code>	""	面板傳送流量更新的目標 URL。儲存時會通過 URL 正確性檢查。提示：「流量更新會傳送至此 URI」。

欄位	鍵	預設	說明
自動停用後重新啟動 Xray (Restart Xray After Auto Disable)	<code>restartXrayOnClientDisable</code>	<code>true</code>	當客戶因有效期限到期或超過流量上限而自動停用時，重新啟動 Xray。提示：「當客戶因有效期限到期或流量上限而自動停用時，重新啟動 Xray。」此切換開關位於「面板」分頁 (General) ——見第 13.2 節；此處為求完整而列出。

13.8. 其他：Xray 組態範本與檢查 URL

Xray 組態範本 (`xrayTemplateConfig`)

- 鍵：`xrayTemplateConfig`
- 預設：內建 (embedded) 的 JSON 範本，隨建置一起提供。
- 用途：Xray-core 組態的基底 JSON 範本，面板會在其之上構建 inbound/outbound。此值在一般取得所有設定的輸出中不會回傳，而是在獨立的 Xray 組態頁面上編輯，而非在面板設定的一般欄位清單中。預設的標準範本可透過 `GET /panel/setting/getDefaultJsonConfig` 取得。

outbound 檢查 URL (`xrayOutboundTestUrl`)

- 鍵：`xrayOutboundTestUrl`
- 預設：`https://www.google.com/generate_204`
- 用途：在檢查 outbound 連線可用性時所使用的 URL。設定時會作為 HTTP(S) URL 進行清理。

13.9. 管理員帳號與 API 權杖

這些參數位於相鄰的分頁（「帳號」 / *Authentication*）上，並在安全相關區段中有詳細說明；此處為各鍵的簡要彙總。

- 變更登入憑證（「目前登入名稱」、「目前密碼」、「新登入名稱」、「新密碼」欄位）以單獨的請求 `POST /panel/setting/updateUser` 儲存。需要正確在目前登入名稱與密碼；新的登入名稱與密碼不得為空。訊息：「您已成功變更管理員的登入憑證。」 / 「使用者名稱或密碼不正確」。
- 雙因素驗證 (2FA) ——鍵 `twoFactorEnable`（預設 `false`）與機密的 `twoFactorToken`。權杖為機密：在已啟用 2FA 的情況下，儲存時的空欄位不會覆寫既有的權杖。在首次啟用 2FA 時，面板會使目前的工作階段失效（提升「登入紀元」）。
- API 權杖 由獨立的端點（`/panel/setting/apiTokens...`）管理：列出、建立（`apiTokens/create`）、刪除、啟用/停用。權杖本身僅在建立時顯示一次，且不會以可讀形式儲存：「請立即複製此權杖。基於安全考量，它不會以可讀形式儲存，且不會再次顯示。」

有關 2FA、密碼、LDAP 同步以及訂閱格式 (JSON/Clash、fragmentation、noises、mux) 的詳情，請參見手冊中相應的獨立區段。

13.10. 3.3.0 中的 API 變更（對整合很重要）

在 3.3.0 版本中，伺服器 API 的路徑結構發生了變更。若您有透過 HTTP 連線面板的外部整合（指令碼、機器人、中央面板、CI 任務），需要對其進行修改，否則它們將停止運作。

⚠ BREAKING CHANGE: `/panel/setting/*` 與 `/panel/xray/*` 端點已移至 `/panel/api` 之下

以前，面板設定管理與 Xray 組態各自獨立存放，位於 `/panel/setting/*` 與 `/panel/xray/*` 路徑下。現在兩組都已註冊在共用的 API 群組 `/panel/api` 內。舊路徑已完全移除——對其的請求將回傳 404。

為何如此做：整個 `/panel/api` 群組都會經過統一的存取檢查，也就是說這些端點現在接受與其餘 API 相同的 `Authorization: Bearer <token>` 標頭。API 權杖即為完整的管理員存取權限，如此一來整個 API 介面便變得一致。

未變更的部分：網頁介面頁面（SPA 路由）`/panel/settings` 與 `/panel/xray` 維持原位——這只涉及伺服器 API 端點。

路徑對應表（舊 → 新）

下方所有路徑的前綴變更——只是在 `/panel/` 之後加上了 `api/`。

原本 (≤ 3.2.x)	變更後 (3.3.0)	方法
<code>/panel/setting/all</code>	<code>/panel/api/setting/all</code>	POST
<code>/panel/setting/defaultSettings</code>	<code>/panel/api/setting/defaultSettings</code>	POST
<code>/panel/setting/update</code>	<code>/panel/api/setting/update</code>	POST
<code>/panel/setting/updateUser</code>	<code>/panel/api/setting/updateUser</code>	POST
<code>/panel/setting/restartPanel</code>	<code>/panel/api/setting/restartPanel</code>	POST
<code>/panel/setting/getDefaultJsonConfig</code>	<code>/panel/api/setting/getDefaultJsonConfig</code>	GET
<code>/panel/setting/apiTokens</code>	<code>/panel/api/setting/apiTokens</code>	GET
<code>/panel/setting/apiTokens/create</code>	<code>/panel/api/setting/apiTokens/create</code>	POST
<code>/panel/setting/apiTokens/delete/:id</code>	<code>/panel/api/setting/apiTokens/delete/:id</code>	POST
<code>/panel/setting/apiTokens/setEnabled/:id</code>	<code>/panel/api/setting/apiTokens/setEnabled/:id</code>	POST
<code>/panel/xray/</code>	<code>/panel/api/xray/</code>	POST
<code>/panel/xray/update</code>	<code>/panel/api/xray/update</code>	POST
<code>/panel/xray/getDefaultJsonConfig</code>	<code>/panel/api/xray/getDefaultJsonConfig</code>	GET
<code>/panel/xray/getXrayResult</code>	<code>/panel/api/xray/getXrayResult</code>	GET
<code>/panel/xray/getOutboundsTraffic</code>	<code>/panel/api/xray/getOutboundsTraffic</code>	GET

原本 (≤ 3.2.x)	變更後 (3.3.0)	方法
/panel/xray/resetOutboundsTraffic	/panel/api/xray/resetOutboundsTraffic	POST
/panel/xray/testOutbound	/panel/api/xray/testOutbound	POST
/panel/xray/warp/:action	/panel/api/xray/warp/:action	POST
/panel/xray/nord/:action	/panel/api/xray/nord/:action	POST
/panel/xray/outbound-subs (與 /outbound-subs/*)	/panel/api/xray/outbound-subs (與 /outbound-subs/*)	GET/POST/DELETE

子路徑的名稱、請求主體與回應格式本身並未變更——只變更了前綴。

如何修復既有的整合

1. 在您的指令碼/組態中找出所有 `/panel/setting/` 與 `/panel/xray/` 的出現之處。
2. 替換前綴：在 `/panel/` 之後立即加上 `api/`（例如 `/panel/setting/all` → `/panel/api/setting/all`）。
3. 請求主體、參數與回應格式無需修改——只變更 URL。
4. 由於設定與 Xray 組態現在位於 `/panel/api` 之下，可以（也應該）使用與 `/panel/api/inbounds/*` 等其他端點相同的 API 權杖 `Authorization: Bearer <token>` 來連線。別忘了套用在整個 `/panel/api` 群組上的 CSRF middleware。

範例：透過 API 讀取所有設定。 以前 (≤ 3.2.x)：

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

現在 (3.3.0) ——在 `/panel/` 之後加上了 `api/`：

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

重新啟動面板同理：POST `/panel/api/setting/restartPanel`。舊路徑 `/panel/setting/restartPanel` 現在會回傳 404。

具型別的 API：結構描述與文件 (Swagger / OpenAPI)

在 3.3.0 中，OpenAPI 規格已完整具備型別。以前，具型別的回應以空物件 `{}` 描述；現在元件與結構描述 (components.schemas) 直接從資料模型產生。因此：

- Swagger UI 顯示真實的資料模型，而非空泛的佔位符。
- 外部產生器 (openapi-generator 等) 可依規格建構出所需語言的現成客戶端。
- 每個具型別的回應都附有指向具體模型的 `$ref`，並附上回應範例。

何處查看 API 文件：

- **內建的 Swagger 頁面**。面板選單中有「API 文件」項目（SPA 路由 `/panel/api-docs`）。此處互動式地列出所有端點及其說明、請求主體與回應範例。
 - **原始的 OpenAPI 3.0 規格** 由 `/panel/api/openapi.json` 位址提供。此 URL 可直接餵給 Postman、Insomnia 或 `openapi-generator`。規格在建置階段內嵌於二進位檔中；當面板在非標準的 `webBasePath` 下運作時，規格中的 `servers` 欄位會自動改寫為目前的基本路徑，以便「Try it out」按鈕與外部產生器命中正確的前綴。
-

14. Telegram 機器人

3X-UI 面板內建 Telegram 機器人，可透過它接收伺服器與用戶端狀態通知，並直接在通訊軟體中管理個別用戶端。機器人採用 long polling（持續輪詢 Telegram）技術，無需外部網域或開放連接埠，只需伺服器能向外連線至 Telegram 伺服器即可。

機器人區分兩類對話者：

- **管理員** – 其 Telegram User ID 已填入機器人設定（「機器人管理員 User ID」欄位）的使用者。可存取所有功能：伺服器統計、備份、用戶端管理、重啟 Xray。
- **用戶端** – 其 Telegram User ID 已綁定至某個 inbound 用戶端（用戶端的 `tgId` 欄位）的任何其他使用者。僅能查看自己的訂閱資訊。

範例：將用戶端綁定至 Telegram。 若要讓使用者收到自己訂閱的統計資料，需將其數字型 Telegram User ID 填入用戶端的 `tgId` 欄位。在用戶端的 JSON 設定中如下所示：

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

完成後，User ID 為 123456789 的使用者即可向機器人傳送 `/usage ivan` 來查看自己的統計資料。管理員也可透過用戶端卡片中的「 設定 Telegram 使用者」按鈕填入同一個 ID，無需手動編輯 JSON。

14.1. 啟用與設定機器人

所有機器人參數均在面板的 **設定** → **Telegram 機器人** 區段中設定。修改設定後直接儲存即可——面板會立即套用，無需重啟面板。若變更了啟用旗標（`tgBotEnable`）、令牌、管理員 User ID 或 API 伺服器位址，面板會自動以新參數停止並重新啟動機器人。先前需在更換令牌後重啟面板的規定已不再適用。

欄位 (UI)	設定鍵值	預設值	說明
啟用 Telegram 機器人	<code>tgBotEnable</code>	<code>false</code>	主開關。提示說明：「透過 Telegram 機器人存取面板功能」。關閉時，機器人不會啟動，通知排程也不會建立。
Telegram 令牌	<code>tgBotToken</code>	(空白)	機器人令牌。提示說明：「請向 Telegram 機器人管理員 @botfather 取得令牌」。令牌為空時機器人無法啟動。
SOCKS 代理伺服器	<code>tgBotProxy</code>	(空白)	連線至 Telegram 的代理伺服器。提示說明 (RU)：「如果您需要使用 Socks5 代理伺服器連線至 Telegram，請依照指南進行設定」。
Telegram API 伺服器	<code>tgBotAPIServer</code>	(空白)	備用 Telegram API 伺服器。提示說明：「所使用的 Telegram API 伺服器。留空則使用預設伺服器」。

欄位 (UI)	設定鍵值	預設值	說明
機器人管理員 User ID	tgBotChatId	(空白)	一個或多個管理員的 Telegram User ID, 以逗號分隔。提示說明：「請使用 @userinfobot 或在機器人中輸入 /id 指令來取得 User ID」。
管理員通知頻率	tgRunTime	@daily	週期性報告排程，採用 crontab 格式。提示說明：「請以 Crontab 格式指定通知間隔」。
備份資料庫	tgBotBackup	false	提示說明：「隨週期性報告附上資料庫備份檔案」。
登入通知	tgBotLoginNotify	true	提示說明：「當有人嘗試登入您的面板時，顯示使用者名稱、IP 位址和時間」。
CPU 使用率通知閾值	tgCpu	80	CPU 使用率閾值（百分比），驗證範圍 0-100。提示說明：「當 CPU 使用率超過此閾值時，向 Telegram 管理員發送通知（單位：%）」。設為 0 時停用 CPU 檢查。
Telegram 機器人語言	—	—	機器人產生所有訊息所使用的語言。

透過 @BotFather 取得令牌

1. 在 Telegram 中開啟與 @BotFather 的對話。
2. 傳送 /newbot 指令並依照說明操作（輸入機器人名稱和以 bot 結尾的唯一 username）。
3. BotFather 會提供格式為 123456789:AA... 的令牌。將其複製至 **Telegram 令牌** 欄位。

取得管理員 User ID

User ID 是帳號的數字識別碼（非 username）。有兩種方式可取得：

- 傳送訊息給 @userinfobot。
- 啟動已設定好的機器人並傳送 /id 指令——機器人會回傳您的 ID。

將取得的數字填入 **機器人管理員 User ID** 欄位。若要指定多位管理員，以逗號分隔各 ID（例如 11111111,22222222）。每個 ID 都會驗證為整數；輸入無效值將導致機器人啟動錯誤。

範例：「機器人管理員 User ID」欄位的值。 單一管理員——只需填入數字：

123456789

兩位管理員以逗號分隔（可不加空格）：

123456789,987654321

每個值必須為整數。@username 或 123 456（數字中含空格）等格式均不適用——機器人將無法啟動。

代理伺服器

支援 `socks5://`、`http://` 和 `https://` 方案。若代理伺服器欄位留空，機器人會嘗試使用面板的通用代理伺服器（若已設定且方案受支援）。URL 方案不受支援或語法不正確時將被忽略——機器人直接連線。當伺服器無法直接存取 Telegram API 時，代理伺服器非常有用。

電子郵件通知 (SMTP)

除 Telegram 外，也可透過電子郵件接收相同事件的通知。此通道在 **設定** → **Email** 的 **SMTP Settings** 分頁中設定：

欄位 (UI)	設定鍵值	預設值	說明
Enable Email Notifications	<code>smtpEnable</code>	<code>false</code>	透過 SMTP 傳送電子郵件通知的主開關。
SMTP Host	<code>smtpHost</code>	(空白)	SMTP 伺服器主機名稱 (例如 <code>smtp.gmail.com</code>)。
SMTP Port	<code>smtpPort</code>	<code>587</code>	SMTP 伺服器連接埠。
SMTP Username	<code>smtpUsername</code>	(空白)	SMTP 驗證使用者名稱，同時用作寄件者 (From) 位址。
SMTP Password	<code>smtpPassword</code>	(空白)	SMTP 驗證密碼。以隱藏方式儲存；若密碼已設定，欄位會顯示「已設定」標記，留空即可保留現有密碼。
Recipients	<code>smtpTo</code>	(空白)	以逗號分隔的收件者清單 (例如 <code>admin@example.com, ops@example.com</code>)。
Encryption	<code>smtpEncryptionType</code>	<code>starttls</code>	連線加密類型： <code>none</code> (無加密)、 <code>starttls</code> (STARTTLS) 或 <code>tls</code> (隱式 TLS)。

Send Test Email 按鈕會傳送測試郵件，並按階段顯示結果：**Connection** (連線)、**Authentication** (驗證) 和 **Send** (傳送)。若有問題，診斷結果會指出哪個階段發生錯誤 (例如「Authentication failed – check username and password」或「Server requires STARTTLS – change encryption type」)，有助於調整設定參數。

第二個分頁 (**Notifications**) 可選擇要接收電子郵件通知的事件——與 Telegram 使用相同的群組卡片 (請參閱第 14.5 節「事件匯流排與通知選擇」)。

Telegram API 伺服器

機器人預設連線至 Telegram 官方 API。在 **Telegram API 伺服器** 欄位中可指定自建 Bot API 伺服器 (`telegram-bot-api`) 的位址。URL 會進行安全性驗證；被封鎖或格式不正確的位址將被捨棄，改用預設伺服器。

14.2. 主選單與按鈕

透過 `/start` 指令呼叫選單。按鈕為附加於訊息的 inline 鍵盤；按鈕組合取決於您是管理員或用戶端。

管理員選單

按鈕	功能
 依流量使用量排序的報告	列出所有用戶端並依流量排序，顯示各用戶端的用量；無資料的 email 標記為「  無結果」。
 伺服器狀態	伺服器摘要資訊（見第 14.5 節）。「  重新整理」按鈕可重新載入資料。
重置所有流量	重置 所有 用戶端的流量計數器。需確認（「您確定嗎？」  ），隨後為每個用戶端顯示「  成功」或「  失敗」，最後顯示「  已完成所有用戶端的流量重置」。
 備份資料庫	傳送資料庫檔案和 <code>config.json</code> （見第 14.6 節）。
 封鎖記錄	傳送因超過 IP 限制而被封鎖的 IP 位址記錄檔。
 Inbound 連線	所有 inbound 摘要：Remark、連接埠、流量、用戶端數量、到期日。
 即將到期	列出流量或有效期即將耗盡的 inbound 和用戶端（見第 14.5 節）。
 指令	顯示管理員指令說明。
 線上	線上用戶端的數量和清單；點擊 email 可開啟用戶端卡片。「  重新整理」按鈕。
 所有用戶端	開啟 inbound 選擇，然後顯示其用戶端清單——供查看/管理使用。
 新增用戶端	啟動新增用戶端精靈（選擇 inbound → 草稿 → 確認）。
訂閱設定 / 個人連結 / QR 碼	選擇 inbound 和用戶端以取得訂閱連結、個人連結或 QR 碼。

用戶端選單

用戶端可使用的按鈕組合較為有限：

按鈕	功能
用戶端統計	顯示與用戶端 Telegram User ID 綁定的所有訂閱資料。
 指令	顯示用戶端指令說明。
訂閱設定	選擇自己的用戶端 → 訂閱連結。
個人連結	選擇自己的用戶端 → 個人連結。
QR 碼	選擇自己的用戶端 → QR 碼。

若使用者沒有任何與其 Telegram User ID 綁定的用戶端，機器人會回覆：「 找不到您的設定！ 請請求管理員在設定中使用您的 Telegram User ID。 您的 User ID：...」。請將此 ID 提供給管理員，由其填入用戶端欄位。

14.3. 機器人指令

自 3.5.0 起，Telegram 的「/」選單中登錄了八個指令：

指令	說明（選單中顯示）	存取權限	功能
/start	顯示主選單	所有人	歡迎訊息；管理員另會看到「  歡迎使用 管理機器人！」和主選單。
/help	機器人說明	所有人	顯示一般歡迎訊息並建議選擇選單項目。
/status	檢查機器人狀態	所有人	回覆「  機器人運作正常」。
/id	顯示您的 Telegram ID	所有人	回傳「  您的 User ID: ...」。方便查詢自己的 User ID。
/usage	顯示用戶端用量：/usage email	所有人（見下文）	3.5.0 起加入選單。
/inbound	搜尋 inbound：/inbound remark (admin)	管理員	3.5.0 起加入選單。
/restart	重啟 Xray Core (admin)	管理員	3.5.0 起加入選單。
/clearall	重設所有用戶端的流量 (admin)	管理員	3.5.0 新增： 會要求確認（「取消」 / 「確認重設流量」按鈕），並將所有用戶端的流量歸零。

參數型指令的細節：

- **/usage [Email]** – 依 email 搜尋用戶端。
 - 對**管理員**：顯示完整用戶端卡片（含管理按鈕）。
 - 對**用戶端**：僅顯示指定 email 的自身訂閱（依 Telegram User ID 綁定）。未輸入參數時，機器人提示：「 請提供搜尋用的 email」。
- **/inbound [連線名稱]** – 僅限管理員。依 Remark 搜尋 inbound，並顯示其參數及所有用戶端的統計資料。未輸入參數（或由用戶端使用時）——「 未知指令」。
- **/restart** – 僅限管理員。重啟 Xray Core。可能的回應：「 Xray Core 已成功重啟」、 Xray Core 未在執行中」（若核心未運作）、「 重啟 Xray Core 時發生錯誤。<錯誤訊息>」。/restart 後若帶有任何參數，將顯示未知指令訊息並提示使用 /restart。

在群組聊天中，/ [?] [?] @botusername 格式的指令僅在 username 與當前機器人名稱相符時才會被處理。

3.5.0 還有三項變更：**線上用戶端**清單中的按鈕標示為 email - inbound [?] [?]（不同 inbound 中的相同名稱不再混淆）；**備份與封鎖日誌**訊息以 Hostname==... 行開頭——當一個機器人服務多個面板時十分方便；依 **Telegram ID** 搜尋用戶端卡片不再受設定 JSON 格式影響（先前來自節點或匯入的「緊湊」記錄會找不到）。

管理員說明（「指令」按鈕）：

```

 [?] [?] [?] [?] Xray Core: /restart
 [?] [?] [?] email [?] [?] [?] [?] [?] /usage [Email]
 [?] [?] [?] inbound [?] [?] [?] [?] [?] [?] [?] [?] /inbound [?] [?] [?] [?]
 [?] [?] [?] Telegram User ID: /id

```

用戶端說明：

```
$ [?] [?] [?] [?] [?] [?] [?] /usage [Email]
[?] [?] [?] [?] Telegram User ID : /id
```

14.4. 用戶端管理（僅限管理員）

開啟用戶端卡片後（透過「所有用戶端」、「線上」、「即將到期」或 `/usage`），管理員可查看用戶端資訊（email、綁定的 inbound、「已啟用」狀態、連線狀態、到期日、流量用量）及 inline 管理按鈕：

按鈕	用途
 重新整理	重新讀取用戶端卡片。
 重置流量	清零用戶端的流量計數器。需確認「  確認重置流量？」。
 流量限制	設定流量限制。預設選項：  無限制（0）、1/5/10/20/30/40/50/60/80/100/150/200 GB，或「  自訂」——在內建數字鍵盤（按鈕 0-9、「  」重置為 0、「  」刪除最後一位、「  確認：N」）上輸入數值。數值單位為 GB。
 修改到期日	預設選項：  無限制、「  自訂」、延長 7/10/14/20 天、1/3/6/12 個月。正數延長有效期（從當前到期日或若已過期則從「現在」起加天數）；0 取消時間限制。
 IP 記錄	顯示用戶端已記錄的 IP 位址（含時間戳，若有）。記錄中可使用「  重新整理」和「  清除 IP」（需確認「  確認清除 IP？」）。
 IP 限制	限制同時連線的 IP 數量。選項：  無限制（0）、1-10 或「  自訂」（數字鍵盤）。
 設定 Telegram 使用者	顯示目前綁定的用戶端 Telegram User ID；允許清除綁定（「  移除 Telegram 使用者」，需確認）。透過系統 Telegram 聯絡人選擇器綁定新使用者。
 啟用/停用	啟用或停用用戶端。需確認「  確認啟用/停用使用者？」。

所有修改設定的操作（流量/IP 限制、到期日、綁定/解除綁定 Telegram 使用者、啟用/停用）在必要時會將 Xray 標記為需要重啟，以使變更生效。操作成功後，機器人顯示「：...」確認訊息，並重新顯示用戶端卡片。

精靈中數字輸入的值限制在 < 999999。

14.5. 通知與報告

通知會傳送給所有管理員（`tgBotChatId` 中的所有 User ID）。

事件匯流排與通知選擇

通知建立在統一事件匯流排之上，提供兩個傳送通道——Telegram 和電子郵件（SMTP）。每個通道可單獨選擇要通知哪些事件。在 設定 → Telegram 的 **Notifications** 分頁，以及 設定 → Email 的同名分頁中進行設定。

事件以卡片群組呈現；每個群組有一個主開關，顯示已啟用事件的計數（n/總計），部分選取時顯示中間狀態。可用群組：

- **Outbound** — 「Down」（`outbound.down`）和「Up」（`outbound.up`）：outbound 斷線與恢復。
- **Xray Core** — 「Crash」（`xray.crash`）：Xray 核心異常終止。
- **Nodes** — 「Down」（`node.down`）和「Up」（`node.up`）：節點變為不可用或恢復。
- **System** — 「CPU high (%)」（`cpu.high`）和「Memory high (%)」（`memory.high`）：CPU 和記憶體高使用率。兩個事件旁均有百分比閾值的 inline 輸入欄位。
- **Security** — 「Login attempt」（`login.attempt`）：嘗試登入面板。

已啟用事件集合分別儲存：Telegram 使用 `tgEnabledEvents`，Email 使用 `smtpEnabledEvents`。預設兩個通道均啟用「Login attempt」和「CPU high」（值為 `login.attempt,cpu.high`）。

面板登入通知

由 **登入通知**（`tgBotLoginNotify`，預設啟用）核取方塊控制。每次嘗試登入網頁面板時，管理員會收到訊息：

- 成功時：「 成功登入面板。」 + 主機、使用者名稱、IP、時間。
- 失敗時：「 面板登入失敗。」 + 主機、**原因**（例如 2FA 錯誤時顯示「2FA 錯誤」）、使用者名稱、IP、時間。

CPU 與記憶體使用率超標

面板每分鐘檢查一次 CPU 和記憶體使用率。若 `tgCpu` > 0 且一分鐘平均 CPU 使用率超過此閾值，管理員會收到：「 處理器使用率為 N%，超過閾值 M%」。同樣地，RAM 使用率也會與 `tgMemory`（預設 80%）閾值比較——事件「Memory high (%)」。

兩個閾值均在 Notifications 分頁的 **System** 群組中，分別位於「CPU high (%)」和「Memory high (%)」事件旁的 inline 欄位設定（見上方「事件匯流排與通知選擇」）。Email 通道使用獨立的鍵值 `smtpCpu` 和 `smtpMemory`。閾值設為 0 時停用對應的檢查。

週期性報告（按排程）

依 **通知頻率**（`tgRunTime`，預設 `@daily`）欄位的 cron 表達式排程。若值為空或無效，則使用 `@daily`。報告內容包含：

排程設定精靈

管理員通知頻率 欄位不需手動輸入字串，而是透過排程設定精靈進行設定。首先在下拉式選單中選擇模式：

- **@every** — 以間隔重複 — 出現數字輸入欄位和單位選擇器（秒 / 分鐘 / 小時）；結果組合為 `@every 6h` 格式的表達式。
- **@hourly** — 每小時、**@daily** — 每天 00:00、**@weekly** — 每週、**@monthly** — 每月 — 預設選項，儲存為對應的巨集（`@hourly`、`@daily`、`@weekly`、`@monthly`）。

- **自訂 (crontab)** – 供輸入自訂 crontab 表達式的欄位。面板的排程器啟用了秒欄位，因此自訂表達式由 6 個欄位組成：秒、分、時、日（月份）、月、日（星期）（例如 `0 30 8 * * *` – 每天 08:30:00）。切換至此模式時，欄位會填入當前選擇的 crontab 等效表達式，作為參考起點。

範例：「通知頻率」（`tgRunTime`）欄位的值。支援預設縮寫和完整 crontab 格式：

值	觸發時機
@daily	每天午夜一次（預設值）
@hourly	每小時
@every 6h	每 6 小時
<code>0 9 * * *</code>	每天 09:00
<code>0 9 * * 1</code>	每週一 09:00
<code>0 */12 * * *</code>	每 12 小時（00:00 和 12:00）

crontab 欄位順序：分、時、日（月份）、月、日（星期）。

1. 「 排程報告：<排程>」文字及當前日期/時間。
2. 伺服器狀態（見下方）。
3. 依 inbound 和用戶端的「即將到期」區塊。
4. 給已綁定 Telegram User ID 的用戶端的個人通知——每位非管理員用戶端會收到其流量/有效期即將耗盡的訂閱清單（含已停用的）。
5. 若已啟用備份資料庫（`tgBotBackup`）——向管理員傳送資料庫備份。

伺服器狀態包含：主機、3X-UI 和 Xray 版本、IPv4/IPv6、運行時間（天數）、平均負載（Load1/2/3）、RAM（當前/總計）、線上用戶端數量、TCP/UDP 連線計數、總網路流量（↑/↓）及 Xray 狀態。

「即將到期」顯示：

- 依 inbound：已停用的數量和「即將耗盡」的數量，然後列出這些 inbound（Remark、連接埠、流量、到期日）；
- 依用戶端：同上，加上用戶端卡片和帶有 email 的按鈕（點擊可開啟用戶端卡片）。

「即將耗盡」的閾值取自面板通用設定：流量餘量（GB）和有效期餘量（天數）。當 inbound/用戶端的剩餘流量低於閾值或距到期日不足閾值天數時，即視為「即將耗盡」。

14.6. 備份與記錄

- **備份資料庫**（ 備份資料庫 按鈕或週期性報告中的核取方塊）：機器人傳送備份時間、資料庫檔案（`x-ui.db`，PostgreSQL 則為 `x-ui.dump`）及 Xray 設定檔 `config.json`。

機器人傳送的備份檔案名稱依伺服器位址而定：使用 `webDomain` 的值，若未設定則使用伺服器的公開 IP。這有助於在從多個面板收集備份時辨識檔案來源。若無法確定位址，則使用通用名稱。

- **封鎖記錄**（ 封鎖記錄 按鈕）：傳送因超過 IP 限制而被封鎖的 IP 位址的當前和前一份記錄檔。空檔案不會傳送。

14.7. 運作特性

- **長訊息**分割為多個部分（閾值約 2000 字元），inline 鍵盤附加至最後一部分。
 - **並行處理**：指令和按鈕點擊以並行方式處理（最多 10 個同時執行的處理器）。
 - **傳送可靠性**：連線錯誤時，訊息以指數退避方式重試傳送（1 秒/2 秒/4 秒，最多 3 次）。
 - **快取**：「伺服器狀態」資料會被快取，以避免頻繁點擊「重新整理」對系統造成負擔。
 - **機器人重啟**：儲存影響機器人的設定（啟用旗標、令牌、管理員 User ID 或 API 伺服器位址）時，面板會自動停止先前的輪詢循環並以最新參數啟動新循環——無需重載面板。同一時間只有一個接收更新的實例在運行。
-

15. 地理資料庫 (geoip / geosite 與自訂)

地理資料庫是 Xray-core 用於依國家歸屬 (IP 區段) 或網域類別來進行流量路由與過濾的 .dat 二進位檔案。面板既能下載與更新標準的地理檔案集合，也能更新以 URL 指定的任意使用者來源。所有檔案都儲存在 Xray 二進位檔旁的 bin 目錄 (預設路徑為 bin，可透過環境變數 XUI_BIN_FOLDER 覆寫)。

15.1. geoip.dat 與 geosite.dat 是什麼

- **geoip.dat** – 「IP 位址 → 國家/地區代碼」的對應資料庫。在路由規則中以 geoip:<[?]> 的形式使用，例如 geoip:ru、geoip:cn，以及特殊標記 geoip:private (私有/本地網路)。本質上它回答的是「這個 IP 位於哪個國家」這個問題。
- **geosite.dat** – 「網域 → 類別/清單」的對應資料庫。以 geosite:<[?]> 的形式使用，例如 geosite:category-ads-all (廣告網域)、geosite:google、geosite:ru。本質上它是分組的網域清單。

這些檔案用於建構諸如「指向俄羅斯 IP/網域的所有流量直連，其餘走 outbound」之類的規則。規則本身在 Xray 的路由區段中設定；地理資料庫只是為其提供資料。沒有最新的地理檔案，引用 geoip: / geosite: 的規則將無法生效，或會依賴過時的清單。

範例：「俄羅斯網域與 IP 直連」規則。路由區段中的此規則會把指向俄羅斯資源的所有流量導向帶有 direct 標籤的 outbound：

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. 標準地理檔案及其更新

面板包含一份固定的「白名單」 (allowlist)，由六個帶有硬編碼下載來源的標準檔案組成。更新透過 POST /panel/api/server/updateGeofile/:fileName 執行 (或不帶檔案名稱以一次更新全部)。

範例：透過 API 更新單一檔案與一次更新全部。只更新 geoip_RU.dat：

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

以一個請求更新全部六個標準檔案 (不指定檔案名稱)：

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

成功回應：

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

檔案名稱	來源（發行版儲存庫）
geoip.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoip.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoip_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geoip.dat)
geosite_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geosite.dat)
geoip_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoip.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

標準檔案更新的特性：

- **更新單一檔案的按鈕**。下載前會顯示確認：「您確定要更新地理檔案嗎？」並附說明「這將更新檔案 #filename#。」（英文 *Do you really want to update the geofile? This will update the #filename# file.*）。成功時會彈出通知「地理檔案更新成功」（英文 *Geofile updated successfully*）。
- **「Update all」按鈕**（英文 *Update all*）會下載全部六個檔案。確認：「這將更新所有地理檔案。」（英文 *This will update all geofiles.*）。
- **條件式下載**。若本地檔案已存在，請求中會帶入 `If-Modified-Since` 標頭，附上檔案的修改時間。伺服器回應 `304 Not Modified` 表示檔案未變更——不會重新下載，只更新檔案的時間戳記。
- **檔案名稱安全性**。只接受 `allowlist` 中的名稱；名稱會檢查是否不含 `..`、路徑分隔符 `/` 與 `\`、絕對路徑，且必須符合 `^[a-zA-Z0-9._-]+\\.dat$` 樣式。任何清單外的名稱都會以錯誤「Invalid geofile name」被拒絕。
- **Xray 重啟**。下載地理檔案後，Xray-core 會重啟以重新讀取更新後的資料庫。若重啟失敗，錯誤訊息中會附加相應的字串。

從命令列更新地理資料庫（x-ui）

地理資料庫也可以不透過面板更新——透過互動式選單 `x-ui`（更新地理檔案的項目），或非互動指令 `x-ui update-all-geofiles`。集合中的每個檔案（geoip/geosite，包含 IR 與 RU 集合）都會輸出各自的狀態：「已更新」、「已是最新」或「下載錯誤」。下載失敗時不會列印虛假的成功訊息。只有在至少有一個檔案確實被更新時，才會發生 Xray 重啟（也就是中斷活躍連線）；若沒有任何檔案變更（全部回傳 `304 Not Modified`），則面板與 Xray 都不會重啟。

15.3. 透過 Xray 自動更新地理資料（Geodata Auto-Update）

以任意 URL 指定的額外 `.dat` 來源，並非透過面板新增，而是透過 Xray-core 原生的 `geodata` 區段。相應的章節被放置於 Xray 更新的模態視窗中（儀表板 → Xray 更新，`xrayUpdates`）——即「自動更新 Geodata」分頁（英文 *Geodata Auto-Update*）。面板在此僅編輯 Xray 設定範本中的 `geodata` 鍵；檔案的下載、驗證與熱重載由 Xray 核心自身負責。

章節上方會顯示提示：「Xray 會依排程下載這些檔案並在不重啟的情況下熱重載它們。URL 必須為 HTTPS。在 Xray 能更新某檔案之前，該檔案必須已存在於 bin 資料夾中。」（英文 *Xray downloads these files on*

schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.) 。

章節欄位

- **排程 (cron)** (英文 *Schedule (cron)*) – 5 個欄位的 cron 字串；預設值為 `0 4 * * *` (每天 04:00) 。儲存時會檢查字串是否剛好包含 5 個欄位，否則會輸出錯誤「Cron 必須包含 5 個欄位，例如 `0 4 * * *`」。
- **透過 outbound 下載 (選填)** (英文 *Download through outbound (optional)*) – 一個下拉清單，列出可用的 outbound 標籤 (外加訂閱的 outbound)，Xray 會透過它來下載檔案；協定為 `blackhole` 的 outbound 不會出現在清單中。此欄位可留空——此時將使用直接連線。此選擇與面板自身請求所用的 outbound 無關 (見 §11)：geodata 自動更新有其專屬、獨立的下載用 outbound。
- **檔案清單** – 每一行設定一組「URL + 檔案名稱」 (英文 *File name*)。URL 必須以 `https://` 開頭 (否則顯示「每個檔案都需要 HTTPS URL。」) 。檔案名稱以單純形式指定，不含路徑與分隔符——只能是 `^[A-Za-z0-9._-]+$` 字元 (否則顯示「檔案名稱必須單純，例如 `geosite_custom.dat` (不含路徑)。」) 。輸入 URL 時，面板會嘗試從路徑的最後一段自動填入檔案名稱。「Add file」按鈕 (英文 *Add file*) 會新增一行，垃圾桶按鈕會刪除該行。

若清單為空，會顯示提示：「尚未設定檔案。在路由規則中以 `ext:geosite_custom.dat:category` 形式引用檔案。」 (英文 *No files configured. Reference files in routing rules as ext:geosite_custom.dat:category.*) 。

儲存

「儲存並重啟 Xray」按鈕 (英文 *Save & Restart Xray*) 會顯示確認「儲存 geodata 設定？」並附說明「Xray 設定範本將被更新，且 Xray 會被重啟。」 (英文 *Save geodata settings? This updates the Xray config template and restarts Xray.*)。儲存後，`geodata` 鍵會寫入設定範本 (`POST /panel/api/xray/update`)，且 Xray 會重啟 (`POST /panel/api/server/restartXrayService`)。若檔案清單為空，`geodata` 鍵會從範本中移除。

重要特性：

- **檔案必須已存在於 bin**。Xray 只更新在啟動時已存在於 `bin` 資料夾中的 `.dat` 檔案。因此新的自訂檔案要先手動放入 `bin` (或至少在那裡以所需名稱建立一個空白/過時版本)，之後 Xray 才會開始依排程維持其最新狀態。
- **熱重載**。在排程下載後，Xray 會重新讀取更新後的資料庫，而不需要完整重啟程序。
- **相容性**。先前下載的地理檔案 (無論標準或自訂) 都會繼續以 `ext:` 語法在路由規則中運作，無需變更。

若清單為空，會顯示提示：「尚無自訂 geo 來源——點擊「新增」來建立一個」 (英文 *No custom geo sources yet – click Add to create one*) 。

表格欄位與來源欄位

欄位 (UI)	JSON	預設值	說明
類型 (<i>Type</i>)	<code>type</code>	– (必填)	資源類型：僅 <code>geosite</code> 或 <code>geoip</code> 。決定最終檔案的名稱。

欄位 (UI)	JSON	預設值	說明
別名 (Alias)	alias	— (必填)	來源的簡短識別碼。由它與類型組成檔案名稱。
URL (URL)	url	— (必填)	指向 .dat 檔案的直接連結 (http/https)。
已啟用 (Enabled)	—	—	來源在清單中是否啟用的標記。
已更新 (Last updated)	lastUpdatedAt	0	上次成功更新的時間 (Unix 時間；0 表示尚未更新)。
路由 (ext:...) (Routing (ext:...))	—	—	路由規則的現成字串：ext:<[?][?].dat>:tag。
動作 (Actions)	—	—	「編輯」、「刪除」、「立即更新」按鈕。

此外，資料庫中還儲存了內部欄位：localPath (檔案在 bin 目錄中的實際路徑)、lastModified (伺服器回傳的 Last-Modified 標頭值，用於條件式下載)、createdAt 與 updatedAt。

檔案命名

最終檔案的名稱會自動由類型與別名組成：

- 類型 geoip → geoip_<alias>.dat；
- 類型 geosite → geosite_<alias>.dat。

例如，類型為 geosite 且別名為 myads 的來源會建立檔案 geosite_myads.dat。

範例：透過 API 新增來源。 將你自己的廣告網域清單新增為別名 myads 的 geosite 資源：

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

面板會將檔案下載到 bin 目錄並命名為 geosite_myads.dat，儲存記錄並重啟 Xray。

按鈕與動作

- **新增** (英文 Add) — 開啟「新增來源」表單 (英文 Add custom geo)。儲存按鈕為「儲存」 (英文 Save)。API: POST /add。
- **編輯** (英文 Edit) — 「編輯來源」表單 (英文 Edit custom geo)。API: POST /update/:id。變更類型或別名時，舊檔案會被刪除，新檔案會重新下載。

- **刪除**（英文 *Delete*）－ 確認「刪除此自訂來源？」（英文 *Delete this custom geo source?*）。刪除資料庫中的記錄以及 `.dat` 檔案本身。API: `POST /delete/:id`。成功時：「自訂 geo 檔案「<名稱>」已刪除」。
- **立即更新**（英文 *Update now*）－ 重新下載特定來源並更新時間戳記。API: `POST /download/:id`。成功時：「Geofile「<名稱>」已更新」。
- **更新全部**－ 一次更新所有自訂來源。API: `POST /update-all`。完全成功時：「所有自訂 geo 來源已更新」（英文 *All custom geo sources updated*）。若至少有一個來源未能更新，操作視為部分失敗，訊息為「無法更新一個或多個自訂來源」（英文 *One or more custom geo sources failed to update*），且回應中會列出成功與失敗的來源。

任一動作（新增、編輯、刪除、更新、有成功項目時的更新全部）之後，Xray-core 都會重啟。

逐步操作：新增來源

1. 點擊「新增」。
2. 在「類型」欄位選擇 `geosite` 或 `geoip`。
3. 在「別名」欄位輸入識別碼（只能是小寫拉丁字母、數字、`-` 與 `_`；佔位提示：`a-z 0-9 _ -`）。
4. 在「URL」欄位指定指向 `.dat` 檔案的直接連結（必須以 `http://` 或 `https://` 開頭）。
5. 點擊「儲存」。面板會立即將檔案下載到 `bin` 目錄、儲存記錄並重啟 Xray。

15.4. 驗證與限制

建立與編輯來源時會執行嚴格的檢查。錯誤訊息：

條件	訊息 (RU)	訊息 (EN)
類型不是 <code>geosite / geoip</code>	Тип должен быть geosite или geoip	<i>Type must be geosite or geoip</i>
別名為空	Укажите псевдоним	<i>Alias is required</i>
別名含不允許字元（不符合 <code>^[a-z0-9_-]+\$</code> ）	Псевдоним содержит недопустимые символы	<i>Alias must match allowed characters</i>
別名被保留	Этот псевдоним зарезервирован	<i>This alias is reserved</i>
URL 為空	Укажите URL	<i>URL is required</i>
URL 無法解析	Некорректный URL	<i>URL is invalid</i>
協定不是 <code>http/https</code>	URL должен использовать http или https	<i>URL must use http or https</i>
主機為空/無效，或被 SSRF 防護封鎖	Некорректный хост URL	<i>URL host is invalid</i>
「類型 + 別名」重複	Такой псевдоним уже используется для этого типа	<i>This alias is already used for this type</i>
找不到來源	Источник не найден	<i>Custom geo source not found</i>
下載錯誤	Ошибка загрузки	<i>Download failed</i>

表單中的提示（用戶端驗證）：「別名：只能用 `a-z`、數字、`-` 與 `_`」（*Alias may only contain lowercase letters, digits, - and _*）以及「URL 必須以 `http://` 或 `https://` 開頭」（*URL must start with http:// or https://*）。

額外的技術限制：

- **保留別名**。不能使用與標準檔案衝突的別名。已保留（比較不分大小寫，連字號等同於底線）：
`geoip`、`geosite`、`geoip_ir`、`geosite_ir`、`geoip_ru`、`geosite_ru`。例如，`geosite-ru` 會被當作 `geosite_ru` 而遭拒絕。
- **SSRF 防護**。URL 的主機會被解析為 IP，若它指向私有/內部位址，下載會被封鎖（使用者會看到「Некорректный хост URL」）。這可防止利用面板存取內部服務。
- **path traversal 防護**。最終檔案路徑必須位於 `bin` 目錄內（含符號連結解析）；嘗試越出其範圍將被拒絕。
- **最小檔案大小**。下載的檔案只有在不小於 64 位元組時才被視為有效；過小的檔案會以下載錯誤被拒絕。
- **代理與條件式下載**。若面板設定中指定了代理，下載會透過它進行；其餘情況則使用具 SSRF 安全傳輸的直接連線。如同標準檔案，會套用 `If-Modified-Since / 304 Not Modified`（未變更的檔案不會重新下載）。下載逾時為 10 分鐘，URL 可用性探測（HEAD，必要時為部分 GET）為 12 秒。

15.5. 面板啟動時的自動檢查

面板啟動時會遍歷所有自訂來源，並對每一個檢查本地檔案是否存在與完整（檔案缺失、為目錄，或小於 64 位元組）。若檔案缺失或損毀，會對來源進行探測並嘗試重新下載。這確保在重新安裝或 `bin` 目錄遺失後，自訂地理檔案會被自動還原。

15.6. 在路由規則中使用地理資料庫

在 Xray 的路由規則中，地理資料庫透過前綴用於 `domain / ip` 之類的欄位：

- **geoip**: 用於 IP 資料庫——`geoip:<[?][?]>`。範例：`geoip:ru`、`geoip:cn`、`geoip:private`。取自 `geoip.dat`（若規則指向特定檔案，則取自 `geoip_RU.dat` 等）。
- **geosite**: 用於網域資料庫——`geosite:<[?][?]>`。範例：`geosite:category-ads-all`、`geosite:google`、`geosite:ru`。取自 `geosite.dat`。

範例：透過 geosite 封鎖廣告。此規則將所有廣告網域送往「黑洞」（假設有帶 `blocked` 標籤、協定為 `blackhole` 的 outbound）：

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

對於自訂檔案，使用外部檔案語法 `ext:`。UI 中的提示：「在路由規則中以 `ext:檔案.dat:標籤` 的形式使用該值（替換標籤）。」（英文 *In routing rules use the value column as `ext:file.dat:tag` (replace tag).*）。格式：

```
ext:<[?][?][?][?].dat>:<[?][?]>
```

其中 `<[?][?][?][?].dat>` 為 `geoip_<alias>.dat` 或 `geosite_<alias>.dat`，而 `<[?][?]>` 為檔案內部的特定清單/類別。面板在「路由 (ext:...)」欄位中會提示形如 `ext:geosite_myads.dat:tag` 的現成範本——只需

把 `tag` 替換為所需的標籤。此類檔案的名稱在「自動更新 Geodata」章節（見 §15.3）的「檔案名稱」欄位中設定——例如 `geosite_custom.dat`；在規則中以 `ext:geosite_custom.dat:category` 引用它。

範例：基於自訂檔案的規則。 若新增了類型為 `geosite`、別名為 `myads` 的來源，而 `.dat` 檔案內部的清單以 `ads` 標籤標記，路由規則如下：

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

對於 IP 來源（類型 `geoip`、別名 `mycorp`、標籤 `office`），欄位會是 `"ip": ["ext:geoip_mycorp.dat:office"]`。

16. 日常運維：備份、日誌、更新、CLI

本章涵蓋面板的日常維護工作：建立與還原資料庫備份、檢視面板及 Xray 的日誌、重啟與停止服務、更新 Xray 及面板本身、週期性排程任務（cron）以及解除安裝面板。部分操作可在網頁介面（「儀表板」和「面板設定」頁面的各分頁）中完成，部分操作則需在伺服器上透過 `x-ui` 主控台選單執行。

16.1. 資料庫備份與還原

面板的所有資料（inbound、用戶端、群組、節點、設定）都儲存在單一資料庫中。備份管理位於「儀表板」頁面的「備份」分頁，區塊標題為「備份與還原」。

面板支援兩種資料庫引擎，備份行為因此有所不同：

- **SQLite**（預設選項）——資料儲存於 `x-ui.db` 檔案中。
- **PostgreSQL**——若面板已設定使用 PostgreSQL，區塊中將顯示提示：

「本面板正在 PostgreSQL 上執行。「備份」會下載 `pg_dump` 封存檔（.dump），而「還原」會透過 `pg_restore` 將其上傳回去。還原也接受 SQLite 資料庫（.db）或 SQLite 遷移轉儲，並將其資料匯入 PostgreSQL。伺服器上必須安裝 PostgreSQL 用戶端工具（`pg_dump` 和 `pg_restore`）。」

匯出（建立備份）

「匯出資料庫」按鈕（Back Up）會將備份檔案下載至您的裝置。

資料庫引擎	檔案名稱	伺服器端行為
SQLite	<code>{host}_YYYY-MM-DD_HHMMSS.db</code>	先執行 WAL checkpoint 以確保檔案包含最新記錄，再將整個檔案讀取後提供下載
PostgreSQL	<code>{host}_YYYY-MM-DD_HHMMSS.dump</code>	執行 <code>pg_dump</code> ，將封存檔提供下載

介面提示：

- SQLite：「點選以將含有您目前資料庫備份的 .db 檔案下載至您的裝置。」
- PostgreSQL：「點選以將目前資料庫的 PostgreSQL 轉儲（.dump）下載至您的裝置。」

從技術角度來看，匯出是對 `GET /panel/api/server/getDb` 的請求。附件名稱由伺服器根據引擎類型設定（Content-Disposition）。

自 3.4.2 起，下載的備份檔案名稱包含您開啟面板所用的位址以及日期時間：`{host}_YYYY-MM-DD_HHMMSS`，副檔名為 `.db`（SQLite）或 `.dump`（PostgreSQL）——例如 `panel.example.com_2026-06-27_000000.db`。如此檔案可依伺服器分組並依時間排序，同一天的多份備份也不會互相覆寫。Telegram 機器人傳送的備份也使用相同名稱（`host` 取自面板的網域/IP，回退值為 `x-ui`）。

範例：透過 API 下載備份。 同一匯出操作也可透過主控台請求取得，例如用於自動備份腳本。需要已認證的會話（登入 cookie）：

```
# 1) 透過 API 下載 session cookie
curl -s -c cookies.txt \
  -d 'username=admin&password=admin' \
  https://panel.example.com:2053/panel/login

# 2) 透過 API 下載 x-ui.db 資料庫備份
curl -s -b cookies.txt -OJ \
  https://panel.example.com:2053/panel/api/server/getDb
```

若面板透過基礎路徑（Web Base Path）開放，需將其加入 URL：...:2053/<base_path>/panel/api/server/getDb。

匯入（還原）

「**匯入資料庫**」按鈕（Restore）開啟檔案選擇器，並將檔案上傳至伺服器進行還原（POST /panel/api/server/importDB，表單欄位 db）。

介面提示：

- SQLite：「點選以從您的裝置選取並上傳 .db 檔案或遷移轉儲 (.dump)，以還原資料庫。」
- PostgreSQL：「點選以選取並上傳 PostgreSQL 備份 (.dump)、SQLite 資料庫 (.db) 或 SQLite 遷移轉儲以還原資料庫。此操作將替換所有目前資料。」

自 3.5.0 起，檔案類型依內容判斷（PGDMP --pg_dump 封存檔；「SQLite format 3」標頭 -- .db 資料庫；開頭為 PRAGMA / BEGIN TRANSACTION -- 文字 SQL 轉儲），而非依副檔名；檔案選擇對話框在兩種資料庫上均接受 .dump, .db。上傳至 PostgreSQL 面板的 SQLite 檔案會以單一交易匯入 PostgreSQL（見 3.14）；匯入前會先驗證檔案確為面板資料庫，舊 schema 會自動補齊遷移。還原 pg_dump 封存檔前，面板會預先（在停止 Xray 之前）檢查其可讀性：若轉儲由較新版本的 PostgreSQL 建立，會顯示帶有確切指令的錯誤 -- 例如「...run 'x-ui pgclient 17'...」。

SQLite 的匯入流程（重要：此流程具原子性且支援回退）：

1. 檢查上傳檔案的格式 -- 必須是有效的 SQLite 資料庫；否則返回錯誤「Invalid db file format」。
2. 將檔案儲存為臨時的 x-ui.db.temp 並進行完整性檢查。
3. 替換資料庫之前，**Xray 會停止**。
4. 目前資料庫重新命名為備用備份 x-ui.db.backup（回退用）。
5. 臨時檔案移動至工作資料庫位置，執行初始化和 schema 遷移，然後進行 inbound 遷移。
6. 若任何步驟失敗 -- 執行回退：從 x-ui.db.backup 還原先前的資料庫，Xray 以舊資料重新啟動。
7. 成功後刪除回退檔案，**Xray 會自動以還原的資料重啟**。

介面操作結果訊息：

結果	訊息
成功	「資料庫已成功匯入」
匯入錯誤	「匯入資料庫時發生錯誤」

結果	訊息
讀取檔案錯誤	「讀取資料庫時發生錯誤」

還原操作會完全替換目前資料。由於 Xray 在此過程中會短暫停止，現有用戶端連線將在匯入期間中斷。

引擎間的遷移檔案 (SQLite ⇌ PostgreSQL)

除了一般備份之外，還有「下載遷移檔案」(Download Migration，請求 `GET /panel/api/server/getMigration`) 功能。它生成一個可攜式檔案，用於切換到另一種資料庫引擎：

自 3.5.0 起，此按鈕僅顯示於 PostgreSQL 面板（在 SQLite 上，一般 `.db` 備份現在可直接還原至 PostgreSQL 面板——不再需要單獨的匯出）：

目前引擎	下載內容	檔案名稱	用途
PostgreSQL	根據 PostgreSQL 資料建立的 SQLite 資料庫	<code>x-ui.db</code>	將面板切換回 SQLite

提示：「點選以下載根據您的 PostgreSQL 資料建立的 SQLite 資料庫 (`.db`)，可直接用於在 SQLite 上執行面板。」

使用 CLI 命令 `x-ui migrateDB [file]` 也可完成 SQLite 的 `.db` ⇌ `.dump` 轉換（見第 16.7 節）。此外，自 3.5.0 起，引擎間的遷移已改為交易式且無損（用戶端群組與全域流量計數器等也會一併轉移；匯入失敗不會變更目標資料庫），並新增 `x-ui pgclient [版本]` 指令與 PostgreSQL 選單中的 **10. Install/Upgrade client tools (pg_dump/pg_restore)** 項目，用於更新 PostgreSQL 用戶端工具（必要時會接入官方 PostgreSQL 套件庫）。

透過 Telegram Bot 備份

若已設定 Telegram Bot（見通知相關章節），它可以直接將備份傳送至管理員聊天室。Telegram 備份包含兩個檔案：資料庫本身（`x-ui.db`，PostgreSQL 為 `x-ui.dump`）和 Xray 設定 `config.json`。訊息前有一行「[?] 備份時間：...」。

有兩種方式可透過 Telegram 取得備份：

1. **按需請求**。Bot 選單中的「 備份資料庫」按鈕——Bot 立即將檔案傳送至目前聊天室。
2. **隨報告自動傳送**。Bot 設定中有「資料庫備份」（Database Backup）開關，描述為「傳送含資料庫備份檔案的通知」。啟用後，每次週期性報告傳送後，Bot 會向所有管理員附上備份。報告傳送的週期由 Bot 的 cron 排程設定（見第 16.6 節）。Bot 在檔案之間及各管理員之間設有暫停，以避免超出 Telegram 的速率限制。

Bot 備份僅在 Bot 已啟動時才會傳送；在 PostgreSQL 上還需要伺服器上安裝 `pg_dump`。

16.2. 日誌檢視

面板有兩個獨立的日誌檢視工具，均可從「儀表板」的「日誌」分頁開啟。每個視窗都可以重新整理（標題中的「重新整理」圖示）並將顯示內容下載為 `x-ui.log` 檔案（下載圖示按鈕）。

面板日誌（應用程式/syslog）

面板日誌視窗（`POST /panel/api/server/logs/{count}`）。控制項：

元素	預設值	說明
行數	20	下拉選單：20 / 50 / 100 / 500 / 1000
等級	Info	最低等級：Debug / Info / Notice / Warning / Error
SysLog（勾選框）	停用	日誌來源：來自應用程式緩衝區或系統日誌
自動更新（勾選框）	停用	每 5 秒重新讀取日誌（見下文）

行為取決於 **SysLog** 勾選框：

- **停用（預設）**：日誌取自面板的內部環形緩衝區，按所選等級過濾。記錄顯示時帶有等級（DEBUG / INFO / NOTICE / WARNING / ERROR）和來源：X-UI：表示面板本身的訊息，XRAY：表示轉發的 Xray 訊息。

不含時間戳記和等級的簡單通知（例如 Windows 上的系統訊息「Syslog is not supported」）現在完整原樣顯示。系統嚴格識別 `YYYY/MM/DD LEVEL - [ ]` 格式；其他所有內容不作解析，因此此類行不再被截斷（之前前三個詞被錯誤識別為日期/時間/等級）。

- **啟用**：面板在伺服器上執行 `journalctl -u x-ui --no-pager -n <count> -p <level>`，即顯示 `x-ui` 服務的系統日誌。允許的行數範圍為 1 至 10000；等級接受 syslog 值（`emerg/0`、`alert/1`、`crit/2`、`err/3`、`warning/4`、`notice/5`、`info/6`、`debug/7`）。在 Windows 上不支援 SysLog 模式——將顯示警告，提示取消勾選並使用應用程式日誌。若 `systemd`/服務不可用，將顯示 `journalctl` 啟動錯誤訊息。

範例：直接從伺服器主控台查看相同的日誌。 當面板不可用時（例如無法啟動），可以直接讀取系統日誌——這正是面板在 SysLog 模式下執行的命令：

```
# [ ] 100 [ ]warning [ ]
journalctl -u x-ui --no-pager -n 100 -p warning

# [ ]
journalctl -u x-ui -f
```

此視窗中的等級用於過濾輸出。實際寫入主控台/syslog 的最低等級由面板的日誌等級設定決定（環境變數，預設為 `Info`；面板始終以 `DEBUG` 等級寫入檔案）。

Xray 存取日誌（存取記錄）

用於 Xray access 日誌的獨立視窗（`POST /panel/api/server/xraylogs/{count}`）。它解析 Xray 存取日誌的行並以表格形式顯示：**Date**、**From**、**To**、**Inbound**、**Outbound**、**Email**。

從 3.4.1 版本開始，此視窗及 Xray 狀態卡片上的呼叫按鈕被標記為「存取日誌」（Access Logs）——之前它們僅被稱為「日誌」。此重新命名是為了避免將 Xray access 日誌檢視器與面板本身的日誌檢視器混淆，後者之前有相同的名稱。

元素	預設值	說明
行數	20	20 / 50 / 100 / 500 / 1000
過濾器	空白	子字符串文字搜尋（按 Enter 套用）
自動更新（勾選框）	停用	每 5 秒重新讀取日誌（見下文）
Direct（勾選框）	啟用	顯示直連連線（透過 freedom outbound 的流量）
Blocked（勾選框）	啟用	顯示被封鎖的連線（進入 blackhole outbound 的流量）
Proxy（勾選框）	啟用	顯示代理流量

事件類型根據日誌行中的出站連線標籤自動確定：符合 freedom 標籤 → 「DIRECT」（綠色），blackhole → 「BLOCKED」（紅色），其他所有 → 「PROXY」（藍色）。api → api 行和空行會被跳過。

自動更新。 兩個日誌視窗（「日誌」和「存取日誌」）都有「自動更新」（Auto Update）勾選框。啟用後，日誌內容每 5 秒自動重新讀取一次，並保留視窗的所有目前設定——所選行數、等級/過濾器以及 Direct / Blocked / Proxy 勾選框。關閉視窗或取消勾選後，輪詢即停止。

若要此視窗顯示記錄，Xray 必須啟用 **存取日誌** 並設定檔案路徑（非 none）——見下文。若 access 日誌已停用或檔案不可存取，視窗將為空（「No Record...」）。

16.3. Xray 日誌等級與設定

Xray 本身的日誌記錄參數在「Xray 設定」頁面的「日誌」（Log）區塊中設定，並帶有警告：

「日誌可能會降低伺服器效能。請僅在必要時啟用您需要的日誌類型！」

欄位	欄位名稱	預設值	說明
日誌等級 (logLevel)	Log Level	warning	Xray 錯誤日誌的詳細程度。允許值：debug、info、notice、warning、error。提示：「用於錯誤日誌的日誌等級，指定需要記錄的資訊。」
存取日誌 (accessLog)	Access Log	none	存取日誌檔案的路徑。特殊值 none 停用存取日誌。提示：「存取日誌檔案的路徑。特殊值「none」停用存取日誌。」

欄位	欄位名稱	預設值	說明
錯誤日誌 (errorLog)	Error Log	空白 (預設路徑)	錯誤日誌檔案的路徑；none 停用。提示：「錯誤日誌檔案的路徑。特殊值「none」停用錯誤日誌。」
DNS 日誌 (dnsLog)	DNS Log	false (停用)	啟用 DNS 請求日誌記錄。提示：「啟用 DNS 請求日誌」。
地址遮罩 (maskAddress)	Mask Address	空白 (停用)	啟用後，日誌中的真實 IP 地址會自動替換為遮罩地址。提示：「啟用後，日誌中的真實 IP 地址將替換為遮罩地址。」

由於預設「存取日誌」= none，「Xray 日誌」視窗（第 16.2 節）初始為空。若要使其正常顯示，請在此設定 access 日誌的路徑並重啟 Xray。

請注意：空的 access 日誌僅影響此視窗。「儀表板」上的線上用戶端清單和用戶端表單中的 IP 數量限制不依賴 access 日誌——面板透過 Xray 核心的 online-stats API（連線統計）確定線上用戶端並統計其 IP 地址。對於不具備此 API 的舊版核心，面板會自動回退到舊方法（讀取 access 日誌），此時 IP 限制仍需要此處設定 access 日誌路徑。

IP 數量限制與 fail2ban。 用戶端的 IP 數量限制本身（用戶端表單和批次新增中的「IP Limit」欄位）只有在安裝了 fail2ban 時才在伺服器上生效——正是 fail2ban 封鎖超出限制的地址。面板會檢查 fail2ban 是否存在（GET /panel/api/server/fail2banStatus）；若不存在，「IP Limit」欄位會變為不可用，並顯示說明提示（Windows 上顯示單獨訊息），而此類伺服器上先前設定的限制會自動歸零，因為它們本來就不會生效。fail2ban 封鎖同時適用於 TCP 和 UDP。自 3.5.0 起，「死」連線（用戶端未正確關閉 TCP 就消失）只會被記錄一次，而非每次 10 秒掃描都記錄：[LIMIT_IP] 行與斷線流程僅在實際重新連線時才會重複，因此 fail2ban 計數器不會虛增，也不再需要調高 maxretry（日誌格式與 failregex 均未變更）。在一般伺服器上，fail2ban 現在會在安裝和更新面板時自動安裝（見第 16.5 節）。

範例：能讓「Xray 日誌」視窗開始顯示記錄的 log 區塊。在 Xray 的 JSON 設定中，如下所示：

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

關鍵是將 "access": "none" 替換為檔案路徑（例如 "./access.log"）。儲存後重啟 Xray，「Xray 日誌」視窗的表格中將出現記錄。

16.4. 管理 Xray：停止與重啟

Xray 的狀態透過「儀表板」上的 Xray 卡片管理。目前狀態顯示為以下值之一：**已執行**（Running）、**已停止**（Stopped）、**未知**（Unknown）、**錯誤**（Error）。出現錯誤時，可使用「Xray 啟動錯誤」工具提示。

按鈕	按鈕名稱	端點	動作
停止	Stop	POST /panel/api/server/stopXrayService	停止 Xray 程序。成功後——警告通知「Xray service has been stopped」。
重啟	Restart	POST /panel/api/server/restartXrayService	以目前設定重啟（或啟動）Xray。成功後——通知「Xray service has been restarted successfully」。

任何操作完成後，面板會透過 WebSocket 廣播新狀態，因此「儀表板」上的狀態無需重新整理頁面即可更新。若操作出錯，Xray 狀態變為「錯誤」，錯誤文字出現在通知中。

除了手動重啟外，面板本身也會檢查是否需要重啟 Xray（每 30 秒的後台任務）以及程序是否崩潰（每秒檢查）——見第 16.6 節。

通道健康監控（Xray 自動重啟）

3.4.1 版本新增了可選的**通道健康監控器**。啟用後，面板會定期檢查指定 URL 的可達性，在連續多次檢查失敗後自動重啟 Xray 核心——這有助於恢復停止通過流量的通道。預設情況下，監控器**已停用**，且**只能透過服務環境變數設定**（網頁介面中沒有其設定——這是作者有意為之）。

變數 `XUI_TUNNEL_HEALTH_MONITOR=true` 啟用監控器。`XUI_TUNNEL_HEALTH_PROXY` 應指向本地 xray inbound（例如 `socks5://127.0.0.1:1080`）——這樣探測會透過 Xray 本身進行，真正檢查通道；若未設定，則僅檢查主機連線，重啟並不能修復伺服器網路連線問題。其他變數設定檢查參數：

變數	用途	預設值
<code>XUI_TUNNEL_HEALTH_MONITOR</code>	啟用監控器（開/關）	<code>false</code>
<code>XUI_TUNNEL_HEALTH_PROXY</code>	探測使用的代理（指定本地 xray inbound）	空白
<code>XUI_TUNNEL_HEALTH_URL</code>	檢查的 URL	<code>https://www.cloudflare.com/cdn-cgi/trace</code>
<code>XUI_TUNNEL_HEALTH_INTERVAL</code>	檢查間隔	<code>30s</code>
<code>XUI_TUNNEL_HEALTH_TIMEOUT</code>	單次檢查逾時	<code>10s</code>
<code>XUI_TUNNEL_HEALTH_FAILURES</code>	觸發重啟前的連續失敗次數	<code>3</code>
<code>XUI_TUNNEL_HEALTH_COOLDOWN</code>	兩次重啟之間的最短間隔	<code>5m</code>

Xray 重啟會中斷所有已連線用戶端的連線，因此建議將間隔和失敗次數閾值設定得足夠大，以避免單次探測的偶發失敗導致不必要的重啟。

16.5. 重啟與更新面板

重啟面板

在「**面板設定**」頁面有「**重啟面板**」（Restart Panel，`POST /panel/api/setting/restartPanel`）動作。確認後，面板將在 **3 秒後** 重啟。

訊息：

- 確認：「您確定要重啟面板嗎？確認後，重啟將在 3 秒後發生。若面板不可用，請檢查伺服器日誌。」
- 成功：「面板已成功重啟」。

從技術上講，在 Linux 上重啟是透過向面板程序發送 `SIGHUP` 信號完成的（或透過已註冊的鉤子）。Windows 不支援發送 `SIGHUP`。

面板自動更新（Update Panel）

「儀表板」提供「**更新面板**」（Update Panel）功能——直接從網頁介面將 3X-UI 更新至最新版本。

更新前，面板會比對版本（`GET /panel/api/server/getPanelUpdateInfo`），從 GitHub 查詢 3x-ui 的最新版本：

欄位	說明
目前面板版本	Current panel version
最新面板版本	Latest panel version
面板已是最新 / 「已更新」	Panel is up to date / Up to date——若無新版本則顯示

啟動更新——`POST /panel/api/server/updatePanel`。確認對話框：

- 「您確定要更新面板嗎？」
- 「這將把 3X-UI 更新至 #version# 版本並重啟面板服務。」

啟動後——彈出訊息「更新面板已開始」（Panel update started）；版本檢查失敗時——「面板更新檢查失敗」（Panel update check failed）。

伺服器端行為：自動更新僅在 **Linux** 上受支援（其他作業系統將返回錯誤「panel web update is supported only on Linux installations」）。面板從 GitHub（raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh）下載官方 `update.sh` 腳本，並在單獨程序中執行：優先透過 `systemd-run` 在獨立 unit（`x-ui-web-update-<timestamp>`）中執行，若無 `systemd` 則以獨立的分離程序執行。腳本完成後，更新元件並重啟面板服務。執行需要 `bash`。

若更新過程中腳本生成了新的隨機網頁基礎路徑（Web Base Path），`x-ui` 服務會自動重啟，使新路徑立即生效。（若不重啟，伺服器將繼續提供舊路徑，而介面顯示新路徑，導致新地址在手動重啟前無法存取。）

Dev 頻道（rolling），自 3.4.2 起。「儀表板」上的面板版本按鈕現在永遠開啟更新視窗（先前在最新穩定版本上會導向 GitHub 發行頁面），且「Dev 頻道」切換開關（`devChannelEnable`）一律顯示——即使在穩定版本上亦然。啟用後，下次更新時即可切換至「rolling」dev 頻道（依 `main` 中每個提交建置）。啟用時會顯示警告：「Dev-сборки отслеживают каждый коммит в main и не являются стабильными релизами — автоматического отката нет」。在使用者未明確操作前，不會更新任何內容。

開發頻道（Dev，持續整合版本）

除了更新至穩定版本外，還有可選的「開發頻道」（Dev）。此切換開關僅在開發版本（按單獨提交建置的 CI 版本）上的更新視窗中顯示；穩定版本上不可見。啟用後，面板將更新至追蹤 `main` 分支每個提交的 `dev-latest` 持續整合版本，這不是穩定版本——會顯示警告，說明開發版本不穩定且沒有自動回退。在開發模式下，視窗顯示「目前提交」/「最新提交」而非版本號。此功能僅在具備 `systemd` 的 Linux 上可用。

在開發版本上，面板將其版本顯示為 `dev+<[?][?][?][?]>` 而非誤導性的穩定版本號——在側邊欄徽章、「儀表板」卡片、更新視窗、Telegram Bot 狀態報告以及 `x-ui -v` 命令輸出中均如此。穩定版本的版本顯示方式不變。

在節點（nodes）上，同一個 3x-ui 面板可透過 `POST /panel/api/nodes/updatePanel` 集中更新——見節點相關章節。

自動安裝 fail2ban

為使用戶端的 IP 數量限制（第 16.3 節）開箱即用，在一般伺服器上安裝和更新面板時，`fail2ban` 現在會自動安裝和設定（之前僅在 Docker 映像中如此）。其行為由環境變數 `XUI_ENABLE_FAIL2BAN` 控制：若未設定或等於 `true`，則執行設定。手動執行可使用命令 `x-ui setup-fail2ban`。`fail2ban` 設定失敗不會中斷面板的安裝或更新。

在純 IPv6 主機上安裝與更新

`install.sh` 和 `update.sh` 腳本現在可在僅有 IPv6 的伺服器上正常工作：下載版本、`x-ui.sh` 腳本和服務檔案不再強制使用 IPv4（`curl -4`），而是使用可用的協議。因此，面板可以在沒有 IPv4 地址的主機上安裝和更新。

3.5.0 的腳本修正

- **RHEL 系**（Rocky/Alma/RHEL/Oracle）：從選單本機安裝 PostgreSQL 現在可正常運作（改用密碼驗證而非 `ident`），IP 限制所需的 `fail2ban` 改從 **EPEL** 安裝。
- **Arch/Manjaro**：安裝與更新不再執行完整的系統升級 `pacman -Syu` ——僅更新套件資料庫。
- **32 位元 ARM**：下載的 Xray 二進位檔命名為 `xray-linux-arm32` ——與面板啟動它時使用的名稱一致（先前更新會以其他名稱存放檔案，面板因此繼續執行舊核心）。

- **IP 憑證**：簽發前會顯示自動偵測到的公網 IPv4 供確認（Enter——接受）；拒絕時改為帶驗證的手動輸入。
- **ACME 連接埠選擇**：按 Enter（接受預設連接埠 80）不再誤報「Your input is invalid」。

透過 XUI_PORT 變數覆蓋面板連接埠

可以透過環境變數 `XUI_PORT` 覆蓋網頁面板的監聽連接埠——它僅在目前程序執行期間有效，**不會修改資料庫中儲存的 `webPort` 值**。允許值為 1 到 65535；空白、不正確或超出範圍的值將被忽略（使用 `webPort`），並在日誌中記錄警告。這在部署時（尤其是 Docker 中）很有用：使用 bridge 網路時，容器發布的連接埠必須與 `XUI_PORT` 一致——例如 `XUI_PORT=8080` 配合 `ports: "8080:8080"`。

更新與切換 Xray-core 版本

在「儀表板」上，還可以獨立於面板管理 Xray-core 的版本。

- **Xray 更新**（Xray Updates）/**選擇版本**（Version）——可用版本的下拉選單。提示：「選擇所需版本」以及警告「重要：舊版本可能不支援目前設定」。
- **安裝/切換版本**——`POST /panel/api/server/installXray/{version}`。對話框：「切換 Xray 版本」/「您確定要切換 Xray 版本嗎？」。成功後——「Xray 已成功更新」。

範例：透過 API 請求切換 Xray-core 版本。版本以 XTLS/Xray-core 的發行標籤指定（帶 `v` 前綴）。例如，切換至 `v1.8.24`：

```
curl -s -b cookies.txt -X POST \
  https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

（`cookies.txt` ——第 16.1 節範例中的 cookie 檔案。）安裝完成後，Xray 將自動以所選版本重啟。

在伺服器端切換版本時，Xray 首先停止，從 GitHub（XTLS/Xray-core）下載所需版本的封存檔，解壓並替換二進位檔案，然後在驗證封存檔/二進位檔案的檢驗大小後重啟 Xray。

16.6. 週期性任務（cron）

面板在啟動時註冊一系列後台任務。其排程是固定的（除 Telegram 報告排程和 LDAP 同步外，無法在介面中設定）。以下是與運維相關的任務：

任務	排程	用途
檢查 Xray 是否正在執行	每 1 秒	監控 Xray 程序是否已啟動
檢查是否需要重啟 Xray	每 30 秒	若設定已標記為已更改，則重啟
收集 Xray 流量	每 5 秒（面板啟動後 5 秒開始）	統計 inbound/用戶端流量
檢查用戶端 IP	每 10 秒	根據日誌監控 IP 限制

任務	排程	用途
節點心跳與流量同步	每 5 秒	與節點交換資料
清理日誌	每日 (@daily)	清理 IP 限制日誌和持久 access 日誌，將目前日誌旋轉至 *.prev.log 。自 3.5.0 起，每日清理也涵蓋 Xray 的 error 日誌（先前僅 access）
限制 Xray 日誌成長	每 10 分鐘 (@every 10m)	3.5.0 新增。 當 access 或 error 日誌任一超過 64 MiB 時將其截斷；已停用的日誌 (none /空白) 不受影響
按週期重置流量	@hourly 、 @daily 、 @weekly 、 @monthly	重置設定了相應自動重置週期的 inbound（及其用戶端）的流量計數器
Telegram 報告	在 Bot 設定中設定（預設 @daily ）	向管理員傳送報告；啟用相應選項時附帶資料庫備份（第 16.1 節）
重置 Telegram 雜湊儲存	每 2 分鐘	僅在 Bot 已啟用時
Telegram CPU 負載監控	每 10 秒	僅在 CPU 閾值 > 0 時

補充說明：

- **週期性流量重置** 僅針對選擇了相應自動重置模式（每小時/每日/每週/每月）的 inbound 觸發。該任務重置 inbound 本身及其所有用戶端的流量。
- **到期和耗盡檢查**。根據到期時間和流量限制耗盡停用戶端的操作在流量統計過程中執行：
`expiry_time` 已過期或配額已耗盡的用戶端被標記並停用，必要時計算下一個到期時間（針對週期性限制和「從首次使用開始計時」模式）。在「儀表板」和列表中以「已到期」/「已耗盡」/「即將到期」狀態反映。
- **Telegram 自動備份** 是報告任務的副作用，沒有單獨的僅用於備份的 cron 排程。因此，自動備份的頻率等於 Bot 報告的頻率。

16.7. 主控台選單與 CLI (x-ui)

在伺服器上，面板透過 `x-ui` 命令管理。不帶參數執行時，開啟互動式選單「3X-UI Panel Management Script」；帶參數執行時，執行特定子命令。與運維相關的選單項目：

選單編號	項目	動作
1	Install	安裝面板（下載並執行 <code>install.sh</code> ）
2	Update	將所有 x-ui 元件更新至最新版本（不丟失資料）；完成後自動重啟
3	Update to Dev Channel (latest commit)	確認後更新至 <code>dev-latest</code> 持續整合版本（ <code>main</code> 分支的最新提交）（見第 16.5 節）
4	Update Menu	僅更新 <code>x-ui</code> 選單腳本本身
5	Legacy Version	按輸入的版本號安裝指定的（舊）版本（例如 <code>2.4.0</code> ）

選單編號	項目	動作
6	Uninstall	完全移除面板和 Xray（見第 16.8 節）
7	Reset Username & Password	重置管理員帳號/密碼
8	Reset Web Base Path	重置網頁面板基礎路徑
9	Reset Settings	將設定重置為預設值
10	Change Port	更改面板連接埠
11	View Current Settings	檢視目前設定
12-14	Start / Stop / Restart	啟動、停止、重啟面板服務
15	Restart Xray	僅重啟 Xray
16	Check Status	目前服務狀態
17	Logs Management	檢視和清理日誌（見下文）
18-19	Enable / Disable Autostart	啟用/停用作業系統啟動時的自動啟動
27	Update Geo Files	更新地理位置檔案（GeoIP/GeoSite）
25	PostgreSQL Management	管理 PostgreSQL

選單項目編號在 3.4.1 版本中有所變化：由於新增了第 3 項「Update to Dev Channel」，所有後續項目的編號遞增了 1。選單項目總數增至 28，選擇範圍為 [0-28]。

CLI 中的日誌管理（第 16 項）

「Logs Management」子選單現在是第 17 項（之前是第 16 項）：

- **Debug Log**——串流查看服務日誌：`journalctl -u x-ui -e --no-pager -f -p debug`（在 Alpine 上——對 `/var/log/messages` 使用 `grep`）。
- **Clear All logs**——清理系統日誌：`journalctl --rotate + journalctl --vacuum-time=1s`，之後服務重啟。（在 Alpine 上不可用。）

x-ui 直接子命令

所有可用子命令：

命令	說明
<code>x-ui</code>	開啟管理選單
<code>x-ui start</code>	啟動面板
<code>x-ui stop</code>	停止面板
<code>x-ui restart</code>	重啟面板

命令	說明
<code>x-ui restart-xray</code>	重啟 Xray
<code>x-ui status</code>	目前狀態
<code>x-ui settings</code>	顯示目前設定
<code>x-ui enable</code>	啟用作業系統啟動時的自動啟動
<code>x-ui disable</code>	停用自動啟動
<code>x-ui log</code>	檢視日誌
<code>x-ui banlog</code>	檢視 Fail2ban 封鎖日誌
<code>x-ui setup-fail2ban</code>	安裝並設定 fail2ban 以用於 IP 限制（見第 16.5 節）
<code>x-ui update</code>	更新面板

| `x-ui update-dev` | 將面板更新至開發頻道（`dev-latest` 持續整合版本） || `x-ui update-all-geofiles` | 更新所有地理位置檔案（並隨後重啟） || `x-ui migrateDB [file]` | 轉換資料庫 `.db` ⇌ `.dump` (SQLite) || `x-ui legacy` | 安裝舊版本 || `x-ui install` | 安裝面板 || `x-ui uninstall` | 移除面板 |

命令 `x-ui update` 下載並執行官方 `update.sh`（與第 16.5 節中的網頁更新相同），並請求確認：「This function will update all x-ui components to the latest version, and the data will not be lost.」完成後，面板自動重啟。

setting 子命令中的 `-webCert` / `-webCertKey` 旗標。可以直接在子命令 `x-ui setting -webCert <[?]?> -webCertKey <[?]?>` 中設定網頁面板的憑證和私鑰路徑——指定任一旗標都會儲存相應路徑（與單獨的 `cert` 子命令相同），面板會立即切換至 HTTPS。

透過 CLI 取得 API Token

透過 CLI 取得 API Token 的命令（選單項目/命令 `x-ui`）不會顯示先前已頒發的 token。API Token 僅以雜湊形式儲存，因此無法以明文形式取得現有 token。若已設定 token，命令會顯示其數量，建議在面板中管理 token（**Settings** → **API Tokens**，見 API Token 相關章節），並立即生成一個名為 `cli-fallback-<[?]?[?]?>` 的 **新備用 token** 並顯示，以確保 CLI 在不進入介面的情況下仍可使用。

16.8. 移除面板

移除操作從 CLI 執行——選單第 5 項（**Uninstall**）或命令 `x-ui uninstall`。移除前需要確認（預設為「否」）：「Are you sure you want to uninstall the panel? xray will also uninstalled!」。

確認後，腳本：

1. 停止服務並停用其自動啟動（`systemctl stop/disable x-ui`，或在 Alpine 上——`rc-service / rc-update`），刪除服務 unit 檔案並重新載入 systemd 設定。

- 刪除資料和應用程式目錄（`/etc/x-ui/`、安裝目錄）以及服務環境檔案（`/etc/default/x-ui`、`/etc/conf.d/x-ui` 或 `/etc/sysconfig/x-ui` ——依發行版而定）。
- 刪除 `x-ui` 腳本本身，並輸出訊息「Uninstalled Successfully.」以及重新安裝的命令。

若面板使用 PostgreSQL（環境檔案中 `XUI_DB_TYPE=postgres`），在刪除面板檔案後，腳本會額外詢問是否也需要刪除 PostgreSQL 伺服器及其所有資料庫：「Also purge PostgreSQL and delete all of its data?」。此請求需要明確確認（預設為拒絕），並附有警告：刪除操作將影響機器上的所有 PostgreSQL 資料庫，包括屬於其他應用程式的資料庫，且不可逆。拒絕時，PostgreSQL 及其資料保持不變。

移除操作不可逆：面板連同 Xray 和所有資料（包括資料庫）一起被刪除。若可能需要資料，請事先匯出資料庫（第 16.1 節）。

16.9. x-ui migrateDB 命令

從 3.3.0 版本開始，管理腳本 `x-ui.sh` 新增了 `migrateDB` 子命令——這是內建二進位檔案 `x-ui`（`x-ui migrate-db`）的封裝，用於在兩種格式之間轉換面板的 SQLite 資料庫：二進位 `.db` 格式和可攜式文字轉儲 `.dump` 格式（普通 SQL 文字）。

命令功能

命令支援兩個方向，自動根據輸入檔案確定方向：

方向	名稱	行為
<code>.db → .dump</code>	dump（匯出）	將 SQLite 二進位資料庫匯出為文字 SQL 檔案
<code>.dump → .db</code>	restore（還原）	從文字 SQL 檔案重建 SQLite 二進位資料庫

底層腳本呼叫面板二進位檔案：

- 匯出：`x-ui migrate-db --src <[?]> --dump <[?]>`
- 還原：`x-ui migrate-db --restore <[?]> --out <[?]>`

呼叫語法

```
x-ui migrateDB [file.db|file.dump] [output]
```

- [file.db|file.dump]** ——輸入檔案（第一個參數）。若未指定，使用預設的面板安裝資料庫：`/etc/x-ui/x-ui.db`。
- [output]** ——輸出檔案路徑（第二個參數）。選填：若未提供，將在輸入檔案旁自動選擇名稱（見下文）。

範例：


```
x-ui migrateDB                               # [?] [?] /etc/x-ui/x-ui.db -> /etc/x-ui/x-
ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db       # [?] [?] [?] [?] [?] .db
```

如何確定方向

腳本根據輸入檔案的副檔名判斷：

- *.db、*.sqlite、*.sqlite3 → **dump** 模式（匯出為文字）；
- *.dump、*.sql → **restore** 模式（重建資料庫）。

若副檔名無法識別，腳本讀取檔案的前 16 個位元組：簽章 `SQLite format 3` 表示二進位資料庫（dump 模式），否則視為轉儲（restore 模式）。

若未提供第二個參數，輸出檔案名稱為：

- 匯出時——與輸入相同的名稱，副檔名改為 `.dump`；
- 還原時——相同名稱，副檔名改為 `.db`。

防護檢查與行為

- **二進位檔案存在性**。若找不到 `x-ui` 二進位檔案或其不可執行——顯示錯誤「`x-ui binary not found ... Is the panel installed?`」。
- **版本中的功能支援**。腳本透過 `x-ui migrate-db -h` 驗證二進位檔案是否支援 `migrate-db --dump/--restore`。若不支援——建議先使用 `x-ui update` 更新面板。
- **輸入檔案存在性**。若輸入檔案不存在，顯示錯誤和呼叫語法說明。
- **覆寫輸出**。若輸出檔案已存在，請求確認（預設為「否」）；若未確認，操作取消。還原時，舊的輸出檔案會預先刪除。
- **保護「活躍」資料庫**。還原至預設資料庫 `/etc/x-ui/x-ui.db` 且面板正在執行時，操作被拒絕，並要求先停止面板（`x-ui stop`）或選擇其他輸出路徑。這防止了對正在執行服務的工作資料庫進行覆寫。
- 若資料庫重建失敗，不完整的輸出檔案將被刪除。

使用場景

- **備份**。文字 `.dump` 格式可供人類讀取，便於在版本控制系統中儲存以及差異化檢視資料庫內容。
- **遷移**。轉儲可在機器間移植，並對 SQLite 檔案格式版本差異具有韌性——可在新伺服器上從中重建可用的 `.db`。
- **診斷**。可以從 `.dump` 直觀地查看面板的結構和資料，而無需 SQLite 工具。

互動式模式

除直接呼叫外，轉換也可從互動式選單中執行。在 PostgreSQL 子選單（`x-ui` → PostgreSQL 管理部分）中有第 9 項「**Convert SQLite .db <-> .dump**」：它詢問輸入檔案路徑（預設 `/etc/x-ui/x-ui.db`）和輸出路徑（可留空以自動命名），方向與 CLI 模式相同，自動確定。

本文件根據 3X-UI 原始碼編寫。若您所用版本中的某個介面項目有所不同，請以面板實際行為及 UI 中的提示為準。